

Table of Contents

1. [Functions • Overview • Palantir](#)
2. [Functions • Overview • Palantir](#)
3. [Functions • Feature support by language • Palantir](#)
4. [Functions • Getting started • Palantir](#)
5. [Functions • Types reference • Palantir](#)
6. [Functions • Branching functions • Palantir](#)
7. [Functions • Function management • Function versioning • Palantir](#)
8. [Functions • Function management • Manage published functions • Palantir](#)
9. [Functions • Function management • Functions in Marketplace • Palantir](#)
10. [Functions • Function management • Function monitoring • Palantir](#)
11. [Functions • Function management • Instrumentation and telemetry • Palantir](#)
12. [Functions • Function management • Permissions • Palantir](#)
13. [Functions • Function consumption • Use functions in the platform • Palantir](#)
14. [Functions • Function consumption • Function metrics • Palantir](#)
15. [Functions • Function consumption • Optimize performance • Palantir](#)
16. [Functions • Function consumption • Version range dependencies • Palantir](#)
17. [Functions • Language agnostic features • Ontology edits • Palantir](#)
18. [Functions • Language agnostic features • Publish and call query functions through API gateway](#)
19. [Functions • Language agnostic features • Configure notifications • Palantir](#)
20. [Functions • Language agnostic features • Make API calls from functions • Palantir](#)
21. [Functions • Language agnostic features • Use platform APIs with the Foundry platform SDK •](#)
22. [Functions • Language agnostic features • Deploy functions • Palantir](#)
23. [Functions • Language agnostic features • User-facing errors • Palantir](#)
24. [Functions • Language agnostic features • Streaming functions • Palantir](#)
25. [Functions • Language agnostic features • Functions settings • Palantir](#)
26. [Functions • TypeScript v2 • Getting started • Palantir](#)
27. [Functions • TypeScript v2 • Ontology edits • Palantir](#)
28. [Functions • TypeScript v2 • Staged writes • Palantir](#)
29. [Functions • Python • Getting started • Palantir](#)
30. [Functions • Python • Functions on objects • Palantir](#)
31. [Functions • Python • Create a custom aggregation • Palantir](#)
32. [Functions • Python • Ontology edits • Palantir](#)
33. [Functions • Python • Use Python functions in Pipeline Builder • Palantir](#)
34. [Functions • Python • Use Python functions in Workshop • Palantir](#)
35. [Functions • Python • Local development • Palantir](#)
36. [Functions • TypeScript v1 • Getting started • Palantir](#)
37. [Functions • TypeScript v1 • Migrate from TypeScript v1 to TypeScript v2 • Palantir](#)
38. [Functions • TypeScript v1 • Decorators • Palantir](#)
39. [Functions • TypeScript v1 • Ontology edits • Palantir](#)

Ontology building / Functions / Overview

Functions

Functions enable code authors to write logic that can be executed quickly in operational contexts, such as dashboards and applications designed to empower decision-making processes. This logic is executed on the server side in an isolated environment.

Notably, functions include first-class support for authoring logic based on the Ontology. This includes support for reading the properties of various object types, traversing links, and flexibly making Ontology edits.

Common use cases for functions include:

- Returning object sets or variable values for use in [Workshop](#).
- Displaying transformed values in a derived table column using [Workshop's function-backed columns](#).
- Aggregating object type values to display as [Workshop charts](#).
- Expressing a complex edit to the Ontology that updates many objects through a [function backed action](#).
- Running logic in the backend to return information to be displayed in the frontend in [Slate](#).
- Computing custom metrics or aggregations for display in [Quiver](#).
- Querying external systems to enrich objects in the Ontology through [external functions](#).
- Using Python functions as a sidecar container in [Pipeline Builder](#).

The languages supported by functions are [TypeScript ↗](#) and [Python ↗](#).

For more information on feature support by language and choosing a language or language version, see [API Reference ↗ language feature support specifications](#).

To learn more about using functions in Foundry, we recommend the following:

Send feedback

- [Getting started with TypeScript v1 functions](#)
- [Getting started with TypeScript v2 functions](#)
- [Getting started with Python functions](#)

Palantir Learning portal

Try our "[Speedrun: Your first Ontology function](#)" course ↗ on learn.palantir.com.

PREVIOUS

← [Action types / Add action types to a Marketplace product](#)

NEXT

[Feature support by language](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Functions / Overview

Functions

Functions enable code authors to write logic that can be executed quickly in operational contexts, such as dashboards and applications designed to empower decision-making processes. This logic is executed on the server side in an isolated environment.

Notably, functions include first-class support for authoring logic based on the Ontology. This includes support for reading the properties of various object types, traversing links, and flexibly making Ontology edits.

Common use cases for functions include:

- Returning object sets or variable values for use in [Workshop](#).
- Displaying transformed values in a derived table column using [Workshop's function-backed columns](#).
- Aggregating object type values to display as [Workshop charts](#).
- Expressing a complex edit to the Ontology that updates many objects through a [function backed action](#).
- Running logic in the backend to return information to be displayed in the frontend in [Slate](#).
- Computing custom metrics or aggregations for display in [Quiver](#).
- Querying external systems to enrich objects in the Ontology through [external functions](#).
- Using Python functions as a sidecar container in [Pipeline Builder](#).

The languages supported by functions are [TypeScript ↗](#) and [Python ↗](#).

For more information on feature support by language and choosing a language or language version, see [API Reference ↗ language feature support specifications](#).

To learn more about using functions in Foundry, we recommend the following:

Send feedback

- [Getting started with TypeScript v1 functions](#)
- [Getting started with TypeScript v2 functions](#)
- [Getting started with Python functions](#)

Palantir Learning portal

Try our "[Speedrun: Your first Ontology function](#)" course ↗ on learn.palantir.com.

PREVIOUS

← [Action types / Add action types to a Marketplace product](#)

NEXT

[Feature support by language](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Feature support by language

Not all features are supported by all languages. Refer to the chart below for feature support by language.

Functions capability by language	AIP Logic	TypeScript v1	TypeScript v2	Python	Description
Ontology object support	Yes	Yes	Yes	Yes	The ability to access Ontology objects in your function.
Ontology interfaces support	No	No	Yes	No	The ability to access and edit Ontology interfaces in your function.
Ontology edits support	Yes	Yes	Yes	Yes	The ability to edit Ontology objects in your function.
Queryable in Workshop	Yes	Yes	Yes	Yes	Invoking a function from a Workshop application .
Usable in Pipeline	No	No	No	Yes	Calling a function from Pipeline Builder pipelines .
API Reference ↗ on models	Yes	Yes	No	No	Send feedback deployment

Functions capability by language	AIP Logic	TypeScript v1	TypeScript v2	Python	Description
support					models from a function .
Semantic search support	Yes	Yes	Yes	Yes	Use functions to create vectors for semantic search .
Webhook support	No	Yes	No	No	The ability to call webhooks from functions .
External API call support	No	Yes	Yes	Yes	Querying external services from within functions .
Serverless execution support	Yes	Yes	Yes	Yes	A serverless function will be spun up on demand when invoked. Refer to serverless functions below for more information.
Deployed execution support	No	No	Yes	Yes	A deployed function will have dedicated resources allocated to it, ready to serve requests.
Call function from API	Yes	Yes	Yes	Yes	The ability to hit a query function from the API gateway.
Marketplace support	Yes	Yes	Yes	Yes	

API Reference ↗

Send feedback

Functions capability by language	AIP Logic	TypeScript v1	TypeScript v2	Python	Description
					functions in Marketplace .
Bring-your-own-model	Yes	Yes	No	No	The ability to register a function as a model . The function interface method is a legacy approach that applies to TypeScript v1 only.

Ontology SDK support

Python and TypeScript v2 functions support the [Ontology SDK](#) (OSDK). The OSDK allows you to leverage the Ontology directly from your development environment and provides [benefits](#) such as compatibility with Developer Console and OSDK versioning. We recommend using Python or TypeScript v2 to access these benefits in your functions repository.

TypeScript v1 vs. TypeScript v2

Both TypeScript v1 and TypeScript v2 allow users to leverage TypeScript's core language features, but there are differences in supported platform features, as shown in the feature support table above. We recommend building workflows using TypeScript v2 functions to take advantage of several key improvements over TypeScript v1:

- Serverless execution in a full Node.js runtime:** TypeScript v2 functions run in a Node.js environment, supporting core modules like `fs`, `child_process`, and `crypto`. This enables greater compatibility with NPM libraries that interact with the file system, [API Reference](#) [↗](#)ensive tasks in parallel, or require other system-level operations.
- First-class OSDK support:** The OSDK can now be used seamlessly in TypeScript v2 functions, making it easy to reuse code, both in and out of the platform. We also have more efficient APIs for working with large-scale Ontology data.

Send feedback

- **Configurable resource requests:** TypeScript v2 functions allow you to request up to 8 vCPUs and 5GB of memory, offering greater control over performance and scalability.

Serverless functions

If serverless functions are enabled for your enrollment, new repositories will use serverless functions by default. We recommend using serverless functions instead of deployed functions for most use cases. With serverless functions, you can have multiple versions of a single function available on demand, making upgrades safer.

When using Python or TypeScript v2 functions, there are also [some cases where deployed functions are preferred](#) or must be used instead of serverless, but these are not common. If available, serverless functions are preferred for several reasons:

- Serverless functions enable different versions of a single function to be executed on demand, making upgrades safer. With deployed functions, you can only run a single function version at a time.
- Serverless functions only incur costs when executed, while deployed functions incur costs as long as the deployment is running.
- Serverless functions require less upfront setup and long-term maintenance, as the infrastructure is managed automatically.

← PREVIOUS
Overview

Getting started → NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

API Reference ↗

Send feedback

Ontology building / Functions / Getting started

Getting started with functions

There are three language options for getting started with functions in Foundry; TypeScript v1, TypeScript v2, and Python. For more information on supported features for each language, review the [language feature support](#) specifications.

Although each language has a different set of supported features, you will be able to access the same basic platform functionality for each language, including running, testing, and publishing functions. This page provides an overview of these features to help you understand how to use functions repositories, regardless of which language you will be working with.

For more detailed instructions on getting started with a specific language, refer to the tutorials below:

- [Getting started with TypeScript v1 functions](#)
- [Getting started with TypeScript v2 functions](#)
- [Getting started with Python functions](#)

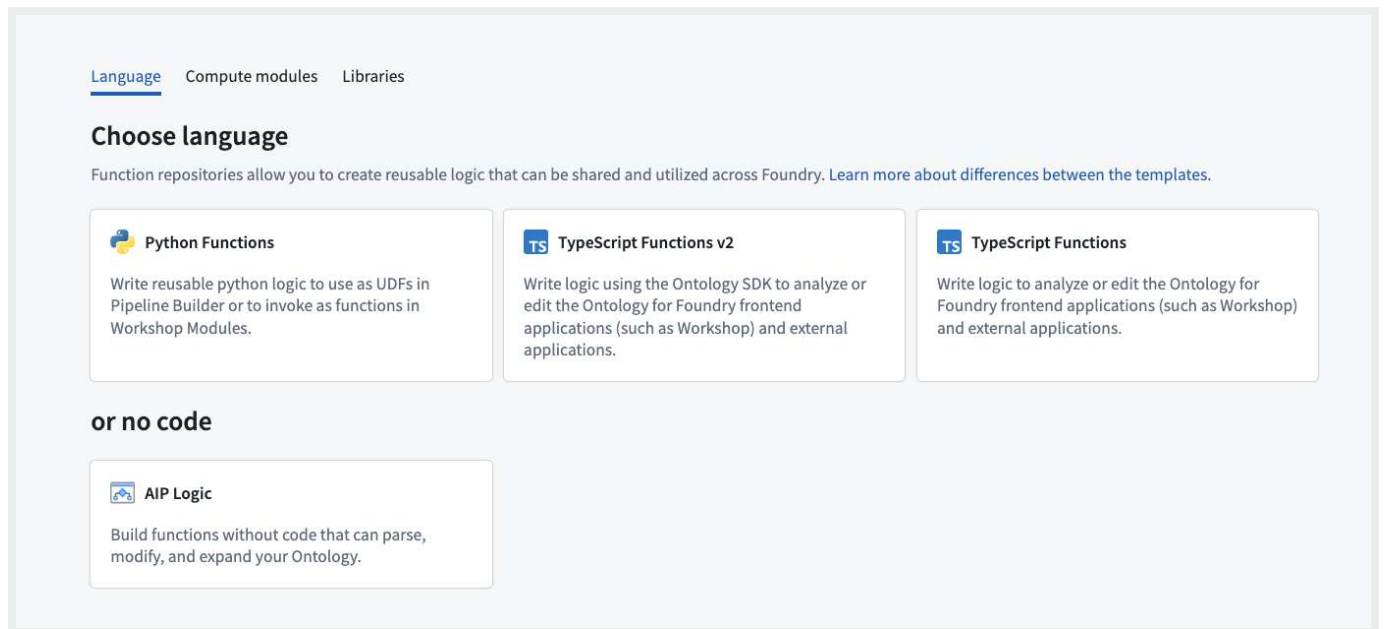
Review the sections below for general information on functions repository creation and usage.

Create a functions repository

When creating a functions repository, you will be able to choose the language that best suits your needs. You can initialize functions repositories directly from a project of your choice by selecting **API Reference** in the top right. Once the repository has been initialized, you can add and run



Send feedback

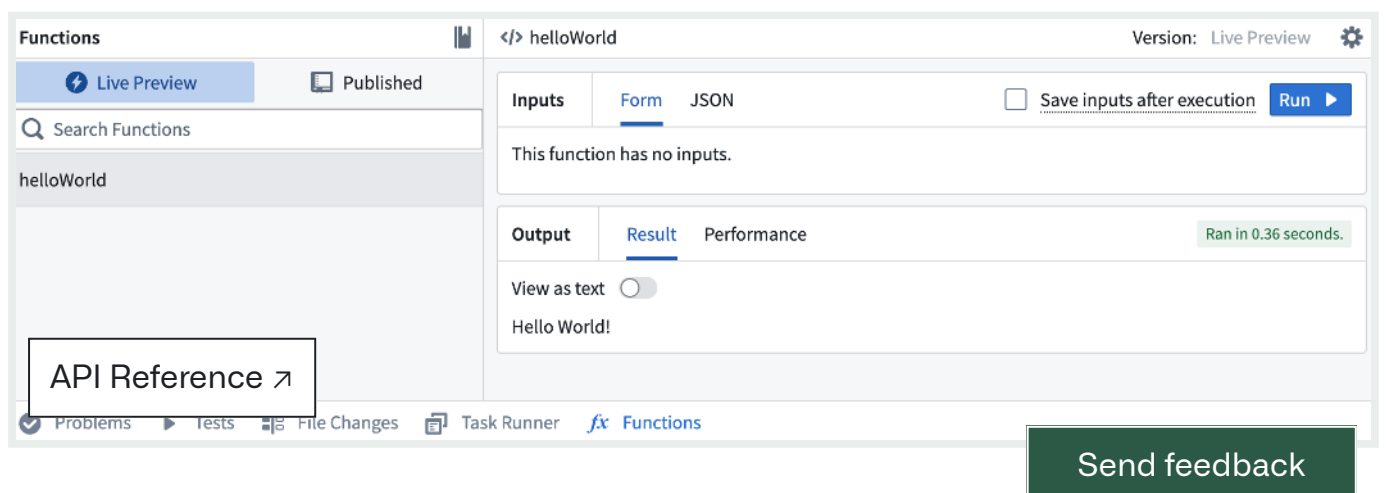


For detailed instructions on how to create a functions repository for a specific language, refer to the tutorial sections below:

- [Create a TypeScript v1 functions repository](#)
- [Create a TypeScript v2 functions repository](#)
- [Create a Python functions repository](#)

Test in live preview

The functions live preview allows you to test your functions before committing them to your repository. You can run a function in live preview after adding it to your repository. To do so, open **Functions** on the bottom toolbar and select **Live Preview**. Choose a function, enter input values, and select **Run** to run the function.





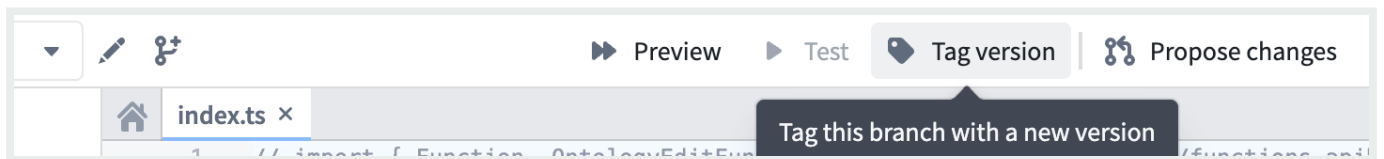
Live preview runs in a different runtime environment than published functions. Differences include CPU resources, available memory, and how long a function can run before timing out.

[Manage the runtime environment for published functions.](#)

Select **Commit** in the upper right to commit your changes to the `master` branch of your repository.

Publish your functions

Once you commit your work, you will see the option to **Tag version**. This will publish all of the functions in your repository to the functions registry, [making them accessible across the platform](#).



Select **Tag version** to tag a release off of the `master` branch. Set the tag name based on the extent of your changes, then choose **Tag and release**.

[API Reference ↗](#)

[Send feedback](#)

Tag and release

You can release functions by tagging a branch or a commit.

Select a branch or enter a commit hash:

 Branch

master

Tag name (latest released tag: 0.0.3)

☐ Patch 0.0.4

Patches are used to signal backwards compatible bug fixes.

☒ Minor 0.1.0

Minor versions are used to signal changes that do not require consumers adjustments.

☐ Major 1.0.0

Major versions are used to signal breaking changes that require consumers' attention.

☐ Custom

0.0.4

Enter a custom tag name that follows your desired naming convention.

Releasing: 0.1.0

Cancel

Tag and release

To view progress as your functions are tagged and released, select the **View** pop-up or navigate to the **Tags** tab. Once **Step 2: Release** is completed, select the published functions to view them in the functions registry.

 Functions may not be immediately searchable by name in Workshop or the Functions Registry while permissions propagate.

[API Reference ↗](#)

[Send feedback](#)

Tags and releases

BranchesTags and releases+ Tag and release

Tag 0.1.0 Succeeded

8599f11ade1: Autogenerated commit message at Jan 9, 2024, 5:01 PM

Jan 9, 2024, 5:01 PMView commit ^

STEP 1

Tag

refs/tags/0.1.0

Tue, Jan 9, 2024, 5:01:21 PM

STEP 2

Release

Succeeded at Jan 11, 2024, 12:07 PM

[functions-publish task succeeded](#)

fx Published functions

myFunction

Run your function

After the checks for your tag have passed, navigate back to the **Code** tab in Code Repositories and select **Functions** from the bottom toolbar. You should see your new function under the **Published** section. Select it, and try running the new function:

Functions

Live Preview

Published

Search Functions

myFunction0.2.0

range0.2.0

MyFunctions.myFunction functions-typescript/src/index.ts Version: 0.2.0 Evals ⚙

Inputs

FormJSON

☐ Save input after execution

Run ▶

n

1

Output

ResultPerformance

Ran in 0.39 seconds.

View as text

☐

2

Problems

Debugger

Tests

File Changes

Task Runner

fx Functions

[Learn more about leveraging functions across the platform.](#)

Types reference

In order to be published to the registry, a TypeScript function must have explicit type annotations on all input parameters and specify an explicit return type. Below is the full list of currently supported function registry types and their corresponding language types.

Function registry type	TypeScript v1 type	TypeScript v2 type
Attachment	Attachment	Attachment
Boolean	boolean	boolean
Binary	Not supported	Not supported
Byte	Not supported	Not supported
Classification marking	ClassificationMarking	ClassificationMarking
Date	LocalDate	DateISOString
Decimal	Not supported	Not supported
Double	Double	Double
Float	Float	Float
GeoPoint	GeoPoint	Point
GeoShape	GeoShape	Geometry
Group	Group	GroupId
Integer	Integer	Integer
Interface	Not supported	0sdk.Instance<MyInterface>
API Reference ↗ List	Not supported	ObjectSet<MyInterface>
	T[] or Array<T>	T[] or Array<T>

Send feedback

Function registry type	TypeScript v1 type	TypeScript v2 type
Long	Long	Long
Mandatory marking	MandatoryMarking	MandatoryMarking
Map	FunctionsMap<K, V>	Record<K, V>
Media reference	MediaItem	Media
Notification	Notification	Notification
Object	MyObjectType	0sdk.Instance<MyObjectType>
Object set	ObjectSet<MyObjectType>	ObjectSet<MyObjectType>
Ontology edit	void	Edits
Optional	T undefined	T undefined
Principal	Principal	Principal
Range	IRange<T>	Range<T>
Set	Set<T>	Not supported
Short	Not supported	Not supported
String	string	string
Struct/custom type	interface	interface
Timestamp	Timestamp	TimestampISOString
Two-dimensional aggregation	TwoDimensionalAggregation<K, V>	TwoDimensionalAggregation<K, V>
Three-dimensional aggregation	ThreeDimensionalAggregation<K, S, V>	ThreeDimensionalAggregation<K, S, V>
User	User	UserId

API Reference ↗

Although both `Integer` and `Long` correspond to the Python type `int`, any fields marked as `int` directly in your function signature will be registered as `Integer`. Therefore, we instead recommend using either the `Integer` or `Long`

Send feedback

types from the API to register numerical data types. Similar guidelines apply to `Float` and `Double`; if the Python type `float` is directly in your function signature, it will be registered as `Float` by default.

Scalar types

Scalar types represent a single value and are commonly used to hold textual, numeric, or temporal data.

In JavaScript and TypeScript, there is one `number` type that is commonly used to represent both integer and floating-point values. In order to provide further type validation and structure, we only support the numeric aliases exported from the `@foundry/functions-api` package for TypeScript v1 functions and the `@osdk/functions` package for TypeScript v2 functions. Similarly, when working with numeric types in Python functions we recommend using the type aliases exported from the `functions.api` module.

Boolean

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Function, Integer } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public isEven(num: Integer): boolean {
6         return num % 2 === 0;
7     }
8 }
```

String

API Reference ↗

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

Send feedback

```

1 import { Function } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public greet(name: string): string {
6         return `Hello, ${name}!`;
7     }
8 }

```

Short

Represents integer values from -32,768 to 32,767.

In Python functions, the `Short` type is an alias for the built-in `int` type.

PYTHON

```

1 from functions.api import function, Short
2
3 @function
4 def increment(num: Short) -> Short:
5     return num + 1

```

Integer

Represents integer values from (-2^{31}) to $(2^{31} - 1)$.

- In both TypeScript v1 and v2 functions, the `Integer` type is an alias for the built-in `number` type.
- In Python functions, the `Integer` type is an alias for the built-in `int` type.

API Reference ↗

TYPESCRIPT V2

PYTHON

```

1 import { Function, Integer } from "@foundry/functions-api";
2
3 export class MyFunctions {

```

Send feedback


```

4   @Function()
5   public sum(a: Integer, b: Integer): Integer {
6       return a + b;
7   }
8 }

```

Long

Represents integer values from $-(2^{53} - 1)$ to $(2^{53} - 1)$. These bounds are equivalent to `Number.MIN_SAFE_INTEGER` and `Number.MAX_SAFE_INTEGER` in JavaScript to prevent precision loss when functions are called from browser contexts.

- In TypeScript v1 functions, the `Long` type is an alias for the built-in `number` type. In TypeScript v2 functions, the `Long` type is an alias for the built-in `string` type.
- In Python functions, the `Long` type is an alias for the built-in `int` type.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```

1 import { Function, Long } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public subtract(a: Long, b: Long): string {
6         return (BigInt(a) - BigInt(b)).toString();
7     }
8 }

```

Float

Represents a 32-bit floating-point number.

- In both TypeScript v1 and v2 functions, the `Float` type is an alias for the built-in `number`

API Reference ↗

ns, the `Float` type is an alias for the built-in `float` type.

Send feedback

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Float, Function } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public multiply(a: Float, b: Float): Float {
6         return a * b;
7     }
8 }
```

Double

Represents a 64-bit floating-point number.

- In both TypeScript v1 and v2 functions, the `Double` type is an alias for the built-in `number` type.
- In Python functions, the `Double` type is an alias for the built-in `float` type.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Double, Function } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public divide(a: Double, b: Double): Double {
6         return a / b;
7     }
8 }
```

Decimal

API Reference ↗

```
1 from decimal import Decimal
2 from functions.api import function
```

Send feedback

```
3
4 @function
5 def return_pi() -> Decimal:
6     return Decimal('3.1415926535')
```

Date

Represents a calendar date.

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Function, LocalDate } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public returnDate(): LocalDate {
6         return LocalDate.fromString("1999-10-17");
7     }
8 }
```

Timestamp

Represents an instant in time.

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Function, Timestamp } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public getCurrentTimestamp(): Timestamp {
6         return Timestamp.now();
7     }
8 }
```

API Reference ↗

Send feedback

Binary

In Python functions, the `Binary` type is an alias for the built-in `bytes` type.

PYTHON

```
1 from functions.api import function
2
3 @function
4 def encode_utf8(param: str) -> bytes:
5     return param.encode('utf-8')
```

Byte

In Python functions, the `Byte` type is an alias for the built-in `int` type.

PYTHON

```
1 from functions.api import function, Byte
2
3 @function
4 def get_first_byte(param: str) -> Byte:
5     if len(param) == 0:
6         raise Exception("String length cannot be zero.")
7     return param.encode('utf-8')[0]
```

Mandatory marking

Markings are mandatory controls that restrict access by requiring a user to have a particular marking in order to access data.

API Reference ↗

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

Send feedback

```

1 import { OntologyEditFunction, MandatoryMarking } from "@foundry/function
2 import { Employee, Objects } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Edits(Employee)
6     @OntologyEditFunction()
7     public async editMandatoryMarkings(markings: MandatoryMarking[]): Pro
8         const employeeOne = Objects.search().employee().filter(e => e.id.
9         employeeOne.markingsProperty = markings;
10    }
11 }

```

Classification marking

Classification-based access controls (CBAC) are mandatory controls used to protect sensitive government information. They restrict access by requiring a user to have a particular classification marking in order to access information.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```

1 import { OntologyEditFunction, ClassificationMarking } from "@foundry/fun
2 import { Employee, Objects } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Edits(Employee)
6     @OntologyEditFunction()
7     public async editClassificationMarkings(markings: ClassificationMarki
8         const employeeOne = Objects.search().employee().filter(e => e.id.
9         employeeOne.markingsProperty = markings;
10    }
11 }

```

Collection types

API Reference ↗ parameterized by other types. For example, `Array[String]` is a list of strings and `Map[String, Integer]` is a dictionary with string keys and in

Send feedback

parameterized types must be specified, and the type must be another supported type. Map keys can only be scalar types or an Ontology object type.

List

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Function, Integer } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public filterForEvenIntegers(nums: Integer[]): Integer[] {
6         return nums.filter(num => num % 2 === 0);
7     }
8 }
```

Map

Maps are commonly used to key by a scalar type and access an associated value that can be of any other function registry type.

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Function, FunctionsMap } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public getMap(): FunctionsMap<string, string> {
6         const myMap = new FunctionsMap<string, string>();
7
8         myMap.set("Name", "Phil");
9         myMap.set("Favorite Color", "Blue");
10
11         return myMap;
12     }
13 }
```

API Reference ↗

Send feedback

Additionally, maps support Ontology objects as keys.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Function, FunctionsMap } from "@foundry/functions-api";
2 import { Airplane } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Function()
6     public getObjectMap(aircraft: Airplane[]): FunctionsMap<Airplane, Integer | undefined> {
7         const myMap = new FunctionsMap<Airplane, Integer | undefined>();
8
9         aircraft.forEach(obj => {
10             myMap.set(obj, obj.capacity);
11         });
12
13         return myMap;
14     }
15 }
```

Set

TYPESCRIPT V1

PYTHON

```
1 import { Function, Integer } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public getSizeOfSet(mySet: Set<Integer>): Integer {
6         return mySet.size;
7     }
8 }
```

Optional

[API Reference ↗](#)

- In TypeScript functions, optional parameters are declared as `varName?: T | undefined`. For example, a function that has an optional integer parameter named

[Send feedback](#)

`value` could be declared as either `value?: Integer` or `value: Integer | undefined`. TypeScript functions can also declare an optional return type by specifying a type of `T | undefined`. For example, a function that may return either an `Integer` or no value would have a return type of `Integer | undefined`.

- In Python functions, optional parameters and return values can be declared using `typing.Optional[T]` or `T | None`. The `T | None` syntax requires Python 3.10 or greater.
- In both TypeScript and Python functions, the parameterized type `T` must be specified, and it must be another supported type.

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Function } from "@foundry/functions-api";
2
3 export class MyFunction {
4     @Function()
5     public greet(name?: string): string | undefined {
6         if (name === undefined) {
7             return undefined;
8         }
9         return `Hello, ${name}!`;
10    }
11 }
```

Functions also support default values in the function signature.

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Double, Function } from "@foundry/functions-api";
2 import { Customer } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Function()
6     public computeRiskFactor(customer: Customer, weight: Double = 0.75): D
7         // ...
8     }
9 }
```

API Reference ↗

Send feedback

Struct/custom type

Custom types are composed of other supported types (including other custom types) and can be used in function signatures.

- In TypeScript functions, custom types are user-defined TypeScript interfaces using the `interface` keyword.
 - Optional fields are supported through either the `?` optional token or a union with `undefined`.
- In Python functions, custom types are user-defined Python classes.
 - To be a valid custom type, the class must adhere to the following:
 - The class must have type annotations on all of its fields.
 - The field types must be supported types; either the primitive API types or native Python types (as defined in the table above) may be used.
 - The `__init__` method must accept only named arguments with the same names and type annotations as the fields.
 - The `dataclasses.dataclass` [↗](#) decorator can be used to automatically generate an `__init__` method that conforms with these requirements.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Function, Integer } from "@foundry/functions-api";
2 import { Passenger } from "@foundry/ontology-api";
3
4 interface PassengerInfo {
5     name?: string;
6     age?: Integer;
7 }
8
9 export class MyFunctions {
10     @Function()
11     public getPassengerInfo(passenger: Passenger): PassengerInfo {
12         return {
13             name: passenger.name,
14             age: passenger.age,
15         };
16     }
17 }
```

API Reference [↗](#)

Aggregation types

Send feedback

Aggregation types can be returned from functions for use in other parts of the platform, such as Charts in [Workshop](#).

There are two supported aggregation types:

- A [two-dimensional aggregation](#) maps from a single bucket key to a numeric value. For example, this could be used to represent an aggregation such as a count of employees with a specific job title.
- A [three-dimensional aggregation](#) maps from two bucket keys to a numeric value. This could be used to represent an aggregation such as a count of employees by each employee's job title and home office.

Aggregations can be keyed by several types:

- [Boolean](#) buckets represent values that are either `true` or `false`.
- [String](#) buckets can be used to represent categorical values.
- [Range](#) buckets represent aggregations where the bucket key is a range of values. These can be used to represent a histogram or date axis in a chart.
 - Numeric ranges, including [Integer](#) and [Double](#), represent bucketed aggregations on numeric values.
 - Date and time ranges, including on [Date](#) and [Timestamp](#), represent bucketed aggregations on date ranges.

Range

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Function, Integer, IRange } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public getRange(min: Integer, max: Integer): IRange<Integer> {
6         return {
7             min,
8             max,
9         }
10    }
11 }
```

API Reference ↗

Send feedback

Two-dimensional aggregation

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Double, Function, TwoDimensionalAggregation } from "@foundry/fur
2
3 export class MyFunctions {
4   @Function()
5   public myTwoDimensionalAggregation(): TwoDimensionalAggregation<string> {
6     return {
7       buckets: [
8         { key: "bucket1", value: 5.0 },
9         { key: "bucket2", value: 6.0 },
10      ],
11    };
12  }
13 }
```

Three-dimensional aggregation

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Double, Function, ThreeDimensionalAggregation } from "@foundry/f
2
3 export class MyFunctions {
4   @Function()
5   public myThreeDimensionalAggregation(): ThreeDimensionalAggregation<string> {
6     return {
7       buckets: [
8         {
9           key: "group-by-1",
10          value: [
11            { key: "partition-by-1", value: 5.0 },
12            { key: "partition-by-2", value: 6.0 },
13          ],
14        },
15        {
16          key: "group-by-2",
17          value: [
18            { key: "partition-by-1", value: 7.0 },

```

API Reference ↗

Send feedback

```

19         { key: "partition-by-2", value: 8.0 },
20     ],
21 },
22 ]
23 };
24 }
25 }

```

Ontology types

i Ontology imports

Object types must be imported into your repository to use them in function signatures. [Learn more about Ontology imports.](#)

Object

Object types from your ontology can be used as both inputs and outputs in your function signature. To either accept or return a single instance of an object type, import the object type from your Ontology SDK and use that type to annotate your function.

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```

1 import { Function, Integer } from "@foundry/functions-api";
2 import { Airplane } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Function()
6     public getCapacity(airplane: Airplane): Integer {
7         return airplane.capacity;
8     }
9 }

```

API Reference ↗

In TypeScript v2, references to object types are supported as [struct](#) parameter fields. To accept a struct or struct list with object type instances, create a custom object type field from your Ontology SDK. This can be paired with [Ontology Edits](#) to support

Send feedback

workflows such as creating multiple instances of an object type derived from other object types.

TYPESCRIPT V2

```
1 import { Osdk } from "@osdk/client";
2 import { Integer } from "@osdk/functions";
3 import { Airplane, Passenger, Ticket } from "@ontology/sdk";
4
5 type TicketEdit = Edits.Object<Ticket>
6
7 interface TicketInfo {
8     airplane?: Osdk.Instance<Airplane>;
9     passenger?: Osdk.Instance<Passenger>;
10    seat?: String;
11 }
12
13 function createTickets(ticketInfo: TicketInfo[]): TicketEdit[] {
14     const batch = createEditBatch<TicketEdit>(client);
15
16     ticketInfo.forEach(i => batch.create(TicketEdit, {
17         flightNumber: i.airplane.flightNumber,
18         passengerName: i.passenger.name,
19         seat: i.seat}))
20
21     return batch.getEdits();
22 }
23
24 export default createTickets;
```

Object set

There are two ways to pass a collection of objects into and out of a function: concrete collections of objects (such as arrays), or object sets.

Passing an array of objects to a function enables executing logic over a concrete list of objects at the expense of loading the objects into the function execution environment up front. [API Reference ↗](#) however, allows you to perform filtering, search-around, and aggregation operations and only load the final result when you request it.

Send feedback

- i** We recommend using an object set instead of an array because an object set often achieves better performance and allows more than 10,000 objects to be passed into the function.

The below example shows how to filter an object set without loading the objects into memory, allowing you to return the filtered object set to a different part of the application.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Function } from "@foundry/functions-api";
2 import { Airplane, ObjectSet } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Function()
6     public filterAircraft(aircraft: ObjectSet<Airplane>): ObjectSet<Airplane> {
7         return aircraft.filter(a => a.capacity.range().gt(200));
8     }
9 }
```

Interface

Interface types from your Ontology can be used as both inputs and outputs in TypeScript v2 function signatures. They are not supported in TypeScript v1 or Python.

TYPESCRIPT V2

```
1 import { Osdsk } from "@osdk/client";
2 import { Integer } from "@osdk/functions";
3 import { Person } from "@ontology/sdk";
4
5 function getAge(person: Osdsk.Instance<Person>): Integer {
6     return person.age;
7 }
8
9 export default getAge;
```

[API Reference ↗](#)

[Send feedback](#)

Interface object set

Interface object sets can be both inputs and outputs in TypeScript v2 function signatures.

TYPESCRIPT V2

```
1 import { ObjectSet } from "@osdk/client";
2 import { Person } from "@ontology/sdk";
3
4 function filterPeople(people: ObjectSet<Person>): ObjectSet<Person> {
5     return people
6         .where({
7             age: {
8                 $gt: 200,
9             }
10        });
11 }
12
13 export default filterPeople;
```

Ontology edit

In addition to writing functions that read data from the Ontology, you can also write functions that create objects and edit the properties and links between objects. For more details about how edit functions work, refer to the [overview page](#).

To be registered as edit functions, TypeScript v1 functions require a `void` return type in the signature. TypeScript v2 and Python functions, however, require explicitly returning a list of Ontology edits.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Edits, OntologyEditFunction } from "@foundry/functions-api";
2 import { Employee, LaptopRequest, Objects } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5
6     @Edits(Employee, LaptopRequest)
```

API Reference ↗

Send feedback

```

7   @OntologyEditFunction()
8   public assignEmployee(newEmployee: Employee, leadEmployee: Employee):
9
10      const newLaptopRequest = Objects.create().laptopRequest(Date.now(
11      newLaptopRequest.employeeName = newEmployee.name;
12
13      newEmployee.lead.set(leadEmployee);
14  }
15 }

```

Attachment

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```

1  import { Attachment, Function } from "@foundry/functions-api";
2
3  export class MyFunctions {
4      @Function()
5      public loadAttachmentContents(attachment: Attachment): Promise<string>
6          return attachment.readAsync().then(blob => blob.text());
7  }
8  }

```

Notification

Notification types can be returned from a function to flexibly configure notifications that should be sent in the platform. For example, you can author a function that takes in parameters such as a `User` and an object type and returns a Notification with a configured message.

- A `Notification` consists of two fields: a `ShortNotification` and `EmailNotificationContent`.
- A `ShortNotification` represents a summarized version of the notification, which will be

API Reference ↗

Foundry platform. It includes a short `heading`, `content`, and a

- `EmailNotificationContent` represents a rich version of the notification sent externally via email. It includes a `subject`, a `body` consisting of headless HTML,

Send feedback

and a collection of `Link`s.

- A `Link` has a user-facing `label` and a `linkTarget`. The `LinkTarget` can be a URL, an `OntologyObject`, or a `rid` of any resource within Foundry.

For an example of how to use the Notifications API, review [our guide](#).

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import {
2     EmailNotificationContent,
3     Function,
4     Notification,
5     ShortNotification,
6 } from "@foundry/functions-api";
7
8 export class MyFunctions {
9     @Function()
10    public buildNotification(): Notification {
11        return Notification.builder()
12            .shortNotification(ShortNotification.builder()
13                .heading("Issue reminder")
14                .content("Investigate this issue.")
15                .build())
16            .emailNotificationContent(EmailNotificationContent.builder()
17                .subject("New issue")
18                .body("hello")
19                .build())
20            .build();
21    }
22 }
```

Media types

Media

For more information, see the [API Reference](#) to get and return media items. In TypeScript v1, use the `MediaItem` type. In TypeScript v2 and Python, use `Media` as both the input and output type. Callers can pass an existing `MediaReference`, for example, a media property on an object.

Send feedback

Downstream consumers can use the returned value to fetch contents, fetch metadata, or attach it to another object. For more information, review the guide on [media](#).

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Function, MediaItem } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public async echoMedia(media: MediaItem): Promise<string | undefined> {
6         const mimeType: string = media.mimeType;
7         // Fetch type-specific metadata (page count, dimensions, duration)
8         const metadata = await media.getMetadataAsync();
9         // Read the binary contents as a Blob
10        const contents: Blob = await media.readAsync();
11        // Narrow to a specialized subtype to call type-specific methods
12        if (MediaItem.isDocument(media)) {
13            const text = await media.extractTextAsync({ startPage: 0, endPage: 100 });
14        } else if (MediaItem.isAudio(media)) {
15            const transcript = await media.transcribeAsync();
16        }
17        return undefined;
18    }
19 }
```

Users, groups, and principals

A Principal represents either a Foundry user account or group. These types can be passed into a function to allow accessing information associated with a user or group, such as a group's name or a user's first and last name or email address. All Principal types are exported from the `@foundry/functions-api` package.

- A `User` always has a username, and may have a `firstName`, `lastName`, or `email`. It also includes all fields associated with a `Principal`.
- A `Group` has a name. It also includes all fields associated with a `Principal`.

API Reference ↗

be either a `User` or a `Group`. You can inspect the `type` field to determine if a given `Principal` is a `User` or a `Group`. In addition to `User` and `Group`-specific fields, a `Principal` has an `id`, `realm`, and dictionary of `metadata`.

Send feedback

User

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Function, User } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public getUserEmail(user: User): string {
6         return user.email;
7     }
8 }
```

Group

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Function, Group } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public getGroupName(group: Group): string {
6         return group.name;
7     }
8 }
```

Principal

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { Function, Principal } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     @Function()
5     public getPrincipalType(principal: Principal): string {
6
7     }
8 }
```

API Reference ↗

Send feedback

```

7         switch (principal.type) {
8             case "user":
9                 return "User";
10            case "group":
11                return "Group";
12            default:
13                return "Unknown";
14        }
15    }
16 }

```

Geometry types

Geometry types represent spatial data and geographic shapes in your functions. Two geometry types are supported:

- **GeoPoint** represents a single geographic point with latitude and longitude coordinates.
- **GeoShape** represents any valid GeoJSON geometry, including Points, Polygons, LineStrings, and other shapes.

These types follow the [GeoJSON specification ↗](#) and can be used for spatial operations, mapping, and geographic analysis. Note that positional arguments follow longitude, latitude order as per the GeoJSON spec.

GeoPoint

The following example shows how to create and return a GeoPoint.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```

1 import { Function, GeoPoint } from "@foundry/functions-api";
2
3 export class MyFunctions {
4     on()
5     createPoint(): GeoPoint {
6         return GeoPoint.fromCoordinates({
7             latitude: 37.7749,
8             longitude: -22.4194

```

API Reference ↗

Send feedback

```
9         });  
10     }  
11 }
```

GeoShape

The following example shows how to create and return a Polygon.

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1  import { Function, Polygon, GeoPoint } from "@foundry/functions-api";  
2  
3  export class MyFunctions {  
4      @Function()  
5      public createPolygon(): Polygon {  
6          const ring: GeoPoint[] = [  
7              GeoPoint.fromCoordinates({ latitude: 37.8, longitude: -22.4 }  
8              GeoPoint.fromCoordinates({ latitude: 37.8, longitude: -22.5 }  
9              GeoPoint.fromCoordinates({ latitude: 37.7, longitude: -22.5 }  
10             GeoPoint.fromCoordinates({ latitude: 37.7, longitude: -22.4 }  
11             GeoPoint.fromCoordinates({ latitude: 37.8, longitude: -22.4 }  
12         ];  
13         return Polygon.fromLinearRings([ring]);  
14     }  
15 }
```

PREVIOUS

← Getting started

NEXT

Branching functions →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Policy ↗](#)

[API Reference ↗](#)

[Terms of Use ↗](#)

Send feedback

Ontology building / Functions / Branching functions

Branching functions

You can develop, publish, and consume functions on a global branch. This is currently supported for TypeScript v1 functions and AIP Logic functions.

Developing functions

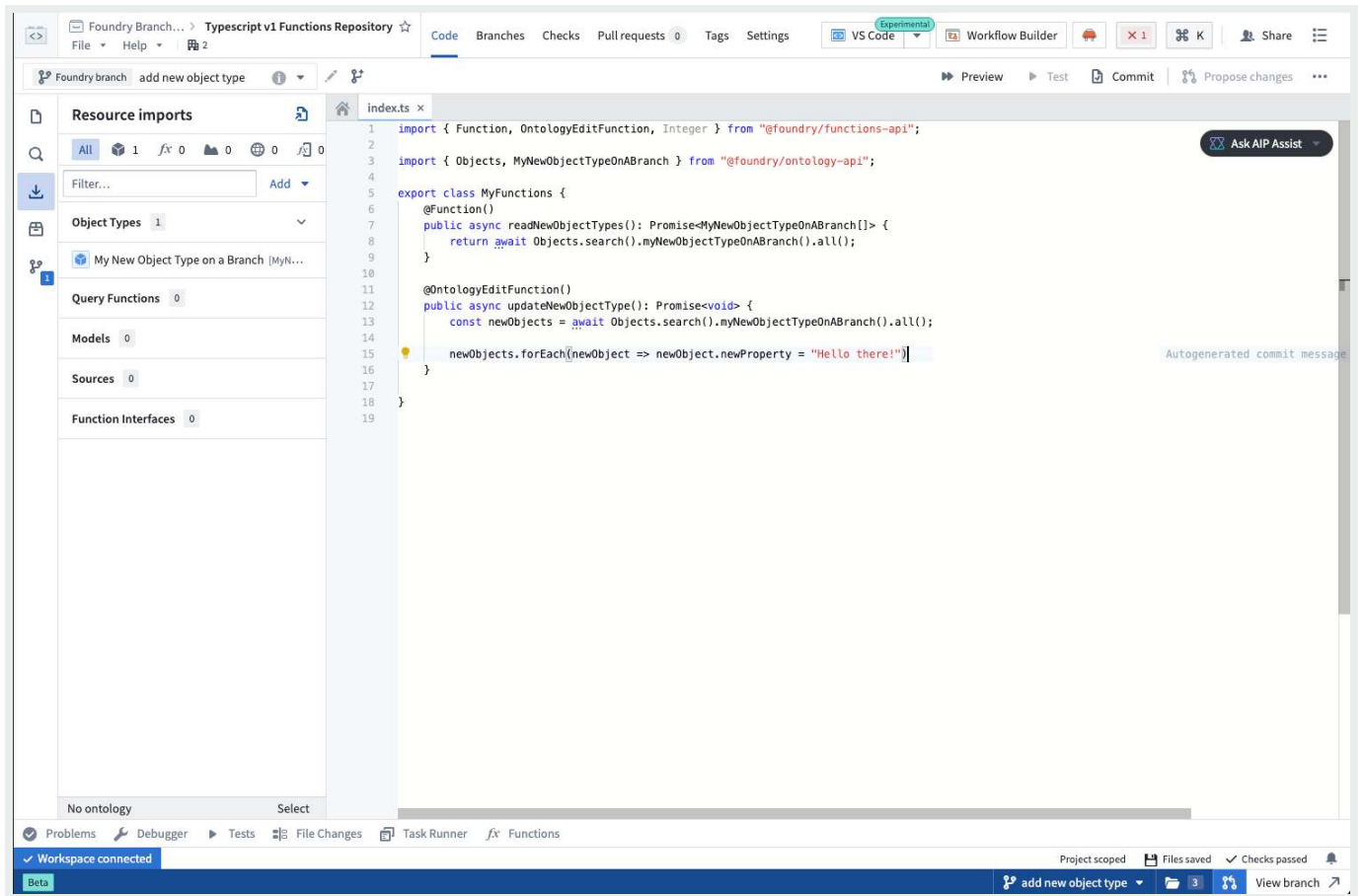
i To use Global Branching for TypeScript v1, upgrade your repository template version to version `0.903.0` of the `functions-typescript` child template or higher. Review [the documentation on repository upgrades](#) for instructions.

You can develop a function that depends on changes made to resources on your global branch, such as newly created or modified ontology entities.

API Reference ↗



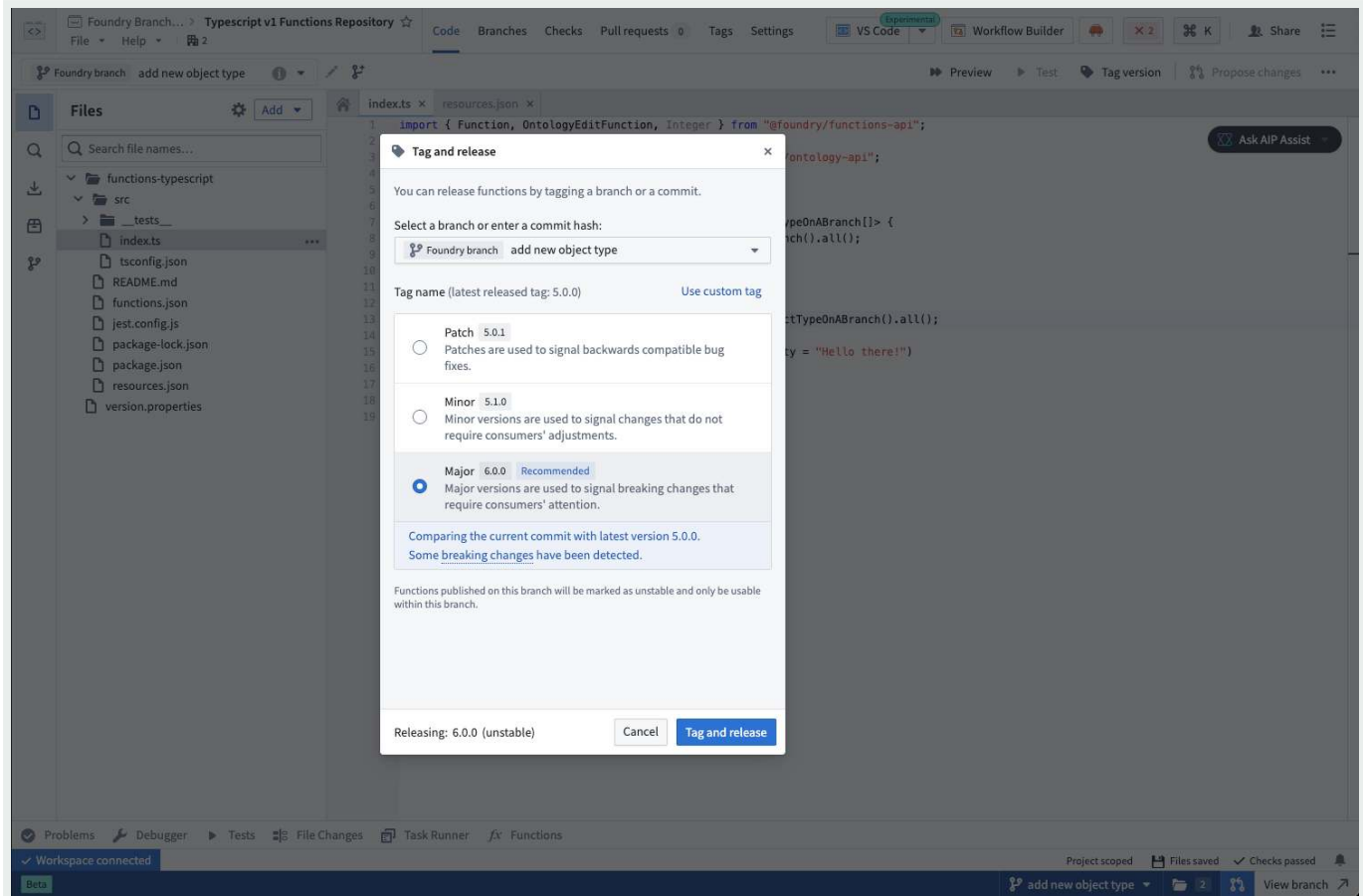
Send feedback



When you are ready, publish your function with a **version target** — the stable version published to `main` when the global branch merges. During development on your branch, the function is published as unstable preview versions.

[API Reference ↗](#)

[Send feedback](#)



Once you have successfully published your function on the branch, you can use the new function version on your branch in Workshop, AIP Logic, and function-backed actions. This function version is labeled with the `Branched pre-release` tag. Note that functions published on your branch will not be accessible from other branches, including `main`.

API Reference ↗

Send feedback

The screenshot shows the Foundry Branch interface. On the left, a sidebar lists variables: 'Object table 1 Active object', 'Object table 1 Object set', and 'var1'. The main area displays the configuration for 'var1', which is a function 'readNewObjectTypes' on 'My New Object Type on a Branch'. The function has a version of '1.0.0 Branched pre-release'. Below the function configuration, a table shows the current value of the variable, which is a list of 367 items. The table has columns for 'Quantity', 'Parts In Batch', and 'Plant'. The data is as follows:

Quantity	Parts In Batch	Plant
.987	42	Denver Production
.9477	45	Atlanta Production
.9681	40	Denver Production
.8424	113	Denver Production
.9236	157	Denver Production
.954	162	Atlanta Production
.8257	100	Atlanta Production
.9218	160	Atlanta Production
.8333	107	Atlanta Production
.9268	160	Denver Production
.8372	118	Atlanta Production
.9216	160	Atlanta Production
.935	45	Atlanta Production
.9644	37	Denver Production
.9569	48	Atlanta Production
.8606	101	Denver Production

On the right, a sidebar shows the 'Button' widget setup, including 'Widget setup', 'Metadata', and 'Display & Formatting' options.

The screenshot shows the Ontology Manager interface. On the left, a sidebar lists various settings: 'Update all new object types', 'Overview', 'Rules', 'Parameters', 'User Interface', 'Capabilities', 'Security & Submission Criteria', 'Automations', and 'History'. The main area displays the configuration for a rule named 'Run function'. The rule has a version of '1.0.0 Branched pre-release'. Below the rule configuration, a code preview shows the following TypeScript code:

```

11 @ontologyEditFunction()
12 public async updateNewObjectType(): Promise<void> {
13     const newObjects = await Objects.search().myNewObjectTypeOnABranch().all();
14     newObjects.forEach(newObject => newObject.newProperty = "Hello there!");
15 }

```

At the bottom, there is a button to 'Add new rule'.

API Reference ↗

It is currently not possible to depend on branched versions of q TypeScript v1 repositories.

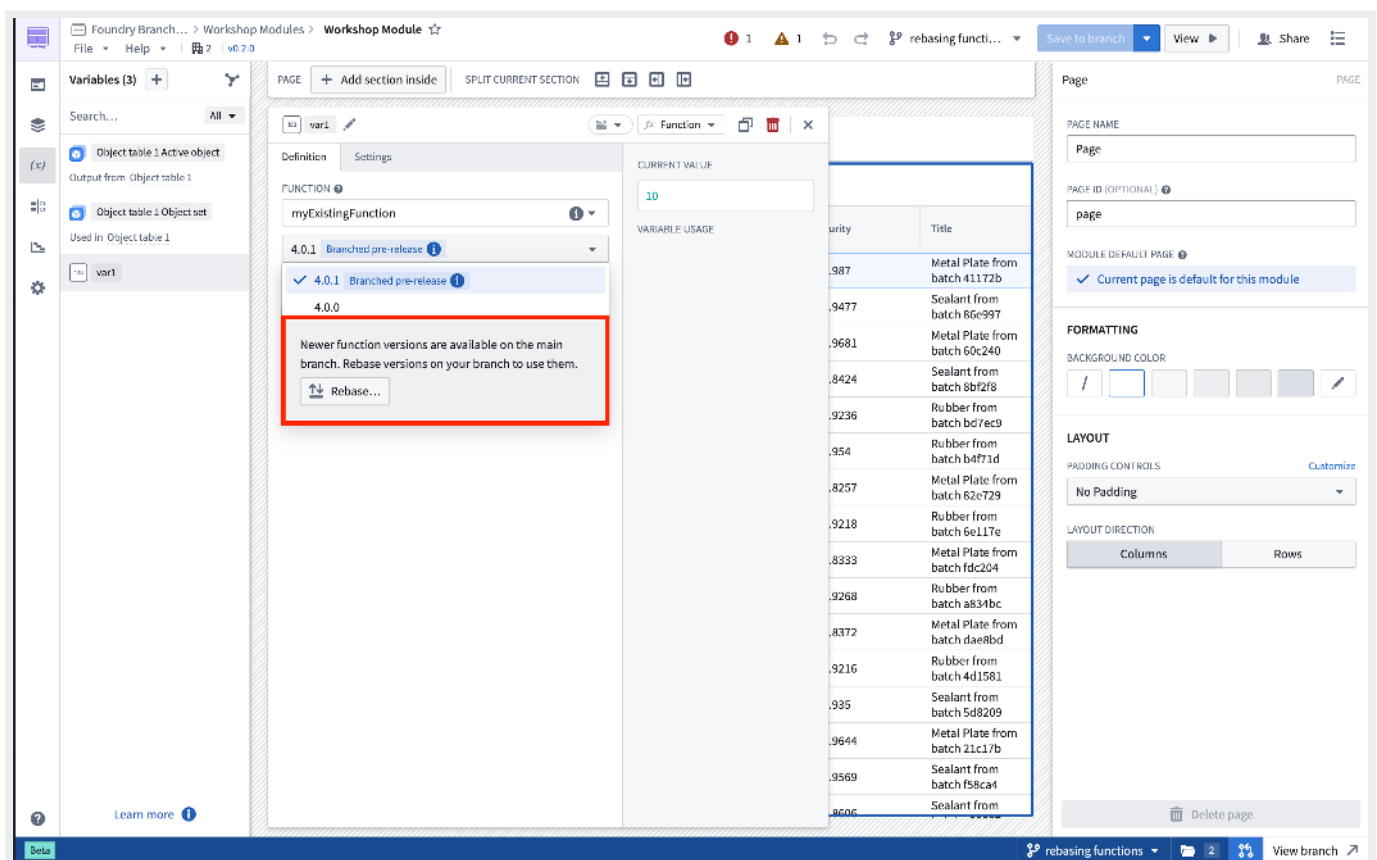
Send feedback

As you continue developing on your branch, you can continue publishing function versions to the same version target. As long as your version target remains the same, any resource using your function will automatically pull the latest published version. This lets you iterate quickly without manually updating function references after each publish.

If the version target on the branch changes, you must update all dependents of the function that were using the old version on the branch to use the new version target. This will also be displayed as a [check](#).

Conflict resolution and rebasing functions

When developing on a branch, you will not automatically receive newer function versions published to `main`. This prevents main development from disrupting your branch work. If newer versions are available on `main`, you will see notifications in function version selectors and Ontology Manager. You can then rebase your function versions to pull in all newer versions from `main`.



[API Reference ↗](#)

If your version target gets published to `main` while you are developing, you must select a new version target before merging. Once you do, update any dependent version target.

[Send feedback](#)

Merging

When merging your global branch, modified functions will be published on `main` with the stable version target. Resources using the version published on your branch will automatically start using the new stable version that was published on `main` as part of the merge.

 Foundry only merges your version target into `main` and does not merge other versions published on the branch. Those versions will not exist on `main` after your branch merges.

PREVIOUS
← [Types reference](#)

NEXT
[Function management / Function versioning](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Function management / Function versioning

Function versioning

This document describes the versioning system used for functions. Versions for function releases are chosen by their publishers and are immutable after creation. Applying appropriate versions is critical to providing consumers of your functions with a stable and reliable experience.

Backward compatible changes vs. breaking changes

The versioning system for functions distinguishes between backward compatible changes and breaking changes. *Backward compatible changes* are changes that do not disrupt existing consumers of your functions. A change that is not backward compatible can be referred to as a *backward incompatible* or a *breaking* change.

Some examples of backward compatible changes are:

- Adding an optional input to a function's signature.
- Optimizing a function's performance without changing its expected behavior.
- Fixing a bug in your function without changing its expected behavior.

Some examples of breaking changes are:

- Adding a required input to a function's signature.
- Changing the output type of a function's signature from an integer to a string.
- Deleting a function.

Whether a change to an existing version is backward compatible, ask yourself if the change would cause disruption to or require explicit attention from consumers of the existing version.

API Reference ↗



Send feedback

Remember that you are ultimately in charge of dictating the expected consumption patterns of your functions.

The semantic versioning system

Functions are versioned according to the [Semantic Versioning 2](#) system.

In Semantic Versioning, versions take the form `X.Y.Z` where `X`, `Y`, and `Z`—known as the major, minor, and patch versions respectively—are non-negative integers (for instance, `1.2.3`). A version may also include a prerelease identifier comprised of alphanumeric characters by appending a hyphen immediately following the patch version (for example, `1.2.3-rc1`).

i This page provides a brief summary of Semantic Versioning. We encourage you to read the [full specification 2](#) since adhering to the specification is an important aspect of publishing functions that can be reliably consumed in other applications.

Choosing a release version

When publishing a new version of a function, consider the following points from the Semantic Versioning specification:

- Major version `0` (`0.y.z`) is for initial development. During initial development, the function may change at any time and your functions should not be considered stable by consumers.
- The major version should be incremented when you make backward incompatible changes.
- The minor version should be incremented when you add functionality in a backward compatible manner.
- The patch version should be incremented when you make backward compatible bug fixes.

[API Reference 2](#) version indicates that the version is unstable and might not satisfy the intended compatibility requirements as denoted by its associated normal version.

Send feedback

Backward compatibility checks

Backward compatibility checks are performed for your functions before you publish a new version. In particular, you will be warned about any of the following breaking changes:

- Dropping a function. This includes deleting a function in your Python or TypeScript function code repository.
- Dropping an input (even an optional one) on a function's signature.
- Reordering an input on a function's signature.
- Adding a required input to a function's signature.
- Bad input type changes (such as integer to string). Note that widening a numeric input type (like integer to float) will result in a warning.
- Bad output type changes (such as string to optional string).

If these checks fail for any reason, it is recommended that you release a major version. However, this does not apply if you are still in the initial development phase (that is, you are still at major version `0`).

 Palantir's built-in checks are not exhaustive of all types of breaking changes. For instance, breaking changes from your internal implementation may not be detected. It is not safe to release a minor or patch version based solely on a successful outcome from these checks.

Caveat: Custom types

The internal functions data type representation currently lacks sufficient information regarding the optionality of custom type fields. As a result, you may notice that for custom type inputs and outputs, the backward compatibility checks will warn you when removing or adding any fields, including optional fields (such as `quantity?: Integer` in Typescript or `quantity: Integer = 0` in Python).

We are currently making changes so that going forward you will not be warned when removing an optional field on an output custom type or adding an optional field on an input

c [API Reference ↗](#)

[Send feedback](#)

i You will still receive a warning when removing an optional field on an input custom type. It is generally considered bad practice to ignore any fields provided by a consumer as they likely expect the provided fields to tell the behavior of your function.

Restrict stable version tags

Stable Semantic Version releases (non-prerelease versions) may be immediately consumed by downstream production applications if the applications have been configured to reference the Function by a version range (for example, `>=1.2.3 <2.0.0`). This makes it important to review and test code changes before releasing them in a new stable version.

It is possible to enforce restrictions on the release of stable versions of your Functions by [enabling a toggle](#) in the repository settings for protected branches.

Frequently asked questions

Choosing release versions in the 0.y.z initial development phase

It is common practice that any breaking changes be made in a minor release and any backward compatible changes be made in a patch release. This is an assumption made by consumers in many development spheres, such as the Node/NPM ecosystem, as demonstrated by their wide use of [caret ranges](#).

Accidentally releasing a backward incompatible change as a patch or minor version

After you realize that you've released a breaking change, you should correct the problem and restore backward compatibility in a new minor version.

API Reference ↗

Send feedback

Consider the following example.

1. You have a function called `myFunction` at version `1.0.0` which takes a single string input.
2. You add a new required input to `myFunction` and accidentally release this change in a minor version release `1.1.0`.

To remediate this, you can revert the breaking change to the signature (that is, remove the new required input you added in `1.1.0`) and release this change in version `1.2.0`.

Checking backward compatibility when a release fails or has not yet been published

In the case of TypeScript or Python functions, your functions may fail or take a few minutes to publish. In either of these cases, the built-in backward compatibility checks will be unable to run. If you want to see the results of these checks before making a new release, you have the following options:

- If the last release failed, you should use the “custom tag” option to compare against the last successful tag.
- If the last release has not been published yet, you should wait for it to finish.

PREVIOUS
← [Branching functions](#)

NEXT
[Manage published functions](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

[API Reference](#) ↗

[Send feedback](#)

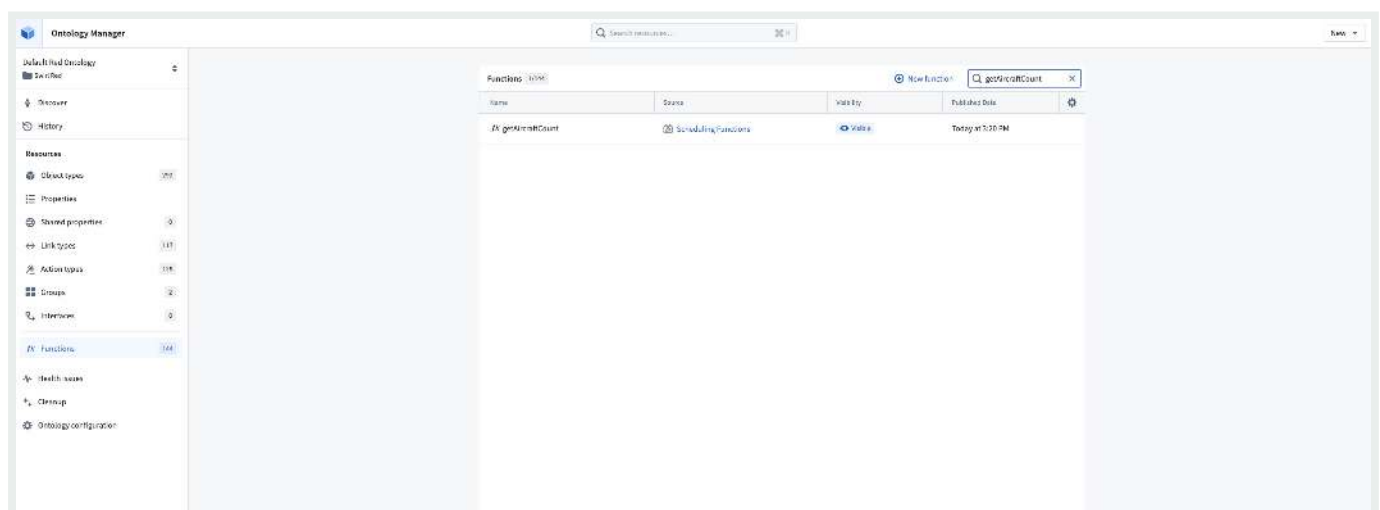
Ontology building / Functions / Function management / Manage published functions

Manage published functions

Once published, all types of functions can be viewed and managed using the **Ontology Manager**.

Searching for functions

To search for functions, navigate to the **Ontology Manager** and select the **Functions** tab. You can search for functions by most metadata on the function, including but not limited to the function name, description, API name and RID.

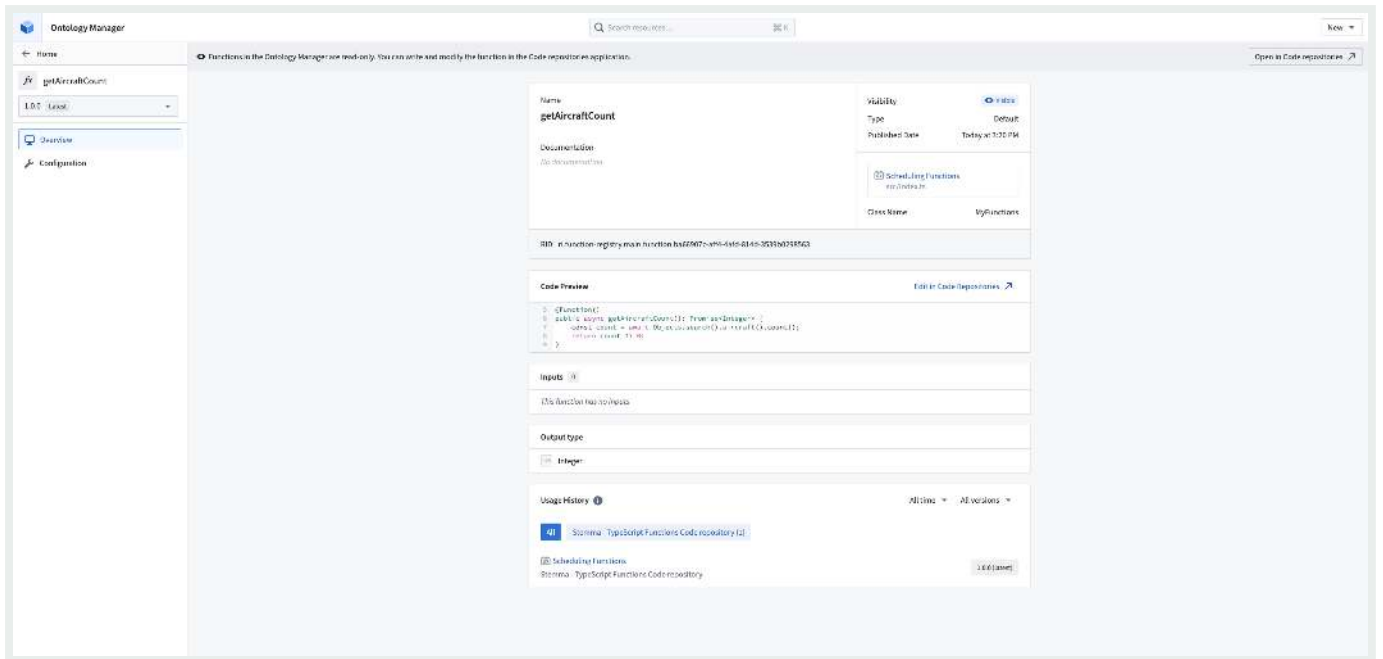


Function overview page

API Reference ↗ In the Ontology Manager, you can view basic information about the function, including its inputs and outputs and any associated usage history for the function.



Send feedback



Function configuration page

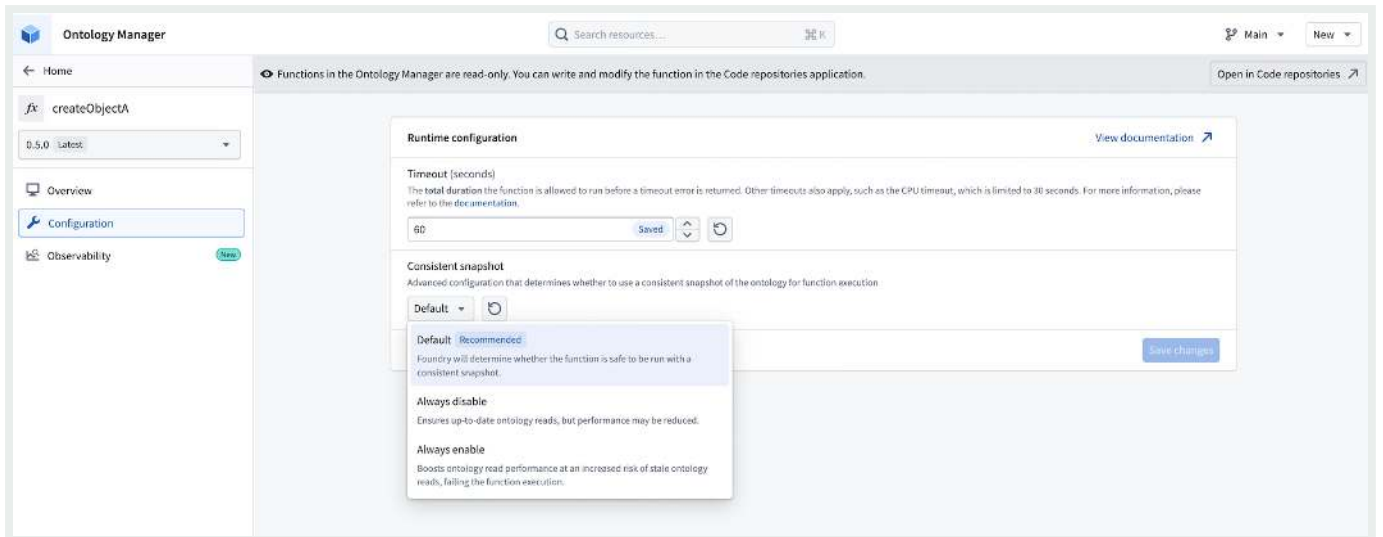
Some types of functions allow you to configure resources such as timeouts or memory limits. If your function supports any configuration options, you can view and edit them in the **Configuration** tab. If this tab is not present, the function does not support any configuration options.

- i** Configuration overrides are applied on a per-function version basis. Depending on the application from which you publish your function, new versions may have the default configuration and you may need to reapply any configuration overrides.

For example, you can configure the timeout on a TypeScript function as seen in the following image.

[API Reference ↗](#)

[Send feedback](#)



Configuration inheritance

Functions provide out-of-the-box support for inheriting configuration overrides when publishing new versions. The configuration is inherited from the prior stable version according to the semantic version specification. If publishing a non-stable version, configurations will be inherited from the prior version, regardless of whether it is a stable release.

Configuration inheritance requires your repository to contain updated template configurations. You can check the hidden `templateConfiguration.json` file to confirm the version your repository is on.

- For TypeScript v1 functions repositories, you must have `parentTemplateVersion >= 3.512.0`
- For Python functions repositories, you must have `parentTemplateVersion >= 0.423.0`

Consistent snapshots

Function-backed actions automatically use one ontology snapshot for all read requests in a single run.

Our [API Reference](#) provides the following qualities:

- **Data consistency:** Without snapshots, sequential ontology queries could return different versions of data if the underlying data changed between requests.

[Send feedback](#)

With snapshots, your function operates on a consistent view of the ontology, similar to snapshot isolation in a database transaction.

- **Improved performance:** Reusing a single snapshot across all ontology requests significantly improves ontology read performance. A single function-backed action, along with any queries within, receives this benefit.

Snapshot configuration

If you need to explicitly manage snapshots for advanced use cases, you can configure the snapshot behavior on the [function configuration page](#) using the following options:

- **Default (recommended):** Leave this option selected unless you encounter snapshot-related errors.
- **Disable snapshots:** Use when you need the freshest data on each query during a run, or when you are hitting snapshot errors due to long-running workloads.
- **Enable snapshots:** Use when you need a consistent point-in-time view across all reads and want better read performance, and your function can tolerate data not updating mid-run. This is not recommended for most use cases.

 By default, functions with sources run against live data without snapshots. Enforcing snapshots is not recommended since functions can perform writes or invoke external systems.

Enforced limits

Several limits are in place to prevent functions from consuming too many resources when they are executed.

Time limit

For API Reference > to **60 seconds** of elapsed run time by default. These limits can be modified on the [function configuration page](#).

Send feedback

Functions are allowed to run for up to **280 seconds** when running in live preview, even if modified on the function configuration page.

 TypeScript v1 functions are additionally limited to **30 seconds** of CPU time, which is not configurable. When a function exceeds this threshold, the cause is often inefficient data loading logic. Refer to the section on [optimizing performance](#) for tips on how to avoid CPU timeouts.

Memory limit

Memory limits differ between TypeScript v1, TypeScript v2, and Python functions.

TypeScript v1

Function execution is limited to **128 Megabytes** of memory usage. This limit is rarely reached; often, functions run into time limits or object loading limits before memory limits.

Deployed Python functions

Deployed Python functions have **2 Gigabytes** of memory usage by default. Currently, deployed Python functions cannot configure memory usage on the function configuration page.

Serverless Python and TypeScript v2 functions

Serverless functions have **1024 Mebibytes** of memory usage by default. This can be configured from **512 Mebibytes** to **5120 Mebibytes** on the [function configuration page](#).

Multithreading

For TypeScript v1, function execution is on a single thread, allowing only one computation at a time. However, you can parallelize loading of object sets or links. Refer to [optimizing performance](#) for more information.

API Reference ↗

Send feedback

For TypeScript v2 and Python functions, you can use multithreading with the built-in Node.js `worker_threads` and Python `threading` libraries.

Object set limits with TypeScript v1

When using [object sets](#), calling `.all()` or `.allAsync()` will throw an error if:

- More than **100,000 objects** are loaded at once from the object set. In general, even loading tens of thousands of objects will run into time limits or memory limits. For use cases where you are running into this limit, consider fetching summary data using [aggregations](#) or fetching a subset of objects using [ordering and limiting](#).
- More than **3 [search arounds](#)** are used at once.

Some aggregation and bucketing operations have limits. See the [aggregations](#) section for details.

PREVIOUS
← [Function versioning](#)

NEXT
[Functions in Marketplace](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

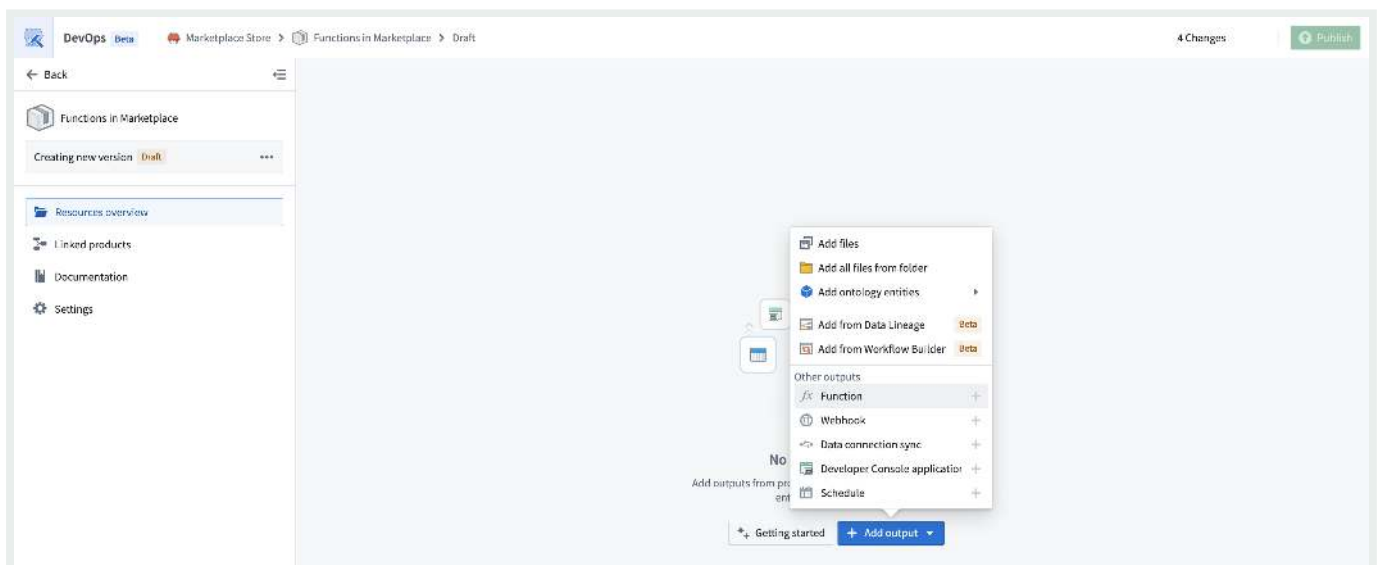
Ontology building / Functions / Function management / Functions in Marketplace

Functions in Marketplace

You can use [Foundry DevOps](#) to include your functions in [Marketplace products](#) for other users to install and reuse.

Adding functions to products

To add a function to a product, [create a product](#). Then, add a function output as shown below.



You will be prompted to choose a function and a version.

[API Reference ↗](#)



[Send feedback](#)

All ontologies
python_osdk_function
Clear

Filters

Input object type
Filter by input object types...

Output object type
Filter by output object types...

Source
Select source to filter by

Supports repositories and compute modules

fx python_osdk_function

python_osdk_function
Last published Yesterday at 11:45 AM
Select

0.1.0

No description

Metadata (4)

RID

ri.function-registry.main.function.05

Repository

Python Functions Repository /Namespace/Marketplace Project

API name

No API name

Visibility

Visible

Inputs (1)

employee

Employee

Selection overview

Cancel
Confirm selection

Including source code for code repositories

When packaging a function authored in a code repository, the backing repository will be automatically included as an additional output in the product. While you can choose to package the code repository with its source code, we generally discourage including source code for products not intended to be installed in [bootstrap mode](#). This is because a Marketplace-installed function does not need its backing source code to execute successfully. Additionally, code repositories are not guaranteed to compile or build out of the box.

If you want to make changes to a function post-installation (for example, to fix a bug or augment functionality), we recommend making those changes to the original function, releasing a new version of the Marketplace product, and then upgrading the installation. If you do not own the installed product, you should reach out to its maintainers with a bug report or feature request.

API Reference ↗ API name resolution

Send feedback

When a function is installed through Marketplace, there are two ways in which its version and API name are resolved: deduplication mode and stable mode.

Deduplication mode

This is the historical and default behavior for resolving a function's version and API name when installed through Marketplace.

In deduplication mode, a function's version is resolved as follows:

- When a function is initially created by an installation, its first version is published at `0.1.0`.
- Upon subsequent installations such as upgrades, the function is published at its latest version incremented by a minor version. For example, if the latest version of the installed function is `1.1.0`, the next installation will publish at `1.2.0`.

i Function versions are immutable. In other words, once a version of a function is published, it cannot be mutated or overridden.

The way a function's API name is resolved is as follows:

- If the API name is already bound to another function in the installation's Ontology, the API name will be deduplicated by appending an incrementing integer suffix. For example, if the API name `myFunction` is already taken, the function will be installed with API name `myFunction1`. If that API name is also taken, it will be installed with API name `myFunction2`, and so on.
- Once an API name exists on the installed function, the function maintains that API name through subsequent installations such as upgrades.

i Function API names are unique per-Ontology. To be precise, if there already exists a function with an API name `myFunction` in an Ontology, no other function with the same API name can exist in that same Ontology.

API Reference ↗

Send feedback

Stable mode

Beta

Stable mode is in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development. Contact Palantir Support to enable this on your enrollment.

Versions and API names are an integral part of your function's API. It is therefore desirable in many cases to preserve them when packaging and installing them through Marketplace. It is especially important when installing functions alongside upstream applications that reference function dependencies statically, as in [Developer Console applications](#).

In stable mode, a function's version is resolved as follows:

- The installed function is always published at its packaged version.
- If the version already exists for the installed function, an entirely new function will be created; the old function will be hidden and its API name will be removed.

 Creating new functions to resolve version conflicts affects upstream applications. If an upstream application is part of the same installation, it is automatically updated to reference the new function. Otherwise, you will need to update it manually.

A function's API name is resolved as follows:

- The installed function is always published with the API name it was packaged with.
- If the API name is already taken by another function, an installation error will occur. To resolve this conflict, you must delete the existing function or change its API name.

Static function inputs

 This feature is only supported in [TypeScript v1](#).

API Reference ↗

It is possible to modify parts of a function's behavior at install time by defining a function which overrides the "static" function input that is shared by all functions in the same installation.

Send feedback

Marketplace product. To do this, you can specify that a particular function may be overridden by using the `@Static` decorator.

```
import { Function, Static, Double } from "@foundry/functions-api";

export class MyFunctions {

    @Function()
    public async modifyNumberByStaticFoo(
        n: Double,
        @Static() staticFunctionInput: (num: Double) => Promise<Double> = this.c
    ): Promise<Double> {
        return await staticFunctionInput(n);
    }

    private async defaultFoo(n: number) {
        return -n;
    }

}
```

When packaging a function, any static inputs will appear as function inputs during installation. Installers can then provide their own function logic that will override the default behavior.

 Calling [queries](#) or [making API calls](#) within overridden static functions is not supported.

← PREVIOUS
Manage published functions

Function monitoring → NEXT

API Reference ↗

© 2020 Foundry Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

Send feedback

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Function management / Function monitoring

Function monitoring

Functions in Foundry can be monitored to track performance and reliability. This page explains the available monitoring capabilities for functions.

Available monitoring rules

Function monitoring in Foundry supports the following rule types:

1. **Function duration p95:** Alerts when the 95th percentile execution time exceeds thresholds.
2. **Number of function failures in window:** Alerts when the total failure count exceeds thresholds within a timeframe. This rule tracks all failure types.
3. **Number of user-facing function failures in window:** Alerts when the count of user-facing failures exceeds thresholds within a timeframe. This rule tracks only user-facing errors thrown by function code.
4. **Number of non-user-facing function failures in window:** Alerts when the count of non-user-facing failures exceeds thresholds within a timeframe. This rule excludes user-facing errors, making it useful for monitoring infrastructure and system-level failures.

For detailed configuration options and parameters for each rule type, review the [monitoring rules reference documentation](#).

Set up function monitoring

To

API Reference ↗

 for your functions, follow the standard process for creating monitoring rules:

☰

🔍

Send feedback

1. Create a monitoring view as described in the [monitoring views overview documentation](#).
2. Add a monitoring rule for functions as described in the section on [adding a monitoring rule](#).
3. Configure appropriate thresholds and severity levels.
4. Set up alert notifications following the [alert subscription guide](#).

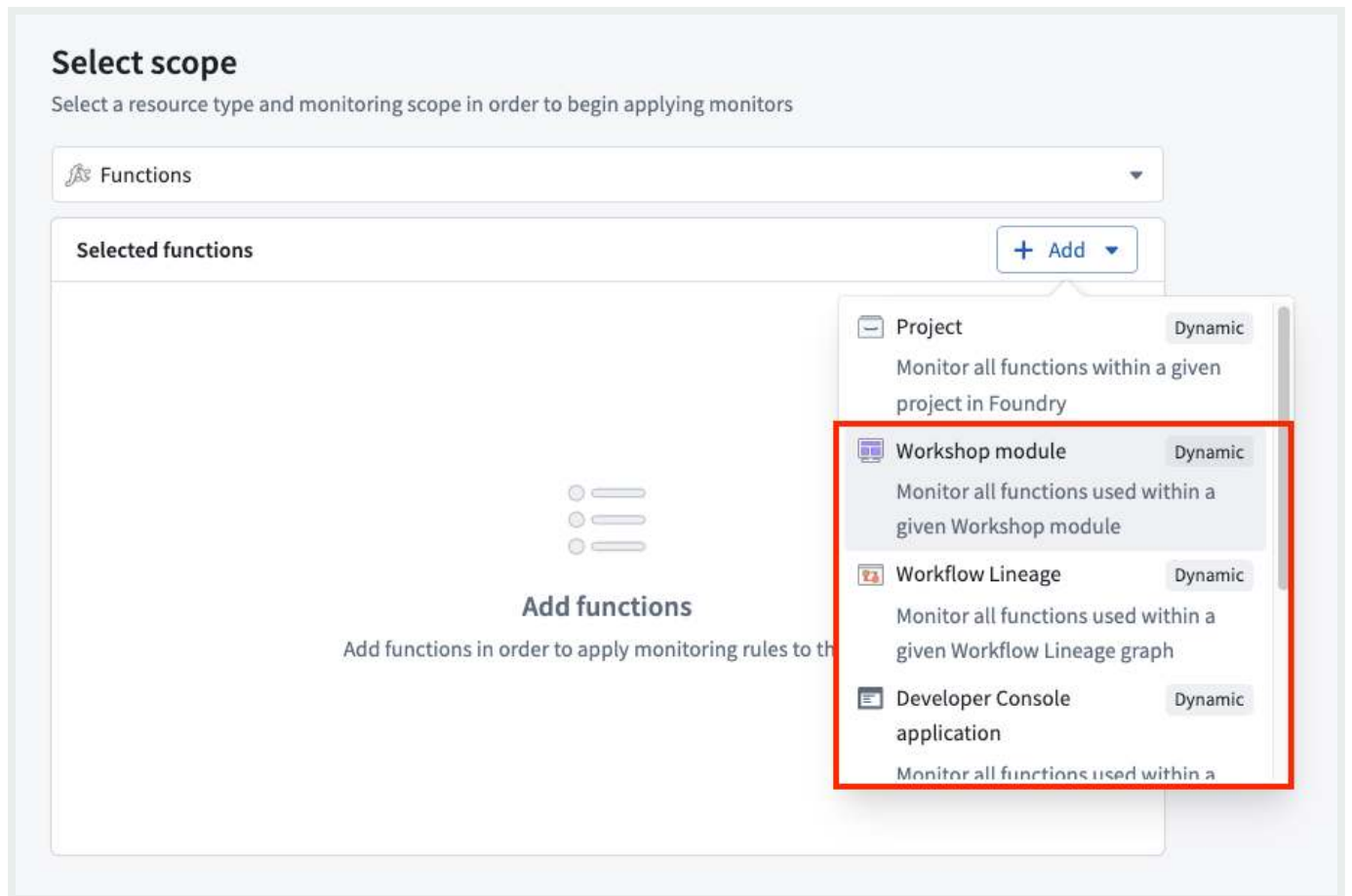
Foundry Notifications This will trigger an in-platform notification and might trigger an email, based on your Foundry Notification settings.				
	Low	Medium	High	Resolution
User name userId	☐	☐	☒	☒
⊕ Add users or groups				
PagerDuty Notifications This will route a page to PagerDuty. Who gets paged is based on your on-call settings configured in PagerDuty.				
		Severity ⓘ Severity type of the PagerDuty integration		
My Pagerduty	Medium	-	High	▶ Test
⊕ Create new PagerDuty integration				
Slack Notifications This will route an alert to the specified channels in your Slack source.				
⊕ Create new Slack integration				
Webhook Notifications This will trigger an alert by calling your custom webhooks.				
⊕ Create new Webhook integration				

Dynamic scopes

Function monitors support **Workflow Lineage**, **Workshop**, and **OSDK application** as dynamic scopes. When you select one of these scopes, the monitor automatically tracks all functions the scoped resource uses and adjusts as functions are added or removed without requiring further intervention.

API Reference ↗

Send feedback



Related documentation

- [Monitoring rules reference](#)
- [Monitoring views overview](#)
- [External system integration](#) for alerts

PREVIOUS
← [Functions in Marketplace](#)

NEXT
[Instrumentation and telemetry](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Functions / Function management / Instrumentation and telemetry

Instrumentation and telemetry in functions

It is possible to emit certain types of telemetry from your functions to allow for monitoring and debugging of production workflows.

To learn how to view telemetry emitted by your functions, see our [AIP observability documentation](#).

Supported telemetry types

The following table provides an overview of the types of telemetry supported by each function language. Note that a single span over the total execution duration of the function, along with a single request log, is automatically created for all function types.

Language	Logs	Spans	Metrics
TypeScript v1	Yes	Only product-defined spans[1]	Only product-defined metrics[2]
TypeScript v2	Yes	Yes[3]	Only product-defined metrics[2]
Python	Yes	Yes[3]	Only product-defined metrics[2]

API Reference ↗

[1] spans in TypeScript v1 functions include operations like object loads and function executions.

Send feedback

[2] Foundry records the total execution duration for all types of functions.

[3] TypeScript v2 and Python functions automatically instrument all outbound network requests, but custom spans can also be added.

Logs

You can emit custom logs from functions and view them retroactively. The following examples demonstrate how to emit logs from TypeScript v1, TypeScript v2, and Python functions.

In TypeScript v2 functions, Foundry will set up the OpenTelemetry SDK's global logger provider, and you will be able to retrieve a logger from it. If you want to use third-party libraries for logging, you must configure them to emit logs through a logger obtained from the global logger provider.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 export class MyFunctions {
2   @Function()
3   public myFunction(name: string): string {
4     console.log(`This is a custom log line, ${name}.`);
5     return `Hello, ${name}!`;
6   }
7 }
```

Spans

You can also create custom spans in TypeScript v2 and Python functions to track the duration of specific operations. The following example demonstrates how to create a custom span.

In TypeScript v2 and Python functions, Foundry will set up the OpenTelemetry SDK's global tracer provider, and you will be able to retrieve a tracer from it. If you want to use third-party libraries for tracing, you must configure them to emit traces through a tracer obtained from the global tracer provider.

[API Reference ↗](#)

[Send feedback](#)

```

1  import { trace } from "@opentelemetry/api";
2  import { Integer } from "@osdk/functions";
3
4  const tracer = trace.getTracer("my-function");
5
6  export default function sqrt(n: Integer): Integer {
7      const sqrt = tracer.startActiveSpan("my-custom-span", (span) => {
8          try {
9              return Math.sqrt(n);
10             } finally {
11                 span.end();
12             }
13         });
14
15         return sqrt;
16     }

```

PREVIOUS

← Function monitoring

NEXT

Permissions →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— Cookies Settings

[API Reference ↗](#)

Send feedback

Ontology building / Functions / Function management / Permissions

Permissions

Authoring and executing functions in the platform is subject to many kinds of permission checks. This section outlines the different types of permissions you should be aware of and common issues you may run into.

Function authoring

Functions repositories must be granted appropriate permissions to:

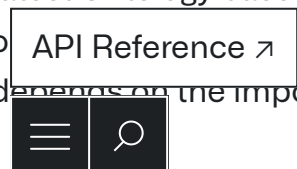
1. Access the Ontology so that proper code bindings can be generated.
2. Load objects in order to run a live preview of a function execution.

Note that repository permissions must be explicitly granted, and are not the same as the permissions granted to your user account. As a result, you have to take specific steps to import object types, link types, and backing datasources into the Project that contains your repository.

For a tutorial on these steps, see [this section](#). Below, we explain the specific resources that are imported and the permissions granted for those resources.

Ontology entity permissions

In a repository, whenever checks run or Code Assist starts up, the functions plugins load the latest Ontology based on the repository's permissions and generate code bindings for every object that was loaded. The set of object and link types that are loaded depends on the imports of the following resource types:

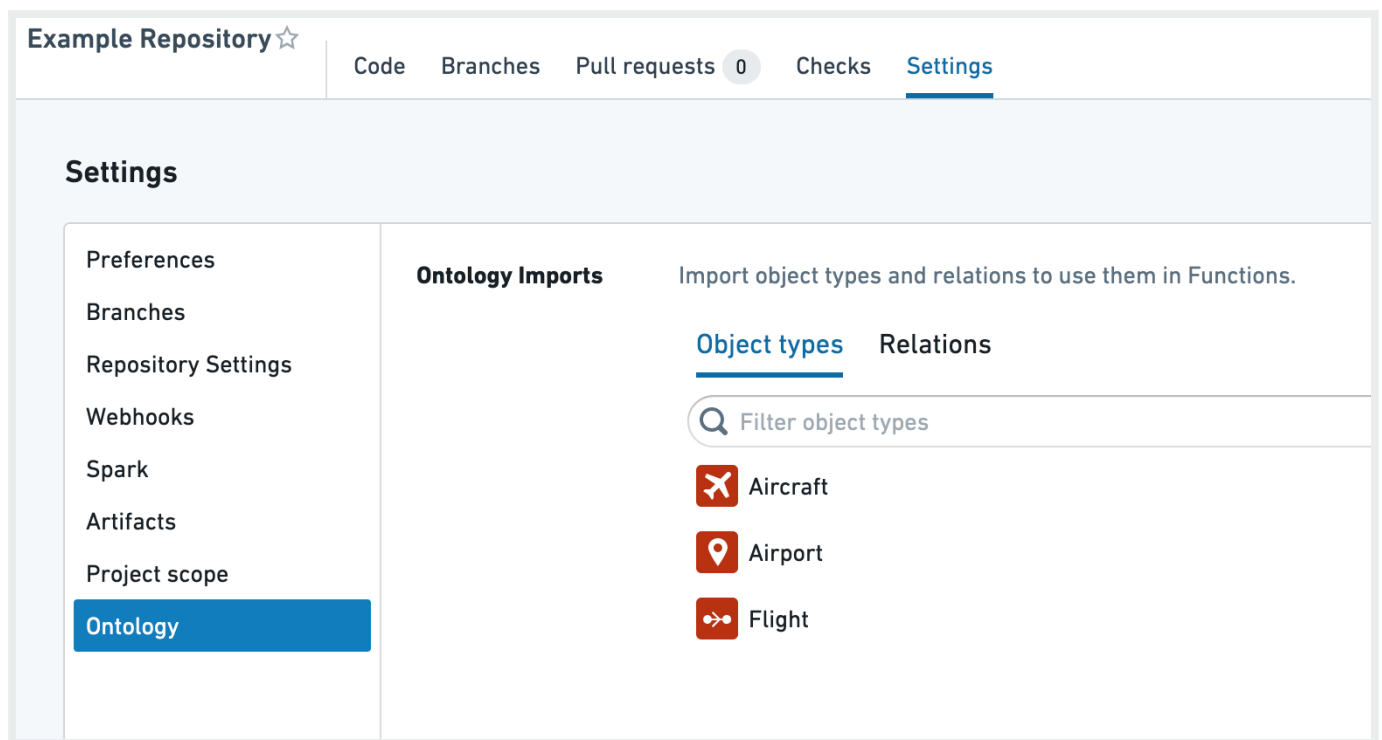


- Ontologies

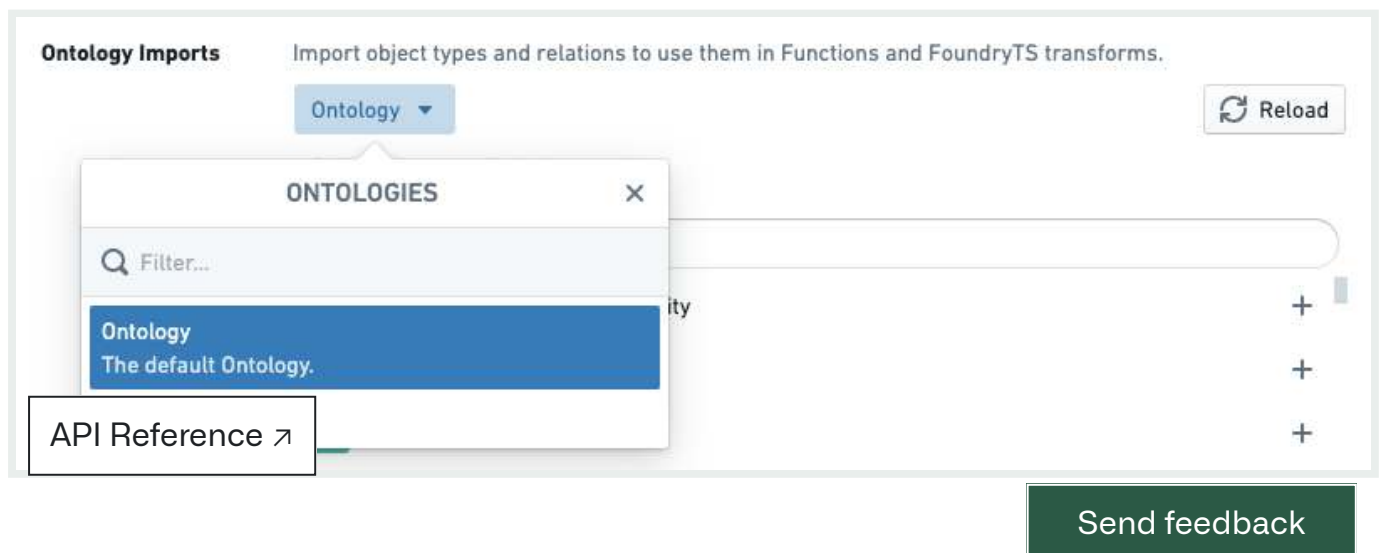
Send feedback

- Ontology branches
- [Object types](#)
- [Link types](#)

In a functions repository, you can import the needed Ontology resources by navigating to **Settings > Ontology**. This interface allows you to choose object and link types to import into your Project.



If your user account has access to multiple Ontologies, you can also choose which Ontology you'd like to use. Currently, importing multiple Ontologies into a single Project is unsupported.



⚠ Warning

Although the above interface shows up within functions repositories, any Ontologies, object types, and link types you import are added at the **Project** level. This means that changing imports in one repository can affect other repositories in the same Project. If you want to have two repositories that rely on different Ontology entities, you should separate them into different Projects.

Object loading permissions

The **functions helper** in a repository allows users to execute functions in two ways: by executing a published function, or by executing code in live preview. When executed in a live preview, functions code is compiled and executed in Code Assist, which is infrastructure designed to enable quick iteration for code authors.

Because it is tied to the repository, Code Assist is subject to the same permissions requirements as code generation, as described above. This means that when running a function in live preview, the backing datasources underlying each object type you wish to use must be imported into the Project.

In the functions helper, if there are object types imported into your Project without the corresponding datasource being imported, a warning will be displayed in live preview prompting you to update the imports:



In the case of most object types, the **Import backing datasources** dialog will prompt you to import the **API Reference** dataset. For object types that have [row-level security](#) enabled, you will be prompted to import a [Restricted View](#).

Send feedback

Published function execution

Once a function has been published, it is ready for use by a broader audience of users and can be configured to execute in applications such as [Workshop](#) and [Actions](#). There are still some considerations to keep in mind for permissions to execute a published function.

Function permissions

In order to execute a function, a user must have **Viewer** role on the repository from which the function was published. Typically, it is best to locate functions repositories in the same Project as end-user applications that rely on functions in that repository, whether those applications are created using Workshop, Slate, or some other tool. If users encounter errors indicating that they lack permissions to read a function (`ReadFunctionsPermissionDenied`), check whether they have read access to the repository. [Learn more about how to move and share resources.](#)

i The **Check access** panel in the sidebar can be used to check someone's access to a Workshop or Slate application, including access to dependent functions. For more information, see the [Check access panel documentation](#).

[Function-backed Actions](#) are a special case in which end users do not necessarily need read access to the function in order to apply an Action that uses it. An administrative user must have read access to a function when configuring an Action to use it. Afterwards, users will be able to apply the Action based on [Action-level permissions](#), regardless of their access to the function.

Object loading permissions

When a function loads object data, either as a parameter or via an [Object search](#), the permissions of the end user running the function determine which objects are loaded. In the case of object types secured using row-level permissions, this means that different users running the same function may receive different results. This behavior is intended—users should only see the objects they have access to, and this behavior ensures the function to work for users with differing access to individual objects.

[API Reference](#) ↗

[Send feedback](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Function consumption / Use functions in the platform

Use functions in the platform

This section documents various ways that you can use functions throughout the Foundry platform. This list is kept mostly up to date, but there may be additional ways to use functions that are not captured here.

Workshop

[Workshop](#) supports integration with functions in a variety of ways, enabling the use of custom logic throughout modules built in Workshop.

Variables

Most Workshop [Variables](#) can be Function-backed, allowing an application builder to use functions to compute values that can then be used throughout Workshop. By default, the value for a variable is recomputed when another variable it depends on is updated. This enables flexible recomputation of values in response to user feedback—for example, when a user edits an input component, a dependent Function-backed Object Set variable will be recomputed automatically.

To learn more, take a look at the [tutorial on how to use functions to back Workshop variables](#).

Below is the mapping between Workshop variable types and their equivalents in TypeScript.

A [API Reference ↗](#) of each given type can be backed by a Function that returns one of the variable types listed. [Learn more about the available Function types are documented.](#)



- Boolean: `boolean`

Send feedback

- String: `string`
- Numeric: `Integer`, `Long`, `Float`, `Double`
- Date: `LocalDate`
- Timestamp: `Timestamp`
- Array: `BaseType[]` or `Set<BaseType>`
- Object Set: `ObjectSet<ObjectType>` (recommended), `ObjectType[]`, or `Set<ObjectType>`

Object Table: Derived properties

Workshop's **Object Table** widget can be configured to compute a Function-backed column, which can update based on user input and will be recomputed on the fly as end users scroll through the table. You can see a [full tutorial for using this functionality](#).

Chart: Derived aggregations

Workshop's **Chart: XY** widget supports using a Function-backed aggregation to derive aggregated values on demand. This can be useful if you want to have aggregation data be based on user selection. To use functions in a Chart widget, simply click to configure a chart layer and select *Function aggregation*.

API Reference ↗

Send feedback

Chart: XY 2WIDGET

Widget setup

Metadata

Display

< Chart Editor

Layer (bar)

Title

Layer (bar)

DATA INPUT

Object set

Function aggregation

Select a Function that returns a Two-dimensional or Three-dimensional aggregation below:

Select a function...

A [reference for the Aggregation API](#) is available. For more advanced use cases, you may want to read the documentation about [how to compute custom aggregations](#).

Actions

[Action types](#) enable applications to make changes to the objects in the Foundry ontology and to dispatch external notifications and side effects in a way that is flexible and secure. Within Actions, functions provide complete flexibility, enabling code authors to define how objects should be updated or how side effects should be configured.

Function-backed Actions

Function-backed Actions use the [Ontology edits](#) API to define the logic for how objects should be updated. This allows you to express complex edits in code—for example, updating every objected linked to some starting object. [See a tutorial for how to use Function-backed](#)
[API Reference](#) ↗

Send feedback

Side effects: Notifications

An Action can be configured to send a Notification to a specified user. You can use functions to compute which users should receive a Notification, as well as the contents of the Notification itself. This provides flexibility such as loading recipient user IDs that are stored within objects, or rendering email content based on object data. To learn more, consult the [full documentation about Notifications](#) and a [guide for how to use functions to configure Notifications](#).

Side effects: Webhooks

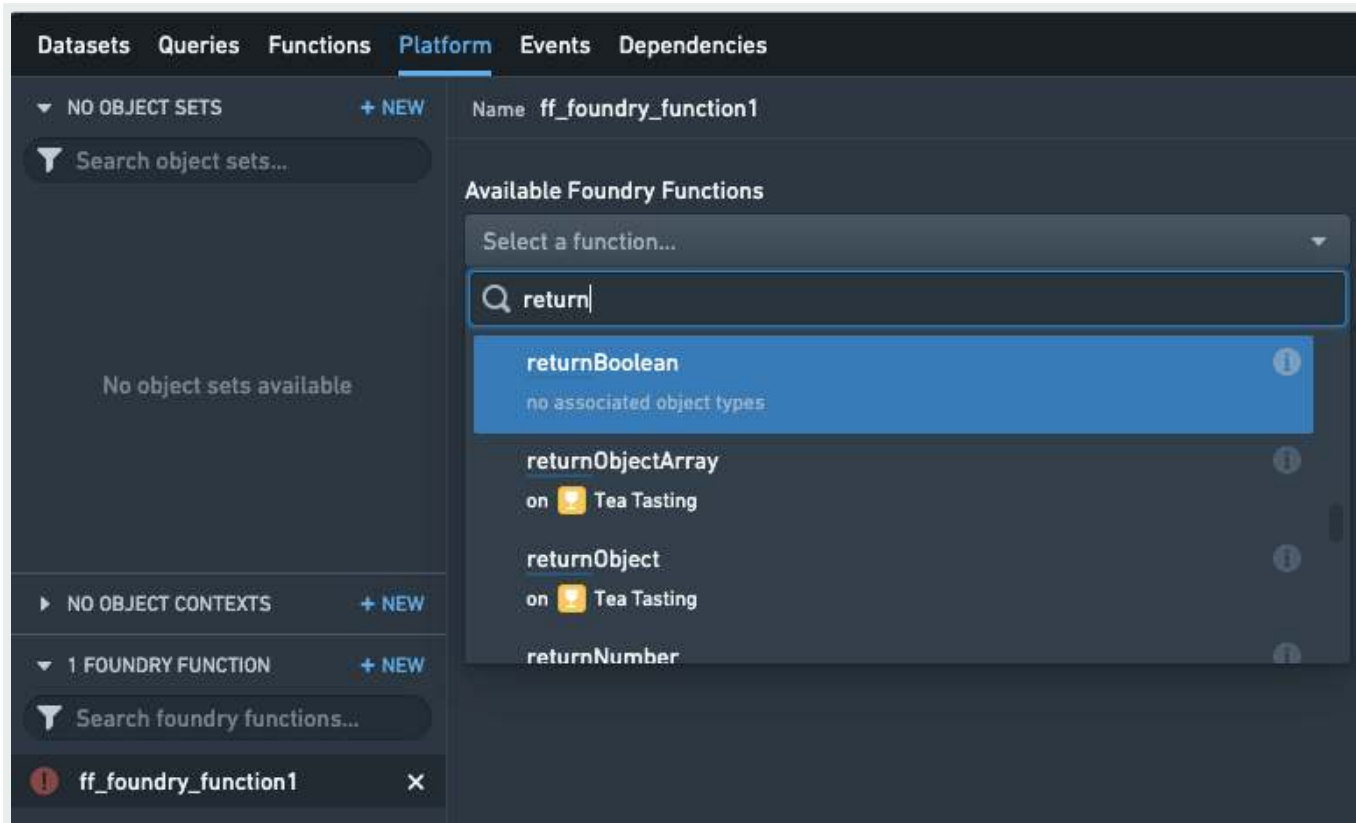
An Action can also be configured to trigger a Webhook when it is applied. Webhooks enable integration of Foundry with other systems, enabling user-applied Actions to write back to APIs outside of Foundry. You can use functions to compute the parameters that should be sent to the Webhook that will be executed, enabling workflows like populating Webhook parameters based on object data. [View full documentation about Webhooks](#).

Slate

[Slate](#) includes native support for finding and using functions within the **Platform** tab. When editing a Slate document, open the Platform tab and add a **Foundry Function** in the bottom-left. Now, you can search for a Function, configure parameters, and use the result in your Slate document.

API Reference ↗

Send feedback



Note that for historical reasons, the Slate product has its own notion of "functions", which are snippets of JavaScript logic located within each Slate document. This is why the functions product is called "Foundry functions" and is located under the **Platform** tab. Slate's functions capability allows for quick, easy data manipulation within a document, but do not have native support for objects.

You can use Slate's functions and Foundry functions in combination with each other—for example, you could return data from a Foundry Function and manipulate it in a Slate Function, or use a Slate Function to compute parameters that should be passed into a Foundry Function.

Quiver

[Object set plots](#) in Quiver use the same underlying component as Workshop's Chart: XY widget. As such, you can use Function-backed Aggregations in Quiver analyses as well.

[API Reference ↗](#)

[Send feedback](#)

[Automate](#) allows you to create function-backed automations, which automatically execute functions when a specified condition is met.

When configuring a Function effect, you can select a function and specify its version. For stable versions (1.0.0 and greater), you can enable automatic upgrades to compatible versions. Functions in Automate execute asynchronously and can run for up to 4 hours.

Note that functions with ontology edit return types will not have the edits applied when used as effects in Automate. To learn more, see the [Function effects documentation](#).

PREVIOUS

← Function management / Permissions

NEXT

Function metrics →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Function consumption / Function metrics

Function metrics

Function metrics display the near real-time usage of a function type over the last 30 days. You can access these metrics from a [function type's overview](#) page in [Ontology Manager](#), or in [Workflow Lineage](#) by selecting the function node for a given execution. The following metrics are available:

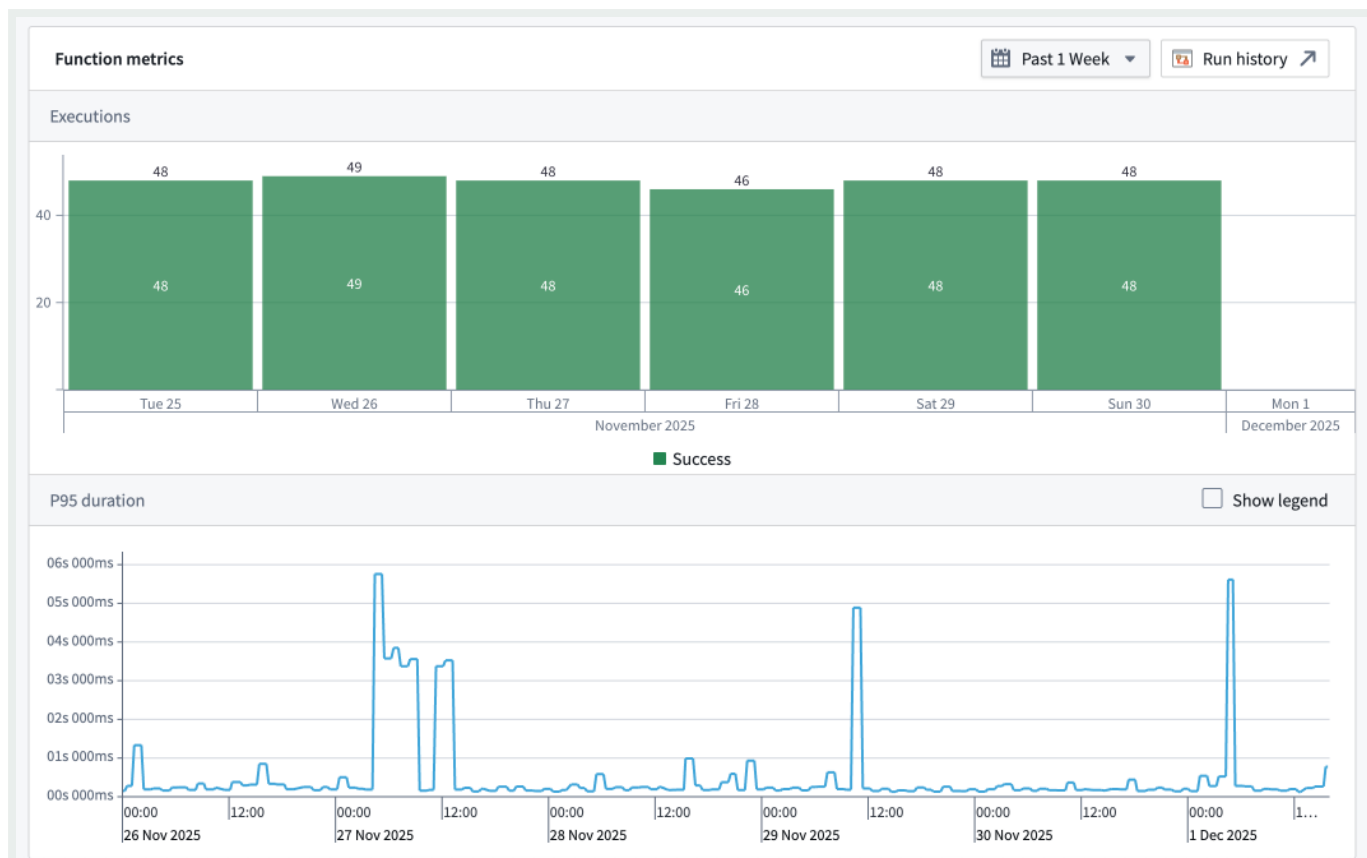
- **Success/failure metrics:** Monitor the current status of your functions with success and failure counts. This enables rapid identification of issues and supports proactive troubleshooting, allowing you to address failures as soon as they occur.
- **P95 duration metric:** Track the 95th percentile (P95) execution duration for each function type. This metric highlights the upper range of execution times, helping you detect performance bottlenecks and optimize workflows for consistent and efficient operation.

You are also able to access [run history](#), which provides a complete view of a given function's executions over the past seven days. Learn more about [AIP observability](#).

API Reference ↗



Send feedback



All metrics are updated in near real-time using the latest data from the Foundry Telemetry Service (FTS). This ensures you have access to the most current information for monitoring, debugging, and maintaining the health of your functions.

You can also embed function metrics directly inside an operational application using the [Observability Chart widget](#) in [Workshop](#) to monitor resource health alongside the rest of a workflow.

Function failure types

Function metrics have a variety of categories of failures that may be displayed. These categories are:

- **Runtime failure:** An unexpected error occurred while executing the function, often due to a bug or unhandled situation in the function's code.
- **Resource limit exceeded:** The function affected more than the permitted limit of object (usually 10,000).
- **User-facing error:** An error occurred that is specifically intended to be shown to the user, often providing guidance on what went wrong or how to fix it.

[API Reference ↗](#)

[Send feedback](#)

- **Invalid inputs error:** One or more of the inputs provided to the function were not valid or did not meet the required criteria.
- **Invalid output error:** The function produced output that was not valid or did not conform to the expected format or rules.
- **Data loading not allowed error:** The function execution attempts to load data, including objects, object sets, users, or groups, but is not allowed to do so.
- **Undeclared object types edited error:** The function execution attempts to update, create or delete an object whose object type is not declared in the function spec.
- **Structured error:** The function execution encounters a structured error as defined on its spec.
- **Deployment error:** The function execution failed due to an error with the function's deployment.
- **Consistent snapshot error:** The function failed to execute due to a consistent snapshot error.

Permissions

To view function metrics, you must be a **viewer** on the function.

PREVIOUS

← [Use functions in the platform](#)

NEXT

[Optimize performance](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

—[Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Functions / Function consumption / Optimize performance

Optimize performance

This page describes best practices for optimizing function performance and resource usage. Following these guidelines helps minimize compute consumption and ensures your functions run efficiently.

i For information about compute costs and how functions are metered, see [Compute usage with Ontology queries](#).

Understand function compute costs

The cost of a function has multiple components:

- **Overhead:** Each function execution has a fixed overhead of 4 compute-seconds, regardless of what it does.
- **Compute time:** The vCPU time the function needs to execute.
- **External calls:** Calls to other parts of the platform (Ontology queries, model inference, LLM calls) incur their own costs.

For more information about how compute-seconds are calculated and measured in the platform, see [Usage types](#).

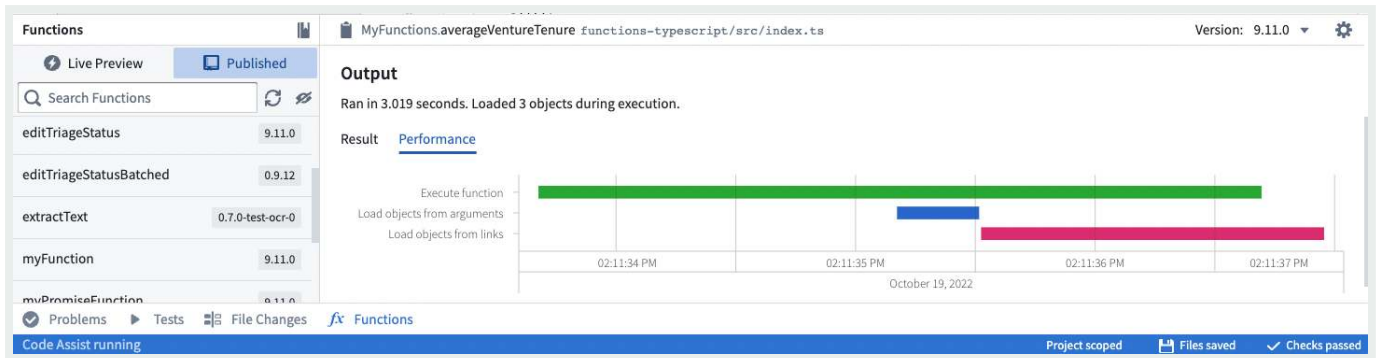
Use the Performance tab

API Reference ↗

The performance tab provides a tool to analyze and identify performance issues with your



Send feedback



The waterfall graph represents operations as horizontal bars stretched out across time on the X-axis. There are markers for each operation to indicate how time is spent.

- **Execute function** indicates CPU time spent executing the function code.
- **Load objects from arguments** and **Load objects from links** indicate the time spent calling the underlying Ontology backend database service (OSS).

To improve function performance:

- Use the Objects API to aggregate and traverse links more quickly than within function context (as described in [Prefer using the Objects API where possible](#)).
- Ensure Ontology backend service calls are done in parallel to avoid sequential loads. If you have multiple `async / await` calls, use `Promise.all` to await all the calls in parallel.
 - For example, a common pattern is to use `.map()` on a list to create Promises, then use `Promise.all` on the resulting list.
 - **Important:** Using `Promise.all()` improves execution speed but does not reduce resource consumption or cost. You still make the same number of operations—they just run in parallel. Bulk operations are both faster and more cost-effective.
- Avoid unnecessary nested loops, which can increase execution time.

Choose efficient input types

When designing functions, the type of input parameter you choose significantly affects performance. Use the most efficient input type that meets your requirements.

Best practice: Use object sets when possible for maximum efficiency and scalability.

API Reference [↗](#) object arrays when you need to work with objects in memory. **Anti-**

pattern: Single object parameters should only be used when your object has one instance or when specific business logic genuinely requires per-

Send feedback

Input type	Efficiency	Use case
Object set	Best	Queryable set of objects; no upfront loading cost if you only need aggregations
Object array	Good	When you need to iterate over specific objects
Single object	Least efficient	When business logic requires processing one object at a time

Best practice: Object sets

Objects passed as parameters trigger Ontology queries to load the object data. Even a single object input triggers a call to load that object into memory.

Object sets are preferable because they defer loading until you actually need the data. If you only need an aggregation (like count or sum), the Ontology backend computes it without loading individual objects.

PYTHON

TYPESCRIPT V2

TYPESCRIPT V1

```

1  from functions.api import function, Float
2  from ontology_sdk.ontology.objects import ExampleDataAircraft
3  from ontology_sdk.ontology.object_sets import ExampleDataAircraftObjectSet
4
5  # Less efficient: Single object triggers upfront loading
6
7  @function()
8  def get_aircraft_name(aircraft: ExampleDataAircraft) -> str:
9      return aircraft.display_name
10
11 # Moderate: Array of objects triggers upfront loading
12
13 @function()
14 def get_aircrafts_names(aircraft_array: list[ExampleDataAircraft]) -> list[str]:
15     return [aircraft.display_name for aircraft in aircraft_array]
16
17 # More efficient: Object set defers loading until needed
18
19 # Here, only the aggregated value is loaded in memory
20 @function()

```

API Reference ↗

Send feedback

```
21 def count_aircrafts(aircraft_set: ExampleDataAircraftObjectSet) -> Float:
22     return aircraft_set.count().compute()
```

Load objects efficiently

The common source of performance issues in functions comes from loading objects inefficiently. Loading objects one at a time in a loop causes a round-trip to the ontology on each iteration.

Anti-pattern: Loading objects one by one

Loading objects inside a loop is an anti-pattern that significantly impacts performance. Each iteration makes a separate query to the Ontology:

PYTHON

TYPESCRIPT V2

TYPESCRIPT V1

```
1 from functions.api import function
2 from ontology_sdk import FoundryClient
3 from ontology_sdk.ontology.objects import ExampleDataAircraft
4
5 # Anti-pattern: Objects loaded one by one in a loop
6
7 @function()
8 def for_loop_worst(pks_list: list[str]) -> int:
9     client = FoundryClient()
10    seats_count = 0
11
12    for current_pk in pks_list:
13        aircrafts = client.ontology.objects.ExampleDataAircraft.where(
14            ExampleDataAircraft.object_type.tail_number == current_pk
15        ).page()
16        aircraft = aircrafts.data[0]
17        seats_count += aircraft.number_of_seats or 0
18
19    return seats_count
```

API Reference ↗

```
21 # Better: Load all objects in one call, then iterate
```

```
22
```

```
23 @function()
```

Send feedback

```

24 def for_loop_better(pks_list: list[str]) -> int:
25     client = FoundryClient()
26     seats_count = 0
27
28     aircrafts = client.ontology.objects.ExampleDataAircraft.where(
29         ExampleDataAircraft.object_type.tail_number.in_(pks_list)
30     )
31
32     for aircraft in aircrafts:
33         seats_count += aircraft.number_of_seats or 0
34
35     return seats_count
36
37 # Best: Let the backend perform the aggregation
38
39 @function()
40 def for_loop_best(pks_list: list[str]) -> int:
41     client = FoundryClient()
42
43     result = client.ontology.objects.ExampleDataAircraft.where(
44         ExampleDataAircraft.object_type.tail_number.in_(pks_list)
45     ).sum(ExampleDataAircraft.object_type.number_of_seats).compute()
46
47     return int(result or 0)

```

Best practice: Backend aggregations

When you need to compute aggregates like counts, sums, or averages, use the Ontology backend's aggregation capabilities instead of loading objects and computing in your function.

Prefer using the Objects API where possible

A common paradigm when using [Workshop's derived properties](#) is to calculate the property value by aggregating over each object's links (for example, counting the number of related objects).

API Reference ↗

Although the code below works, the function itself must retrieve all linked objects, and then perform an aggregation (in this case, calculating the length):

Send feedback

TYPESCRIPT V1

TYPESCRIPT V2

```

1 import { Function, FunctionsMap, Integer } from "@foundry/functions-api";
2 import { Employee } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Function()
6     public async getEmployeeProjectCount(employees: Employee[]): Promise<
7         const promises = employees.map(employee => employee.workHistory.a
8         const allEmployeeProjects = await Promise.all(promises);
9         let functionsMap = new FunctionsMap();
10        for (let i = 0; i < employees.length; i++) {
11            functionsMap.set(employees[i], allEmployeeProjects[i].length)
12        }
13        return functionsMap;
14    }
15 }

```

While the above takes advantage of the async API and asynchronous functions (see [Optimizing link traversals](#)), it is often beneficial to use the aggregation methods provided by the Objects API:

TYPESCRIPT V1

TYPESCRIPT V2

```

1 import { Function, FunctionsMap, Integer } from "@foundry/functions-api";
2 import { Employee, Objects } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Function()
6     public async getEmployeeProjectCount(employees: Employee[]): Promise<
7         const result: FunctionsMap<Employee, Integer> = new FunctionsMap(
8         // Get all projects that have an employeeId matching from the emp
9         const aggregation = await Objects.search().project()
10            .filter(project => project.employeeId.exactMatch(...empho
11            .groupBy(project => project.employeeId.byFixedWidths(1))
12            .count();
13
14        const map = new Map();
15        aggregation.buckets.forEach(bucket => {
16            // bucket.key.min represents the employeeId as each bucket si
17            map.set(bucket.key.min, bucket.value);
18        });

```

API Reference ↗

Send feedback

```

19     employees.forEach(employee => {
20         const value = map.get(employee.primaryKey);
21         if (value === undefined) {
22             return;
23         }
24         result.set(employee, value);
25     });
26
27     return result;
28 }
29 }

```

In this way, you can perform the aggregation without needing to pull in all linked projects first.

i Note that the usual limitations of aggregations still apply. In particular, `.topValues()` on string IDs will only return the top 1000 values. Aggregations are currently limited to a maximum of 10K buckets, so you may need to perform multiple aggregations to retrieve the desired result. See [Computing Aggregations](#) for more details.

Optimizing link traversals

The most common source of performance issues in functions comes from traversing links in an inefficient manner. Often, this occurs when you write code that loops over many objects and calls an API to load related objects on every iteration of the loop.

TYPESCRIPT V1

TYPESCRIPT V2

```

1 for (const employee of employees) {
2     const pastProjects = employee.workHistory.all();
3 }

```



API Reference ↗

In this example, each iteration of the loop will load an individual employee's past projects causing a round-trip to the database. To avoid this slowdown, you can

Send feedback

s

link traversal APIs when traversing many links at once. Below is an example of a function that is written to load links asynchronously:

TYPESCRIPT V1

TYPESCRIPT V2

```
1 import { Function } from "@foundry/functions-api";
2 import { Employee } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Function()
6     public async findEmployeeWithMostProjects(employees: Employee[]): Promise<Employee> {
7         // Create a Promise to load projects for each employee
8         const promises = employees.map(employee => employee.workHistory.a
9         // Dispatch all the promises, which will load all links in parallel
10        const allEmployeeProjects = await Promise.all(promises);
11        // Iterate through the results to find the employee who has the g
12        let result;
13        let maxProjectsLength;
14        for (let i = 0; i < employees.length; i++) {
15            const employee = employees[i];
16            const projects = allEmployeeProjects[i];
17
18            if (!result || projects.length > maxProjectsLength) {
19                result = employee;
20                maxProjectsLength = projects.length;
21            }
22        }
23
24        return result;
25    }
26 }
```

The TypeScript v1 example uses an ES6 [async function ↗](#), which makes it convenient to handle the `Promise` return values that are returned from the `.getAsync()` and `.allAsync()` methods. The TypeScript v2 example uses `Promise.all()` together with `$link` and `asyncIter()` to load each employee's linked projects in parallel.

[API Reference ↗](#) async operations and resource usage

Asynchronous operations can speed up function execution, but they resource usage. Understanding this distinction is important for cost optimization.

Send feedback

PYTHON

TYPESCRIPT V2

TYPESCRIPT V1

```
1 from functions.api import function, Float
2 from ontology_sdk.ontology.object_sets import ExampleDataAircraftObjectSet
3
4 # Best: Bulk operation using search around
5
6 @function
7 def bulk_processing(aircraft_set: ExampleDataAircraftObjectSet) -> Float:
8     all_maintenance_events = aircraft_set.search_around_example_data_airc
9     return all_maintenance_events.count().compute()
```

⚠ Async operations improve speed, not cost

Using asynchronous operations like `Promise.all()` can improve execution speed by running operations in parallel. However, it is important to understand that async operations **do not reduce resource consumption or cost**—they just make things faster.

For example, parallelizing a loop of individual queries is faster than running them sequentially, but you are still making the same number of queries. **Bulk operations that push computation to the backend are both faster and more resource-effective** than either approach.

Write efficient ontology edits

When writing functions that edit objects, apply the same bulk-loading principles. Load all objects upfront rather than one at a time.

Editing large set of objects

When editing large numbers of objects, use pagination (explicit or implicit via `iterate` or `asyncIterator`) to process them in manageable chunks without loading everything into memory

API Reference ↗

Send feedback

```
1 from functions.api import function, OntologyEdit
2 from ontology_sdk.ontology.objects import ExampleDataAircraft
3 from ontology_sdk.ontology.object_sets import ExampleDataAircraftObjectSet
4 from ontology_sdk import FoundryClient
5
6 # Single object edit
7
8 @function(edits=[ExampleDataAircraft])
9 def edit_aircraft_name(aircraft: ExampleDataAircraft) -> list[OntologyEdit]:
10     ontology_edits = FoundryClient().ontology.edits()
11     editable = ontology_edits.objects.ExampleDataAircraft.edit(aircraft)
12     editable.display_name = "new display name"
13     return ontology_edits.get_edits()
14
15 # Bulk edit using object set with iteration
16
17 @function(edits=[ExampleDataAircraft])
18 def edit_all_aircrafts(aircraft_set: ExampleDataAircraftObjectSet) -> list[OntologyEdit]:
19     ontology_edits = FoundryClient().ontology.edits()
20
21     for aircraft in aircraft_set.iterate():
22         editable = ontology_edits.objects.ExampleDataAircraft.edit(aircraft)
23         editable.display_name = "new display name"
24
25     return ontology_edits.get_edits()
26
27 # Alternative: Pagination
28
29 # This processes objects in chunks. The iterate() method above takes care of this
30 @function(edits=[ExampleDataAircraft])
31 def edit_all_with_pagination(aircraft_set: ExampleDataAircraftObjectSet) -> list[OntologyEdit]:
32     edits = FoundryClient().ontology.edits()
33
34     next_token = None
35     while True:
36         page = aircraft_set.page(1000, next_token)
37         for aircraft in page.data:
38             editable = edits.objects.ExampleDataAircraft.edit(aircraft)
39             editable.status = "reviewed"
40             next_token = page.next_page_token
41         if not next_token:
42             break
```

[API Reference ↗](#)[Send feedback](#)

```
44
45     return edits.get_edits()
```

Optimize derived column generation

Workshop supports computing derived properties using functions on objects (FOO).

Workshop applications typically call these functions with a few dozen rows of content from an object table. The function then returns a map where each object is mapped to the display value in the derived column.

Base implementation without optimization

Below is a non-optimized implementation that serves as the base case:

TYPESCRIPT V1 TYPESCRIPT V2

```
1  import { Function, FunctionsMap, Double } from "@foundry/functions-api";
2  import { Objects, ObjectSet, objectTypeA } from "@foundry/ontology-api";
3
4  export class MyFunctions {
5      /**
6       * This Function takes an ObjectSet as input, and generates a derived
7       * This derived column maps each object instance to the numeric value
8       * This implementation is a trivial for-loop that multiplies an object
9       * This serves as the base case that we will improve below.
10     */
11     @Function()
12     public getDerivedColumn_noOptimization(objects: ObjectSet<objectTypeA>) {
13         // Define the result map to return
14         const resultMap = new FunctionsMap<objectTypeA, Double>();
15
16         /* There is a limit to the number of objects that can be loaded in
17          * See enforced limit documentation for current object set load limit
18          */
19         const allObjs: objectTypeA[] = objects.all();
20
21         // For each loaded object, perform the computation. If the result
22         allObjs.forEach(o => {
23             const result = this.computeForThisObject(o, scalar);
```

API Reference ↗

Send feedback

```

24         if (result) {
25             resultMap.set(o, result);
26         }
27     });
28
29     return resultMap;
30 }
31
32 // An example of a function that computes the required value for the
33 private computeForThisObject(obj: objectTypeA, scalar: Double): Double {
34     if (scalar === 0) {
35         // Division by zero error
36         return undefined;
37     }
38     // Checks if exampleProperty is defined, and divides if so. If no
39     return obj.exampleProperty ? obj.exampleProperty / scalar : undefined;
40 }
41 }

```

Parallel execution optimization

If the computation is complex, it is possible to reduce compute time by using asynchronous execution. This way, computations for each object are executed in parallel:

TYPESCRIPT V1

TYPESCRIPT V2

```

1 import { Function, FunctionsMap, Double } from "@foundry/functions-api";
2 import { Objects, ObjectSet, objectTypeA, objectTypeB } from "@foundry/or
3
4 /**
5  * This function takes a list of strings that are object primaryKeys as i
6  */
7 @Function()
8 public async getDerivedColumn_parallel(objects: ObjectSet<objectTypeA>, s
9     // Define the result map
10     const resultMap = new FunctionsMap<objectTypeA, Double>();
11
12     // There is a limit to the number of objects that can be loaded in me
13     // enforced limit documentation for current object set load limit
14     * This should not be a problem as Workshop can lazy load as users ar
15     */
16     const allObjs: objectTypeA[] = objects.all();

```

API Reference ↗

Send feedback

```

17
18 // Launch parallel computations for each object in the array
19 const promises = allObjs.map(currObject => this.computeForThisObject(
20
21 // Use Promise.all to parallelize async execution of helper function
22 const allResolvedPromises = await Promise.all(promises);
23
24 // Populate resultMap with results
25 for (let i = 0; i < allObjs.length; i++) {
26     resultMap.set(allObjs[i], allResolvedPromises[i]);
27 }
28
29 return resultMap;
30 }
31
32 // An example of a function that computes the required value for the prov
33 private async computeForThisObject(obj: objectTypeA, scalar: Double): Pro
34     if (scalar === 0) {
35         // Division by zero error
36         return undefined;
37     }
38     // Checks if exampleProperty is defined, and divides if so. If not, i
39     return obj.exampleProperty ? obj.exampleProperty / scalar : undefined
40 }

```

Advanced: Ontology filtering within computation

For more complex cases where each object requires querying the Ontology, see the below examples. **Note:** The same applies with a `TwoDimensionalAggregation` that would populate a [Chart XY widget](#) in Workshop. You can pass a list of category strings (buckets) to compute, instead of a list of object instances. Below is an example:

TYPESCRIPT V1

TYPESCRIPT V2

```

1 /**
2  * An example of a function that computes the required value for the prov
3  * For a given object, query the Ontology (filter for other objects, search

```

API Reference ↗

```

6 private async computeForThisObject_filterOntology(obj:
7 // Create an object set by filtering on some property
8 const currObjectSet = await Objects.search().objectTypeB().filter(o

```

Send feedback


```

9      // Note: If there is an existing link between the ObjectTypes, an alt
10     // const currObjectSet = await Objects.search().objectTypeA([obj]).se
11
12     // Compute the aggregation for this object set
13     return await this.computeMetric_B(currObjectSet);
14 }
15
16 @Function()
17 public async computeMetric_B(objs: ObjectSet<objectTypeB>): Promise<Double
18     // Set up calls to different parts of the equation
19     const promises = [this.sumValue(objs), this.sumValueIfPresent(objs)];
20
21     // Execute all promises
22     const allResolvedPromises = await Promise.all(promises);
23
24     // Get values from the promises
25     const sum = allResolvedPromises[0];
26     const sumIfPresent = allResolvedPromises[1];
27
28     // Perform calculation
29     return sum / sumIfPresent;
30 }
31
32 @Function()
33 public async sumValue(objs: ObjectSet<objectTypeB>): Promise<Double> {
34     // Sum the values of the objects
35     const aggregation = await objs.sum(o => o.propertyToAggregateB);
36     const firstBucketValue = aggregation.primaryKeys[0].value;
37     return firstBucketValue;
38 }
39
40 @Function()
41 public async sumValueIfPresent(objs: ObjectSet<objectTypeB>): Promise<Dou
42     // Sum the object values if they are not null
43     const aggregation = await objs.filter(o => o.metric.hasProperty()).su
44     const firstBucketValue = aggregation.primaryKeys[0].value;
45     return firstBucketValue;
46 }

```

Converting to TwoDimensionalAggregation

API Reference ↗

For use with [ChartXY widgets](#) in Workshop, you can produce a
[TwoDimensionalAggregation](#) from your computed results:

Send feedback

```
1 @Function()  
2 public async getDerivedColumn_parallel_asTwoDimensional(objects: ObjectSe  
3     const resultMap: FunctionsMap<objectTypeA, Double> = await this.getDe  
4  
5     // Create a TwoDimensionalAggregation from the resultMap  
6     const aggregation: TwoDimensionalAggregation<string> = {  
7         // Map the entries (object -> Double) of resultMap to (string ->  
8         buckets: Array.from(resultMap.entries()).map(([key, value]) => ({  
9             key: key.pkProperty, // Use the primary key property  
10            value  
11        })),  
12    };  
13  
14    return aggregation;  
15 }
```

PREVIOUS

[← Function metrics](#)

NEXT

[Version range dependencies →](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)[Privacy Statement ↗](#)[Terms of Use ↗](#)[—Cookies Settings](#)[API Reference ↗](#)[Send feedback](#)

Ontology building / Functions / Function consumption / Version range dependencies

Version range dependencies for functions

In addition to depending on a pinned version of a Function, some applications like Workshop and Actions allow you to depend on a Function at a version range. Doing so enables automatic upgrades at runtime, which can save you time in your development cycle and provide a downtime-less upgrade experience for [deployed functions](#).

While version range dependencies are a powerful feature, they also carry certain risks (for example, there are [permissioning consequences specific to Actions](#)). This documentation explains the mechanics behind version range resolution so that you can better understand these risks and make an informed decision on whether version range dependencies are suitable for your application.

i This documentation page assumes prior knowledge on topics like backward compatibility and the Semantic Versioning system. If you are not familiar with these topics, review our documentation on [functions versioning](#).

You should also be familiar with the rules around version precedence as defined in the [Semantic Versioning specification 7](#). In other words, you should be able to determine, given two distinct versions, which one has lower precedence. For example, `1.0.0-rc.1 < 1.0.0 < 1.0.1 < 1.1.0 < 2.0.0`.

Version ranges

API Reference ↗

In its simplest form, a version range is a collection of version inequalities. A version is said to "satisfy" a range if it satisfies all of its inequalities. For example, version `1.2.0`

Send feedback

satisfies the range `>=1.0.0 <2.0.0`.

- i** Internally, the semantics of Function version ranges are adopted from NPM, a popular package manager for the JavaScript ecosystem. Review the [NPM documentation on version ranges ↗](#) for a rigorous definition.

Applications like Workshop and Actions currently only allow version ranges that comprise backward compatible versions (that is, minor or patch upgrades).

- i** The NPM equivalent of this backward compatible range used by Workshop and Actions is the [caret range ↗](#).

Version range resolution

With the exception of deployed functions, when you depend on a Function at a version range, a concrete version that satisfies the range will be chosen at runtime during execution. In particular, the *maximum* satisfying version will be chosen on an eventual basis (it can take a few minutes to pick up new releases).

Deployed functions

For deployed functions, a concrete version is instead resolved to the currently deployed version, if it satisfies the range. If the deployed version does not satisfy the range, an error will be returned.

Risks

While functions developers are guided towards the Semantic Versioning specification and [API Reference ↗](#), it is always possible for breaks to be accidentally introduced in non-

Send feedback

If your application picks up a breaking change, it can manifest in any number of problems, like runtime failures or unexpected behavior.

Upon noticing a breaking change, you should immediately contact the developer of the Function so that they can [release a fix](#), and in the meantime, you should pin your Function dependency to the last working version.

 With the caveat of deployed Function dependencies, if your application has strict uptime requirements and cannot tolerate any breaks, you should use pinned version dependencies.

Permissions and provenance in Actions

When using Function version ranges in [Function-backed Actions](#), there are important considerations around permissions and provenance that can affect Action behavior. For more information about these implications, refer to the Actions documentation on [auto upgrades](#).

PREVIOUS
← [Optimize performance](#)

NEXT
[Language agnostic features / Ontology edits](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

[API Reference](#) ↗

[Send feedback](#)

Ontology edits

An **Ontology edit** is the act of creating, modifying, or deleting an object. Functions support returning [Ontology edits](#) for use in a [function-backed action](#).

- TypeScript v1 functions are authored using the `@OntologyEditFunction` decorator, which provides special semantics to simplify your code. TypeScript v1 functions also use the `@Edits` decorator to provide actions with provenance information, which the actions may use to [enforce permissions](#). You can write unit tests for TypeScript v1 Ontology edit functions using the APIs available for [verifying Ontology edits](#).
- TypeScript v2 functions are authored using the `createEditBatch` function exported from the `@osdk/functions` package. These functions rely on the `Edits` type to provide actions with provenance information.
- Python functions are authored by creating an edits container using the `FoundryClient` exported from the Ontology SDK. These functions rely on the `edits` parameter of the `@function` decorator to provide actions with provenance information.

The rest of this document describes how Ontology edit functions work behind the scenes to provide you with a better understanding of the underlying infrastructure.

When edits are applied

A common misunderstanding about Ontology edit functions is whether or not running them will update objects in the Ontology. When you run an Ontology edit function in the functions helper in **Authoring**, edits are not applied to the actual objects. The only way to update objects using a function is by configuring an action to use the function as described in the [API Reference](#) [function-backed actions](#).

That you can freely run Ontology edit functions in the functions helper to validate results on various inputs, without concern that the objects themselves will be

Send feedback

updated.

Output
Ran in 673ms. Loaded 182 objects during execution.

RESULT

```
[
  {
    "Ontology edit (": {
      "Modified object": {
        "Primary Key": {id: 10},
        "Object Type": aircraft-object,
        "Properties": {wifi: false}
      }
    },
    "Ontology edit (": {
      "Modified object": {
        "Primary Key": {id: 11},
        "Object Type": aircraft-object,
        "Properties": {wifi: false}
      }
    }
  },
  ...
]
```

Caveats

Edits and object search

Changes to objects and links are propagated to the object set APIs *after* your function has finished executing. This means that any `Objects.search()` query whose results depend on an edited value, such as a filter, search around, or aggregation on the property or link you edited, will use the old objects, properties, and links, and may not reflect the edits to the Ontology, including creation and deletion. Your function will need to handle this case manually.

This is distinct from retrieving a specific object that you have already edited, for example by its primary key. In that case, the functions infrastructure applies your pending edits when the object is materialized, so the edited property and link values are returned. See [The @Edits decorator](#) for an example.

For the following example, assume there is an Employee with ID 1.

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { OntologyEditFunction, Edits } from "@foundry/functions-api";
2 import { Employee, Objects } from "@foundry/ontology-api";
3
4 $CaveatEditFunctions {
5   @Edits(Employee)
6   @OntologyEditFunction()
7   public async editAndSearch(): Promise<void> {
8     const employeeOne = Objects.search().employee().filter(e => e.id,
```

API Reference ↗

Send feedback


```
9      employeeOne.name = "Bob";
10
11      const count = await Objects.search().employee().filter(e => e.name === "Bob");
12      console.log(count);
13      // Expected: 1, Actual: 0
14  }
15 }
```

Optional arrays in function-backed actions

While omitted optional arrays are handled as `undefined` when running an `@ontologyEditFunction` in code repositories, they are passed as empty arrays when executing the function through an action.

PREVIOUS

← [Function consumption / Version range dependencies](#)

NEXT

[Publish and call query functions through API gateway](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Language agnostic features / Publish and call query functions through API gateway

Queries

Queries are the read-only subsets of functions that may be optionally exposed through the [API gateway](#). They cannot have any side effects, such as modifying the Ontology or altering external systems. You should use an [Action](#) if you need those additional editing capabilities through the API gateway.

Query decorator

Use the following syntax to define a query function.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Query } from "@foundry/functions-api";  
2  
3 @Query({ apiName: "myTypeScriptV1Function" })
```

For Python and TypeScript v1 functions, the decorator accepts an API name parameter of type `string`, which is required to define an API name. When using TypeScript v1, the query will behave similarly to the existing [@Function decorator](#) if the `apiName` parameter is not defined. Note that the corresponding Python syntax is `api_name`.

Example: API-named query

[API Reference ↗](#)

The example below demonstrates how to expose a query through the API gateway.

[Send feedback](#)

```
1 import { Query, Double } from "@foundry/functions-api";
2 import { Objects, Aircraft } from "@foundry/ontology-api";
3
4 export class PublishedQueries {
5     @Query({ apiName: "getReschedulableAircraftCount" })
6     public async countAircraftTakingOffAfter(minimumTimeInMinutes: Double
7         const aircraftCount = await Objects.search().aircraft()
8             .filter(aircraft => aircraft.timeUntilNextFlight.range()
9             .count();
10
11     return aircraftCount!;
12 }
13 }
```

API name validations

The `apiName` of a query must be a string that meets the following requirements:

- Be in `lowerCamelCase`.
- Be under 100 characters.
- Not contain leading numbers.
- Be unique among all Ontologies imported into the repository.
 - The [tagging process](#) will fail if the `apiName` is not unique, requiring you to change the name.

Additionally, a repository containing API-named queries must import entities from at least one ontology.

Version and update API-named queries

API-named queries will always use the **latest tagged version** of the published query and do not follow the semantic versioning paradigm as other Foundry functions.

To disassociate the API name from the query and break it in the API g
remove the API name from the query decorator and release a new tag from the repository.

[Send feedback](#)

i Changing the API name in the decorator and publishing a new tag will break the consumer. Only the latest published version of the query is supported. To allow consumers to upgrade at their convenience without breaking changes, you can support multiple versions of the same API name. To do this, you must make a copy of the query code in your repository and give it a different API name, for example `getReschedulableAircraftCountV2`.

Search and view queries

As with other functions, you can search for and manage your queries in [Ontology Manager](#). You can search for the query name or the API name. In the example [above](#), the queries are `getReschedulableAircraftCount` for the API name and `countAircraftTakingOffAfter` for the query name, respectively.

The screenshot displays the Ontology Manager interface. On the left sidebar, there's a search bar with 'countAircraftTakingOffAfter' entered, a version dropdown set to '0.0.2 Latest', and an 'Overview' tab. The main content area shows the query details for 'countAircraftTakingOffAfter'. It includes a 'Name' field, a 'Visibility' toggle set to 'Visible', a 'Type' of 'Default', and a 'Published Date' of 'Yesterday at 2:05 PM'. Below this, there's a 'Code Preview' section with a code editor showing a TypeScript function. The 'Inputs' section shows a single input 'minimumTimeInMinutes' of type 'Double'. The 'Output type' section shows 'Double'. At the bottom, there's a 'Usage History' section with a button 'All' and a message: 'No recorded usage history found, but some applications may still depend on this Function.'

API Reference ↗

Send feedback

- i** When using TypeScript v1 functions, you may need to update the `functions.json` file in your repository to enable queries by setting the `enableQueries` property to `true`:

TYPESCRIPT V1

```
1 {
2   "enableQueries": true
3 }
```

Call a query function

After publishing your TypeScript or Python query function, navigate to the code repository where you want to consume the function, and import it using the [Resource imports sidebar](#).

Your function will be callable from the consuming repository. For example:

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Queries } from "@foundry/ontology-api";
2
3 export class MyFunctions {
4   @Function()
5   public callQueryFunction(): Promise<Double> {
6     return Queries.getReschedulableAircraftCount(10);
7   }
8 }
```

For TypeScript v1 functions, Foundry must know which query functions you call from a published function. We automatically provide static analysis to try and detect queries that are called. However, this static analysis may occasionally miss certain calls leading to a runtime error. [API Reference ↗](#) prompting you to add the `@Uses` decorator. This decorator serves to augment the automatically detected query usage.

Send feedback

The following example demonstrates the usage of the `@Uses` decorator:

TYPESCRIPT V1

```
1 import { Uses } from "@foundry/functions-api";
2 import { Queries } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Uses({ queries: [Queries.getReschedulableAircraftCount] })
6     @Function()
7     public callQueryFunction(): Promise<Double> {
8         return Queries.getReschedulableAircraftCount(10);
9     }
10 }
```

PREVIOUS

← [Ontology edits](#)

NEXT

[Configure notifications](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Language agnostic features / Configure notifications

Configure notifications

Functions can be used to flexibly configure notifications that should be sent in the platform, including notifications that are sent externally to a user's email address.

Configuring a notification in a function makes use of the `Principal` (representing a `User` or `Group`) and `notification` types. These references may be useful while working through this section:

- Reference for [Principal, User, and Group types](#)
- Reference for [notification types](#)

Define a custom notification

Suppose that the ontology includes an `Issue` object that can be assigned to a `User`. You can create a function that defines the notification that should be sent to the given `User` with details about the `Issue`.

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { EmailNotificationContent, Function, Notification, ShortNotification } from "@foundry/ontology-api";
2 import { Issue } from "@foundry/ontology-api";
3
4 export class NotificationFunctions {
5   @Function()
6   createIssueNotification(issue: Issue, user: User): Notification {
7     // Create a short notification that will be shown within the platform
8     const shortNotification = ShortNotification.builder()
9       .heading("New issue")
10      .content("A new issue has been assigned to you.")
```

API Reference ↗



Send feedback

```

11         // Link to the Issue object in the platform
12         .addObjectLink("Issue", issue)
13         .build();
14
15         // Define the email body. The email body may contain headless HTML
16         // Note that you can access properties of both the user and the issue
17         const emailBody = `Hello, ${user.firstName},
18
19 A new issue has been assigned to you: ${issue.description}.`;
20
21         const emailNotificationContent = EmailNotificationContent.builder()
22             .subject("New issue")
23             .body(emailBody)
24             .addObjectLink("Issue", issue)
25             .build();
26
27         return Notification.builder()
28             .shortNotification(shortNotification)
29             .emailNotificationContent(emailNotificationContent)
30             .build();
31     }
32 }

```

Retrieve users and groups

In addition to having a `User` passed into the function, you may retrieve a `User` or `Group` on demand. Suppose that the `Issue` object has an `assignee` field that contains a user ID. In the example below, the function returns a notification that reminds the user about the issue:

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```

1  import { EmailNotificationContent, Function, Notification, ShortNotification } from "@foundry/ontology-api";
2  import { Issue } from "@foundry/ontology-api";
3
4  export class NotificationFunctions {
5      @Function()
6      public async createIssueReminderNotification(issue: Issue): Promise<Notification> {
7          if (!issue.assignee) {
8              throw new UserFacingError("Cannot create notification for issue without assignee");
9          }
10
11          const user = await Users.getUserByIdAsync(issue.assignee);

```

API Reference ↗

Send feedback


```

12
13     const emailBody = `Hello, ${user.firstName},
14
15 This is a reminder to investigate the following issue: ${issue.description}
16
17 // You can also use this structure to build the entire notification
18 return Notification.builder()
19     .shortNotification(SHORT_NOTIFICATION.builder()
20         .heading("Issue reminder")
21         .content("Investigate this issue.")
22         .addObjectLink("Issue", issue)
23         .build())
24     .emailNotificationContent(EMAIL_NOTIFICATION_CONTENT.builder()
25         .subject("New issue")
26         .body(emailBody)
27         .addObjectLink("Issue", issue)
28         .build())
29     .build();
30 }
31 }

```

Return recipients

The `Notification` API documented above allows you to return custom notification content. Another way you can use functions to configure notifications is by returning a list of recipients for the notification. To do so, create a function that returns one or more `Principal` objects, such as `User` or `Group` objects.

In the example below, the function returns both the user who reported the issue and the user who is currently assigned to the issue:

[TYPESCRIPT V1](#)
[TYPESCRIPT V2](#)
[PYTHON](#)

```

1 import { Function, User, Users } from "@foundry/functions-api";
2 import { Issue } from "@foundry/ontology-api";
3
4 ...
5 } NotificationFunctions {

```

API Reference ↗

```

6     * Given an Issue, returns users representing the current assignees of
7     * who originally reported the issue.
8     */

```

Send feedback

```
9      @Function()
10     public async getIssueAssigneeAndReporter(issue: Issue): Promise<User[]> {
11         if (!issue.assignee || !issue.reporter) {
12             throw new UserFacingError("Cannot create notification for iss
13         }
14
15         const user = await Users.getUserByIdAsync(issue.assignee);
16         const issueReporter = await Users.getUserByIdAsync(issue.reporter
17
18         return [user, issueReporter];
19     }
20 }
```

PREVIOUS

← [Publish and call query functions through API gateway](#)

NEXT

[Make API calls from functions](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Language agnostic features / Make API calls from functions

Make API calls from functions

It is possible to make API calls to external sources from TypeScript v1, TypeScript v2 and Python functions, but doing so requires additional configuration. This configuration and external source usage are detailed below.

Source aliases

For TypeScript v2 and Python functions, we recommend referencing sources through [source aliases](#). A source alias is a portable, named reference that you can use as the source identifier in place of a specific source. When your function is distributed through a [Marketplace product](#), the alias can be remapped to a different source per environment, keeping your function code portable. TypeScript v1 functions use generated source symbols and do not support aliases.

Configure access to external APIs

By default, functions are not allowed to call external APIs. To enable calling external systems from your function, you must [configure a source](#) in [Data Connection](#) to allow Foundry to connect with an external system.

For functions to connect to your source's external system securely, your source must be configured to [enable exports](#) and allow the [import of your source into Code Repositories](#). Both of these can be configured by navigating to the source in Data Connection and

○ [API Reference ↗](#) **tion settings** section.

For TypeScript v1 functions, the source's API name (configured on the **configuration** tab under **Connection settings**) is the identifier you reference in code.

[Send feedback](#)

i Make sure to fully configure the certificate chain in your source.

Webhook and function runtime environments are not identical.

Sometimes, a webhook will work correctly while the API call from a function might encounter an `UNABLE_TO_GET_ISSUER_CERT` error.

Refer to our documentation on the [openssl command in the source terminal](#) to verify certificates.

Use an external source in a function

To make API calls from a function, you must first import your source into a functions repository using the [resource imports sidebar](#). For TypeScript v2 and Python functions, we recommend then [creating a source alias](#) and using its alias key as the source identifier. TypeScript v1 functions reference the imported source directly. You must then declare that your function uses the source, as shown in the examples below.

Examples of this are shown below:

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 import { ExternalSystems } from "@foundry/functions-api";
2 import { MySource } from "@foundry/external-systems/sources";
3
4 export class MyExternalFunctions {
5     @ExternalSystems({ sources: [MySource] })
6     @Function()
7     public async myExternalFunction(): Promise<string> {
8         const { url } = MySource.getHttpsConnection();
9         const response = await MySource.fetch(url);
10
11         return response.text();
12     }
13 }
```

API Reference ↗

Send feedback

You can test your function in live preview and use it to make external calls once published.



Third-party clients are not yet supported for serverless execution or live preview without overriding the fetch function or HTTP agent. To ensure your API calls function properly across all environments, you must use the relevant library methods to make requests with the correct configuration. Direct API calls to external sources or internal Foundry URLs are not guaranteed to work in all environments.

Access source attributes and credentials

You can access source attributes provided by each function type's corresponding library.

The example below shows how to obtain the base URL of the source in the example above.

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 const { url } = MySource.getHttpsConnection();
```



You can also access additional secrets or credentials stored on the source by using the following syntax to access secrets:

TYPESCRIPT V1 TYPESCRIPT V2 PYTHON

```
1 const secret = MySource.getSecret("MySecret");
```



Use the pre-configured clients

For sources that provide a REST API, the source object allows you to retrieve a client. This client will be pre-configured with the server and client certificates specified on the source. It

with [API Reference ↗](#) additional proxy configurations which allow egress from the environment functions are executed in. You should always use this client, if possible. If the function can egress to the source from all environments.

Send feedback

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 const fetch = MySource.fetch;
```



Alternatively, you can use your own client or third-party libraries which make external requests, and use the source object to [retrieve attributes and credentials](#).

TypeScript v2 functions provide a pre-configured HTTP agent as an additional integration point for usage with third party libraries which accept a custom HTTP agent.

The following example demonstrates retrieving this agent and using it with [axios](#).

TYPESCRIPT V2

```
1 import { getHttpAgent, getHttpsConnection } from "@palantir/functions-source";
2 import axios from 'axios';
3
4 const agent = await getHttpAgent(source);
5 const { url } = getHttpsConnection(source);
6
7 const response = await axios.get(url, {
8     httpsAgent: agent,
9 });
```

i Currently, it is impossible to access source attributes that are not credentials unless the source provides an HTTPS client. For example, you will not be able to access the `hostname` or other non-secret attributes on a [PostgreSQL source](#).

Use OAuth 2.0 with outbound applications

If [API Reference](#) requires OAuth 2.0 authorization, you can configure an [outbound](#) Panel and use it as the authentication method for a REST API source.

When your function runs, the source exposes the calling user's OAuth

Send feedback

session credentials. Your function can then use the token to call the external API on the user's behalf.

This pattern is supported in Python and TypeScript v2 functions. See [Use the source's pre-configured client](#) below for code examples.

Limitations

- **TypeScript v1:** TypeScript v1 functions cannot retrieve OAuth tokens directly from a source. To authenticate with an OAuth 2.0 API from a TypeScript v1 function, wrap the call in a [webhook](#) on a REST API source configured with the outbound application. Consider [migrating to TypeScript v2](#) for direct token access.
- **Deployed mode:** OAuth token refreshing is not available when the function is running in [deployed mode](#). If the calling user's access token expires during execution, the function cannot refresh it automatically. Run your function in [serverless mode](#) to use OAuth-backed outbound applications.
- **Direct function usage in Workshop:** Functions used directly in a [Workshop](#) module, such as [function-backed variables](#) or functions that populate widget content, cannot trigger the OAuth 2.0 interactive authorization prompt. If a user has not already authorized the outbound application, the function will fail rather than display the prompt. To use an OAuth-backed function from Workshop, wrap it in a [function-backed action](#). Alternatively, ensure the user completes the authorization flow from another interactive interface (such as a function-backed action against the same outbound application) before the function is invoked directly in Workshop.

Use the source's pre-configured client

The simplest approach is to use the HTTP client provided by the source. The `Authorization` header is injected automatically.

```
PYTHON  TYPESCRIPT V2

ns.api import function
ns.sources import get_source

3
4 @function(sources=["myOAuthSourceAlias"])
5 def call_external_api() -> str:
```

API Reference ↗

Send feedback

```

6     source = get_source("myOAuthSourceAlias")
7     url = source.get_https_connection().url
8     client = source.get_https_connection().get_client()
9
10    response = client.get(url + "/api/v1/resource", timeout=10)
11    return response.text

```

Use a native HTTP client with manual token injection

If you need to use your own HTTP client instead of the source-provided one, retrieve the OAuth token from session credentials and set the `Authorization` header manually.

PYTHON

TYPESCRIPT V2

```

1  import requests
2  from functions.api import function
3  from functions.sources import get_source
4  from external_systems.sources import OAuthCredentials, Refreshable, Source
5
6  @function(sources=["myOAuthSourceAlias"])
7  def call_external_api() -> str:
8      source = get_source("myOAuthSourceAlias")
9      url = source.get_https_connection().url
10
11     refreshable_credentials: Refreshable[SourceCredentials] = source.get_
12     session_credentials: SourceCredentials = refreshable_credentials.get(
13
14     if not isinstance(session_credentials, OAuthCredentials):
15         raise ValueError("Expected OAuth credentials")
16
17     access_token: str = session_credentials.access_token
18
19     response = requests.get(
20         url + "/api/v1/resource",
21         headers={"Authorization": f"Bearer {access_token}"},
22         timeout=10,
23     )
24     return response.text

```

API Reference ↗

Send feedback

Use OAuth-backed functions in actions

A common pattern is to call an OAuth-backed external API and feed the result into an [Ontology edit](#). You can then expose that function through a [function-backed action](#). When a user runs the action from Workshop or AIP Studio, their OAuth token is used to make the API call, and the resulting object edits are attributed to them.

For example, the function below uses an OAuth token to fetch the calling user's profile from a third-party identity service. It then creates a new ontology object with that information:

PYTHON TYPESCRIPT V2

```
1  from functions.api import function, OntologyEdit
2  from functions.sources import get_source
3  from ontology_sdk import FoundryClient
4  from ontology_sdk.ontology.objects import UserProfile
5
6
7  @function(sources=["myOAuthSourceAlias"], edits=[UserProfile])
8  def link_user_profile() -> list[OntologyEdit]:
9      source = get_source("myOAuthSourceAlias")
10     url = source.get_https_connection().url
11     client = source.get_https_connection().get_client()
12
13     response = client.get(url + "/v1/me", timeout=10)
14     response.raise_for_status()
15     profile = response.json()
16
17     ontology_edits = FoundryClient().ontology.edits()
18     ontology_edits.objects.UserProfile.create(
19         profile["id"],
20         display_name=profile["display_name"],
21     )
22     return ontology_edits.get_edits()
```

Troubleshoot common errors

[API Reference ↗](#)

For OAuth authorization errors, such as HTTP 401: Unauthorized, C and no refresh handler provided, or Resolved source credential

[Send feedback](#)

the `Source`, see [OAuth and outbound applications](#) in the Data Connection troubleshooting reference.

HTTP 407: Proxy authentication required

Function network requests must be covered by your source's [egress policies](#). If the destination hostname does not match an allowed policy, the request may return `HTTP 407: Proxy Authentication Required`.

If your egress policies look correct, check how the request URL is built. The URL from `getHttpsConnection()` has no trailing slash, so an appended path that omits the leading `/` is fused to the hostname:

```
1 "https://example.com" + "api/v1"  
2 → "https://example.comapi/v1"
```

The resulting hostname (`example.comapi`) is not covered by any egress policy, so the request is rejected. Prefix the path with `/` (for example, `url + "/api/v1"`).

PREVIOUS

← [Configure notifications](#)

NEXT

[Use platform APIs with the Foundry platform SDK](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Functions / Language agnostic features / Use platform APIs with the Foundry platform SDK

Use platform APIs with the Foundry platform SDK

Foundry APIs expose a variety of functionality that you can leverage with the [Foundry platform SDK](#) library through functions. You can use the platform SDK to build functions for administrative or governance workflows, interact with schedules and builds, access media sets, and more.

⚠ First-class authentication is not supported for TypeScript v1 functions. We recommend using Python functions and TypeScript v2 functions for these workflows.

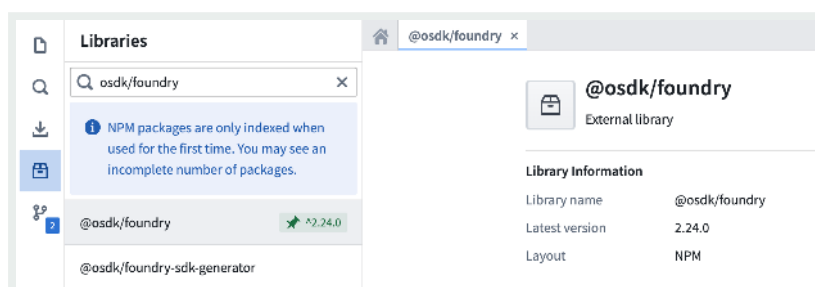
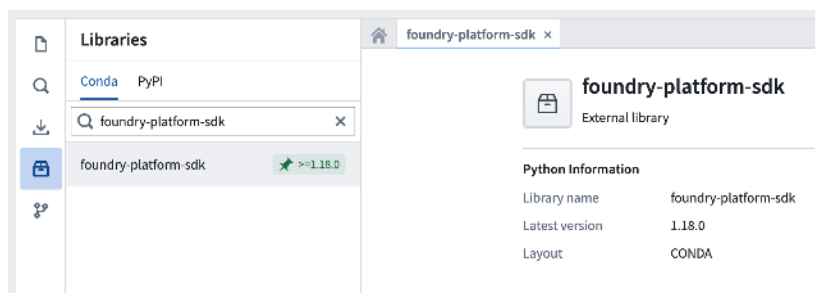
Install the SDK

To install the Foundry platform SDK, navigate to the **Libraries** side panel in your code repository and search for the SDK name: `foundry-platform-sdk` for Python, or `@osdk/foundry` for TypeScript.

API Reference ↗

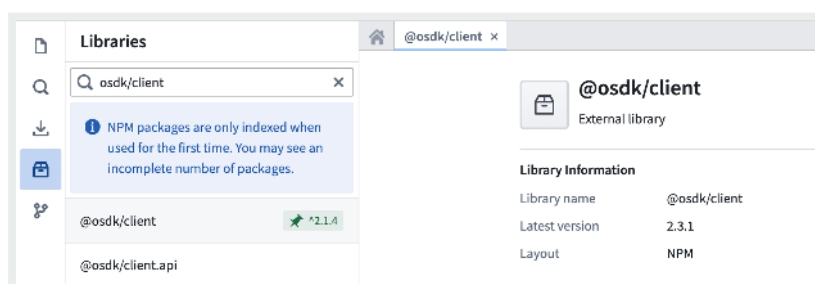


Send feedback



Initialize your client

Your function requires authentication to interact with Foundry APIs. This process involves instantiating an authenticated “client” through which you can make requests to the Foundry APIs through the SDK. In TypeScript v2 repositories, this requires the `@osdk/client` library, which should be pre-installed. You can verify this by looking for the green pin:



Use platform APIs

[API Reference ↗](#)

Once your function is authenticated, you can start using Foundry APIs. The examples below show how to call a language model or query media sets in both Python and JavaScript.

[Send feedback](#)

```

1  import {
2      Client, // Use Client if your function interacts with the Ontology
3      PlatformClient // Use PlatformClient if your function does not interact with the Ontology
4  } from "@osdk/client";
5  import { Functions } from "@osdk/foundry";
6
7  export default async function useLlm(
8      client: PlatformClient, // This parameter gets populated by Foundry at runtime
9      prompt: string
10 ): Promise<string> {
11
12     const promptMessage = [
13         {
14             role: "USER",
15             content: prompt
16         }
17     ];
18     const result = await Functions.Queries.execute(
19         client,
20         "com.foundry.languagemodelservice.models.gpt41.CreateChatCompletion",
21         {
22             parameters: {
23                 messages: promptMessage
24             }
25         },
26         {
27             preview: true, // Required only for unstable endpoints, see API Reference
28         }
29     );
30     return result.value["completion"] as string;
31 }

```

Client permissions

The TypeScript v2 client (passed to the function by Foundry at runtime) and the Python client initialized in code have the following permissions scope:

API Reference ↗

- api:admin-read
- api:functions-read
- api:ontologies-read

Send feedback

- `api:orchestration-read`
- `api:usage:mediaset-read`
- `api:usage:ontologies-write`

Each platform API endpoint requires certain scopes to hit the endpoint. Documentation on these scopes can be found in the [API reference](#)

PREVIOUS

← [Make API calls from functions](#)

NEXT

[Deploy functions](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Deploy functions

Prerequisites

This guide requires that you have already authored and published a Python or TypeScript v2 function. Review the [getting started with Python functions](#) or [getting started with TypeScript v2 functions](#) documentation for a tutorial.

Choose between deployed and serverless execution modes

If serverless functions are enabled for your enrollment, new repositories will use it by default. We generally recommend serverless functions for most use cases. While a deployed function may be useful in some circumstances, the serverless execution mode requires less maintenance and avoids incurring the costs associated with long-lived deployments.

Deployed functions have some capabilities that are not available to serverless functions:

- The long-lived nature of deployed functions means that local caching may be possible if the function is tolerant to restarts.
- Serverless functions support [external sources](#) using the client from the provided source object, but they do not support third-party clients. You must deploy your function to make external API calls with third-party clients.
- Deployed functions support GPU allocations to accelerate computationally intensive API Reference ▸ and inference workflows through parallel processing, while serverless functions do not.

Send feedback

Deployed functions have some limitations that do not apply to serverless execution:

- Serverless functions enable different versions of a single function to be executed on demand, making upgrades safer. With deployed functions, you can only run a single function version at a time.
- Serverless functions only incur costs when executed, while deployed functions incur costs as long as the deployment is running.
- Serverless functions require less upfront setup and long-term maintenance, as the infrastructure is managed automatically.

To enable serverless functions for your enrollment, contact your Palantir administrator.

Architecture

Functions can be run in a serverless mode, leveraging on-demand resources, or they can be deployed to a long-lived container.

 We recommend using serverless functions if enabled on your enrollment, rather than deployed functions. While there are some cases where deployed functions are useful, the serverless executor is generally more flexible.

When your function is deployed, a long-running environment will be created to handle incoming execution requests. The environment will be scaled according to the request volume and occasionally restarted by automated processes. All functions from a single repository are hosted by a single deployment.

Compute costs

Deployed functions will incur compute costs for the running deployment.
Serverless functions will only incur costs when executed.

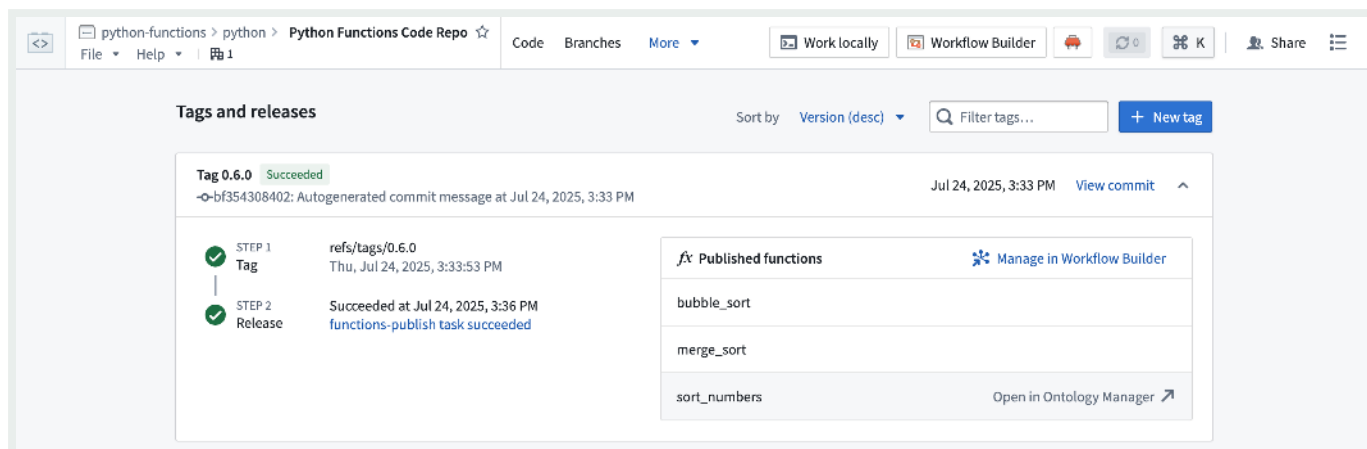
Deploy a function

API Reference ↗

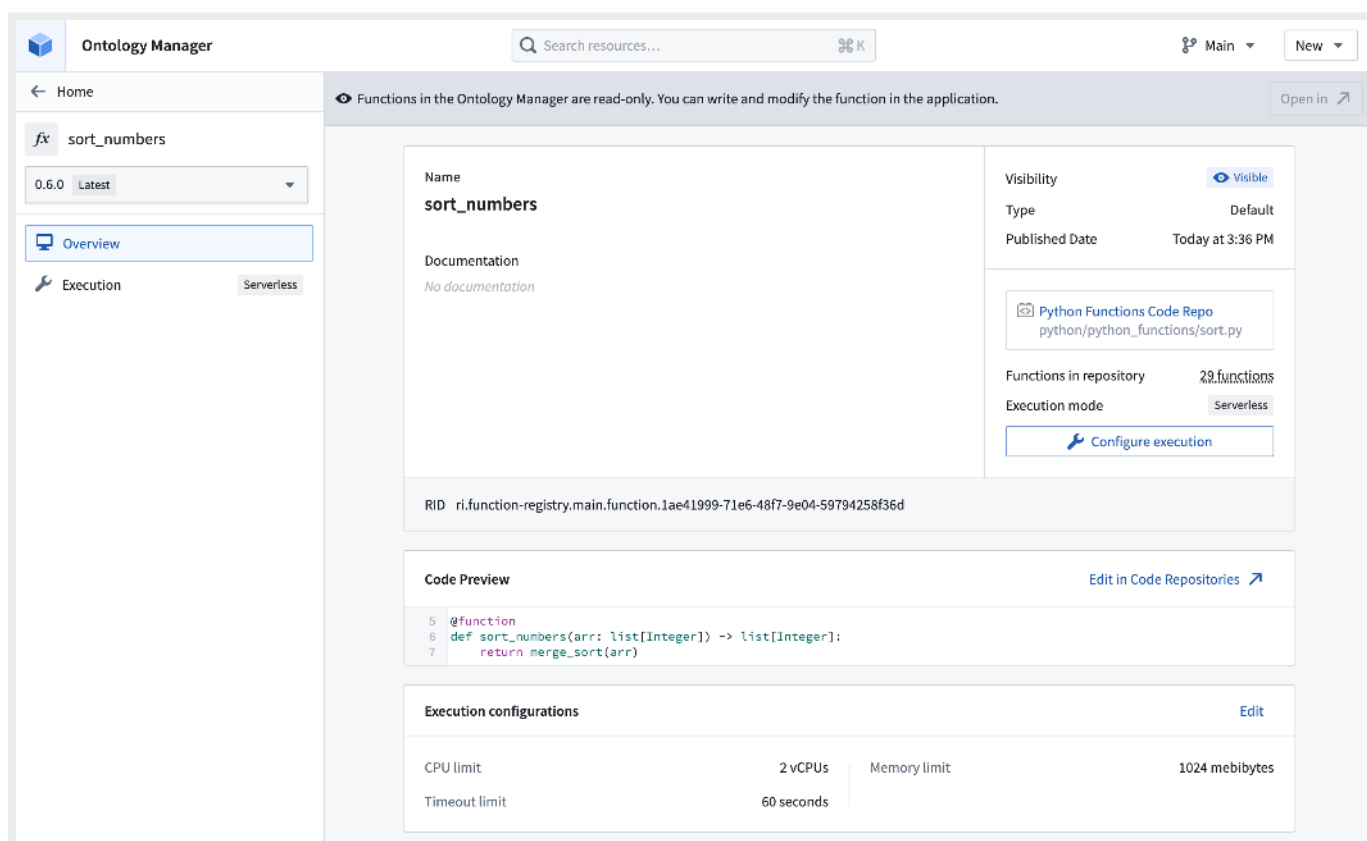
Follow the steps below to configure and deploy a function:

Send feedback

1. Open your function repository and navigate to the **Branches** tab, then select **Tags and releases**.
2. Hover over the function you want to deploy, then select **Open in Ontology Manager**.

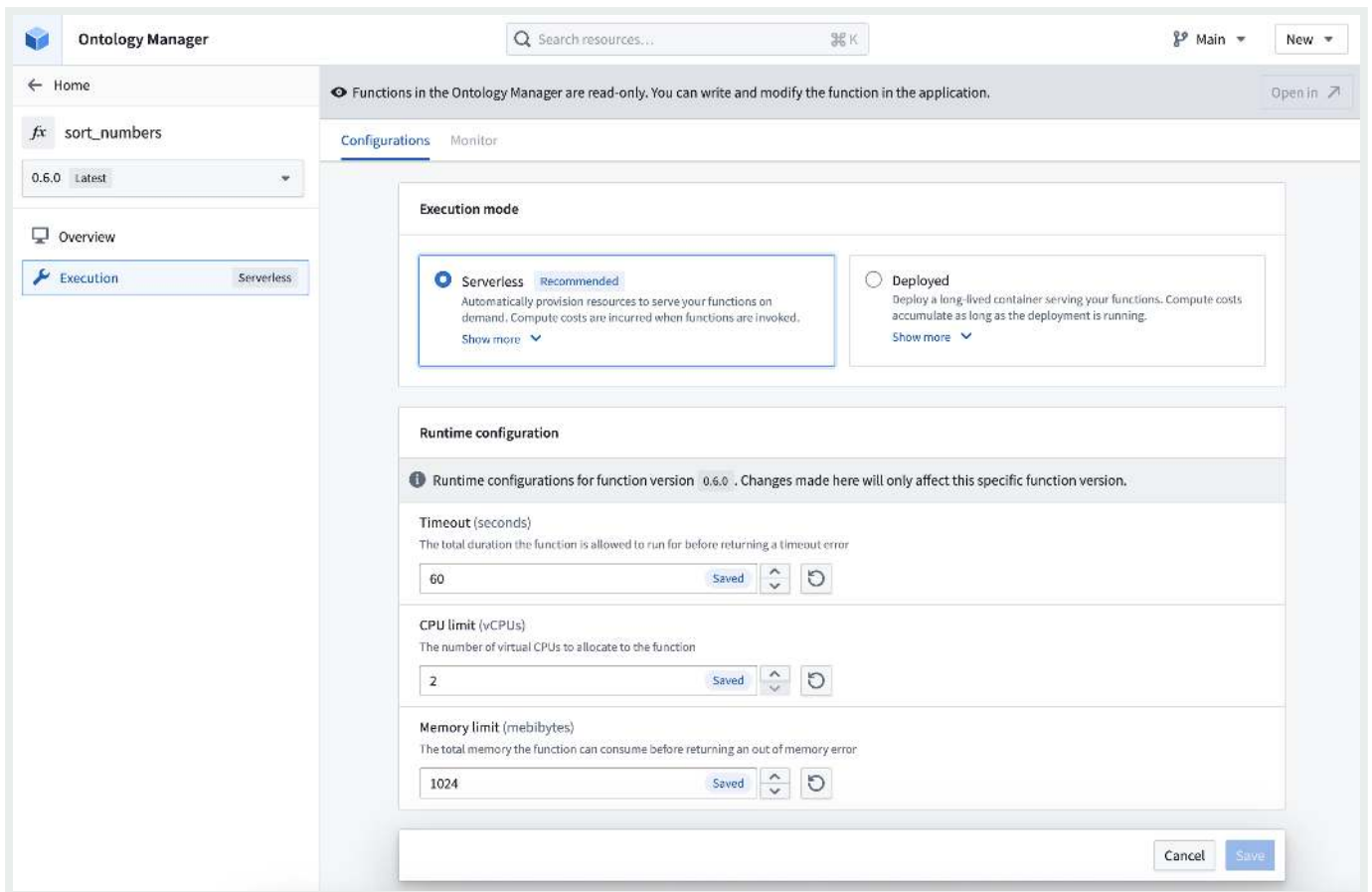


3. Select the version of the function you want to use from the version selector on the left.
4. Select **Configure execution**.



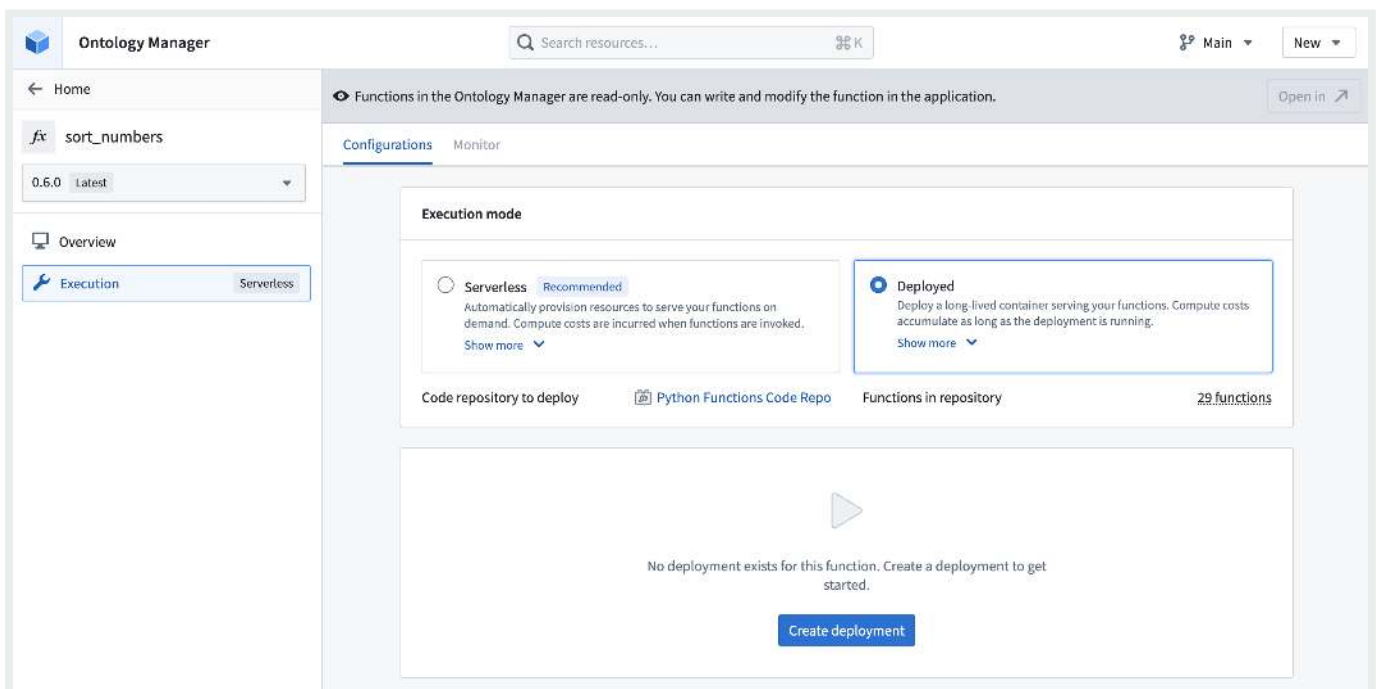
5. If serverless functions are enabled in your environment, you will see an option to switch **API Reference** and deployed. If unselected and no deployment exists, serverless will be used by default.

[Send feedback](#)



6. Select the **Deployed** execution mode option.

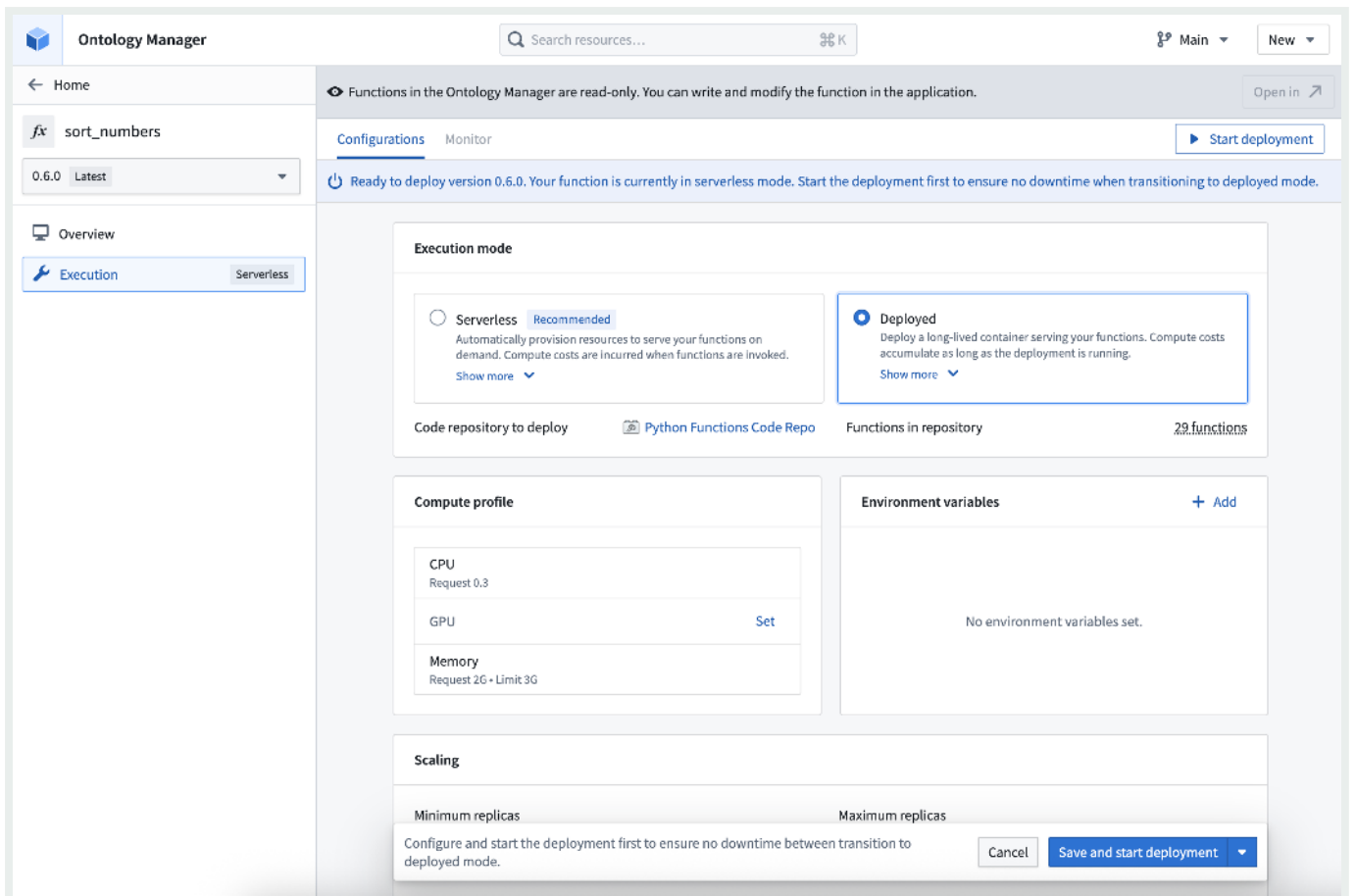
7. If no deployment exists for the function, select **Create deployment**.



API Reference ↗

applied for the configuration when the deployment is first created. You can view the entire configuration by scrolling down to the end of the

Send feedback

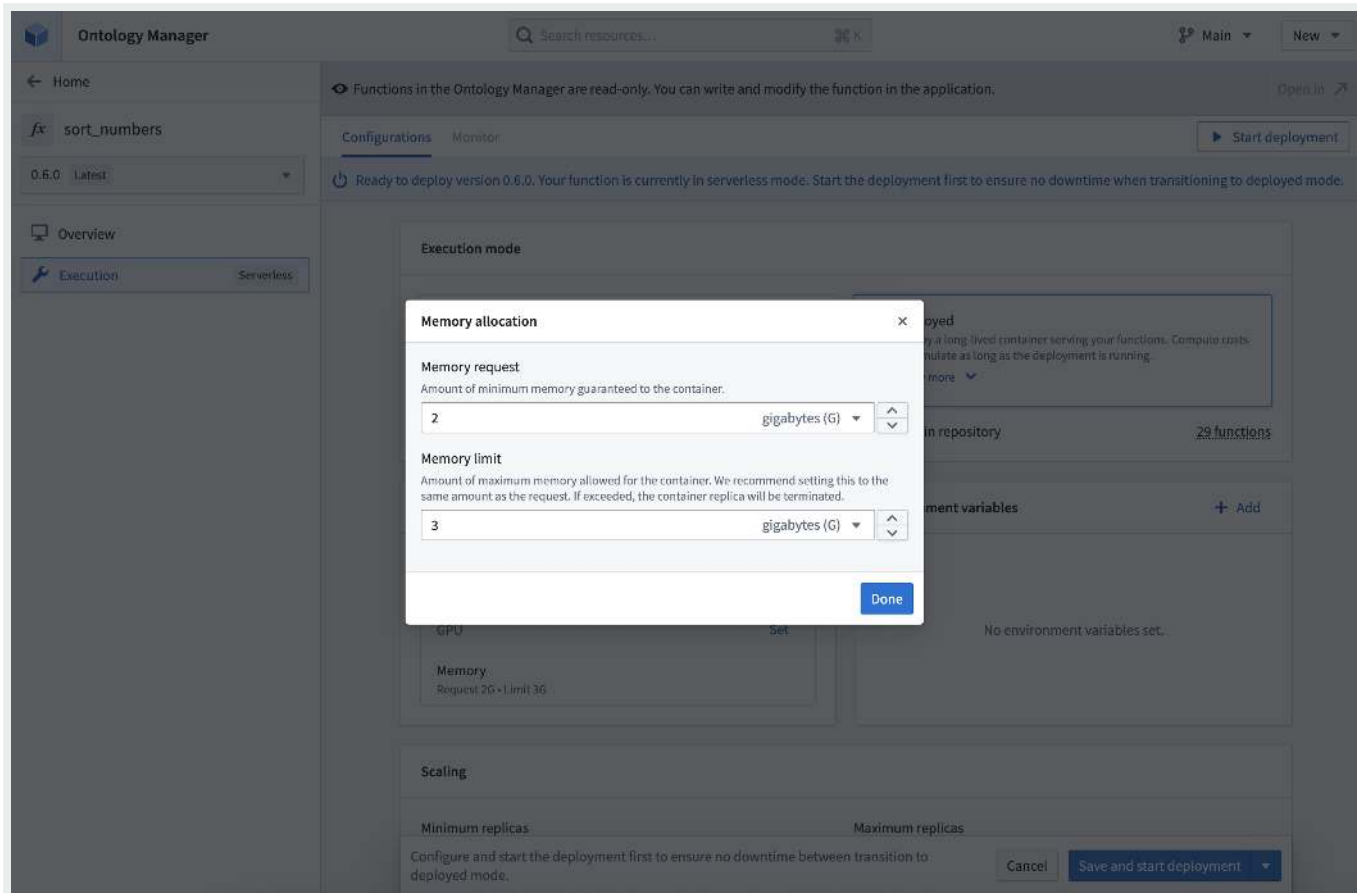


9. Modify the deployment configuration as needed. You can configure the following:

- The compute resources allocated to the deployment, including CPU, GPU, and memory.
- The minimum limit and the maximum limit for autoscaling based on request load.
- The environment variables that will be set for the deployment upon startup.
- The total duration the function is allowed to run before returning a timeout error. Unlike the other deployment settings, timeout is configured individually for each function version.

[API Reference ↗](#)

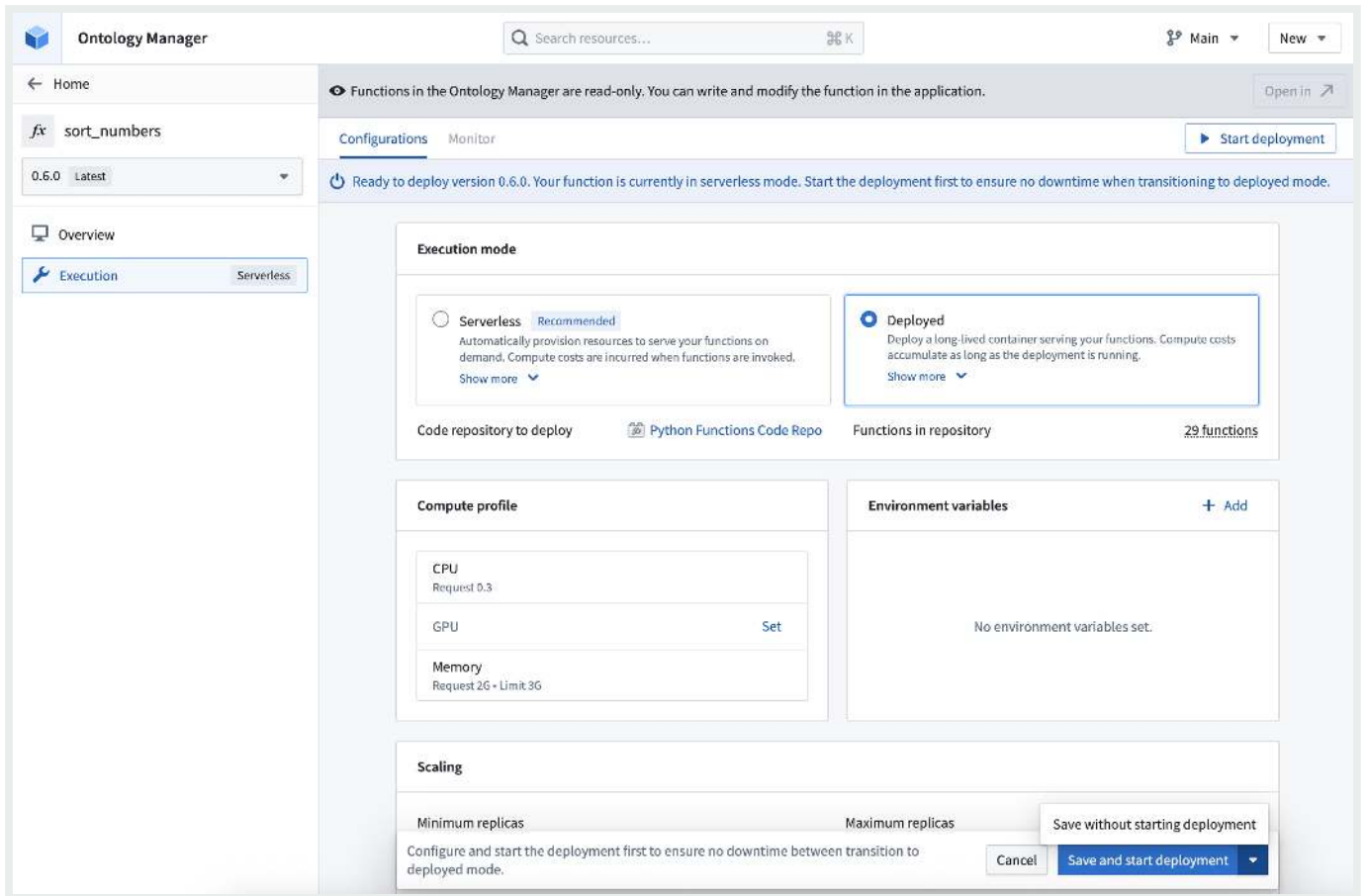
[Send feedback](#)



10. Select **Save and start deployment** to save any changes and launch the deployment. You may also select **Save without starting deployment** to save the configuration without launching the deployment.

[API Reference ↗](#)

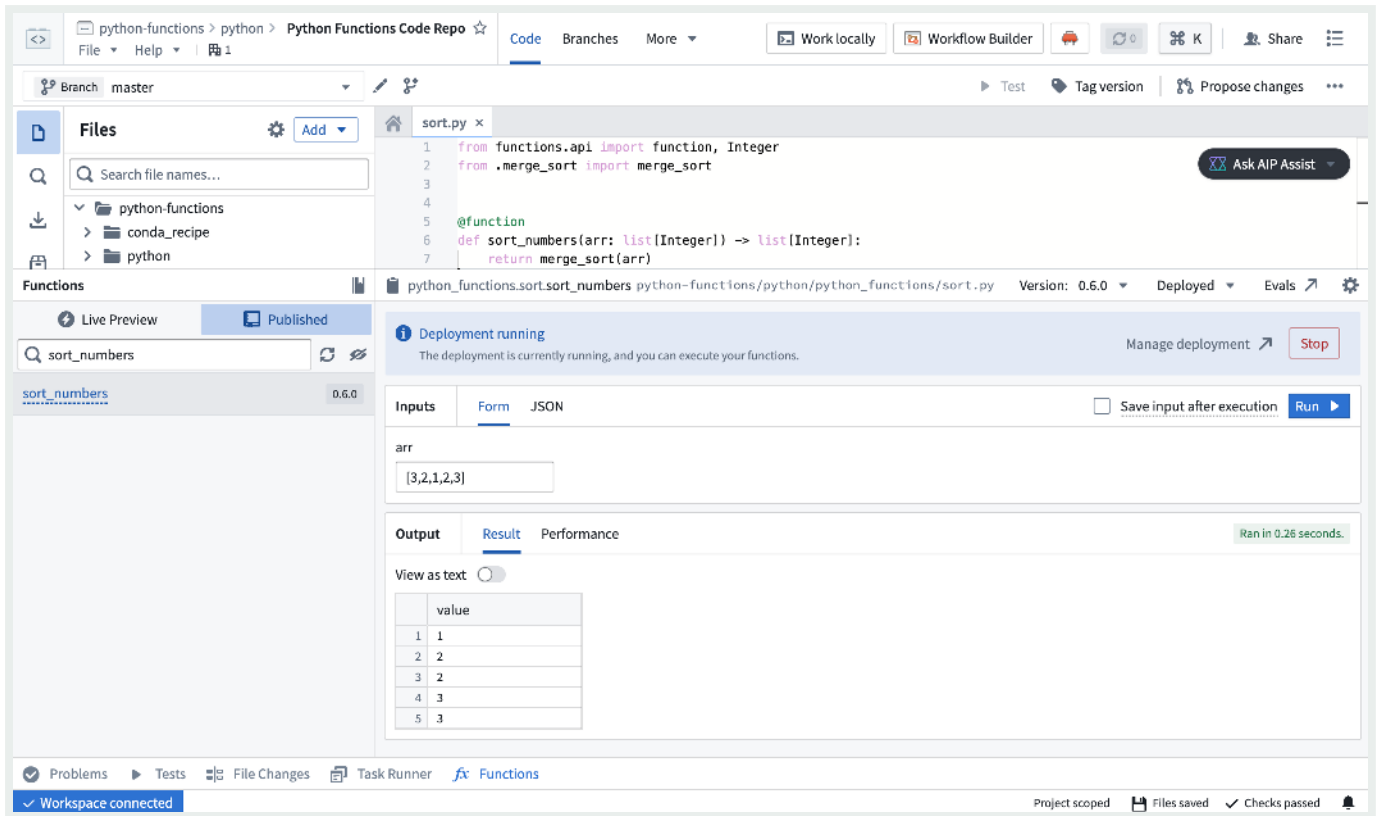
[Send feedback](#)



11. If you chose to **Save and start deployment** option is selected, you will need to wait for the deployment that is hosting the function to start up. This may take a few minutes.
12. To verify that the deployment is running, navigate to the code repository containing the function and run the function. The function should execute successfully and return the expected result.

[API Reference ↗](#)

[Send feedback](#)



PREVIOUS

← [Use platform APIs with the Foundry platform SDK](#)

NEXT

[User-facing errors](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#)

[Privacy Statement](#)

[Terms of Use](#)

—[Cookies Settings](#)

[API Reference](#)

[Send feedback](#)

User-facing errors

When running functions in other parts of the platform, such as Workshop or actions, you may want to throw an error with a detailed message. To do so, throw a `UserFacingError`. For example:

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 import { Function, UserFacingError } from "@foundry/functions-api";
2 import { Employee } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @Function()
6     public async searchExactlyFiveEmployees(employees: Employee[]): Promise<void> {
7         if (employees.length !== 5) {
8             throw new UserFacingError(`Pass in exactly 5 employees. Received ${employees.length}`);
9         }
10
11         // search employees
12     }
13 }
```

When running this as a [Function-backed Action](#) in a [Workshop application](#) with an incorrect number of employees, users will see the following error:

[API Reference ↗](#)[Send feedback](#)



By adding a detailed user facing error message, you can help other users of your Function quickly identify and fix the issue.

PREVIOUS
← Deploy functions

NEXT
Streaming functions →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Streaming functions

Python and TypeScript v2 functions can stream the result of an execution in chunks. This is useful when data becomes available over time and lets consuming applications begin processing the result before it is complete.

Write a function with a streamed response

To stream a response, Python functions must be annotated with an `Iterable[T]` return type and use the `yield` keyword to produce values as they become available. In TypeScript, you must declare an async generator function by adding a `*` character after the `function` keyword, specify a return type of `AsyncIterable<T>`, and use the `yield` keyword to produce values as they become available.

The following example demonstrates how a function can return a stream of integers where one second elapses between each integer:

PYTHON TYPESCRIPT V2

```
1 from functions.api import function
2 from typing import Iterable
3 import time
4
5 @function
6 def my_lazy_number_generator(n: int) -> Iterable[int]:
7     for i in range(n):
8         time.sleep(1)
9         yield i
```

API Reference ↗



Send feedback

Functions with streamed responses are useful when working with language models that may take time to generate the entire output. By yielding each chunk of the response as it is produced by the model, you can provide a real-time experience that does not block on the full completion of the response. Refer to [Language models in TypeScript v2 and Python functions](#) for more information on calling language models in functions.

The following example uses the `openai` SDK to call a language model and stream the response back by enabling the `stream` flag in the request:

PYTHON

TYPESCRIPT V2

```
1 from openai import OpenAI
2 from functions.api import function
3 from functions.aliases import model
4 from foundry_sdk.v2.language_models import (
5     get_openai_base_url,
6     get_foundry_token,
7     get_http_client,
8 )
9 from typing import Iterable
10
11 @function
12 def create_chat_completion(prompt: str) -> Iterable[str]:
13     client = OpenAI(
14         api_key=get_foundry_token(preview=True),
15         base_url=get_openai_base_url(preview=True),
16         http_client=get_http_client(preview=True),
17     )
18
19     stream = client.chat.completions.create(
20         model=model("gpt55").rid,
21         messages=[
22             {
23                 "role": "user",
24                 "content": prompt,
25             },
26         ],
27         stream=True
28     )
```

API Reference ↗

```
31         if event.choices:
32             content = event.choices[0].delta.content
```

Send feedback

```
33         if content:
34             yield content
```

Call streaming functions through the Ontology SDK

Once you have tagged and published your streaming function, you can execute it through the [Ontology SDK](#) in a React application, a custom widget in Workshop, or another function.

Beta

Executing functions with streamed responses through the Ontology SDK is in the [beta](#) phase of development. Functionality may change during active development.

PYTHON TYPESCRIPT

```
1 from foundry_sdk_runtime import AllowBetaFeatures
2
3 with AllowBetaFeatures():
4     with client.ontology.queries.create_chat_completion_streaming(prompt="
5         for text in stream:
6             # ...
```

PREVIOUS

← [User-facing errors](#)

NEXT

[Functions settings](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[API Reference](#) ↗

[Send feedback](#)

Functions settings

The **Functions settings** page in [Control Panel](#) allows administrators to manage a collection of settings that govern how functions behave within a specific [space](#), with each setting controlling whether a particular capability is enabled as detailed in the sections below. To access the page, navigate to **Control Panel > Functions settings** for the space you want to configure.

Extended function execution

The **Extended function execution** modal allows specific functions to execute with certain extended capabilities:

- Call [actions](#) from within the function.
- Obtain an authentication token with an extended time-to-live (TTL) of up to four hours.

Extended function execution

Allow specific functions to execute with certain extended capabilities - call Actions from within the Function, and get an auth token that has an extended TTL of up to 4 hours. Impose granular control over the enablement of these capabilities by configuring the below allow-lists.

Repositories allowed to publish extended functions ▾

Functions allowed to execute with extended capabilities ▾

Projects allowed for installing extended functions via Marketplace ▾

API Reference ↗

By enabling these capabilities grant functions elevated access, the ability to execute with extended capabilities. Extended function execution provides granular control over where they are enabled through a set of allowlists

[Send feedback](#)

configured in the modal's collapsible sections. A function must satisfy the relevant allowlists before it can publish, execute, or install with those extended capabilities. Configure the following allowlists to control which repositories, functions, and projects can use extended function execution.

Repositories allowed to publish extended functions

The repositories in this allowlist are the only repositories in the space that can publish functions with extended execution capabilities. To add a repository, select **Add** and search for the repository you want to allow.

Functions allowed to execute with extended capabilities

The functions in this allowlist are the only functions that execute with extended capabilities. This list is checked at execution time, so a function must remain in the allowlist to continue executing with extended capabilities. New functions published from an allowlisted repository will automatically propagate to the functions allowlist. To add a function, select **Add** and search for the function you want to allow.

Projects allowed for installing extended functions via Marketplace

The projects in this allowlist are the only projects where [Marketplace](#) installation succeeds for functions that have extended execution capabilities. To add a project, select **Add** and search for the project you want to allow.

PREVIOUS

← [Streaming functions](#)

NEXT

[TypeScript v2 / Getting started](#) →

[API Reference ↗](#)

Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Send feedback](#)

[Terms of Use ↗](#)

[— Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / TypeScript v2 / Getting started

Getting started with TypeScript v2 functions

TypeScript v2 allows users to take advantage of several key [improvements over TypeScript v1](#), including a Node.js runtime and first-class OSDK support. Review the sections below to get started.

Create a TypeScript v2 functions repository

Navigate to a project of your choice and create a new code repository by selecting **+ New > Repository**. Select the TypeScript v2 functions template to initialize your repository.

The screenshot shows the 'Choose language' section of the Foundry interface. At the top, there are tabs for 'Language', 'Compute modules', and 'Libraries'. Below the tabs, the heading 'Choose language' is followed by a subtext: 'Function repositories allow you to create reusable logic that can be shared and utilized across Foundry. [Learn more about differences between the templates.](#)'

There are three main template cards visible:

- TS TypeScript Functions**: Write logic to analyze or edit the Ontology for Foundry frontend applications (such as Workshop) and external applications.
- Python Functions**: Write reusable python logic to use as UDFs in Pipeline Builder or to invoke as functions in Workshop Modules.
- TS TypeScript Functions v2**: Write logic using the Ontology SDK to analyze or edit the Ontology for Foundry frontend applications (such as Workshop) and external applications. This card is highlighted with a blue border.

Below these cards, there is a section titled 'or no code' with a card for **AIP Logic**: Build functions without code that can parse, modify, and expand your Ontology.

At the bottom left of the interface, there is a button labeled 'API Reference ↗'.

Once the repository has been created, navigate to the `typescript-functions/src/functions/helloWorld.ts` file.

Send feedback

Write a function

To write a new function, create a new file in the `typescript-functions/src/functions` directory of your repository and give it a descriptive name, for example, `helloWorld.ts`. Write your function using `export default` for Foundry to detect it.

TYPESCRIPT V2

```
1 export default function helloWorld(): string {  
2     return "Hello World!";  
3 }
```

You must satisfy the following conditions for your function to be published to Foundry:

1. Define your function in a `.ts` file in the `typescript-functions/src/functions` directory. In this directory, you can also group related functions in subdirectories.
2. The name of your file must match the name of your function. For a function called `myFunction` to be published, it must be defined in a file called `myFunction.ts` within the `typescript-functions/src/functions` directory.
3. The TypeScript function must be the default export of your file.
4. Your function's input and output types must follow the supported function types, as detailed in the [type reference](#).

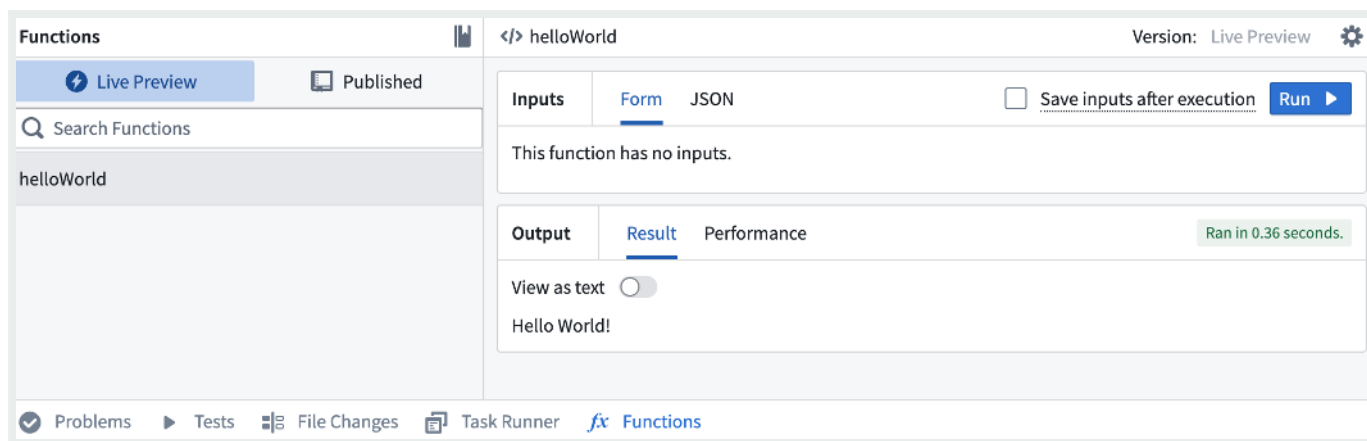
Your function's file path is used to uniquely identify the function that gets published from it. Note that a change in your function's file path will therefore result in a new function being published.

Test in live preview

To test your function in live preview, open the **Functions** helper and select **Live preview**. Choose your function and select **Run** to execute.

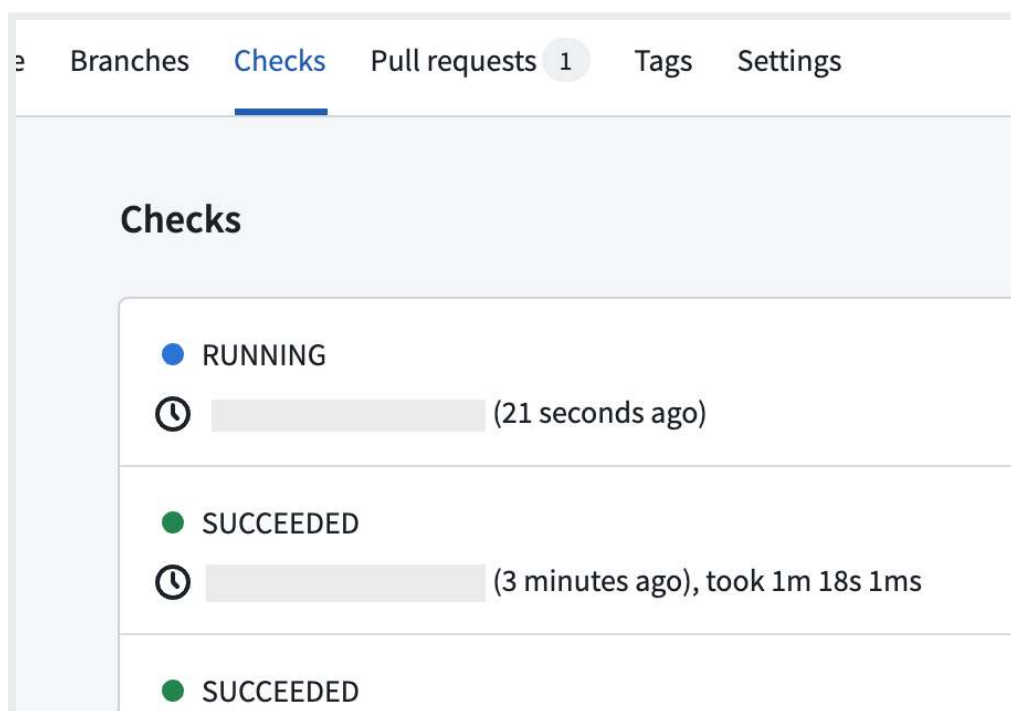
API Reference ↗

Send feedback



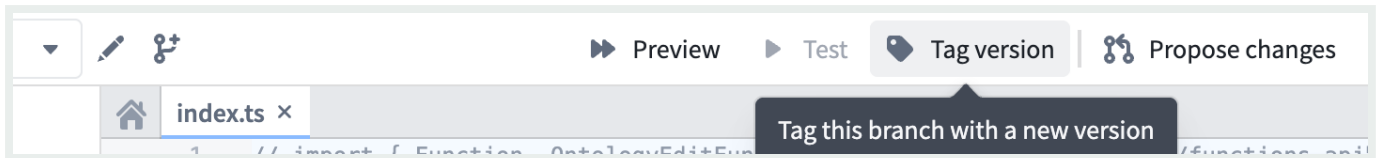
Commit and publish a function

Select **Commit** at the upper right corner of the window to commit your changes to the `master` branch of your repository. To view your function's checks, open the **Checks** tab at the top of the page. Here, after making a commit, you should see a running check.



At [API Reference](#) for work, you will see the **Tag version** option. This will publish all of the functions in your repository.

Send feedback



Select **Tag version** to tag a release off of the `master` branch. Set the tag name based on the extent of your changes, and then select **Tag and release**.

Tag and release

×

You can release functions by tagging a branch or a commit.

Select a branch or enter a commit hash:

Branch `master`

Tag name (latest released tag: 0.0.1) [Use custom tag](#)

☒ Stable

☐ Prerelease ?

☐ Patch `0.0.2` `Development`
Patches are used to signal backwards compatible bug fixes.

☒ Minor `0.1.0` `Development`
Minor versions are used to signal changes that do not require consumers' adjustments.

☐ Major `1.0.0`
Major versions are used to signal breaking changes that require consumers' attention.

Comparing the current commit with latest version 0.0.1.
No breaking changes have been detected.

API Reference ↗
Releasing: 0.1.0

Cancel

Tag and release

Send feedback

To view the progress as your functions are tagged and released, select the **View** pop-up or navigate to the **Tags** tab. Once **Step 2: Release** is completed, select the published functions to view them in the function registry.

 Functions may not be immediately searchable by name in Workshop or the function registry while permissions propagate.

Tags and releases

Sort by Version (desc)

Filter tags...

New tag

Tag 0.1.0 Succeeded

8b4f5723c20: Autogenerated commit message at Jun 11, 2025, 12:16 PM

Jun 11, 2025, 12:16 PM [View commit](#)

STEP 1

Tag

refs/tags/0.1.0

Wed, Jun 11, 2025, 12:16:28 PM

STEP 2

Release

Succeeded at Jun 11, 2025, 12:19 PM

[functions-publish task succeeded](#)

fx

Published functions

helloWorld

findSumOfArray

Manage in Workflow Builder

Use a new function

After the checks for your tag have passed, navigate back to the **Code** tab in **Code Repositories** and select the **Functions** helper. You should now be able to see your functions under the **Published** section. Select it and run the new function:

Functions

Live Preview

Published

Search Functions

findSumOfArray 0.1.0

helloWorld 0.1.0

findSumOfArray typescript-functions/src/functions/findSumOfArray.ts

Version: 0.1.0

Deployed

Evals

Inputs

Form

JSON

numbers

[1,2,3]

Output

Result

Logs

Performance

View as text

6

Save input after execution

Run

Ran in 3.61 seconds.

PREVIOUS

← Language agnostic features / Functions

API Reference ↗

NEXT

Ontology edits →

Send feedback

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology edits

In addition to writing functions that read data from the Ontology, you can also write functions that create objects and edit the properties and links between objects. This page documents the object edit APIs available to you in functions. For more details about how edit functions work, refer to the [overview page](#).

For the edits created in a function to actually be applied, Ontology edit functions *must be configured as a [function-backed Action](#)*. Configuring an Action in this way allows you to provide additional metadata, configure permissions, and access the Action in various operational interfaces. As noted in the [documentation](#), running an edit function outside of an Action will not actually modify any object data.

⚠ Warning

Searching for objects immediately after editing them may return unexpected results. See the [Caveats section](#) for details.

Define an edit function

Functions that edit the Ontology must explicitly declare the entities that will be edited using the `Edits` type exported from the `@osdk/functions` package. The following example declares a new type representing edits to the `Employee` and `Ticket` object types, as well as a link type between `Employee` and `Ticket`. Edits to multiple entities need to be joined with the `|` operator.

API Reference ↗



```
t { Employee, Person, Ticket } from "@ontology/sdk";  
import { Edits } from "@osdk/functions";
```

Send feedback

```
3
4 type OntologyEdit = Edits.Object<Employee> | Edits.Interface<Person> | Edi
```

You must then declare that the function returns an array of edits of the new type.

```
1 export default function createNewTicketAndAssignToEmployee(): OntologyEdit[] {
2     // ...
3 }
```

Construct an Ontology edits batch

To perform Ontology edits in a TypeScript v2 function, first construct an Ontology edits batch using the `createEditBatch` function exported from `@osdk/functions`, passing the previously declared type as a type argument:

```
1 import { Employee, Ticket } from "@ontology/sdk";
2 import { Client } from "@osdk/client";
3 import { createEditBatch, Edits } from "@osdk/functions";
4
5 type OntologyEdit = Edits.Object<Employee> | Edits.Object<Ticket> | Edits
6
7 export default function createNewTicketAndAssignToEmployee(client: Client) {
8
9     const batch = createEditBatch<OntologyEdit>(client);
10    // ...
11 }
```

This batch is used to keep track of all edits made in a function.

Update properties

[API Reference ↗](#)

Object properties

[Send feedback](#)

Use the `update` method available on the created batch to modify one or more object properties:

```
1 batch.update(employee, { lastName: newName });
```

If you have not loaded the `employee` object instance into memory, you can also update it by referencing the object's API name and primary key:

```
1 batch.update({ $apiName: "Employee", $primaryKey: 23 }, { lastName: newName });
```

Subsequent access to the `lastName` property value of `employee` later in the same function execution will *not* reflect the changes that you make when calling `update` on the edit batch.

Sometimes, it is useful to copy all of the property values of one instance of an object type to another instance. The following example assigns the property values of `employee2` to `employee1`:

```
1 batch.update(employee1, employee2);
```

Note that the primary key property value of an existing object cannot be updated.

Interface properties

You can use the `update` method to modify the interface properties of an object through an Ontology interface. In the example below, the type of `person` is an Ontology interface, but the underlying instance is an object that implements the `Person` interface.

The `update` method takes two arguments both for object types and interfaces. For interfaces, it takes the interface that will be modified and the interface properties to be

API Reference ↗

Send feedback

```
1 batch.update(person, { firstName: newFirstName });
```



Note that an interface property that is implemented by the primary key property of the underlying object cannot be updated.

Update links

For many-to-many links, the `link` and `unlink` methods are available on the created batch to add or remove links between objects.

```
1 // Assign an employee to an office.
2 batch.link(employee, "office", office);
3
4 // Unassign an office from an employee.
5 batch.unlink(employee, "office", office);
```



For one-to-one and one-to-many links, use the `update` method available on the created batch to modify the foreign key property of the source object. The example below illustrates a one-to-many link. An employee can have multiple tickets, but each ticket can only have one employee.

```
1 // Assign a ticket to an employee.
2 batch.update({ $apiName: "Ticket", $primaryKey: 13 }, { assignedEmployeeId
3
4 // Unassign a ticket.
5 batch.update({ $apiName: "Ticket", $primaryKey: 13 }, { assignedEmployeeId
```



As with updating properties, you can also reference either side of the link with an API name and primary key if you have not loaded a concrete instance of the object type previously.

API Reference ↗

```
1 // Assign a ticket to an employee.
2 batch.link({ $apiName: "Employee", $primaryKey: 23 }, "
3
```



Send feedback

```
4 // Unassign a ticket from an employee.
5 batch.unlink({ $apiName: "Employee", $primaryKey: 23 }, "assignedTickets",
```

Create objects

Objects

You can create new objects using the `create` method on the edit batch. When creating a new object, you must specify a value for its primary key and can optionally initialize any other properties.

In this example, we create a new `Ticket` object with the given ID, set its `dueDate` property, and assign it to the given `Employee` by modifying the `assignedTickets` link. To simplify the calculation of the new value of `dueDate`, we use the `luxon` library.

```
1 import { Employee, Ticket } from "@ontology/sdk";
2 import { Client, Osdk } from "@osdk/client";
3 import { createEditBatch, Edits, Integer } from "@osdk/functions";
4 import { DateTime } from "luxon";
5
6 type OntologyEdit = Edits.Object<Employee> | Edits.Object<Ticket> | Edits
7
8 export default function createNewTicketAndAssignToEmployee(
9   client: Client,
10   employee: Osdk.Instance<Employee>,
11   ticketId: Integer,
12 ): OntologyEdit[] {
13   const batch = createEditBatch<OntologyEdit>(client);
14
15   batch.create(Ticket, {
16     ticketId,
17     dueDate: DateTime.now().plus({ days: 7 }).toFormat('yyyy-MM-dd'),
18   });
19
20   // The new ticket does not exist in the Ontology as a concrete instance,
21   // so we need to reference its API name and primary key.
22   batch.link(employee, "assignedTickets", { $apiName: "Ticket", $primaryKey: ticketId });
23 }
```

API Reference ➤

Send feedback

```
24     return batch.getEdits();
25 }
```

Interfaces

You can create new object instances through interfaces by calling the `create` method and specifying an interface, underlying object type, and a set of interface properties. One of the interface properties supplied must be implemented by the primary key property of the underlying object type.

```
1 editBatch.create(Person, {
2     $objectType: "Employee",
3     firstName: "John",
4     lastName: "Doe",
5 });
```

Delete objects

Objects

You can delete an object by calling the `delete` method on the edit batch.

In this example, we delete all the tickets assigned to the given employee:

```
1 for await (const ticket of employee.$link.assignedTickets.asyncIter())
2     batch.delete(ticket);
3 }
```

Objects may also be deleted using a primary key instead of an instance:

[API Reference ↗](#)

```
1 batch.delete({ $apiName: "Ticket", $primaryKey: 12 });
```

[Send feedback](#)

Interfaces

You can delete an object through an interface by calling the `delete` method.

```
1 batch.delete(person);
```

Edits on struct properties

Ontology struct properties for both object and interface types can be edited with TypeScript v2 functions. [Struct types](#) in TypeScript v2 are defined using TypeScript interfaces. Struct types in functions can be used to edit Ontology struct properties, as long as they contain the same fields as the struct property, with field names matching the API names of the Ontology struct property fields.

```
1 interface Address {  
2   street: string,  
3   city: string,  
4   state: string,  
5   country: string,  
6   zipcode: string,  
7 }  
8  
9 export default function updateEmployeeAddress(  
10   client: Client,  
11   employee: Osdk.Instance<Employee>,  
12   newAddress: Address  
13 ): OntologyEdit[] {  
14   const batch = createEditBatch<OntologyEdit>(client);  
15   batch.update(employee, { address: newAddress });  
16   return batch.getEdits();  
17 }
```

[API Reference ↗](#)

← [Getting started](#)

NEXT →

[Send feedback](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Staged writes [Beta]

Beta

Staged writes are in the [beta](#) phase of development and may not be available on your enrollment. Functionality may change during active development. Contact Palantir Support to request access.

Live preview and published function preview are only supported in Code Repositories, not in local development or VS Code workspace environments.

Staged writes provide an additional execution model for functions that edit objects in the Ontology. Unlike edits made via regular [Ontology edit functions](#), staged-write functions:

- Provide a read-after-write guarantee for Ontology edits applied in the function. All edits applied within the function are staged and will be reflected in Ontology queries and aggregations later in the function.
- Allow nested calls to other staged-write functions making Ontology edits.

This page shows how to write staged-write functions and documents their unique properties. For more details about how edit functions work, refer to the [overview page](#).

Key differences from regular Ontology edit functions

Staged-write functions differ from regular edit functions in several important ways.

API Reference ↗

Read after write guarantee



Send feedback

Within a staged-write function, any Ontology data read will reflect all edits previously made in the function (and all edits made in a calling staged-write function within the same action execution). Such edits are staged only and are not visible in queries made by other users or through functions outside the context of the execution. This allows you to query the Ontology from within the function using search requests and aggregations with all staged edits being reflected in the result of the query.

No requirement to return edits at the end of the function

Regular Python and TypeScript V2 Ontology edit functions require returning a batch of Ontology edits as the return value of the function for those edits to be applied. In staged-write functions, the Ontology edits are automatically staged and will be applied to the Ontology at the end of the function execution when the action completes. This frees up the return value of the function to return other information to the caller.

For example, you can apply an action that executes a TypeScript V2 staged-write function which then makes some edits and further calls an AIP Logic function. Queries made by the AIP Logic function will return the Ontology changes made in the TypeScript V2 function; any additional edits made by the AIP Logic function will join the same staged edits without a need to return them as part of the Logic function. All staged edits will be applied automatically once the action completes.

Atomic execution

All operations within a staged-write function, including queries, function calls, and AIP Logic executions, stage their edits together. Those staged edits are committed (that is, applied to the Ontology) after the function completes successfully. If the function throws an error, the Ontology remains unmodified and all staged edits are discarded before the function is retried by the action.

WriteableClient

Some [API Reference](#) functions use a `WriteableClient` instead of the standard `Client`. The `WriteableClient` provides direct methods for creating, updating, and deleting objects without needing to construct an edit batch.

[Send feedback](#)

Define a staged-write function

Staged-write functions must explicitly declare the entities that will be edited using the `Edits` type exported from the `@osdk/functions` package. The first parameter must be a `WriteableClient<T>` where `T` is the union of all edit types the function will perform. The return value is no longer constrained to be an array of edits so you can return any value. The following example declares a function that will edit the `Employee` object type:

```
1 import { Employee } from "@ontology/sdk";
2 import { Edits, Integer } from "@osdk/functions";
3 import { WriteableClient } from "@osdk/functions/experimental";
4
5 type OntologyEdit = Edits.Object<Employee>;
6
7 export default async function assignTicket(
8   client: WriteableClient<OntologyEdit>,
9   employeeId: string,
10  ticketId: string
11 ): Promise<Integer> {
12   // ...
13 }
```

Create objects

Use the `create` method on the `WriteableClient` to create new objects. You must specify the object type and provide a value for the primary key, along with any other properties you want to initialize.

```
1 import { Employee } from "@ontology/sdk";
2 import { Edits } from "@osdk/functions";
3 import { WriteableClient } from "@osdk/functions/experimental";
4
5 type OntologyEdit = Edits.Object<Employee>;
6
7 export default async function createEmployee(
8   client: WriteableClient<OntologyEdit>,
9   employeeId: string,
10  firstName: string,
```

API Reference ↗

Send feedback

```

11     lastName: string
12 ): Promise<Integer> {
13     await client.create(Employee, {
14         employeeId: employeeId,
15         firstName: firstName,
16         lastName: lastName
17     });
18
19     return employeeId;
20 }
21
22 export default createEmployee;

```

Creating with generated IDs

When you need to generate an ID and then use it immediately:

```

1 import { Ticket } from "@ontology/sdk";
2 import { Edits, Integer } from "@osdk/functions";
3 import { WriteableClient } from "@osdk/functions/experimental";
4 import { randomUUID } from "crypto";
5
6 type OntologyEdit = Edits.Object<Ticket>;
7
8 async function createTicket(
9     client: WriteableClient<OntologyEdit>,
10     title: string
11 ): Promise<string> {
12     const ticketId = randomUUID();
13
14     await client.create(Ticket, {
15         ticketId: ticketId,
16         title: title,
17         status: "open"
18     });
19
20     return ticketId;
21 }

```

[API Reference ↗](#)

```
export default createTicket;
```

[Send feedback](#)

Update objects

Object properties

Use the `update` method on the `WritableClient` to modify object properties:

```
1 await client.update(employee, { lastName: newName });
```

You can also update an object by referencing its API name and primary key:

```
1 await client.update({ $apiName: "Employee", $primaryKey: 23 }, { lastName:
```

Interface edits are not supported in staged-write functions.

Delete objects

You can delete an object by calling the `delete` method on the `WritableClient`:

```
1 await client.delete(ticket);
```

Objects may also be deleted using a primary key instead of an instance:

```
1 await client.delete({ $apiName: "Ticket", $primaryKey: 12 });
```

[API Reference ↗](#) Delete links

Use the `link` and `unlink` methods on the `WritableClient` to add many links between objects:

[Send feedback](#)

```

1 // Assign a ticket to an employee
2 await client.link(employee, "assignedTickets", ticket);
3
4 // Unassign a ticket from an employee
5 await client.unlink(employee, "assignedTickets", ticket);

```

You can also reference either side of the link with an API name and primary key:

```

1 await client.link(
2   { $apiName: "Employee", $primaryKey: 23 },
3   "assignedTickets",
4   { $apiName: "Ticket", $primaryKey: 12 }
5 );

```

To edit one to many links, edit the foreign key property using a create or update object edit.

Read-after-write within a staged-write function

One of the key advantages of staged-write functions is the ability to read data that was just written in the same execution. This is useful for implementing workflows that require immediate consistency.

```

1 import { Employee, Ticket } from "@ontology/sdk";
2 import { Edits, Integer } from "@osdk/functions";
3 import { WriteableClient } from "@osdk/functions/experimental";
4 import { randomUUID } from "crypto";
5
6 type OntologyEdit = Edits.Object<Ticket> | Edits.Link<Employee, "assignedTickets">;
7
8 async function assignTicketAndCheckWorkload(
9   client: WriteableClient<OntologyEdit>,
10  employeeId: Integer,
11  title: string,
12  { ticketId: string, totalAssignedTickets: number }> {
13    const ticketId = randomUUID();
14
15    // Create the ticket

```

API Reference ➤

Send feedback

```

16     await client.create(Ticket, {
17         ticketId: ticketId,
18         title: title,
19         status: "open"
20     });
21
22     // Assign the ticket to the employee
23     await client.link(
24         { $apiName: "Employee", $primaryKey: employeeId },
25         "assignedTickets",
26         { $apiName: "Ticket", $primaryKey: ticketId }
27     );
28
29     // Query the employee's total workload including the newly assigned t
30     // This works because of the read-after-write guarantee.
31     const result = await client(Ticket).aggregate((tickets) =>
32         tickets
33             .where(ticket => ticket.assignedTo.employeeId.exactMatch(emp
34             .where(ticket => ticket.status.exactMatch("open"))
35             .count()
36     );
37
38     return {
39         ticketId: ticketId,
40         totalAssignedTickets: result
41     };
42 }
43
44 export default assignTicketAndCheckWorkload;

```

Calling other functions from a staged-write function

When you invoke another function or query from within a staged-write function, those operations participate in the same staged edits. Any reads will reflect edits previously staged in the execution, and any edits the called function makes are added to the same staged edits. This applies to:

- Other TypeScript staged-write functions

API Reference ↗

- Ontology queries

Send feedback

If the top-level function completes successfully, every edit staged across the nested calls is committed together. If any call throws, the entire batch is rolled back.

In the example below, `assignTicket` is a separate staged-write function published from the same repository. `bulkAssignTickets` calls it via the OSDK-generated `$Queries` import; each invocation adds its edits to the same staged edits.

```
1 import { Employee, Ticket, $Queries } from "@ontology/sdk";
2 import { Edits, Integer } from "@osdk/functions";
3 import { WriteableClient } from "@osdk/functions/experimental";
4
5 type OntologyEdit = Edits.Object<Ticket> | Edits.Link<Employee, "assigned">;
6
7 async function bulkAssignTickets(
8   client: WriteableClient<OntologyEdit>,
9   employeeId: Integer,
10  ticketIds: string[]
11 ): Promise<Integer> {
12   let assignedCount = 0;
13
14   for (const ticketId of ticketIds) {
15     // Each call to `assignTicket` stages its edits alongside those of
16     // the calling function.
17     await client($Queries.assignTicket).executeFunction({
18       employeeId: employeeId,
19       ticketId: ticketId,
20     });
21
22     assignedCount++;
23   }
24
25   // All staged edits across the nested calls will be committed together
26   // when the top-level function completes.
27   return assignedCount;
28 }
29
30 export default bulkAssignTickets;
```

Understanding when staged edits are committed is important for building reliable functions:

1. **Function execution:** All operations (creates, updates, deletes, reads, nested function calls) are staged to the Ontology. They are visible to the function itself and all nested functions but do not appear outside of the current execution until committed.
2. **Commit:** If the function completes successfully, all staged edits are committed before the action finishes.
3. **Rollback on error:** If the function throws an error, the Ontology remains unmodified and all staged edits are discarded. The function is then retried by the action.

```
1 import { Employee } from "@ontology/sdk";
2 import { Edits, Integer } from "@osdk/functions";
3 import { WriteableClient } from "@osdk/functions/experimental";
4
5 async function updateEmployeeWithValidation(
6   client: WriteableClient<Edits.Object<Employee>>,
7   employeeId: Integer,
8   newSalary: number
9 ): Promise<Integer> {
10   // Validate input
11   if (newSalary < 0) {
12     // Staged edits will be discarded
13     throw new Error("Salary cannot be negative");
14   }
15
16   // Update the employee
17   await client.update(
18     { $apiName: "Employee", $primaryKey: employeeId },
19     { salary: newSalary }
20   );
21
22   // If we reach here, all staged edits will be committed atomically
23 }
24
25 export default updateEmployeeWithValidation;
```

[API Reference ↗](#)

← [Ontology edits](#)

NEXT →

Python

[Send feedback](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

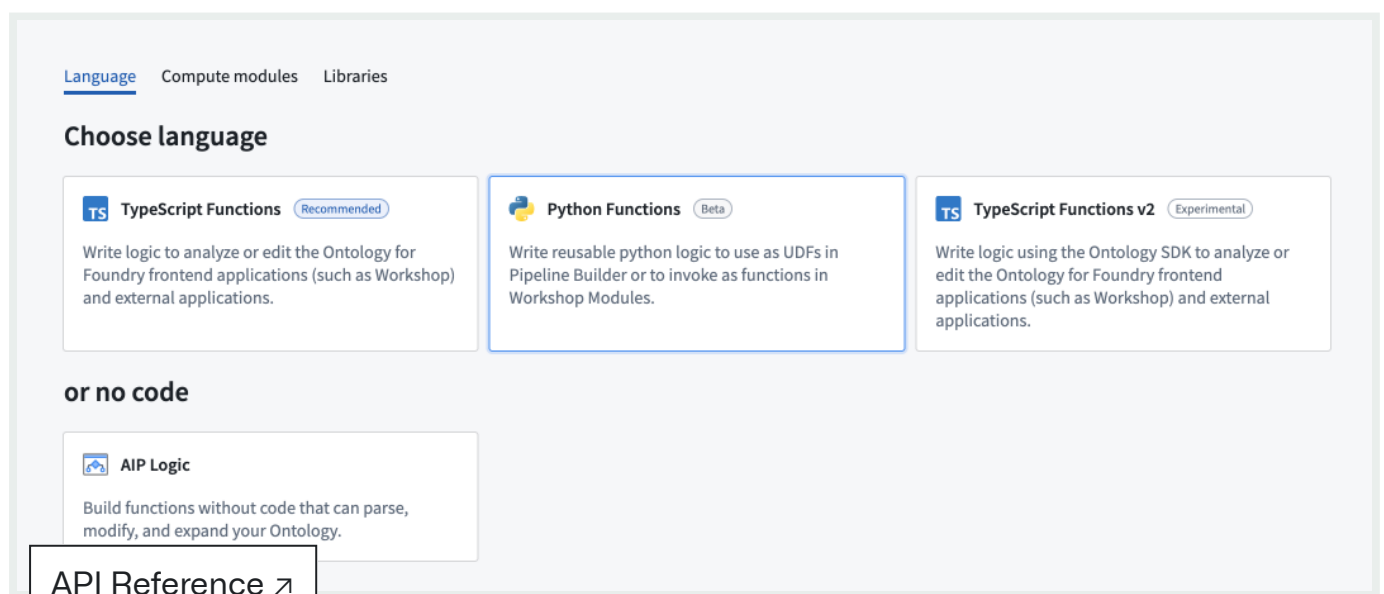
Ontology building / Functions / Python / Getting started

Getting started with Python functions

The following documentation will guide you through the initial steps to prepare Python functions for use in the Palantir platform. You will learn how to create a Python functions repository, commit and publish a function, test in live previews, and more.

Create a Python functions repository

Navigate to a project of your choice and create a new code repository by selecting **+ New > Repository**. Select the **Python functions** template to initialize your repository. We recommend grouping all functions for use in Workshop or Ontology-based applications in a single repository to minimize costs.



Once the repository is created, navigate to the `python-functions/python-functions/my_function.py` file.

Send feedback

Explore the repository

Your repository will be initialized with a `my_function.py` file containing some example functions, including the following:

PYTHON

```
1 from functions.api import function, String
2
3 @function
4 def my_function() -> String:
5     return "Hello World!"
```

Notice that the function adheres to the following constraints:

- The function must be decorated with `@function` from the `functions.api` package to be recognized as a Python function. You may have multiple Python files with multiple functions in each file, but *only the functions with this decorator* will be registered as Python functions.
- The function must declare the types of all of its inputs along with the type of its output, either using the type from the functions API package or its corresponding Python type. For example, the above example's output type is declared as `String` from the functions API, but it may also be declared as the corresponding Python type `str`.

i Even if you declare the type of an argument with the API type (for example, `String`), your function will be passed the corresponding Python type at runtime (in this example, `str`).

For a full overview of types in Python functions see our [type reference documentation](#).

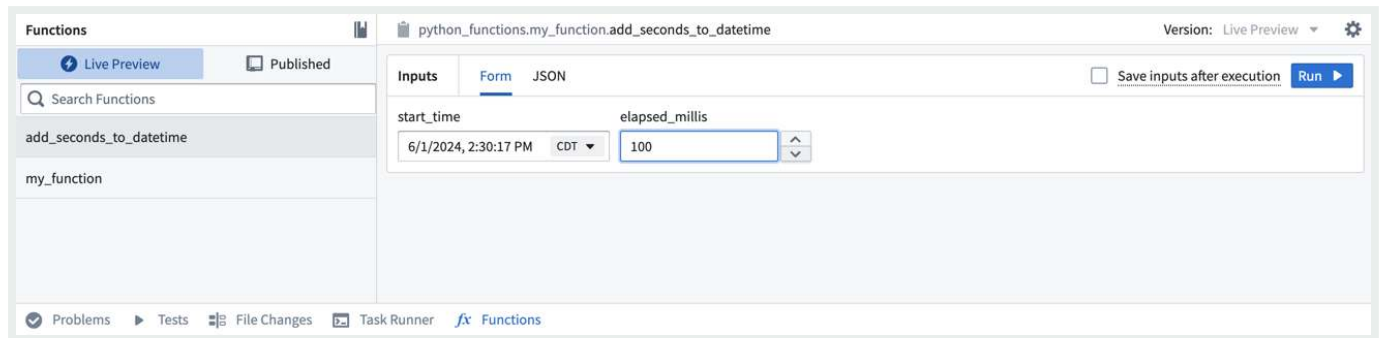
Test in live preview

API Reference ↗

After you add the new function, you can run it immediately in the **Functions** helper from the bottom left of the screen and select **Live**

Send feedback

the `add_seconds_to_datetime` function, enter input values, and select **Run** to run the code.



Select **Commit** in the upper right to commit your changes onto the `master` branch of your repository.

Commit and publish a function

Once you write a function (or uncomment one of the example functions provided), follow the steps below to commit and publish it.

1. Commit your changes by selecting **Commit** in the **Source control** tab and adding a commit message.
2. Select the **Branches** tab from the top center of the screen, then select **Tags and releases**.
3. Choose **New tag** and provide a version for the release.

[API Reference ↗](#)

[Send feedback](#)

Tag and release

You can release functions by tagging a branch or a commit.

Select a branch or enter a commit hash:

Branch

master

Tag name (latest released tag: 0.0.1)

Use custom tag

Stable

Prerelease ?

Patch 0.0.2 Development

Patches are used to signal backwards compatible bug fixes.

Minor 0.1.0 Development

Minor versions are used to signal changes that do not require consumers' adjustments.

Major 1.0.0

Major versions are used to signal breaking changes that require consumers' attention.

Comparing the current commit with latest version 0.0.1.
No breaking changes have been detected.

Releasing: 0.1.0

Cancel

Tag and release

4. Select **Tag and release** and wait for the release step to complete.

5. Once the check is successful, select the **Code** tab, then open the **Functions** tab on the

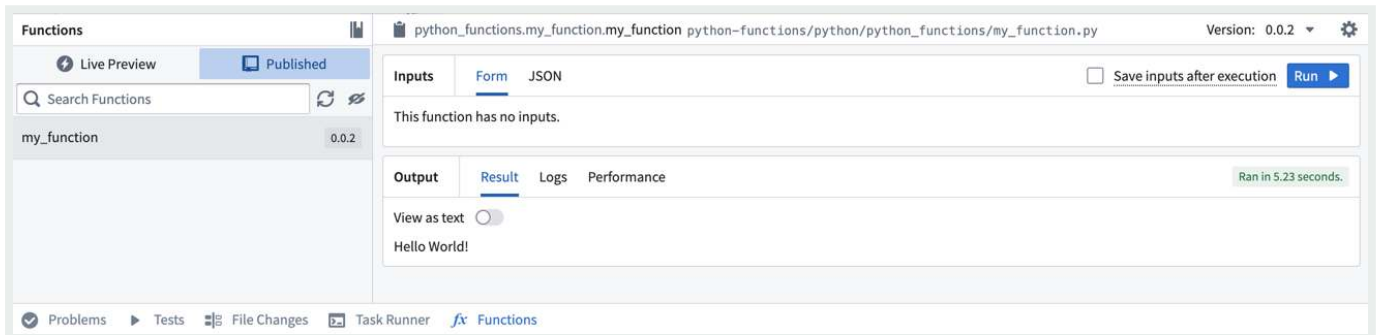
API Reference ↗

se. You will see `my_function` in the results.

Send feedback



6. Select the function, then choose **Run** to execute the function that you just published.



Add another function

Now, we will add a more complex function to this repository to test and publish. Copy and paste the code below to the bottom of the `my_function` file.

PYTHON

```
1 from functions.api import function, String
2 from datetime import datetime, timedelta
3
4 @function
5 def add_seconds_to_datetime(start_time: datetime, elapsed_millis: int) ->
6     dt = start_time + timedelta(milliseconds=elapsed_millis)
7     return dt.isoformat()
```

For more examples of how to use your Python functions in the platform, review our [API Reference](#), [using Python functions in Pipeline Builder](#) and on [using Python functions in Workshop](#).

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Python / Functions on objects

Functions on objects

You can write functions that interact with the Ontology using the Python Ontology SDK.

Generate a Python Ontology SDK

To generate a Python Ontology SDK client, navigate to the [Resource imports sidebar](#) and select **Add > Ontology**. From there, select your desired Ontology and import any objects and links you would like to interact with in your functions. After saving to confirm your selections, a banner will appear to indicate that a corresponding OSDK has not yet been created.

Navigate to the **SDK Generation** tab to generate and install the OSDK.

API Reference ↗



Send feedback

Python Functio... > Code repository - 2025-01-21 15:27:57

File Help 1

Code Version control Checks Settings

Branch master

Resource imports

Data entities SDK generation

No SDK configured

Configure an SDK to use the Ontology within your code repository.

Create new SDK

README.md

Python Functions

To get started, open `python-functions/python/python_fun`

A basic function looks like:

```
from functions.api import function, String

@function
def my_function() -> String:
    return "Hello World!"
```

Notice that the function adheres to the following constraints:

1. It must be annotated with `@function` from the `functions`
1. You may have multiple Python files with multiple func
2. It must declare the types of all of its inputs and the type of its
1. e.g. the example's output type is declared as `String`

Note: Even if you declare the type of an argument with the API type

Supported Types

Primitive API Types

Below is the full list of currently supported primitive Functions API type

Functions API Type	Can Declare as Python Type?	Correspo
Array ¹	✓	list
Binary	✓	bytes
Boolean	✓	bool
Byte ²	✗	int

Problems Tests File Changes Task Runner Functions

Workspace connected

If no OSDK has been generated, you will be prompted to enter a name for the generated package. The package name cannot be changed after the first version has been generated.

After selecting **Create new version**, you can monitor the generation progress from this view.

API Reference ↗

Send feedback

Python Function...

Code repository - 2025-01-21 15:27:57

Code

Version control

Checks

Settings

File

Help

1

Branch

master

Resource imports

Data entities

SDK generation

ontology_sdk

Add

Select a published version to view code snippets which can be used to interact with the generated SDK.

0.1.0

Generator 1.178.0

Started at Jan 21, 2025, 3:32 PM

README.md

Python Functions

To get started, open `python-functions/python/python_fun`

A basic function looks like:

```

from functions.api import function, String

@function
def my_function() -> String:
    return "Hello World!"

```

Notice that the function adheres to the following constraints:

- It must be annotated with `@function` from the `functions`
- You may have multiple Python files with multiple functions
- It must declare the types of all of its inputs and the type of its outputs
 - e.g. the example's output type is declared as `String`

Note: Even if you declare the type of an argument with the API type

Supported Types

Primitive API Types

Below is the full list of currently supported primitive Functions API types

Functions API Type	Can Declare as Python Type?	Corresponding Python Type
Array ¹	✓	list
Binary	✓	bytes
Boolean	✓	bool
Byte ²	✗	int

Problems

Tests

File Changes

Task Runner

Functions

Workspace connected

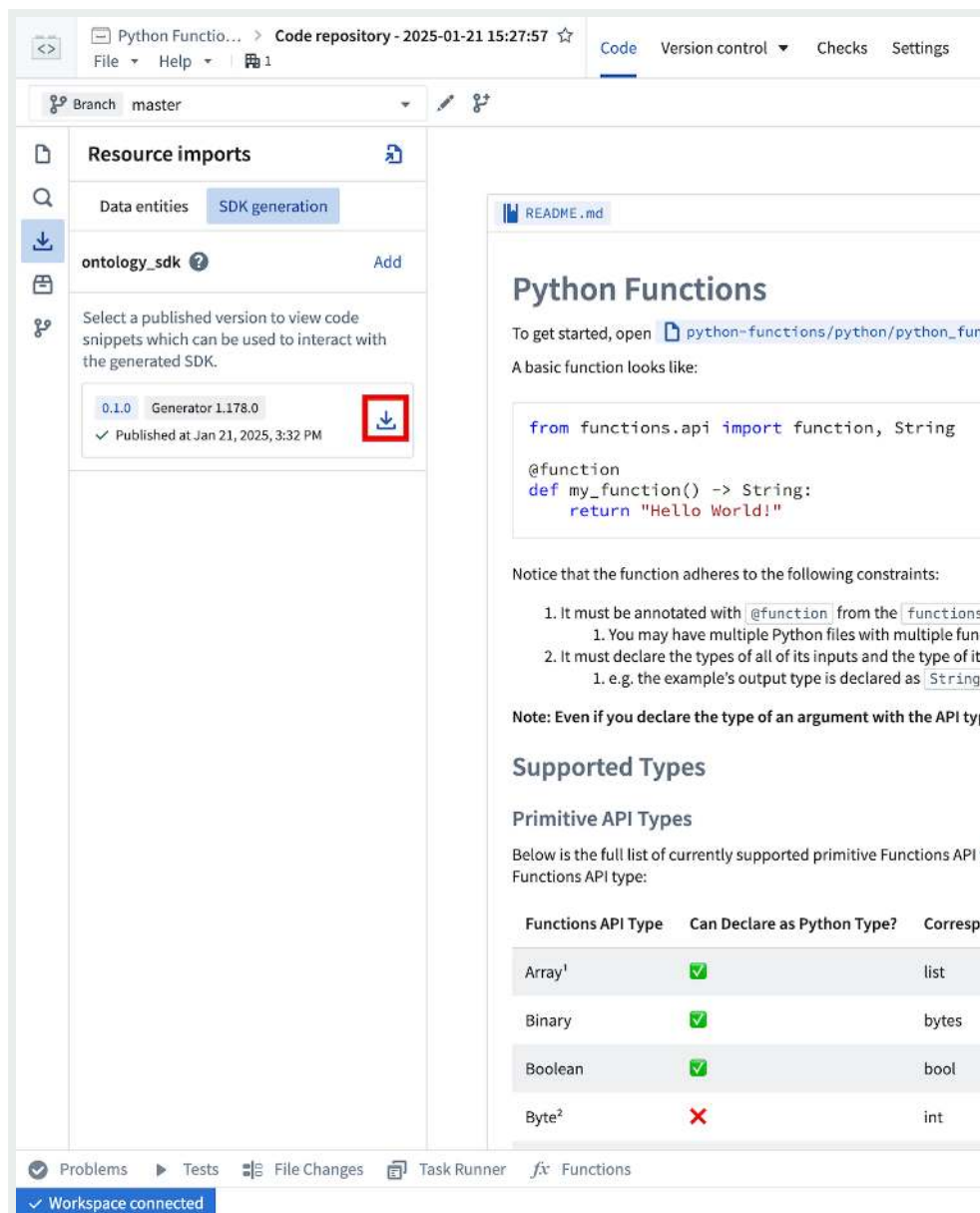
Once generation is complete, you will need to install the newly generated version with the



button.

API Reference ↗

Send feedback

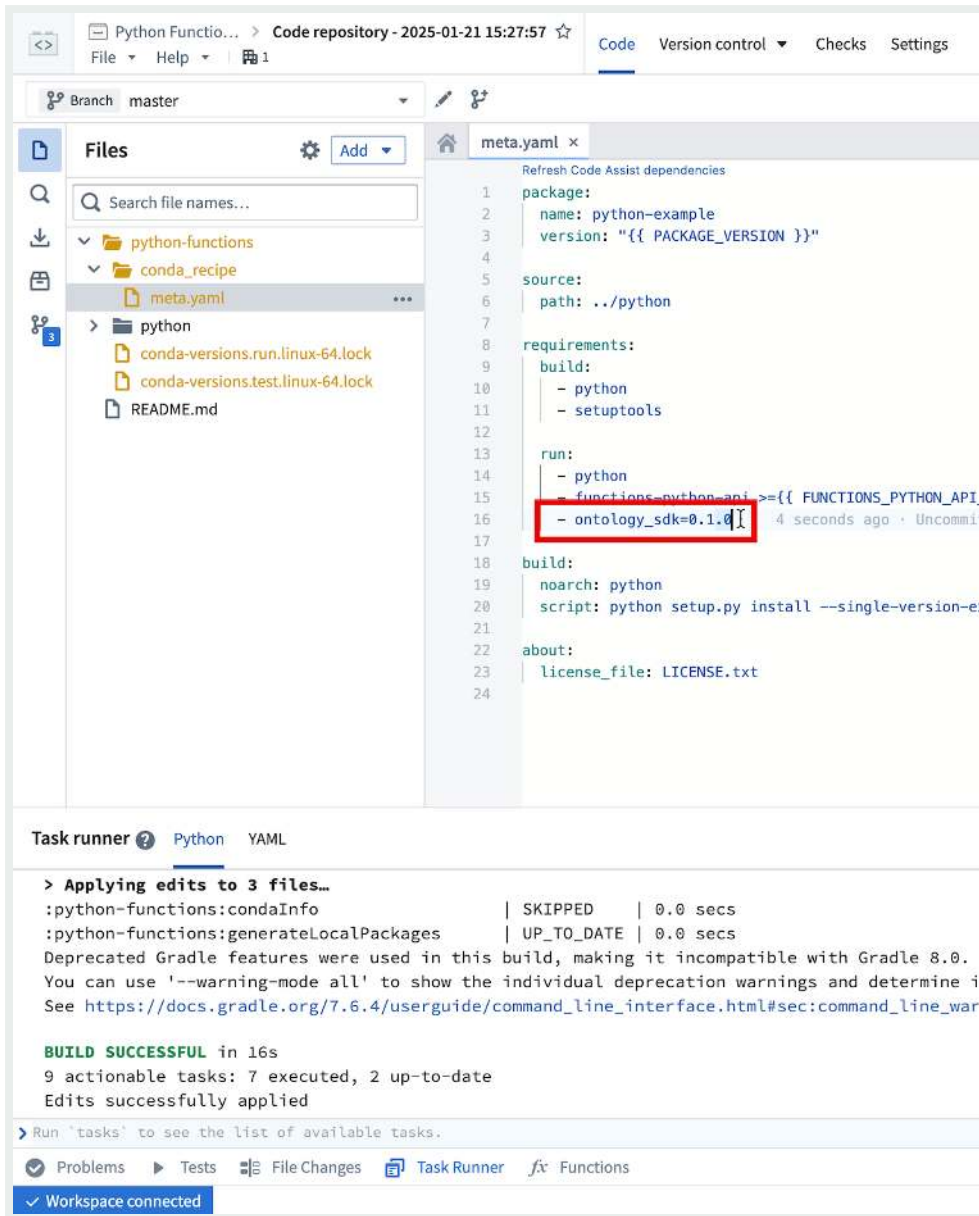


This will trigger an interactive install in the task runner panel. Once that task completes successfully (the Task Runner will display `BUILD SUCCESSFUL`), code completion for the OSDK will be available in your code assist session.

The `meta.yml` file will also be updated to include a reference to the generated package. You can manually update `meta.yml` instead of using the installation helper, but if you manually update `meta.yml`, you will need to rebuild your code assist session to pick up the changes.

API Reference ↗

Send feedback



Any time you import additional resources in the sidebar you will be prompted to generate and install a new version of the OSDK that includes these resources. Additionally, if you modify imported resources (for instance, adding a new property to an already imported object type), you will need to generate a new OSDK version to pick up these changes.

Examples

For an example object type named `Aircraft` with properties `brand` and `capacity`, you could write a function that accepts an `Aircraft` object and summarizes it like so:

API Reference ↗

```

1 from functions.api import function
2 from ontology_sdk.ontology.objects import Aircraft
3

```

Send feedback

```

4 @function
5 def aircraft_input_example(aircraft: Aircraft) -> str:
6     return f"{aircraft.brand} aircraft, holds {aircraft.capacity} passengers"

```

Furthermore, if you wanted to search for `Aircraft` objects satisfying a certain capacity threshold, you could write the following:

```

1 from functions.api import function
2 from ontology_sdk import FoundryClient
3 from ontology_sdk.ontology.objects import Aircraft
4 from ontology_sdk.ontology.object_sets import AircraftObjectSet
5
6 @function
7 def aircraft_search_example() -> AircraftObjectSet:
8     client = FoundryClient()
9     return client.ontology.objects.Aircraft.where(Aircraft.object_type.capacity > 100)

```

The Python OSDK also offers beta features such as interoperability with pandas DataFrames:

```

1 from functions.api import function
2 from ontology_sdk.ontology.object_sets import AircraftObjectSet
3
4 @function
5 def aircraft_dataframe_example(aircrafts: AircraftObjectSet) -> int:
6     df = aircrafts.to_dataframe()
7     return df['capacity'].sum()

```

Review the [Python Ontology SDK documentation](#) for more information.

PREVIOUS

← Getting started

NEXT

Create a custom aggregation →

API Reference ↗

© 2026 Palantir Technologies Inc. All rights reserved.

Send feedback

[Cookies Statement](#) ↗

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Python / Create a custom aggregation

Create a custom aggregation

Functions can be used to compute custom aggregations based on data in the ontology, which can then be surfaced in a chart widget in Workshop. This guide walks through how to write custom aggregation logic that loads aggregated data from the ontology, manipulates the results to create a projection of future results, and returns the modified results.

These references may be useful while working through this section:

- Reference for [Python aggregation types](#)



`.from_osdk()` in `TwoDimensionalAggregation` and `ThreeDimensionalAggregation` are only supported when using v2 of the Python OSDK.

Loading an aggregation

In this example, assume you have an ontology consisting of expenses, with each `expense` object having properties for department name, expense `date`, and expense `amount`. If you want to estimate the monthly spend by department over the next six months, you can begin by loading the aggregated data for the monthly spend:

```
1 client = FoundryClient()
2 result: AggregateObjectsResponse = (
3     client.ontology.objects.Expense
4     .aggregate(
5         Expense.object_type.department_name.exact()
6         Expense.object_type.date.exact()
7     )
8     .sum(Expense.object_type.amount)
```

API Reference ↗



Send feedback

```
7     .compute()
8 )
```

Manipulating aggregation results

Next, you can extrapolate the spend for each department for the next six months. For this example, you can take a simple approach of using the final month's value as the estimate for the next six months.

```
1 current_buckets = ThreeDimensionalAggregation.from_osdk(result, "department")
2 modified_buckets: list[NestedBucket[str, Range[Date], Double]] = []
3 date_format = "%Y-%m-%d"
4 for bucket in current_buckets.buckets:
5     # Find the bucket corresponding to the most recent month
6     last_bucket: SingleBucket[Date, Double] = bucket[-1].value
7
8     next_six_months: list[SingleBucket[Range[Date], Double]] = []
9     # The `date` field has been converted to a string formatted YYYY-MM-DD
10    # Convert to type `Date` from the string. Convert back to a string when needed
11    # creating a SingleBucket object for each month
12    current_month: Date = datetime.strptime(last_bucket.key, date_format)
13
14    # Loop six times
15    for _ in range(6):
16        # Construct the next month from the current month
17        next_month = current_month + relativedelta(months=1)
18        # Add a new bucket which uses the next month as the date range
19        # and the most recent months amount as the value
20        next_six_months.append(SingleBucket(Range(min=current_month, max=
21        current_month + relativedelta(months=6)), last_bucket.value))
22        current_month = next_month
23
24    # Append the modified results as a NestedBucket
25    modified_buckets.append(NestedBucket(bucket.key, next_six_months))
```

Returning the aggregation

[API Reference ↗](#)

Now that you have created an estimate for the next six months, you can return the estimated values as a `ThreeDimensionalAggregation`:

[Send feedback](#)

```
1 return ThreeDimensionalAggregation(modified_buckets)
```



The full example code for this function is as follows:

```
1 from dateutil.relativedelta import relativedelta
2 from datetime import datetime
3
4 from functions.api import (
5     Date,
6     Double,
7     NestedBucket,
8     SingleBucket,
9     String,
10    Range,
11    ThreeDimensionalAggregation,
12    function,
13 )
14 from ontology_sdk import FoundryClient
15 from ontology_sdk.ontology.objects import Expense
16
17
18 @function
19 def estimated_department_expenses() -> ThreeDimensionalAggregation[str, D
20     client = FoundryClient()
21     result = (
22         client.ontology.objects.Expense
23         .group_by(Expense.object_type.department_name.exact())
24         .group_by(Expense.object_type.date.exact())
25         .sum(Expense.object_type.amount)
26         .compute()
27     )
28
29     current_buckets = ThreeDimensionalAggregation.from_osdk(result, "depa
30     modified_buckets: list[NestedBucket[str, Range[Date], Double]] = []
31     date_format = "%Y-%m-%d"
32     for bucket in current_buckets.buckets:
33         # Find the bucket corresponding to the most recent month
34         last_bucket: SingleBucket[Date, Double] = bucket.buckets[-1]
35
36         # Find the bucket corresponding to the month six months ago
37         six_months_ago: list[SingleBucket[Range[Date], Double]] = []
38         # The `date` field has been converted to a string formatted YYYY-MM-DD
39         # Convert to type `Date` from the string.
40         current_month: Date = datetime.strptime(last_bucket.key, date_for
```



API Reference ↗

Send feedback


```

40
41     # Loop six times
42     for _ in range(6):
43         # Construct the next month from the current month
44         next_month = current_month + relativedelta(months=1)
45         # Add a new bucket which uses the next month as the date range
46         # and the most recent months amount as the value
47         next_six_months.append(SingleBucket(Range(min=current_month,
48                                                    current_month = next_month)
49
50         # Append the modified results as a NestedBucket
51         modified_buckets.append(NestedBucket(bucket.key, next_six_months))
52
53     return ThreeDimensionalAggregation(modified_buckets)

```

The resulting aggregation can be used in a Workshop chart to show the monthly spend estimate for the next six months.

PREVIOUS

← [Functions on objects](#)

NEXT

[Ontology edits](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology edits

In addition to writing functions that read data from the Ontology, you can also write functions that create objects and edit the properties and links between objects. This page documents the object edit APIs available to you in functions. For more details about how edit functions work, refer to the [overview page](#).

For the edits created in a function to actually be applied, Ontology edit functions *must be configured as a [function-backed Action](#)*. Configuring an Action in this way allows you to provide additional metadata, configure permissions, and access the Action in various operational interfaces. As noted in the [documentation](#), running an edit function outside of an Action will not actually modify any object data.

⚠ Warning

Searching for objects immediately after editing them may return unexpected results. See the [Caveats section](#) for details.

Define an edit function

Functions that edit the Ontology must:

- Be decorated with the `@function(edits=[MyObjectType])` decorator imported from `functions.api` to specify the object types that will be edited.
- Have an explicit `list[OntologyEdit]` return type hint imported from `functions.api`.

API Reference ↗

Product an Ontology edits container

Send feedback

To perform Ontology edits in a Python function, first construct an Ontology edits container from the [OSDK client](#). For example:

```
1 ontology_edits = FoundryClient().ontology.edits()
```

This container is used to keep track of all edits made in a function.

Update properties

Ontology objects in Python functions are read-only by default. Attempts to modify their properties will raise an exception.

In order to edit an object, first obtain an editable view of that object using an Ontology edits container, either from an existing object instance:

```
1 editable_object = ontology_edits.objects.MyObjectType.edit(my_object)
```

or given an object primary key:

```
1 editable_object = ontology_edits.objects.MyObjectType.edit(object_primary_key)
```

Once you have an editable object, you can edit property values by reassigning the property value for an object. For example:

```
1 editable_employee.last_name = new_name
```

Subsequent access to the `last_name` property value of `editable_employee` later in the script will yield the new value that was just set. However, the original non-editable object will *not* reflect the changes.

[API Reference ↗](#)

Send feedback

Array properties on editable objects are read-only. To modify an array, create a copy of it, modify the copy, then update the property:

```
1 # Copy to a new array
2 array_copy = list(editable_object.my_array_property)
3 # Now you can modify the copied array
4 array_copy.append(new_item)
5 # Then overwrite the property value
6 editable_object.my_array_property = array_copy
```

Note that the primary key property value of an existing object cannot be updated.

Update links

Single-link and multi-link properties have various methods for updating links:

```
1 # Set an Employee's supervisor
2 editable_employee.supervisor.set(new_supervisor)
3
4 # Clear an Employee's supervisor
5 editable_employee.supervisor.clear()
6
7 # Add a new report to the given employee
8 editable_employee.reports.add(new_report)
9
10 # Remove an old report associated with the given employee
11 editable_employee.reports.remove(new_report)
```

As with updating properties, accessing links of `editable_employee` after they have been updated will reflect the updates you have made.

Create objects

[API Reference ↗](#)

You can create new objects using the `MyObjectType.create()` method in the `MyObject` edits container. When creating a new object, you must specify a value

[Send feedback](#)

In this example, we create a new Ticket object with the given ID, set its `due_date` property, and assign it to the given Employee by modifying the `assigned_tickets` link.

```
1 from datetime import datetime, timedelta
2
3 from functions.api import function, Integer, Array, OntologyEdit
4 from ontology_sdk import FoundryClient
5 from ontology_sdk.ontology.objects import Employee, Ticket
6
7 @function(edits=[Employee, Ticket])
8 def create_new_ticket_and_assign_to_employee(
9     employee: Employee,
10    ticket_id: Integer
11 ) -> List[OntologyEdit]:
12    ontology_edits = FoundryClient().ontology.edits()
13
14    new_ticket = ontology_edits.objects.Ticket.create(ticket_id)
15    new_ticket.due_date = datetime.now() + timedelta(days=7)
16
17    editable_employee = ontology_edits.objects.Employee.edit(employee)
18    editable_employee.assigned_tickets.add(new_ticket)
19
20    return ontology_edits.get_edits()
```

Property values may also be passed directly to the create method in addition to the primary key. For example:

```
1 new_due_date = datetime.now() + timedelta(days=7)
2 new_ticket = ontology_edits.objects.Ticket.create(ticket_id, due_date=new_due_date)
```

Delete objects

You can delete an object by calling the `MyObjectType.delete()` method on the Ontology edits container.

[API Reference ↗](#)

In this example, we delete all the tickets assigned to the given employee:

[Send feedback](#)

```
1 for ticket in employee.tickets:
2     ontology_edits.objects.Ticket.delete(ticket)
```



Objects may also be deleted using a primary key instead of an instance:

```
1 ontology_edits.objects.Ticket.delete(ticket_id)
```



PREVIOUS

← [Create a custom aggregation](#)

NEXT

[Use Python functions in Pipeline Builder](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Use a Python function in Pipeline Builder

i Pipeline Builder supports both Java and Python user-defined functions (UDF).
[Learn more about Java UDFs.](#)

Prerequisites

This guide assumes you have already authored and published a Python function. Review our [getting started with Python functions](#) documentation for a tutorial.

Architecture

Python functions run in a Pipeline Builder pipeline as a sidecar container. This means that the function does not need to be deployed and scales dynamically with the size of your pipeline. Embedded functions can be [previewed](#) similarly to other transforms in Pipeline Builder.

Use your function in a Pipeline Builder pipeline

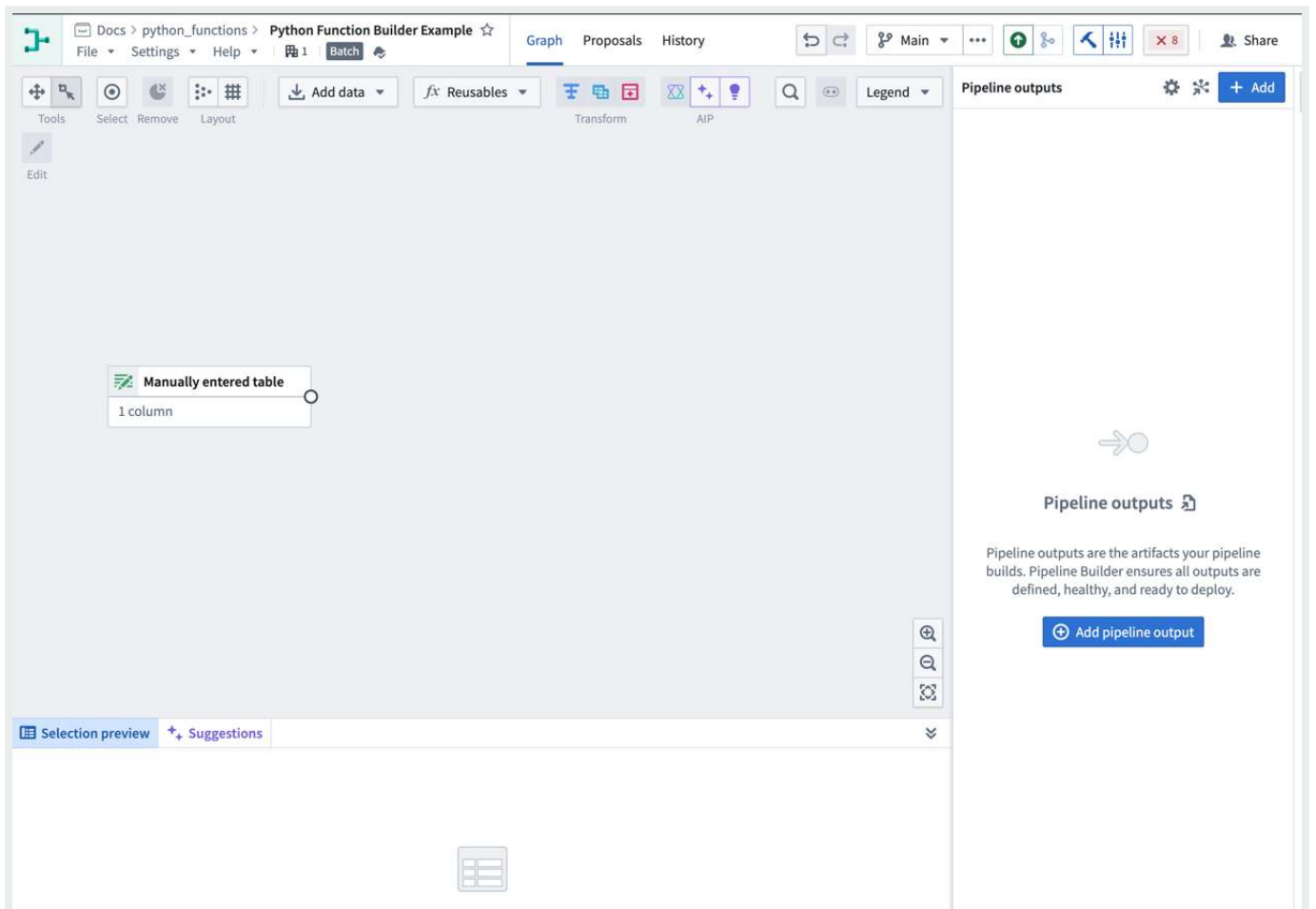
For more information on how to prepare and configure a Python function in your pipeline:

[API Reference ↗](#)



→ Pipeline Builder pipeline in which you want to use your

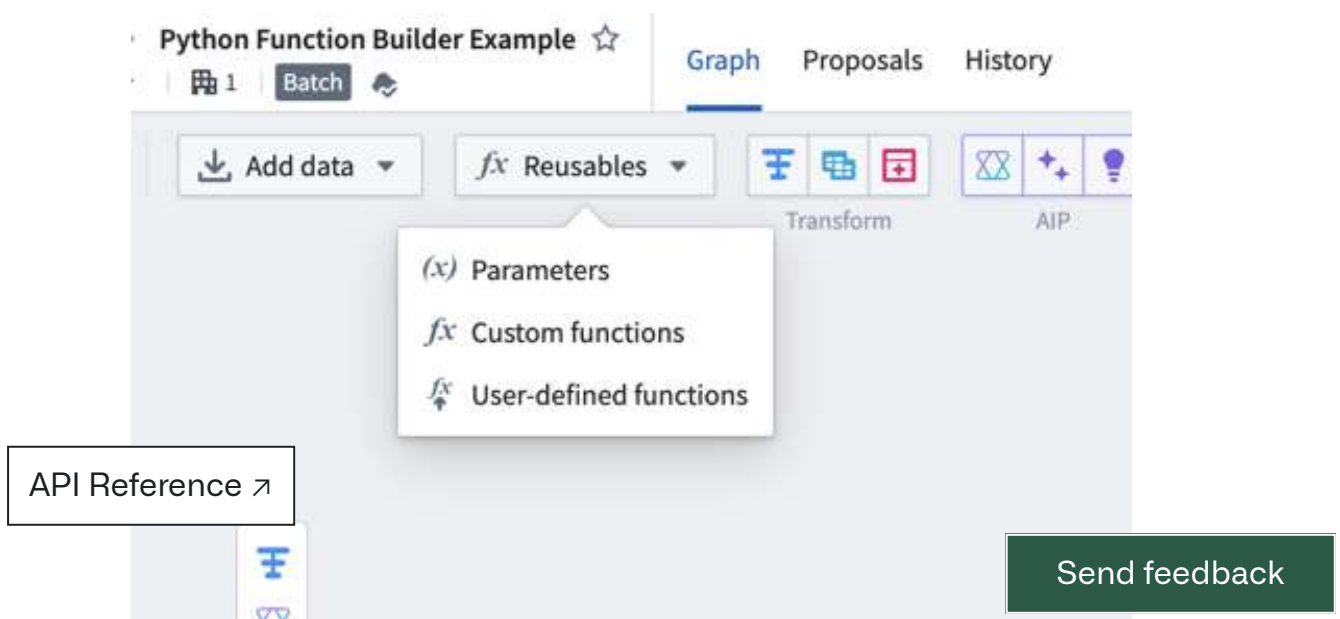
Send feedback



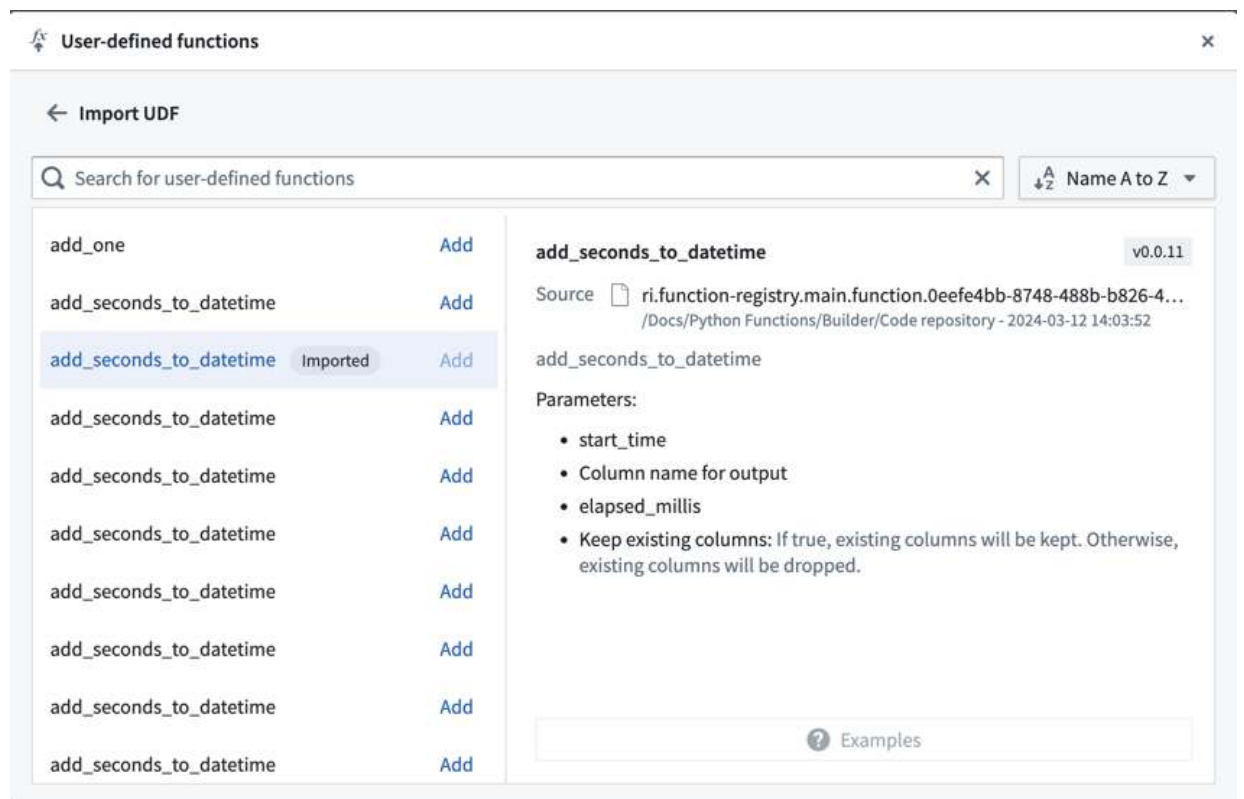
2. Import your UDF into Pipeline Builder using one of two methods:

- **From the graph view:**

1. Select **Reusables** from the upper part of the pipeline graph, then choose **User-defined functions**.



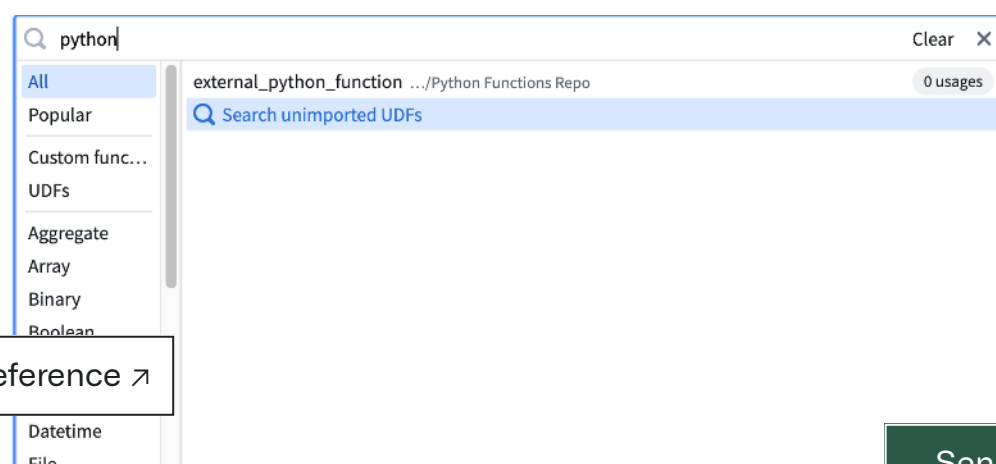
2. Select **Import UDF** and search through the available functions to find the one you want to use 3. Choose **Add** next to the function name. The function should then display an **Imported** tag.



4. Close the import dialogue and select **Transform** on your Pipeline Builder graph where you would like to use the function. 5. From the list of transforms, find the **UDFs** tab to the left to view your imported functions.

◦ **Use the transform picker:**

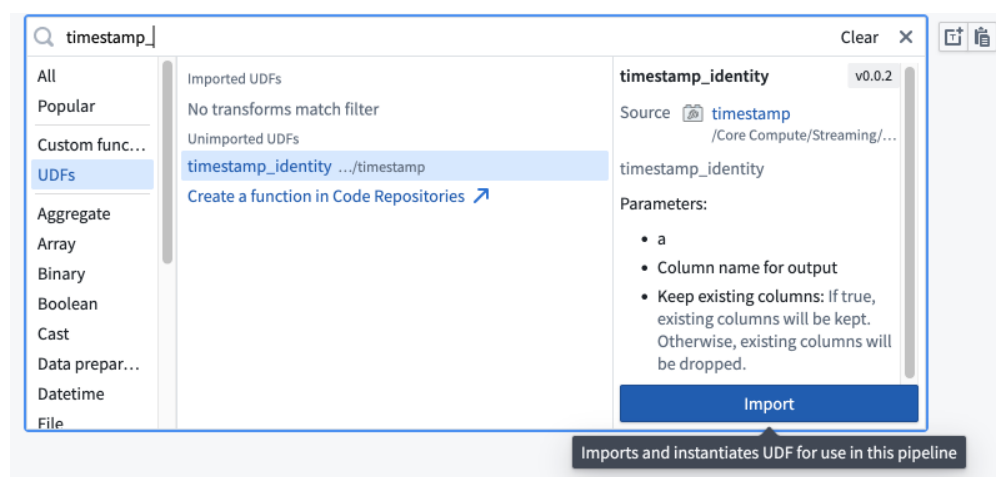
1. Select **Transform** on the pipeline builder graph.
2. Enter the name of the UDF you want to import.



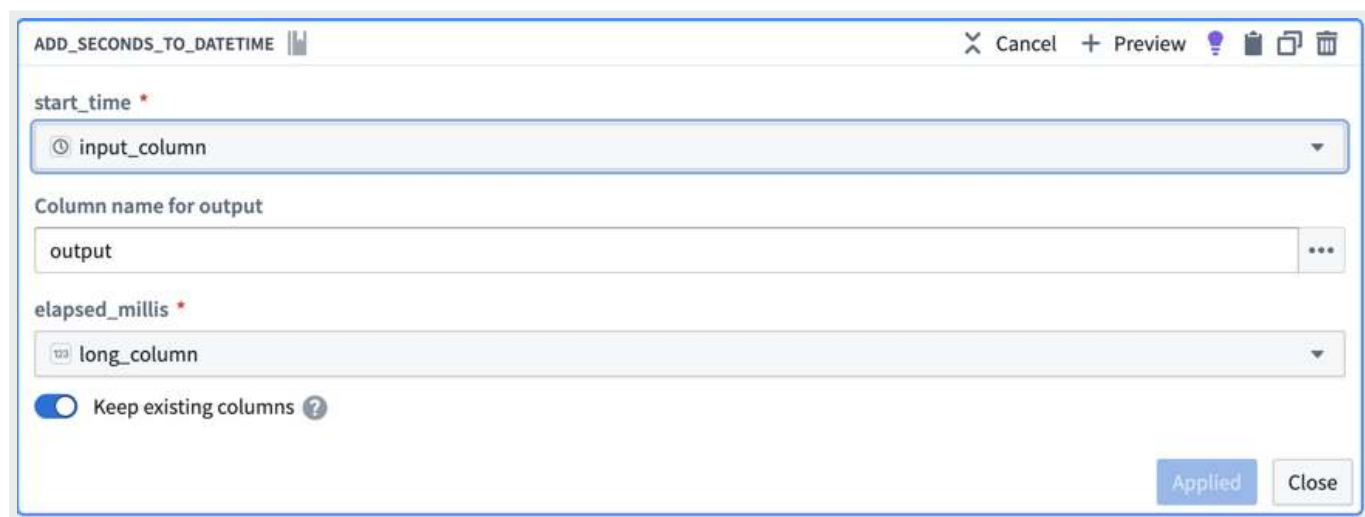
API Reference ↗

Send feedback

3. Select **Search unimported UDFs**. 4. Hover over the desired UDF and select **Import**.



3. Fill out the transform definition specifying the input columns and parameters, then select **Apply**.



You should now see your Python function on your Pipeline Builder graph and can preview the output of the function.

[API Reference ↗](#)

[Send feedback](#)

The screenshot displays the Pipeline Builder interface. The main workspace shows a workflow: 'Manually entered table' (1 column) → 'Transform path' (3 columns) → 'New dataset Sat, Jun 1, 2...' (Not yet deployed). The 'Transform path' node is selected, and the 'Selection preview' tab is active, showing a table with 3 columns: long_column, input_column, and output. The 'Back to outputs' panel on the right shows the output schema: long_column (Long), input_column (Timestamp), and output (String).

	long_column	input_column	output
1	100	2024-06-01T00:00:00.000Z	2024-06-01T00:00:00.100000+
2	100	2024-06-03T00:00:00.000Z	2024-06-03T00:00:00.100000+
3	100	2024-06-04T00:00:00.000Z	2024-06-04T00:00:00.100000+

External API calls in Pipeline Builder

To make API calls to an external system from Pipeline Builder, you can publish a [Python function with access to external systems](#). This will allow you to write logic that communicates with external systems and use it as part of your pipeline.

To be used as a user-defined function (UDF) in Pipeline Builder, all sources used in your function must be configured to be importable into pipelines. To configure this setting, navigate to the source in Data Connection, then to the **Connection settings > Code import configuration** tab:

[API Reference ↗](#)

[Send feedback](#)

Code import configuration

Enable code imports

☒ Allow this source to be imported into code repositories

When a source is imported into a code repository, users with access to the code repository can write custom code using the connection details configured for that source, including accessing secret values.

☐ Allow this source to be imported into compute modules

When a source is imported into a compute module, users with access to the compute module can write and execute arbitrary custom code using the connection details configured for that source, including secret values. Secret values will be available in environment variables as plaintext to the code running in the compute modules.

☒ Allow this source to be imported into pipelines

When a source is imported into a pipeline, the entire pipeline - including any user written code referenced in the pipeline - will be authorized to egress to this source and access secret values.

i Source editors can always remove imports or disallow imports, even if the source is imported into a code resource they don't have access to.

Once you have enabled this option on your source and published your Python function, it can be used in your pipeline in the same way as any other Python function.

PREVIOUS

← [Ontology edits](#)

NEXT

[Use Python functions in Workshop](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Functions / Python / Use Python functions in Workshop

Use a Python function in Workshop

Prerequisites

This guide assumes you have already authored and published a Python function. Review the [getting started with Python functions](#) documentation for a tutorial. For examples of how to query the Ontology using the Python SDK, see the [Python Ontology SDK documentation](#).

Use the Python function in Workshop

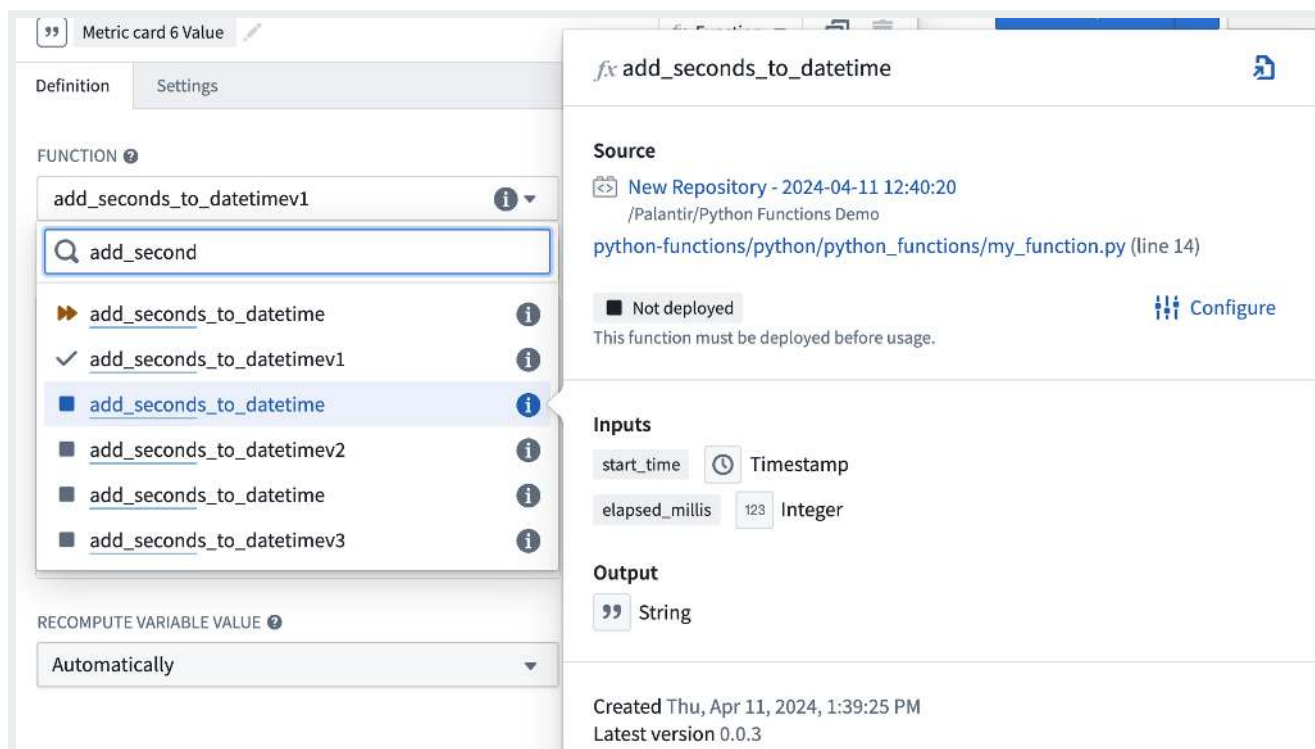
In Workshop, search for the Python function from the **Variables** tab to the left side of the module. Both [serverless and deployed functions](#) will appear here. Serverless functions can always be run at any version and do not need to be manually deployed. Deployed functions will show an icon with one of three states for both the function and the function version:

- **Running:** This function and version can serve requests.
- **Stopped:** This function and version are not available. In the function selector, hover over the information icon, select **Configure** and then **Create and start deployment** to make the function available.
- **Upgrading:** This function and version are not yet available.

API Reference ↗



Send feedback



Cut a new release

Only one version of the function's repository is hosted at a given time. To make changes to functions with limited downtime we recommend adding a new function (like `function_v1`) with the changes and tagging as described [here](#). From your published functions under tags and releases, select **Open in Ontology Manager**.

In Ontology Manager, select the version of the function repository you want to use in applications, then select **Upgrade**.

API Reference ↗

Send feedback

Visibility

Visible

Type

Default

Published Date

Today at 4:32 PM

 New Repository - 2024-04-22 16:26...
python/python_functions/my_fun...

Upgrade

Another version of this function is currently deployed.

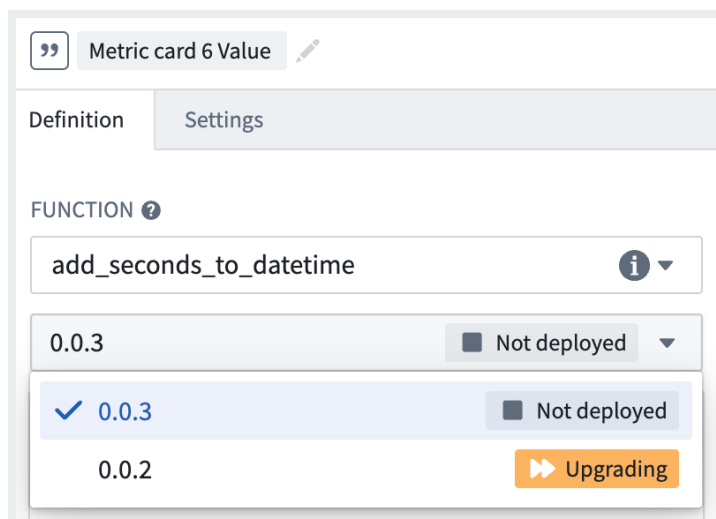
Update all downstream applications using functions from this repository to the new version you have deployed. Note that the previous deployment version will no longer be running so your applications will have a short downtime as you make this change. You will have `function_v0` and `function_v1` available at the same time so while you need to switch to the new deployment version, you do not have to change the function you are using. When `function_v0` is no longer used, you can delete the function.

Debug errors

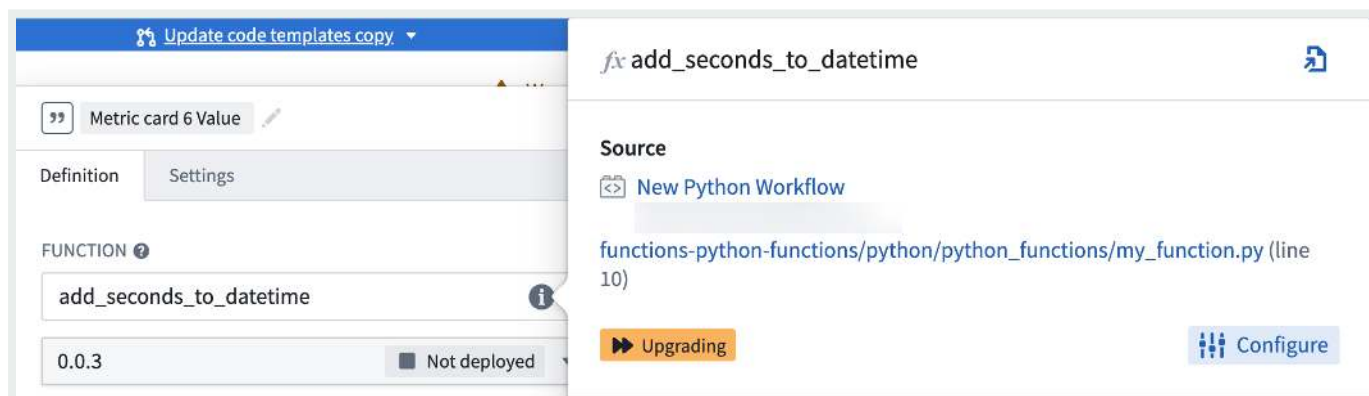
If your function is not working as expected in Workshop, first check if the issue is related to the logic or the responsiveness of the function. If there is an issue with the logic, inspect the source code in the backing code repository. If there is an issue with the function being unresponsive or throwing an error, follow the steps below:

1. Check if the version you selected is currently running in the function selector dropdown menu.

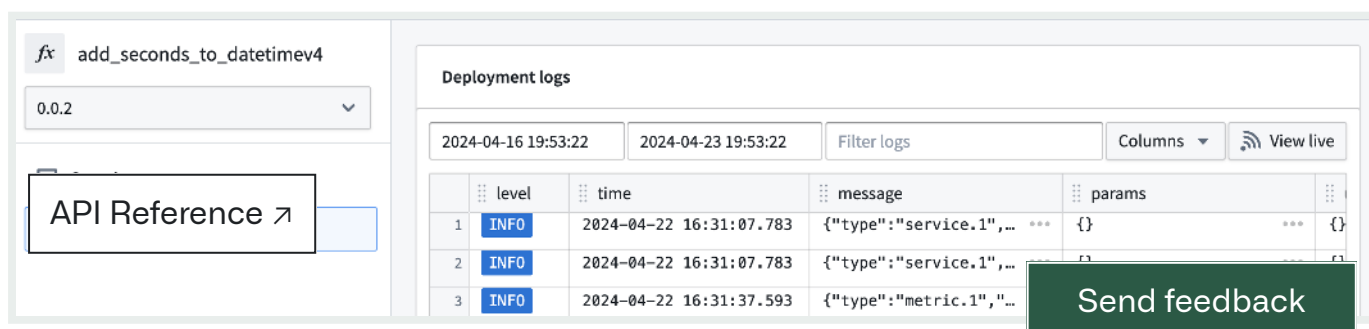
[API Reference ↗](#)[Send feedback](#)



- If the function is not deployed or **Upgrading**, hover over the function's information icon and select **Configure**. This will take you to Ontology Manager where you can select **Start Deployment** to get your function running again.



- If your function is **Running** or you need more information about the deployment's behavior, select **Deployment** from the left panel in Ontology Manager to view detailed logs. SLS logs are also available if you select **View live**.



Create a function-backed column

To create a function-backed column, you must publish a function that meets the following requirements:

- The function's input parameters must include an object set parameter (imported from `ontology_sdk.ontology.object_sets`) and can optionally include other input parameters. This object set parameter will enable the objects displayed in the table to be passed into the function to then generate the desired derived column. Note that a `list[ObjectType]` parameter will also work here, but this less performant option is not recommended.
- The function's return type must be `dict[ObjectType, ColumnType]`, where `ColumnType` is the desired [type](#) for the column, or `dict[ObjectType, CustomType]` to return multiple columns from the function. Learn more about [custom types](#).

Once a function that meets the above criteria is configured and published, you can configure a function-backed property column within the [Object Table](#) widget configuration.

An example of a function returning one column:

```
1 from functions.api import Date, Integer, String, function
2 from ontology_sdk import FoundryClient
3 from ontology_sdk.ontology.object_sets import MyObjectTypeObjectSet
4 from ontology_sdk.ontology.objects import MyObjectType
5
6 @function
7 def function_backed_column_single_col(
8     selected_objects: MyObjectTypeObjectSet
9 ) -> dict[MyObjectType, Integer]:
10     final_dict = {}
11
12     for curr_obj in selected_objects:
13         final_dict[curr_obj] = 10 # The value can be defined for any arbitrary
14
15     return final_dict
```

[API Reference ↗](#)

[Send feedback](#)

Section

Pk	Value	function_backed_column_single_col_list
47deb641-c00f-4110-879f-dd6b9d8653d5	94.76114187206164	10
98950c7a-f9d6-47e7-a6c3-c47ef6966b4e	47.43428949379131	10
35553db2-a9a3-4e92-a928-86535713e979	10.005274913203198	10
8b7b1023-1197-4a1d-b295-06891faad60a	26.254667965267252	10
83827d30-4470-4740-adf1-b73d7275292f	38.87880284261773	10
24dea312-ef56-4ae1-b49a-	34.85836733057483	10

Example Objects

COLUMNS

Pk

Value

function_backed_column_single_col_

function_backed_column_single_c...

on

Example Objects

0.0.2

Function inputs

SELECTED_OBJECTS

Use a variable input

Using runtime input

An example of a function returning multiple columns:

```

1 from dataclasses import dataclass
2
3 @dataclass
4 class CustomType:
5     column_name_a: int
6     column_name_b: int
7
8 @function
9 def function_backed_column_multiple_cols(
10     selected_objects: MyObjectTypeObjectSet, some_other_parameter: String
11 ) -> dict[MyObjectType, CustomType]:
12     final_dict = {}
13
14     for curr_obj in selected_objects:
15         final_dict[curr_obj] = CustomType(column_name_a=10, column_name_b
16
17     return final_dict

```

Pk	Value	column_name_b	column_name_a
47deb641-c00f-4110-879f-dd6b9d8653d5	94.76114187206164	20	10
98950c7a-f9d6-47e7-a6c3-c47ef6966b4e	47.43428949379131	20	10
35553db2-a9a3-4e92-a928-86535713e979	10.005274913203198	20	10
8b7b1023-1197-4a1d-b295-06891faad60a	26.254667965267252	20	10
83827d30-4470-4740-adf1-b73d7275292f	38.87880284261773	20	10
24dea312-ef56-4ae1-b49a-	34.85836733057483	20	10

Pk

Value

function_backed_column_multiple_co

column_name_b

column_name_a

+ Add column

Add all properties Remove all properties

API Reference ↗

Send feedback

← PREVIOUS

NEXT →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

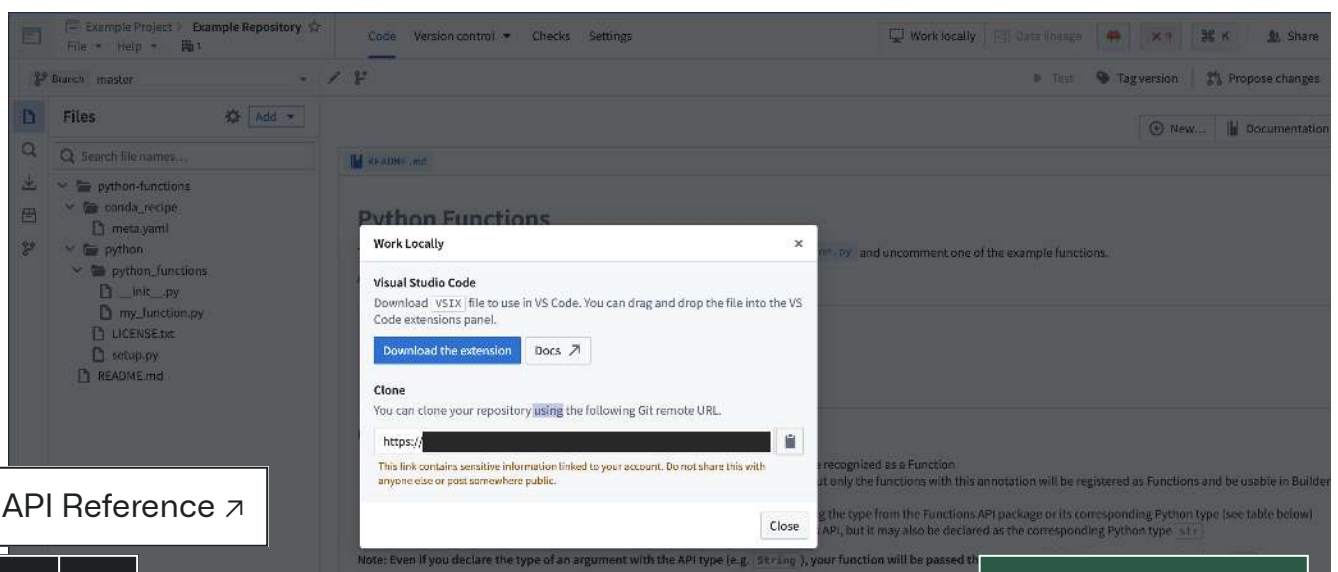
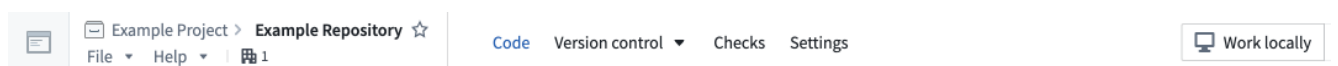
Local development

You can carry out local development of Python functions repositories, allowing for high-speed iteration in your customized environment.

Setting up local development for Python functions repositories

Clone the repository

1. In the menu bar of your repository, select **Work locally** to open the dialog and copy the given repository URL.



API Reference ↗

Send feedback

2. Using the command line, run `git clone <URI>` on your local machine in a directory of your choice. Then use the `cd` command to navigate to the repository.

Limitations

- The token granted for cloning is short-lived and read-only, with the exception of pushing back to your repository.
- You will still need to push your changes to Foundry to publish artifacts, or if you wish to run checks or build.

Set up the development environment

Prerequisites

- Ensure Java 17 is installed and that the environment variable `JAVA_HOME` points to the right Java installation. Java 17 can be downloaded from the [Oracle website ↗](#).

i Setting the `JAVA_HOME` environment variable based on your operating system:

- Windows: Run `SETX JAVA_HOME -m "<java-home-dir>"` in PowerShell. This modifies the system environment variable and you will need to restart the shell for changes to take effect. Alternatively you can run `[System.Environment]::SetEnvironmentVariable("JAVA_HOME", "<java-home-dir>")` to set `JAVA_HOME` in the running process.
- Linux or macOS: Run `export JAVA_HOME=<java-home-dir>`.

- Ensure your repository is upgraded to the latest template version by following the steps outline [here](#).
- Ensure that the environment variables `CI`, `JEMMA`, and `CA` are not set.
- If running on an Apple silicon Mac, ensure that [Rosetta 2 ↗](#) is installed. You can install

API Reference ↗ `sudo /usr/sbin/softwareupdate --install-rosetta --agree-to-license` in the terminal.

Send feedback

⚠ Template version error

If `./gradlew` commands fail with an error similar to the following, your repository is on an outdated template version:

```
A problem occurred evaluating project ':transforms-python'.
```

```
> Failed to apply plugin class 'com.palantir.transforms.lang.python.gradle
```

```
> Could not create task ':transforms-python:createCondaPackLayer'.
```

```
> Task with name 'condaPackRun' not found in project ':transforms-py
```

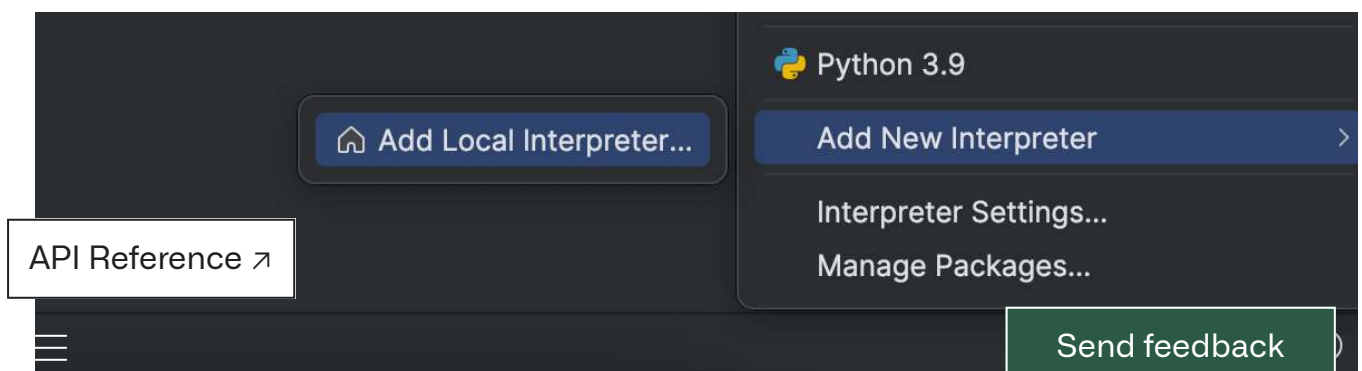
Upgrade your repository to the latest template version as described above to resolve this error.

Visual Studio Code

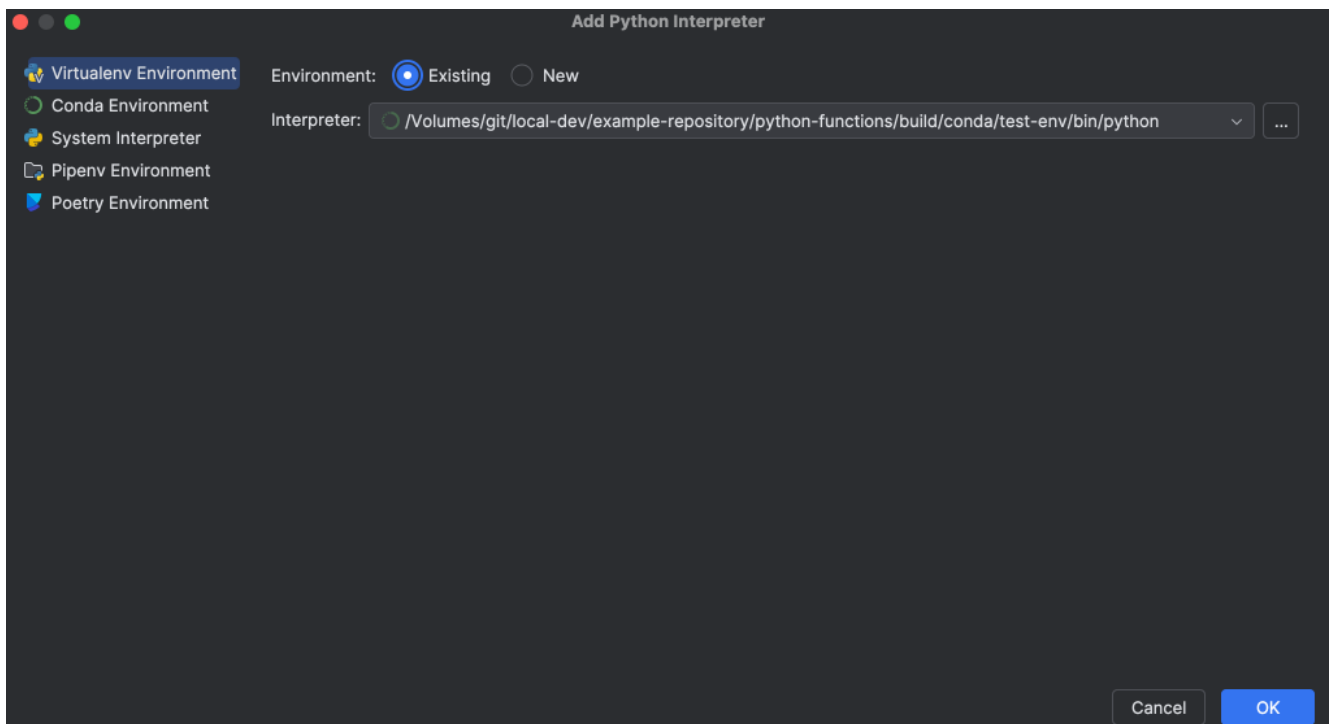
- Ensure you have [Visual Studio Code ↗](#) installed.
- Install the [Python extension ↗](#) from the Visual Studio Code site or from the **Extensions** tab in the application.
- To auto-generate settings files that configure the Python interpreter for Visual Studio Code, run the command `./gradlew vscode`.

PyCharm

- To set up a Python development environment, run the command `./gradlew condaDevelo`.
- Ensure you have [JetBrains PyCharm ↗](#) installed locally.
- Import the project following the steps outlined [here ↗](#).
- Choose **Add New Interpreter** from the [Python Interpreter selector ↗](#) on the status bar.



- In the left-hand pane of the **Add Python Interpreter** dialog, select **Virtualenv Environment**.



- Choose **Existing environment** and set the **Interpreter** field to the Python interpreter from your Conda environment.
 - For Unix, the Python interpreter path is `<your-conda-environment-dir>/bin/python`.
 - For Windows, the Python interpreter path is `<your-conda-environment-dir>\python.exe`.

i Depending on whether the test plugin is enabled, the installed environments would include `./python-functions/build/conda/run-env`, `./python-functions/build/conda/test-env`, or both. You should pick the test environment if you plan on running tests.

- Select **Ok**.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / TypeScript v1 / Getting started

Getting started with TypeScript v1 functions

⚠ The following documentation is specific to TypeScript v1 functions. For the TypeScript v2 version of this page, see [Getting started with TypeScript v2 functions](#). For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

Create a TypeScript v1 functions repository

Navigate to a project of your choice and create a new code repository by selecting **+ New > Repository**. Select the TypeScript functions template to initialize your repository.

The screenshot shows the 'Choose language' section of the Foundry interface. At the top, there are tabs for 'Language', 'Compute modules', and 'Libraries'. Below the tabs, the title 'Choose language' is followed by a description: 'Function repositories allow you to create reusable logic that can be shared and utilized across Foundry. [Learn more about differences between the templates.](#)'

There are three main options for language selection:

- Python Functions**: Write reusable python logic to use as UDFs in Pipeline Builder or to invoke as functions in Workshop Modules.
- TypeScript Functions v2**: Write logic using the Ontology SDK to analyze or edit the Ontology for Foundry frontend applications (such as Workshop) and external applications.
- TypeScript Functions**: Write logic to analyze or edit the Ontology for Foundry frontend applications (such as Workshop) and external applications.

Below these options, there is a section for 'or no code' with an option for 'AIP Logic'.

At the bottom left, there is a search bar with the text 'API Reference' and a magnifying glass icon. At the bottom right, there is a green button labeled 'Send feedback'.

Once the repository is created, navigate to the `functions-typescript/src/index.ts` file.

Write a function

Functions in this repository must be defined within a TypeScript class, and that class must be exported from the `functions-typescript/src/index.ts` file. You can either write your function in the prepopulated examples in `index.ts`, or create a new file. If you create a new file, ensure that you export your class from `index.ts`.

Below is a basic example:

```

TYPESCRIPT V1
1 import { Function, Integer } from "@foundry/functions-api";
2
3 export class ExampleFunctions {
4
5     @Function()
6     public addIntegers(a: Integer, b: Integer): Integer {
7         return a + b;
8     }
9 }

```

If the above code is written in a file called `exampleFunctions.ts`, it must be exported from the index file as shown below:

```

TYPESCRIPT V1
1 // in functions-typescript/src/index.ts
2
3 export * from "../relative/path/to/exampleFunctions";

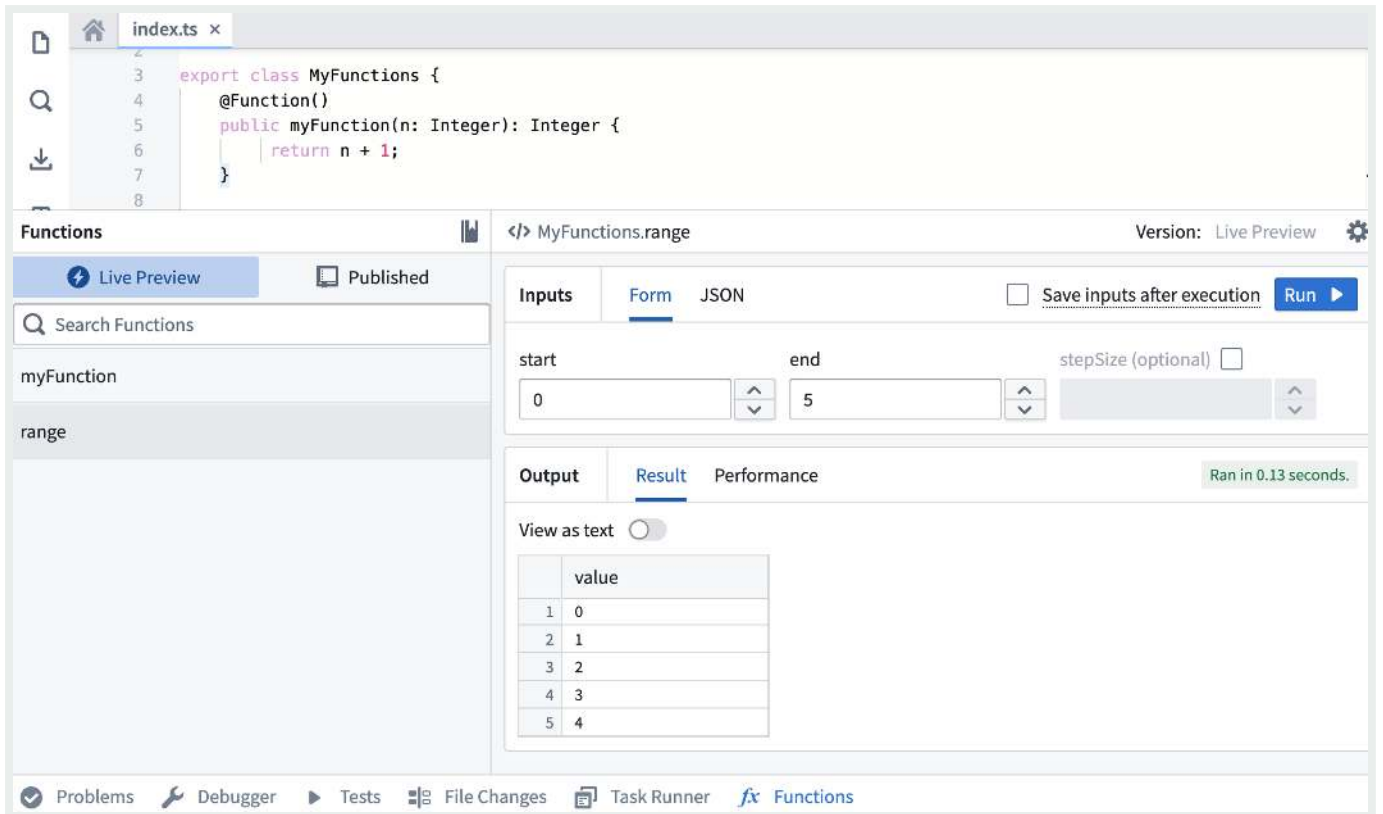
```

Test in live preview

API Reference ↗

Any function, you can run it in the functions helper. Open the **Functions** helper and select **Live Preview**. Choose the `range` function, enter in **Run** to run the code.

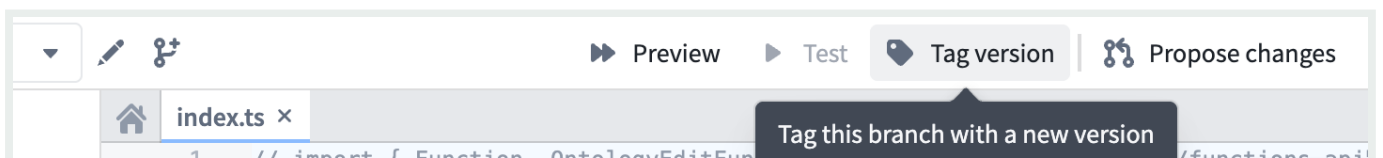
Send feedback



Select **Commit** in the upper right to commit your changes onto the `master` branch of your repository.

Publish a function

After committing your work, you will see the **Tag version** option. This will publish all of the functions in your repository.



Select **Tag version** to tag a release off the `master` branch. Set the tag name based on the extent of your changes, then choose **Tag and release**.

[API Reference ↗](#)

[Send feedback](#)

Tag and release

You can release functions by tagging a branch or a commit.

Select a branch or enter a commit hash:

 Branch

master

Tag name (latest released tag: 0.0.1) [Use custom tag](#)

☒ Stable

☐ Prerelease ?

☐ Patch 0.0.2 Development
Patches are used to signal backwards compatible bug fixes.

☒ Minor 0.1.0 Development
Minor versions are used to signal changes that do not require consumers' adjustments.

☐ Major 1.0.0
Major versions are used to signal breaking changes that require consumers' attention.

Comparing the current commit with latest version 0.0.1.
No breaking changes have been detected.

Releasing: 0.1.0

Cancel

Tag and release

To view the progress as your functions are tagged and released, select the **View** pop-up or navigate to the **Tags** tab. Once **Step 2: Release** is completed, select the published functions to view in the **Function registry**.

API Reference ↗

Send feedback

⚠ Functions may not be immediately searchable by name in Workshop or the function registry while permissions propagate.

Tags and releases

BranchesTags and releases+ Tag and release

Tag 0.1.0 Succeeded

8599f11ade1: Autogenerated commit message at Jan 9, 2024, 5:01 PM

Jan 9, 2024, 5:01 PMView commit ^

STEP 1
Tag

refs/tags/0.1.0
Tue, Jan 9, 2024, 5:01:21 PM

STEP 2
Release

Succeeded at Jan 11, 2024, 12:07 PM
functions-publish task succeeded

fx Published functions

myFunction

Use a new function

After the checks for your tag have passed, navigate back to the **Code** tab in **Code Repositories** and select the **Functions** helper. You should now be able to see your new `range` function under the **Published** section. Select and run the function.

Functions

Live PreviewPublished

Search Functions

myFunction0.2.0

range0.2.0

MyFunctions.range functions-typescript/src/index.ts

Version: 0.2.0Evals

InputsFormJSON

startendstepSize (optional)

1102

OutputResultPerformance

Ran in 0.57 seconds.

View as text

	value
1	1
2	3
3	5
4	7
5	9

Next steps

In [API Reference](#) you learned how to use Code Repositories to write, publish, and test a function from a repository. Next, we recommend learning how to author [functions on objects](#).

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / TypeScript v1 / Migrate from TypeScript v1 to TypeScript v2

Migrate from TypeScript v1 to TypeScript v2

This guide describes the syntax and structure differences you may encounter while migrating existing TypeScript functions from v1 to v2. Refer to the [feature support documentation](#) to learn about v2 enhancements and what each version supports.

Declare a function

To publish a function to the platform in TypeScript v1, you must annotate it with the `@Function()` decorator from the `@foundry/functions-api` package. Additionally, the function must be a method of a class that is exported from the root `index.ts` file of the repository.

```
1 // src/index.ts
2
3 import { Function } from "@foundry/functions-api";
4
5 export class MyFunctions {
6
7     @Function()
8     public reverseStringArray(arr: string[]): string[] {
9         return arr.reverse();
10    }
```

API Reference ↗



Send feedback

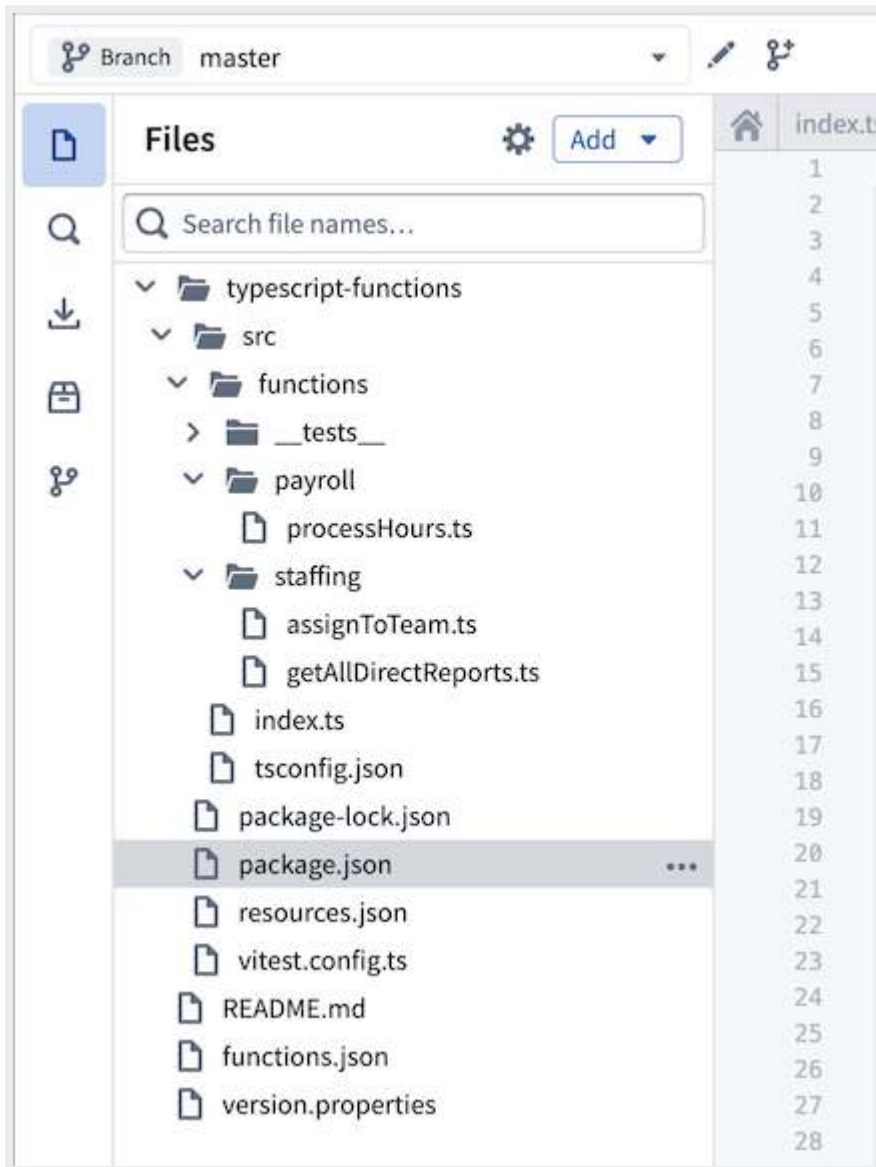
To publish a function to the platform in TypeScript v2, you must write it in a file in the `src/functions` directory and export it using `export default`. Each file can export a single function.

```
1 // src/functions/reverseStringArray.ts
2
3 export default function reverseStringArray(
4   arr: string[]
5 ): string[] {
6   return arr.reverse();
7 }
```

To keep your repository organized, we recommend grouping related functions into subdirectories within the `src/functions` directory. For example, the following folder structure organizes functions into `payroll` and `staffing` subdirectories to make the separation of responsibilities clearer.

[API Reference ↗](#)

[Send feedback](#)



Refer to our documentation on [getting started with TypeScript v2 functions](#) for more information.

Use the `@osdk/functions` package

In TypeScript v1, you must import primitive types like `Integer` and `Double` from the `@foundry/functions-api` package to use them in the signature. In TypeScript v2, however, you must instead use the `@osdk/functions` package.

The following example imports the `Integer` type from the `@foundry/functions-api` package in the signature of a TypeScript v1 function to calculate the greatest common divisor of two integers:

API Reference ↗

Send feedback

```

1 import { Function, Integer } from "@foundry/functions-api";
2
3 export class MyFunctions {
4
5     @Function()
6     public gcd(a: Integer, b: Integer): Integer {
7         if (b === 0) {
8             return a;
9         }
10        return gcd(b, a % b);
11    }
12 }

```

In TypeScript v2, the core TypeScript logic is identical, but you must use the `Integer` type from the `@osdk/functions` package:

```

1 import { Integer } from "@osdk/functions";
2
3 export default function gcd(a: Integer, b: Integer): Integer {
4     if (b === 0) {
5         return a;
6     }
7     return gcd(b, a % b);
8 }

```

Refer to the [types reference](#) for examples of how to import and use types in the signature across both TypeScript v1 and TypeScript v2 functions.

Dates and timestamps

TypeScript v1 uses the `LocalDate` and `Timestamp` types from the `@foundry/functions-api` package for working with temporal data. TypeScript v2 replaces these with the `DateISOString` and `TimestampISOString` types from the `@osdk/functions` package, which represent dates and timestamps as [ISO 8601](#) strings.

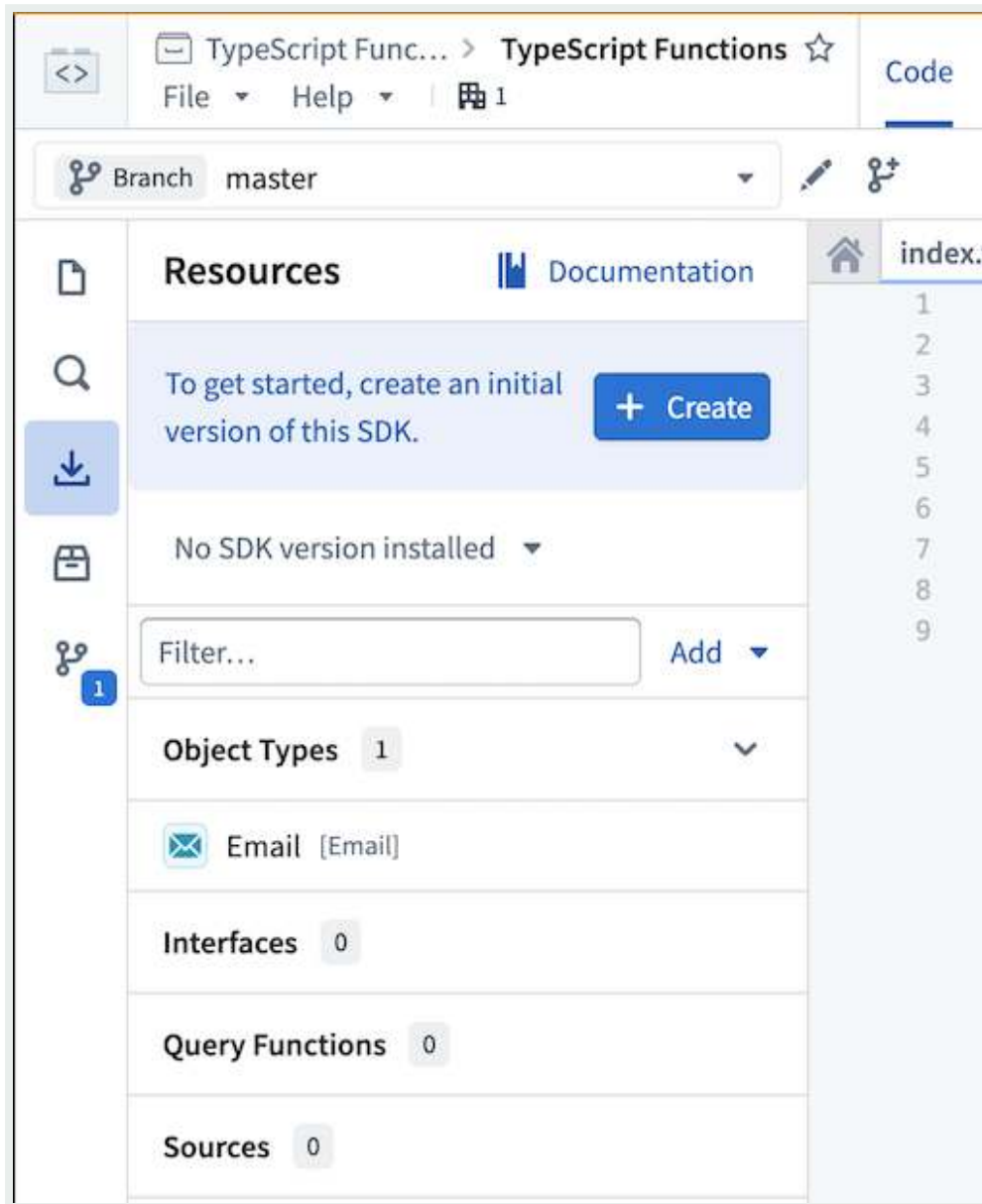
[API Reference ↗](#)

TypeScript v2 functions can use any date and timestamp library available in the ecosystem, such as [dayjs](#), [date-fns](#), and [luxon](#).

[Send feedback](#)

Generate the Ontology SDK

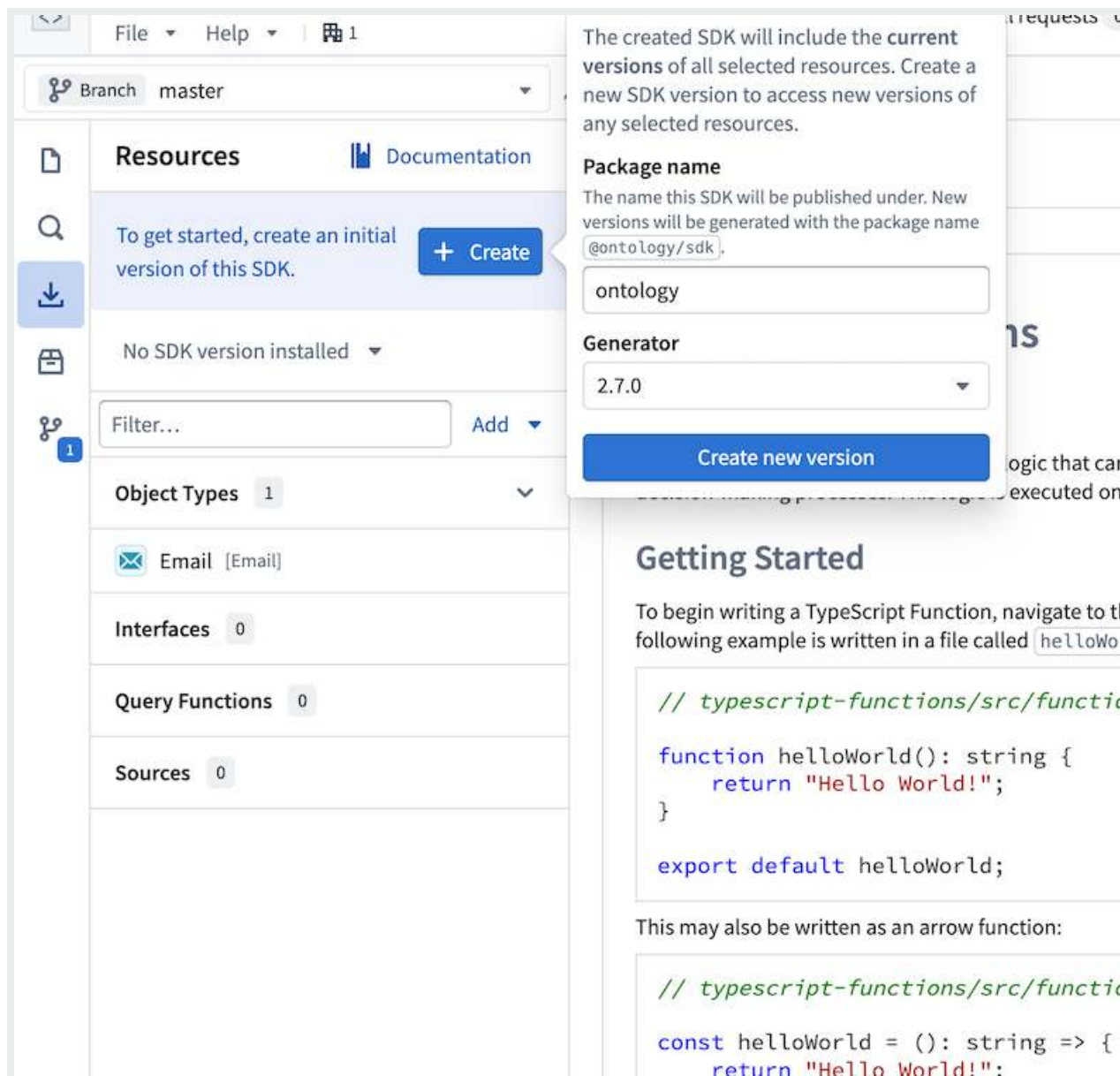
TypeScript v2 functions provide first-class support for querying and editing the Ontology through the [Ontology SDK](#). Like in TypeScript v1, TypeScript v2 repositories allow you to import Ontology entities through the [Resource imports sidebar](#). Once you add your object types and link types, you are prompted to create an initial version of the Ontology SDK.



Select **Create**, then choose a name for the Ontology SDK. This name cannot be changed after the first version is generated. Select **Create new version** to generate the Ontology SDK.

[API Reference ↗](#)

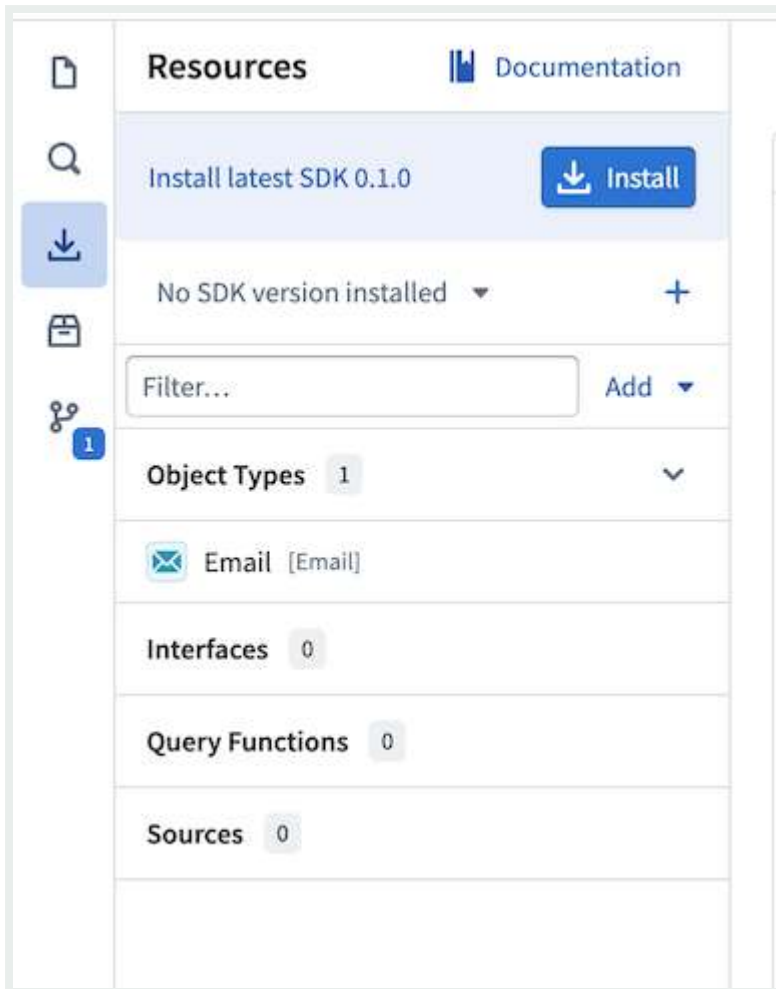
[Send feedback](#)



Once the Ontology SDK is created, you will see an option to install it into the workspace. Selecting **Install** will add the Ontology SDK as a dependency in the `package.json` file and make it available to use in TypeScript code.

[API Reference ↗](#)

[Send feedback](#)



View the **Documentation** tab in the sidebar for comprehensive examples of working with your Ontology in TypeScript.

Query the Ontology

In TypeScript v1, you must import `Objects` from the `@foundry/ontology-api` package to perform searches against your Ontology:



```
11     }  
12 }
```

In TypeScript v2, you must access an Ontology SDK client by specifying it as the first argument in the function signature:

```
1 import { Aircraft } from "@ontology/sdk";  
2 import { Client } from "@osdk/client";  
3 import { Integer } from "@osdk/functions";  
4  
5 export default async function countAircraft(client: Client): Promise<Integer> {  
6     const aircraft = await client(Aircraft).aggregate({  
7         $select: {  
8             $count: "unordered"  
9         }  
10    });  
11  
12    return aircraft.$count;  
13 }
```

Edit the Ontology

To write an Ontology edit function in TypeScript v1, you must annotate it with the `@OntologyEditFunction()` decorator from the `@foundry/functions-api` package and give it a `void` return type. You must also apply the `@Edits decorator` to declare all edited object types up front, allowing permissions on those object types to be enforced before the function-backed action is called.

```
1 import { Edits, OntologyEditFunction } from "@foundry/functions-api";  
2 import { Aircraft, Employee } from "@foundry/ontology-api";  
3  
4 export class MyOntologyEditFunctions {  
5  
6     @Edits(Aircraft, Employee)  
7     @OntologyEditFunction()  
8     public myFunction(aircraft: Aircraft, employee: Employee): void {  
9         aircraft.businessCapacity = 3;  
10        employee.department = "HR";  
11    }  
12 }
```

API Reference ➤

Send feedback

```
11     }  
12 }
```

In TypeScript v2, you must import the `createEditBatch` function from the `@osdk/functions` package to construct a store of edits that is used for the duration of the execution. You must use the `Edits` type to declare which entities your function is permitted to edit. This enforces type safety at compile time; if you attempt to edit an object or link of a type not covered by your `Edits` type, the TypeScript compiler will return an error.

```
1  import { createEditBatch, Edits } from "@osdk/functions";  
2  import { Aircraft, Employee } from "@ontology/sdk";  
3  import { Client, Osdk } from "@osdk/client";  
4  
5  type OntologyEdit = Edits.Object<Aircraft> | Edits.Object<Employee>;  
6  
7  export default function myFunction(  
8      client: Client,  
9      aircraft: Osdk.Instance<Aircraft>,  
10     employee: Osdk.Instance<Employee>  
11 ): OntologyEdit[] {  
12  
13     const batch = createEditBatch<OntologyEdit>(client);  
14  
15     batch.update(aircraft, { businessCapacity: 3 });  
16     batch.update(employee, { department: "HR" });  
17  
18     return batch.getEdits();  
19 }
```

In TypeScript v1, edits *are not* applied to the Ontology during function execution. As described in our [edits and object search documentation](#), changes to objects and links are only propagated after the function finishes executing, and only when called within a function-backed action.

TypeScript v2 makes this behavior more explicit. Rather than implicitly accumulating edits, your function must track them using an edit batch and return them upon completion.

API Reference ↗

Refer to the [ontology edits](#) section in the TypeScript v2 documentation for the full list of supported operations.

Send feedback

Generate unique IDs for objects

To generate unique IDs for newly created objects in TypeScript v1, use the `Uuid` utility from the `@foundry/functions-utils` package.

```
1 import { Edits, OntologyEditFunction } from "@foundry/functions-api";
2 import { Uuid } from "@foundry/functions-utils";
3 import { FlightScenario, Objects } from "@foundry/ontology-api";
4
5 export class ExampleEditFunctions {
6
7     @Edits(FlightScenario)
8     @OntologyEditFunction()
9     public createFlightScenario(): void {
10         const scenario = Objects.create().flightScenarios(Uuid.random());
11         scenario.scenarioName = "New scenario";
12     }
13 }
```

TypeScript v2 runs in a full Node.js environment, so you can use the `node:crypto` core module instead:

```
1 import { FlightScenario } from "@ontology/sdk";
2 import { Client } from "@osdk/client";
3 import { createEditBatch, Edits } from "@osdk/functions";
4 import { randomUUID } from "node:crypto";
5
6 type OntologyEdit = Edits.Object<FlightScenario>;
7
8 export default function createFlightScenario(client: Client): OntologyEdit {
9
10     const batch = createEditBatch<OntologyEdit>(client);
11
12     batch.create(FlightScenario, {
13         id: randomUUID(),
14         scenarioName: "New scenario",
15     });
16 }
```

[API Reference ↗](#)

```
17     return batch.getEdits();
18 }
```

[Send feedback](#)

 Avoid calling `randomUUID` or other random value generators at the top level of a module outside of the function body. TypeScript v2 functions use warm invocations where all module-level code is evaluated once during initialization and then reused across subsequent invocations. This means that a `randomUUID` call at the module level will be evaluated a single time and produce the same value for every warm invocation. Always generate random values inside the function body to ensure uniqueness.

Load objects into memory

TypeScript v1 functions expose the `.all()` and `.allAsync()` APIs to load all objects of a particular type into memory for processing. However, this approach can lead to high memory usage and slower performance as the number of objects in your Ontology grows.

```
1 import { Edits, OntologyEditFunction } from "@foundry/functions-api";
2 import { Aircraft, Objects } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5
6     @Edits(Aircraft)
7     @OntologyEditFunction()
8     public editAircraft(): void {
9
10         const aircraft = Objects.search().aircraft().all();
11
12         aircraft.forEach(a => {
13             a.arrived = true;
14         });
15     }
16 }
```

TypeScript v2 functions support streaming object processing via the Ontology SDK, avoiding the need to hold the entire set of objects in memory at once. We recommend this approach

 [API Reference ↗](#)

[Send feedback](#)

```

1 import { Aircraft } from "@ontology/sdk";
2 import { Client } from "@osdk/client";
3 import { createEditBatch, Edits } from "@osdk/functions";
4
5 type OntologyEdit = Edits.Object<Aircraft>;
6
7 export default async function editAircraft(client: Client): Promise<OntologyEdit> {
8
9     const batch = createEditBatch<OntologyEdit>(client);
10
11     for await (const a of client(Aircraft).asyncIter()) {
12         batch.update(a, { arrived: true });
13     }
14
15     return batch.getEdits();
16 }

```

If data scale is not a concern, the following alternative loads all objects of a particular type:

```

1 import { Aircraft } from "@ontology/sdk";
2 import { Client } from "@osdk/client";
3 import { createEditBatch, Edits } from "@osdk/functions";
4
5 type OntologyEdit = Edits.Object<Aircraft>;
6
7 export default async function editAircraft(client: Client): Promise<OntologyEdit> {
8
9     const batch = createEditBatch<OntologyEdit>(client);
10
11     const aircraft = await Array.fromAsync(client(Aircraft).asyncIter());
12
13     aircraft.forEach(a => {
14         batch.update(a, { arrived: true });
15     });
16
17     return batch.getEdits();
18 }

```

[API Reference ↗](#)

PREVIOUS

← [Getting started](#)

[Send feedback](#)

[Decorators](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Decorators

⚠ The following documentation is specific to TypeScript v1 functions. For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

[TypeScript](#) [↗](#) functions are declared as methods of a [TypeScript class](#) [↗](#). There are a few requirements for a function to be discovered and published:

- The method must be `public`
- The class the method belongs to must be exported from the `functions-typescript/src/index.ts` file
- The method must be decorated with one of the following decorators imported from the `@foundry/functions-api` package:
 - `@Function()` for generic functions.
 - `@OntologyEditFunction()` for functions that will back an Action.
 - Object provenance information may be optionally specified with the `@Edits([object type])` decorator when using the `@OntologyEditFunction()` method.
 - Object provenance information will be inferred on a best-efforts basis using the static analysis of code if the `@Edits([object type])` decorator is absent.
 - `@Query({ apiName: "userDefinedAPIName" })` for read-only queries that you want to execute through [Foundry API](#). Note that this decorator should not be used in addition to the `@Function` decorator; it should be used on its own.

API Reference [↗](#) of functions that are correctly exported in this way:



Send feedback

```

1 import { Function, OntologyEditFunction, Query, Integer, Edits } from '@foundry/ontology-api';
2 import { Employee } from '@foundry/ontology-api';
3
4 export class MyUsefulFunctions {
5     @Function()
6     public incrementNumber(x: Integer): Integer {
7         return x + 1;
8     }
9
10    @Edits(Employee)
11    @OntologyEditFunction()
12    public updateName(employee: Employee, newName: string): void {
13        employee.firstName = newName;
14    }
15
16    @Query({ apiName: "getEmployeesByName" })
17    public async getEmployeesByName(name: string): Promise<ObjectSet<Employee>> {
18        return Objects.search().employee().filter(employee => employee.firstName === name);
19    }
20 }

```

Any method that is private or not decorated with the relevant decorations will not be published to the function registry. This allows users to create helper functions and utilities for reuse or organization.

i Republishing

Note that each function in a TypeScript repository is uniquely defined by its class name and method name—if you change the name of the class or method, the function will be published under a new identifier.

PREVIOUS

← [Migrate from TypeScript v1 to TypeScript v2](#)

NEXT

[Ontology edits](#) →

[API Reference ↗](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Send feedback](#)

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology edits

⚠ The following documentation is specific to TypeScript v1 functions. For the TypeScript v2 version of this page, see [Ontology edits](#). For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

In addition to writing functions that return derived values based on the Ontology, you can also write functions that edit the properties and links between objects in the Ontology. This page documents the object edit APIs available to you in functions. For more details about how edit functions work, refer to the [overview page](#).

To actually be used in an operational context, **Ontology edit functions must be configured as an Action**, known as a [function-backed Action](#). Configuring an Action in this way allows you to provide additional metadata, configure permissions, and access the Action in various operational interfaces. As noted in the [documentation](#), running an edit function outside of an Action will not actually modify any object data.

⚠ Warning

Searching for objects after editing them may return unexpected results. See the [Caveats section](#) for details.

Declaring an edit function

API Reference ↗

For functions that edit the Ontology must:

Send feedback

- Be decorated with the `@OntologyEditFunction()` decorator imported from `@foundry/functions-api`
- Be decorated with the `@Edits([object type])` decorator imported from `@foundry/functions-api` to specify the object types that will be edited
- Have an explicit `void` return type

Updating properties

You can edit property values by simply reassigning the property value for an object. For example:

```
1 employee.lastName = newName;
```

If you access the `lastName` property value later in the same function execution, the new value that you just set will be returned.

[Array properties](#) on objects are generated with the `ReadOnlyArray` type. To modify an array, create a copy of it, modify the copy, then update the property:

```
1 // Copy to a new array
2 let arrayCopy = [...myObject.myArrayProperty];
3 // Now you can modify the copied array
4 arrayCopy.push(newItem);
5 // Then overwrite the property value
6 myObject.myArrayProperty = arrayCopy;
```

Note that you cannot update the primary key property value of an existing object.

Updating links

[API Reference ↗](#) | `MultiLink` interfaces have various methods you can use to update links:

[Send feedback](#)

```

1 // Set an Employee's supervisor
2 employee.supervisor.set(newSupervisor);
3
4 // Clear an Employee's supervisor
5 employee.supervisor.clear();
6
7 // Add a new report to the given employee
8 employee.reports.add(newReport);
9
10 // Remove an old report associated with the given employee
11 employee.reports.remove(oldReport);

```

As with updating properties, accessing links after they have been updated reflects the updates you have made.

Creating objects

You can create new objects using the `Objects.create()` interface available from `@foundry/ontology-api`. When creating a new object, you have to specify a value for its primary key.

In this example, we create a new `Ticket` object with the given ID, set its `dueDate` property, and assign it to the given `Employee` (by modifying the `assignedTickets` link).

```

1 import { OntologyEditFunction, Edits } from "@foundry/functions-api";
2 import { Employee, Objects, Tickets } from "@foundry/ontology-api";
3
4 export class TicketActionFunctions {
5     @Edits(Employee, Tickets)
6     @OntologyEditFunction()
7     public createNewTicketAndAssignToEmployee(employee: Employee, ticketId: string) {
8         const newTicket = Objects.create().ticket(ticketId);
9
10         newTicket.dueDate = LocalDate.now().plusDays(7);
11
12         employee.assignedTickets.add(newTicket);
13     }
14 }

```

API Reference ↗

Send feedback

Deleting objects

You can delete an object by calling the `.delete()` method.

In this example, we delete all the tickets assigned to the given employee.

```
1 const tickets = employee.tickets.all();  
2 tickets.forEach(ticket => ticket.delete());
```



How edits are captured

When an Ontology edit function is executed, all updates to objects are captured by the functions infrastructure and returned at the end of the function execution. This includes new object creations via the `Objects.create()` API, all property updates, and object deletions.

Edits are collapsed intelligently so that the minimal set of edits are applied in an action. For example, if you create a new object and then update its properties, a single **Create Object** edit will be returned containing the property updates. Similarly, updating multiple properties of an existing object will return a single **Update Object** edit containing all of the property edits. Deleting an object will erase any other property edits that were done before the deletion. The entire function must succeed in order to generate the list of edits which is passed to the actions service executing the atomic transaction.

The captured Ontology edits are returned as a list from the function execution, which is why Ontology edit functions must have a return type of `void` or `Promise<void>`. When they are executed, the true return type of the function is a list of Ontology edits, so it is not possible to simultaneously return another value.

Edits are captured in a single edit store over the entire lifecycle of a single function execution. This means that it is possible to call into helper functions which create, update, or delete objects even if those helper functions are not published as Ontology edit

for [API Reference](#) ↗

Send feedback

```

1 export class HelperEditFunctions {
2     @Edits(ObjectA, ObjectB)
3     @OntologyEditFunction()
4     public createAndLink(): void {
5         const objectA = this.createObjectA();
6         const objectB = this.createObjectB();
7         objectA.linkToB.set(objectB);
8     }
9
10    /**
11     * Even though these helper functions are not annotated with @OntologyEditFunction
12     * they can create new objects for use in other edit functions.
13     */
14    private createObjectA(): ObjectA {
15        const objectA = Objects.create().objectA(this.generateRandomId());
16        objectA.prop1 = "example";
17        objectA.prop2 = 42;
18        return objectA;
19    }
20
21    private createObjectB(): ObjectB {
22        const objectB = Objects.create().objectB(this.generateRandomId());
23        objectB.prop1 = "another example";
24        return objectB;
25    }
26
27    /* Generate your primary keys as needed. For example,
28    import { Uuid } from "@foundry/functions-utils";
29    private generateRandomId(){
30        return Uuid.random();
31    }
32    */
33 }

```

Retrieving edited values

When edits are done in a function, the functions infrastructure will return the edited values when you read them. For example, setting a property of an object and then retrieving it will

re [API Reference ↗](#)

Send feedback

```
airplane.departureTime = newDepartureTime;
console.log(airplane.departureTime); // Will log newDepartureTime
```

Deleting an object will remove it from search results and prevent access to its properties.

The @Edits decorator

Actions may require provenance information to enforce its permissions. To provide actions with this information, you can use the `@Edits` decorator and specify the object types for which your function returns edits.

Consider the following when using the `@Edits` decorator:

- When editing properties, the type of the object that was edited should be declared.
- When editing one-to-one or one-to-many links, the type of the object with the foreign key property should be declared.
- When editing join table links, both the source and target object types should be declared.

⚠ Functions perform static analysis of your code to automatically detect referenced object types. However, static analysis *may fail* to properly detect a reference. We strongly recommend that you always use the `@Edits` decorator to provide provenance information about referenced object types.

For the following example, the object types `Employee` and `Aircraft` are edited by a function:

```
1 import { OntologyEditFunction } from "@foundry/functions-api";
2 import { Employee, Aircraft, Objects } from "@foundry/ontology-api";
3
4 export class MyOntologyEditFunction {
5     @ontologyEditFunction(Aircraft, Employee)
6     public myFunction(): void {
7         const x = Objects.search().aircraft().all()[0];
8         x.businessCapacity = 3;
9     }
}
```

API Reference ↗

Send feedback

```

10     const y = Objects.search().employee().all()[0];
11     y.department = "HR";
12 }
13 }

```

If you retrieve (or materialize) a previously edited object through the `Objects.search()` API using an identifier that the edit did not change, such as its primary key, the edited value will be returned. This is because the functions infrastructure applies your pending edits to the object when it is materialized. This differs from filtering, searching around, or aggregating on the value you edited, which will not reflect the edit until your function finishes executing. See [Edits and object search](#) for details.

```

1  import { OntologyEditFunction } from "@foundry/functions-api";
2  import { Employee, Objects } from "@foundry/ontology-api";
3
4  export class CaveatEditFunctions {
5      @Edits(Employee)
6      @OntologyEditFunction()
7      public async editAndSearch(): Promise<void> {
8          const employeeOne = Objects.search().employee().filter(e => e.id.
9              employeeOne.name = "Bob";
10
11          // Retrieve the already edited object
12          const employeeOneAgain = Objects.search().employee().filter(e =>
13              console.log(employeeOneAgain.name); // Prints "Bob"
14      }
15 }

```

← PREVIOUS
Decorators

Generate unique IDs for new objects → NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[API Reference ↗](#)

[Terms of Use ↗](#)

Send feedback

Ontology building / Functions / TypeScript v1 / Generate unique IDs for new objects

Generate unique IDs for new objects

⚠ The following documentation is specific to TypeScript v1 functions. For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

When writing an [Ontology edit function](#) that creates objects, you may want to generate a unique ID for the newly created object. You can set this up in functions by using the `@foundry/functions-utils` package to generate a globally unique identifier.

Import the package

The `@foundry/functions-utils` package is installed by default, but if the package is not present in the `package.json` file:

- In the `"dependencies"` section, add `"@foundry/functions-utils": "0.1.0"`

As mentioned in the [documentation on adding dependencies](#), remember to restart Code Assist to have the new package available for autocomplete.

Use the package in code

To generate a unique ID, you can use the `Uuid.random()` utility function from the `@foundry/functions-utils` package. The below code example shows how you could use the `Uuid.random()` function in an example Ontology edit function.

API Reference ↗

Send feedback


```
1 import { OntologyEditFunction, Timestamp } from "@foundry/functions-api";
2 import { Objects } from "@foundry/ontology-api";
3 import { Uuid } from "@foundry/functions-utils";
4
5 export class ExampleEditFunctions {
6     @Edits(FlightScenario)
7     @OntologyEditFunction()
8     public createFlightScenario(): void {
9         const scenario = Objects.create().flightScenarios(Uuid.random());
10        scenario.scenarioName = "New scenario";
11        scenario.creationTime = Timestamp.now();
12    }
13 }
```

PREVIOUS

← [Ontology edits](#)

NEXT

[Error types](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Functions / TypeScript v1 / Error types

Error types

 The following documentation is specific to TypeScript v1 functions. For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

In addition to declaring an [output type](#) on your TypeScript function, you can also declare an error type. This can be particularly useful for propagating and handling errors in the context of queries.

Define an error type

You can define an error type using the `FunctionsError` type exported from `@foundry/functions-api`. It takes two type parameters; a string name and an optional type that defaults to an empty object. Any [valid output type](#) for a function may be used as an error type, including objects and object sets.

You can union multiple `FunctionsError` types together to define a set of possible errors for your function. For example, you could define the following error type for a function that gets an employee's teammates:

```
1 import { FunctionsError } from "@foundry/functions-api";
2 import { Employee } from "@foundry/ontology-api";
3
```

API Reference ↗

```
  teammatesError =
```

```
    FunctionsError<"EmployeeNotFoundForId", string>
```

```
    FunctionsError<"MultipleEmployeesFoundForId", { e
```

Send feedback

Declare an error type on a function

To declare an error type on a TypeScript function, you can use the `FunctionsResult` type exported from `@foundry/functions-api`. It takes two type parameters; an output type and an error type.

Using the `GetTeammatesError` example from the previous section, you can declare the error type on a function like so:

```
1 import { Function, FunctionsError, FunctionsResult } from "@foundry/functions-api";
2 import { Employee } from "@foundry/ontology-api";
3
4 type GetTeammatesError =
5     | FunctionsError<"EmployeeNotFoundForId", string>
6     | FunctionsError<"MultipleEmployeesFoundForId", { employees: Employee[] }>
7
8 @Function()
9 public getTeammates(employeeId: string): FunctionsResult<Employee[], GetTeammatesError> {
10     ...
11 }
```

Note that by default the `FunctionsResult` type includes a few errors that can be returned by the TypeScript function runtime infrastructure:

- `FunctionsTypeScriptExecutorService:CpuTimeoutError`: Returned when the function exceeds the CPU time limit.
- `FunctionsTypeScriptExecutorService:WallTimeoutError`: Returned when the function exceeds the wall time limit.
- `FunctionsTypeScriptExecutorService:OutOfMemoryError`: Returned when the function exceeds the memory limit.
- `FunctionsTypeScriptExecutorService:RuntimeError`: Returned when the function encounters some other runtime error.

Function outputs and errors in your function

[API Reference ↗](#)

To return outputs and errors, you can use the `FunctionsResult.ok` and `FunctionsResult.err` methods exported from `@foundry/functions-api`:

[Send feedback](#)

- **FunctionsResult.ok**: Takes a single output value as its argument.
- **FunctionsResult.err**: Takes an error name and value as its arguments.

Using the `getTeammates` example from the previous section, you can return outputs and errors like so:

```
1 import { Function, FunctionsError, FunctionsResult } from "@foundry/functions-api";
2 import { Employee } from "@foundry/ontology-api";
3
4 type GetTeammatesError =
5   | FunctionsError<"EmployeeNotFoundForId", string>
6   | FunctionsError<"MultipleEmployeesFoundForId", { employees: Employee[] }>
7
8 @Function()
9 public getTeammates(employeeId: string): FunctionsResult<Employee[], GetTeammatesError> {
10   const employees = await Objects.search().employee([employeeId]).allAsync();
11   if (employees.length === 0) {
12     return FunctionsResult.err("EmployeeNotFoundForId", employeeId);
13   }
14
15   if (employees.length > 1) {
16     return FunctionsResult.err("MultipleEmployeesFoundForId", { employees });
17   }
18
19   const employee = employees[0];
20   const teammates = await employee.teammates.allAsync();
21   return FunctionsResult.ok(teammates);
22 }
```

Handle errors from queries

When a function with an error type is called as a query from another repository, it returns a `Result` type response, which can be an `ok` or `err` result.

You can use the `isOk` or `isErr` type guards exported from `@foundry/functions-api` to differentiate between the two possible results. For instance, call the `getTeammates` example from [API Reference](#) , assuming you also published it as a query, and handle the response like so:

Send feedback

```

1 import { Function, isOk } from "@foundry/function-api";
2 import { getTeammates } from "@foundry/ontology-api/queries";
3
4 @Function()
5 public async myFunction(employeeId: string): Promise<string> {
6     const result = await getTeammates({ employeeId });
7
8     if (isOk(result)) {
9         const teammates = result.value;
10        // Do something with "teammates" here
11        ...
12    }
13
14    // You can inspect the "name" field to case on each error by name and
15    switch (result.error.name) {
16        case "EmployeeNotFoundForId": ...
17        case "MultipleEmployeesFoundForId": ...
18        case "FunctionsTypescriptExecutorService:OutOfMemoryError": ...
19        ...
20    }
21 }

```

PREVIOUS

← Generate unique IDs for new objects

NEXT

Handle undefined values →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—Cookies Settings

[API Reference ↗](#)

Send feedback

Ontology building / Functions / TypeScript v1 / Handle undefined values

Handle undefined values

⚠ The following documentation is specific to TypeScript v1 functions. For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

Below are two useful patterns for handling `undefined` values which may be returned from accessing properties or links.

Explicit checks

```
1 @Function()
2 public getFullName(employee: Employee): string {
3     if (!(employee.firstName && employee.lastName)) {
4         throw new UserFacingError("Cannot derive full name because either
5     }
6     return employee.firstName + " " + employee.lastName;
7 }
```

By checking that both the `firstName` and `lastName` fields are defined, the TypeScript compiler knows that the final line with the `return` statement can compile correctly. The benefit of this approach is that type checking is more explicit, and in the case where `undefined` values are present, you can throw a more explicit error about what went wrong.

[API Reference ↗](#)

Not an assertion operator

Send feedback

You can use the TypeScript [non-null assertion operator](#) `!` (`!`) to ignore the `undefined` case.

```
1 @Function()  
2 public getFullName(employee: Employee): string {  
3     return employee.firstName! + " " + employee.lastName!;  
4 }
```

This approach simply overrides the TypeScript compiler and asserts that the fields you're accessing are defined. Although this makes for more concise code, this can lead to cryptic errors in the case when one of the fields turns out to be `undefined`. We recommend making explicit checks when possible.

← PREVIOUS
[Error types](#)

NEXT →
[Debug functions](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#)

[Privacy Statement](#)

[Terms of Use](#)

— [Cookies Settings](#)

[API Reference](#)

[Send feedback](#)

Ontology building / Functions / TypeScript v1 / Debug functions

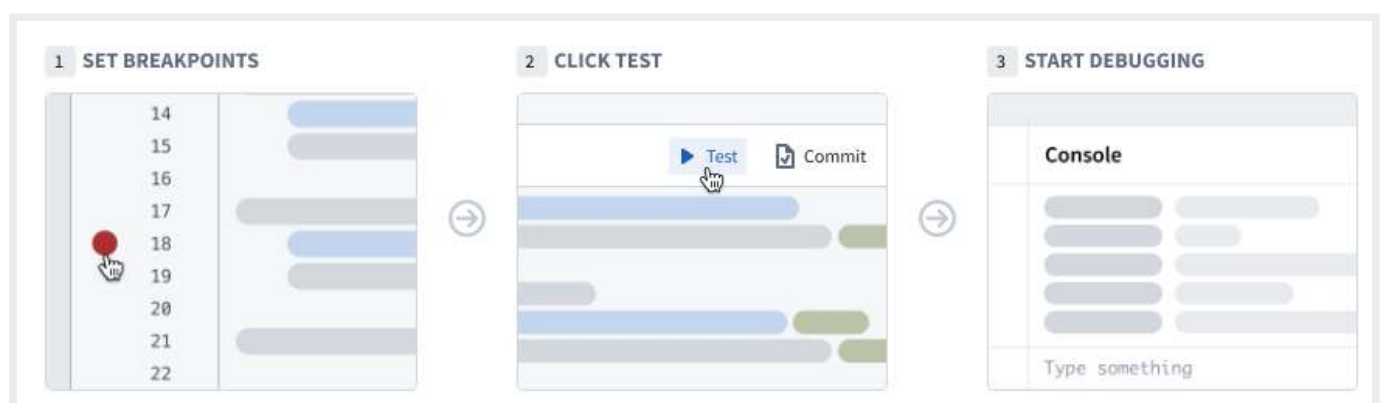
Debug functions

⚠ The following documentation is specific to TypeScript v1 functions. For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

As you write functions, you will likely need to inspect the state of your execution to fix issues with code correctness or performance. Below are features you can use to do this. Note that these debugging steps also apply to [unit tests](#).

Authoring debugger

Use the debugger tool in Code Repositories to examine the behavior of your unit test while it runs. Set breakpoints to pause the execution of the unit test in order to examine variables, and understand functions and libraries.



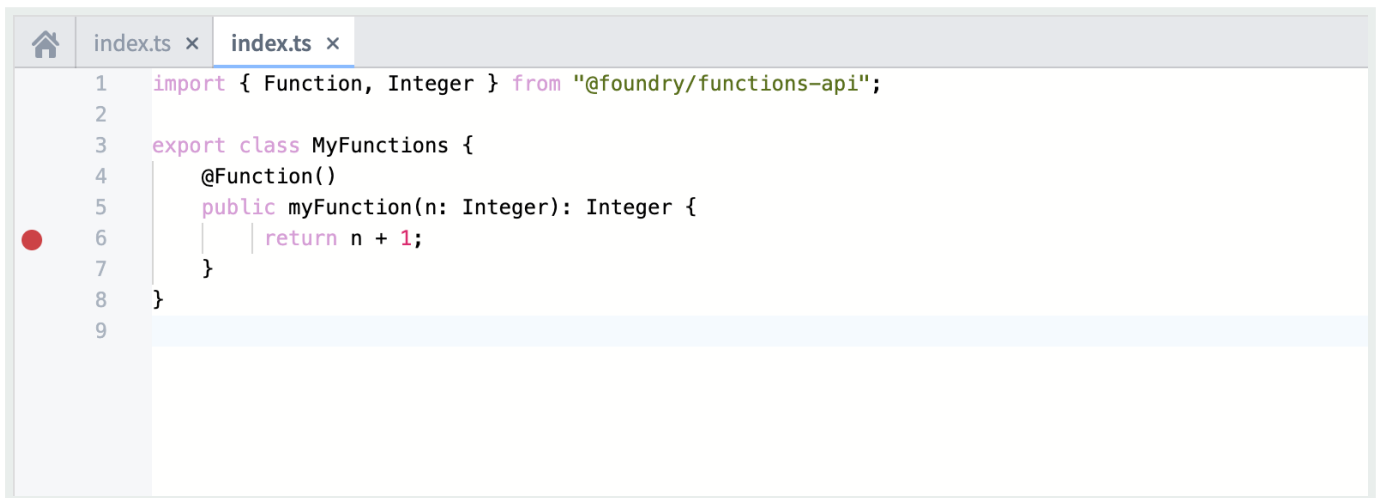
[API Reference ↗](#)

Set breakpoints

[Send feedback](#)

To use the debugger, you need to set breakpoints. These breakpoints indicate the specific points where the debugger should pause the code execution, enabling you to interact with variables.

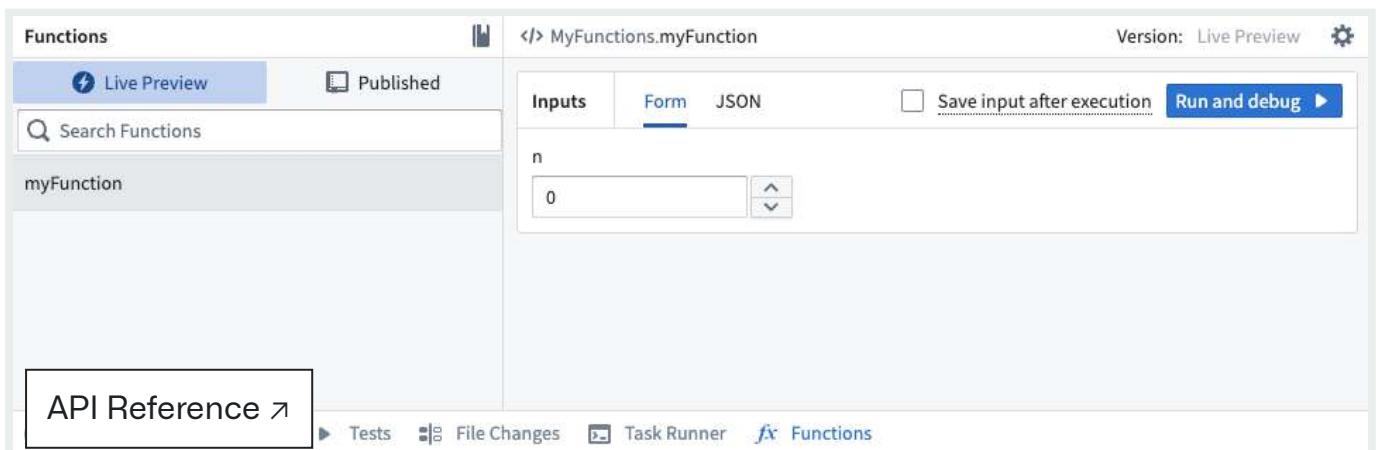
Set a breakpoint by selecting the faded red dot in the margins of each line of code. The debugger suspends the execution *before* the marked line runs. You can set multiple breakpoints across several files, if needed.



Run the debugger

During live preview

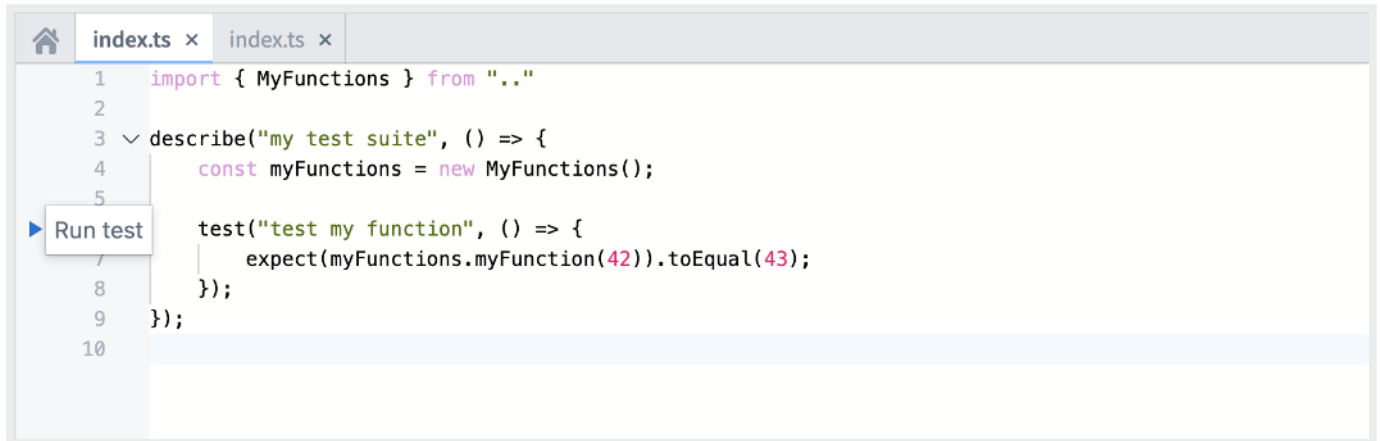
After adding breakpoints in your code, select **Run and debug**, located in the functions panel.



Send feedback

During testing

After adding breakpoints in your code, select **Run test**, located next to the unit test in the code editor.



```
1 import { MyFunctions } from ".."
2
3 describe("my test suite", () => {
4   const myFunctions = new MyFunctions();
5
6   test("test my function", () => {
7     expect(myFunctions.myFunction(42)).toEqual(43);
8   });
9 });
10
```

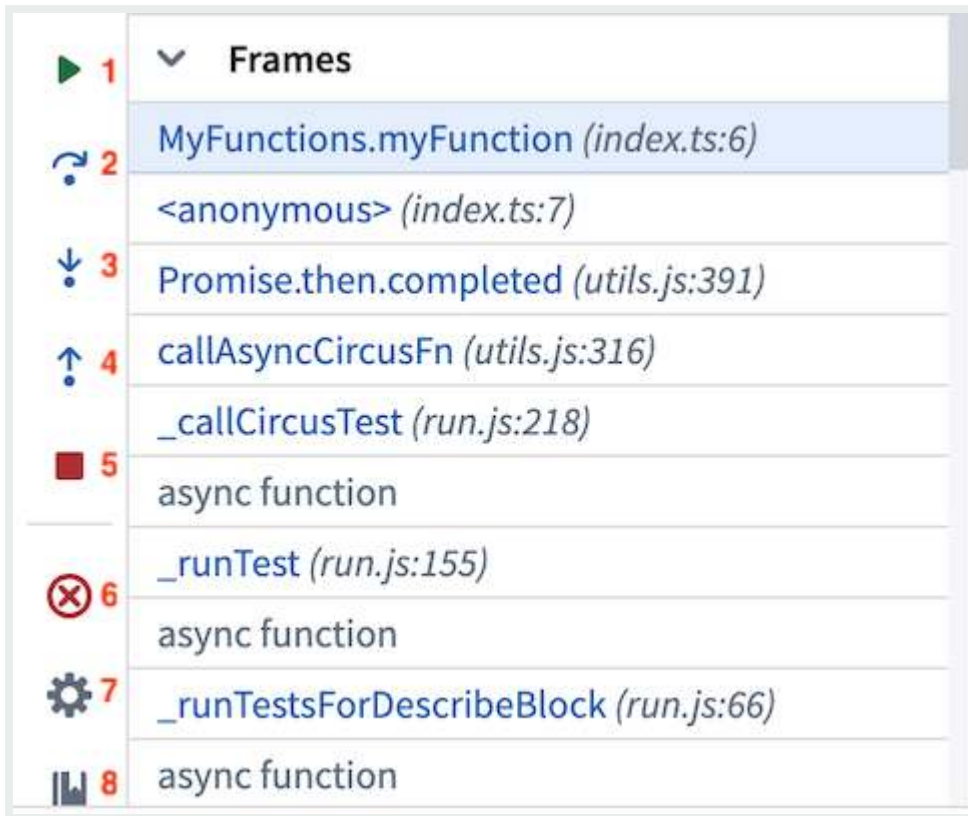
Use the debugger

Once the debugger has started, the debugger panel will open and pause on the first breakpoint it encounters. The left bar of the debugger allows you to navigate the code, remove breakpoints, and finish or stop the debugging session.

As you navigate the code, the editor highlights the line of code to be executed next. Use the following buttons to advance the debugger:

[API Reference ↗](#)

[Send feedback](#)



1. **Resume execution:** Continue execution until completion or until paused by the next breakpoint.
2. **Step over:** Execute the line of code without stepping into internal functions.
3. **Step into:** Navigate into internal functions if they exist in that line of code.
4. **Step out:** Navigate out of an internal function and advance the debugger.
5. **Stop execution:** Stop the debugger completely.
6. **Remove breakpoints:** Remove all breakpoints from the repository and run the unit test without pausing the execution.
7. **Settings:** Toggle the debugger on/off (without clearing the breakpoints).
8. **Documentation:** Open the documentation for additional details.

Examine variables

While the debugger is running, you can examine the variables and data at the exact point of code execution.

Frames represent the functions in which the debugger is active or in which breakpoints exist. Each frame indicates the name of the function followed by the name of the file and the line number in which the function is written.

Select a frame to examine the variables within that frame and run console commands against it.

Variables

The variables section displays the values stored in both local and global variables while the transform is executed.

Variables

```
n: (number) 42
▼ this: (MyFunctions) MyFunctions
  ▼ [[Prototype]]: (Object) Object
    > constructor: (Function) class MyFunc...
    > myFunction: (Function) f myFunction(...)
    > [[Prototype]]: (Object) Object
```

Console

[API Reference ↗](#)

The console allows you to interact with your data using JavaScript console commands while running the debugger.

[Send feedback](#)

i Note that the console operates within the context of the selected frame. Attempting to execute commands on variables local to a different frame will lead to an error.



Console logging

Functions supports emitting console logs during execution for debugging purposes. To do so, simply use the `console.log` command to emit logs. For example:

```
1  @Function()  
2  public testConsoleLogging(n: Integer): Integer {  
3      for (let i = 0; i < n; i++) {  
4          console.log(`Iteration ${i}`);  
5      }  
6      return n;  
7  }
```

Using console logs in this way can be useful for debugging correctness issues. You can also add console logs to identify performance bottlenecks in your code. See the guide for [optimizing performance](#) for more information on how to improve the performance of link traversal logic.

During testing

When using the **Tests** helper in **Authoring**, console logs will be captured and displayed below:

Send feedback

Tests

index.ts

my test suite test my function

console.log
Hello world

at Object.<anonymous> (src/__tests__/index.ts:7:17)

console.log
hello

at MyFunctions.myFunction (src/index.ts:12:18)

console.log
This is the value 43

Problems

Debugger

Tests

File Changes

Task Runner

Functions

During live preview

When you run a function using the **Functions** helper in **Authoring**, console logs will be captured and displayed below, along with timestamps:

MyFunctions.testConsoleLogging functions-typescript/src/index.ts

Output

Loaded in 1154ms.

RESULT

5

LOGS

LOG [2020-03-26T18:50:01.837Z] Iteration 0

LOG [2020-03-26T18:50:01.837Z] Iteration 1

LOG [2020-03-26T18:50:01.837Z] Iteration 2

LOG [2020-03-26T18:50:01.837Z] Iteration 3

LOG [2020-03-26T18:50:01.837Z] Iteration 4

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / TypeScript v1 / Import resources into Code Repositories

Import resources into Code Repositories

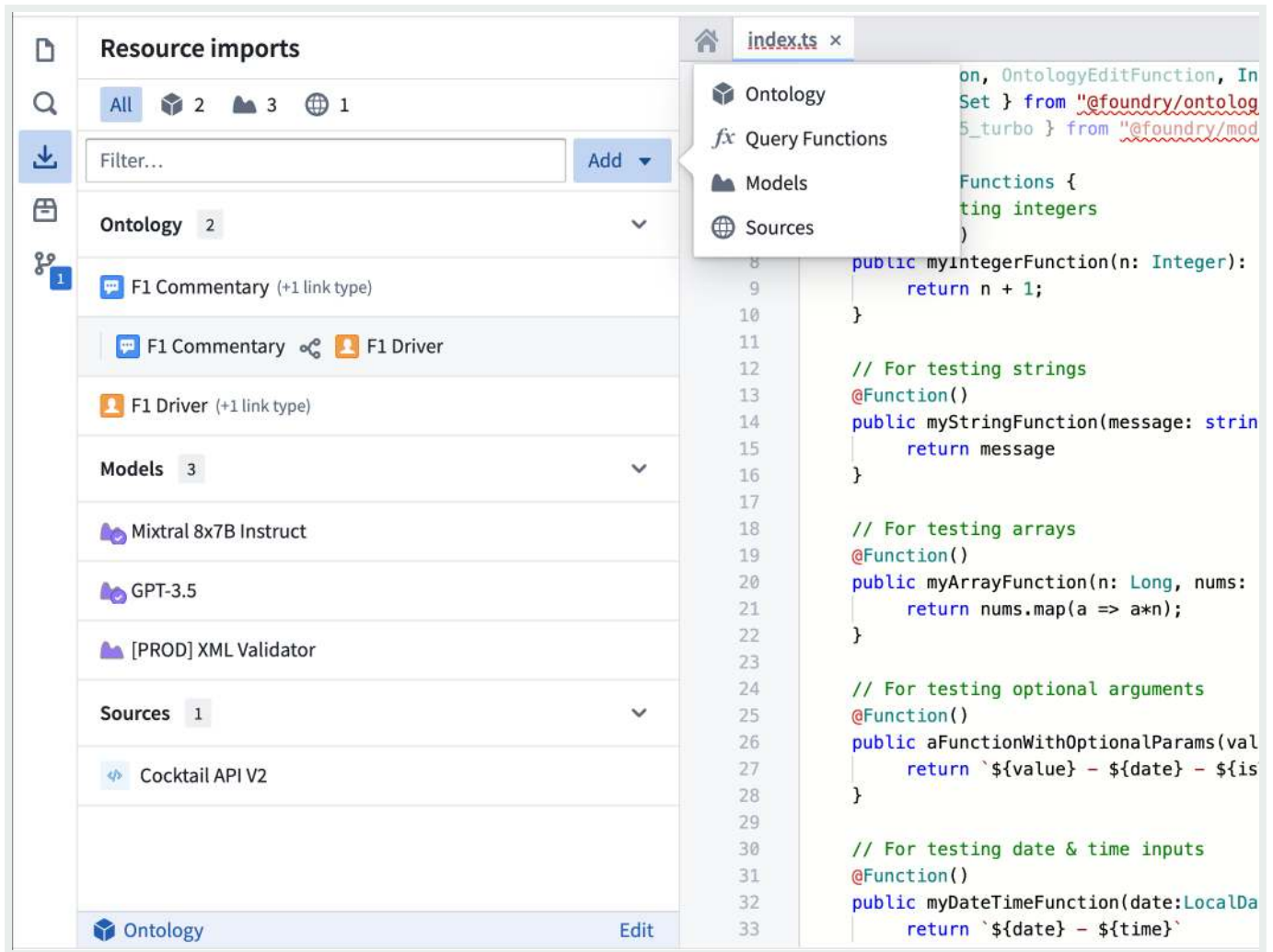
⚠ The following documentation is specific to TypeScript v1 functions. For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

The **Resource imports** sidebar in Code Repositories offers a centralized interface to manage imported Foundry resources within your TypeScript functions repository. The sidebar allows you to import, remove, and view details of various resources, including Ontology types, LMS language models, live deployments, and external systems such as REST APIs.

API Reference ↗



Send feedback



Select an Ontology

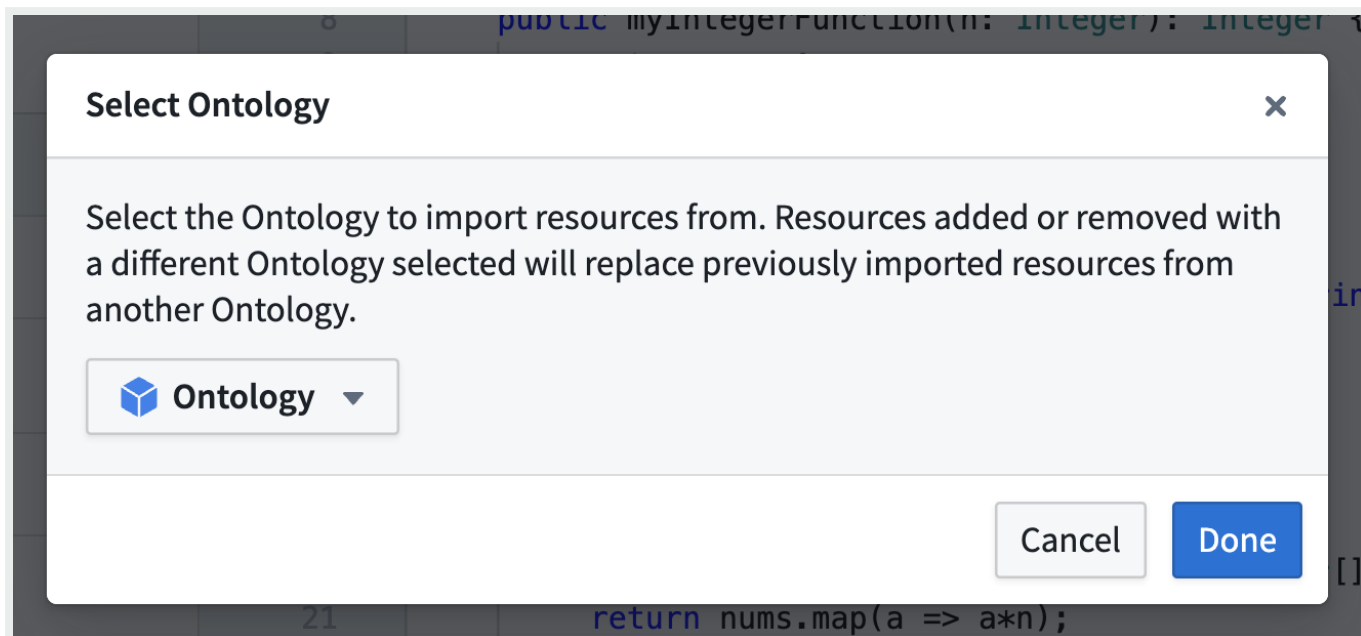
An Ontology is required to import object and link types. To choose an Ontology:

1. Choose **Add** to open the resource selector menu, and then choose **Ontology** to begin importing Ontology types. If no Ontology is selected, this will automatically open the Ontology selector dialog.

If you have already imported at least one Ontology type, that type's Ontology is automatically selected. To change the Ontology, choose the **Edit** button next to the selected Ontology's name to open the Ontology selector dialog.

[API Reference ↗](#)

[Send feedback](#)



All imported resources within your repository must be associated with the same Ontology. Note that importing resources after changing the Ontology will overwrite any existing imports from other Ontologies.

Import resources

⚠ Modern versions of the TypeScript v1 template maintain the current state of repository imports in a `resources.json` file checked into your repository. If you encounter warnings about an unresolvable file in the sidebar, see the [file-based ontology imports](#) section for information about the expected file format and troubleshooting steps for resolving the issue.

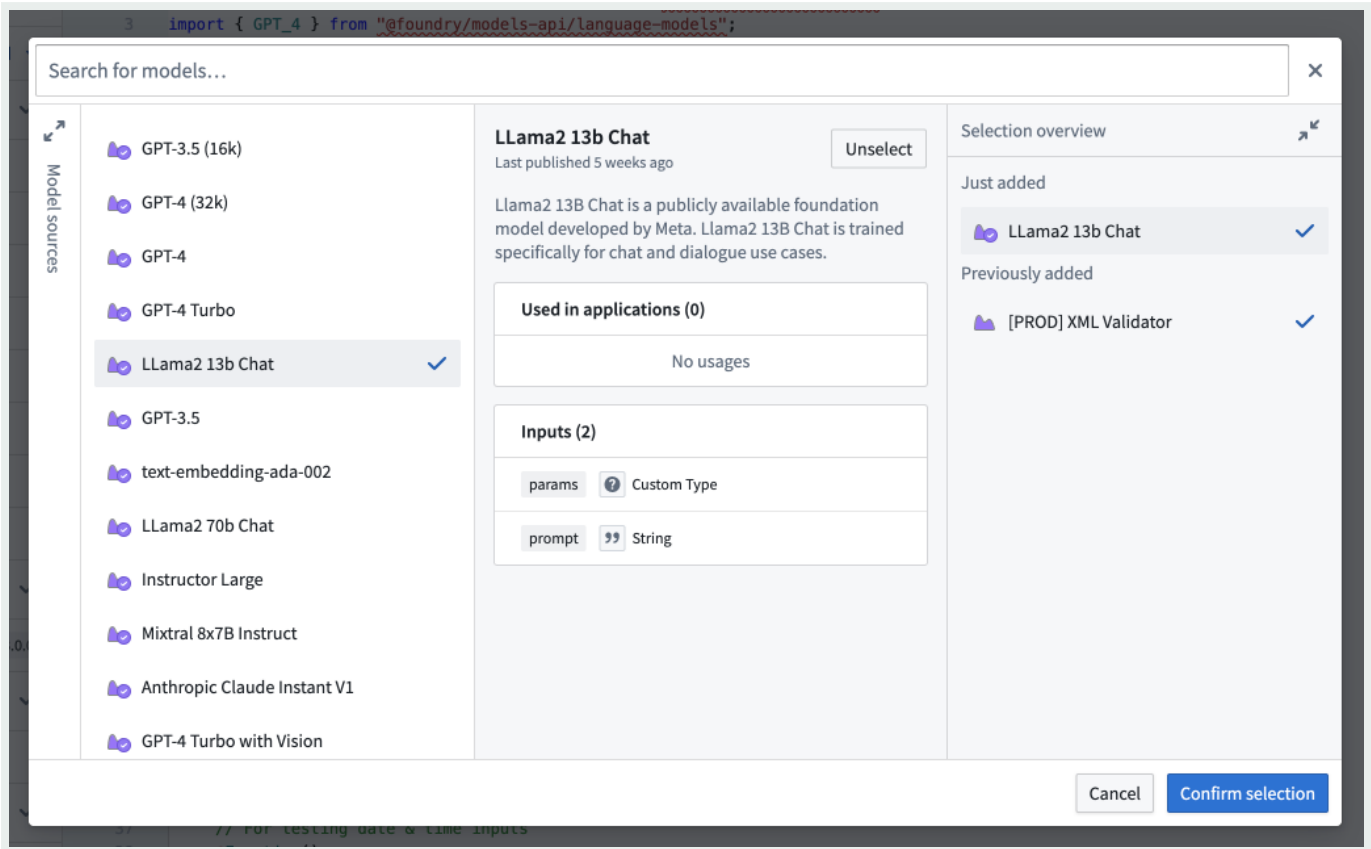
To import resources using the sidebar:

1. Use the **Add** button in the top right of the sidebar and select the desired resource type. This will open the selector dialog for that resource.
2. Use the search bar and filters to locate the resources you want to import.
3. Choose a resource to display its preview panel with detailed information.

4. Click the **Confirm** button to add resources to your selection. The **API Reference** panel to review your selection and confirm by choosing **Confirm selection**.

Send feedback

After confirming your selection, Code Assist will be restarted to re-run the necessary code generation tasks to apply your changes.



Learn more about importing resources of a specific type:

- [Ontology types](#)
- [Language models](#)
- [Live deployments](#)
- [External sources](#)

Manage imported resources

Resources are categorized by type in the sidebar:

- Ontology: Object, interface, and link types
- Models: LMS models and live deployments
- API Reference ↗ systems such as REST APIs

Choose the corresponding resource icon at the top of the sidebar to

Send feedback

 text input to search by name. To remove a resource, hover over the resource icon and

choose the **Remove** button. To add or remove multiple resources simultaneously, use a selector dialog. To view more details, select an imported resource to open its preview panel.

Some resource types may have dependencies between other resources. For instance, link types are organized under their respective object types. If an imported resource has dependencies, a message like "(1 link type)" will be displayed next to the resource title. To view a resource's dependencies, hover over the resource icon and select the chevron that appears.

The screenshot shows the 'Resource imports' panel on the left and a code editor on the right. The 'Resource imports' panel has a header 'Resource imports' and a sub-header 'All' with counts for different resource types: 2 cubes, 3 folders, and 1 globe. Below this is a 'Filter...' input field and an 'Add' button. A section titled 'Ontology' shows two items: 'F1 Commentary (+1 link type)' and 'F1 Driver (+1 link type)'. The code editor on the right shows a file named 'index.ts' with the following code:

```
1 import { Function, Ontology } from '...'
2 import { ObjectSet } from '...'
3 import { GPT_3_5_turbo } from '...'
4
5 export class MyFunctions {
6   // For testing integers
7   @Function()
8   public myIntegerFunction(n: number): number {
9     return n + 1;
10  }
11
12   // For testing strings
13   @Function()
14   public myStringFunction(s: string): string {
```

Importing resources without API names

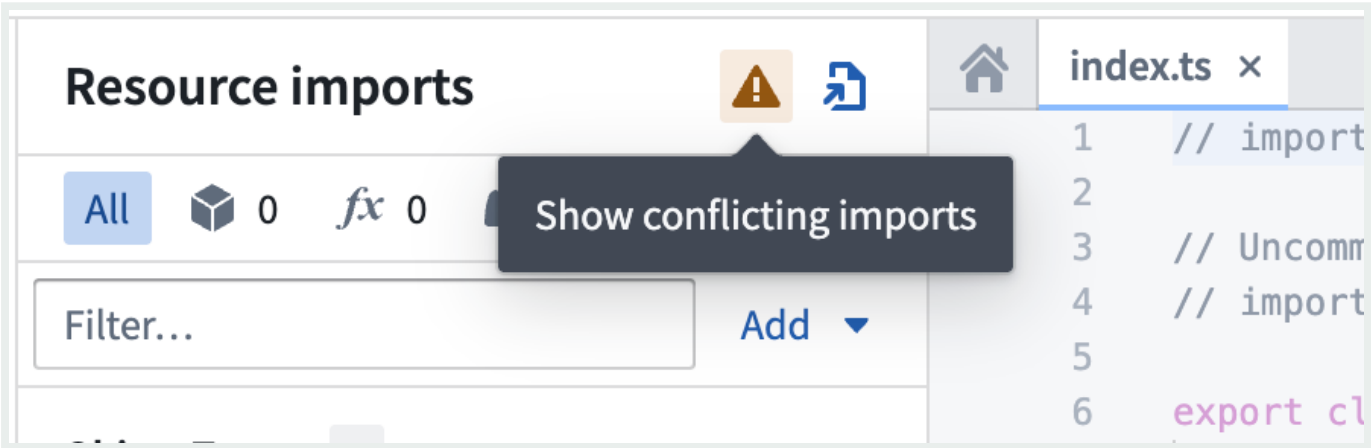
Resources must have an API name to be referenced within code in TypeScript functions repositories. If a resource lacks an API name, a warning is displayed. Hover over the warning sign to learn more or easily configure an API name by choosing **Add API name**. Alternatively, choose **Learn more** to see documentation about adding an API name tailored to the specific resource type.

The screenshot shows a list of resources. The first resource is '[PROD] XML Validator'. The second resource is 't5-small-pytorch (Wed, Jan 31, ...)' with a warning icon. A dialog box is open over the warning icon, containing the text: 'This resource has no API name. An API name is required to use this resource in your repository.' Below the text are two buttons: 'Learn more' and 'Add API name'. At the bottom right of the screenshot is a 'Send feedback' button.

Import resources with value type dependencies

Some resources depend on [value types](#) to define the datatypes used to interact with them, for example, function interfaces. For these resources, their value type dependencies are imported into the repository automatically so that they are available to use along with the resource.

In some cases, importing a combination of such resources can result in a value type dependency conflict. This occurs when different resources have a common value type they depend on at differing versions. It is not possible to have both versions of the same value type imported, and this causes a compilation error. This error is accompanied by a warning in the sidebar, allowing you to view the resources with conflicting dependencies.



File-based repository imports

Modern versions of the TypeScript v1 template maintain the current state of repository imports using a `resources.json` file checked into your repository. This gives you full Git semantics, allowing you to review, branch, and revert changes to your imports. The resource import sidebar helps you update this file by automatically inserting entries into the `resources.json` file.

If the `resources.json` file is in an invalid state, a warning will appear in the sidebar informing you that the file cannot be processed. If you encounter this error, ensure that your file is a valid JSON object with the following data:

API Reference ↗

Field	Type	Send feedback
objectTypes	Array of { rid: string }	

Field	Type
linkTypes	Array of { rid: string }
sources	Array of { rid: string }
functions	Array of { rid: string, version: string }
valueTypes	Array of { rid: string, version: string }
functionInterfaces	Array of { rid: string, version: string }
_comment	String
version [1]	Integer

[1] The `version` field is used to express the version and format of the `resources.json` file. Currently only `version: 1` is supported. `version: 0` is used to indicate that your repository must undergo a migration from the prior, repository-wide imports workflow. This migration is handled automatically with a patch applied to your repository when a commit with `version: 0` is made.

Note that because the `resources.json` file is checked into your repository, you can view the commit history and use the history to revert the file back to a working state.

Enable resource types

By default, some resource types may not be enabled for use in your repository. The enabled resource types are determined by your `functions.json` file. This is the contents of a typical default `functions.json` file.

```

1 {
2   "useOntologyApiNames" : true,
3   "enableModelFunctions" : false,
4   "enableModelGraphFunctions" : false,
5   "enableDiscoverImproperOntologyAccess": false,
6   "enableQueries": false,
7   "enableModelMetadata": false,
8   "useDeploymentApiNames": true,
9   "enableActorProperties": true,
10  "enableTimeSeriesProperties": false,
11  "enableExternalSystems": false,

```

API Reference ↗

Send feedback

```
12  "enableMediaReferenceProperties": false
13 }
```

Importing resources without enabling the corresponding flag in your `functions.json` file may cause checks to fail in your repository. To use imported live deployments, set `enableModelFunctions` to true. To use imported sources, set `enableExternalSystems` to true.

PREVIOUS

← [Debug functions](#)

NEXT

[Add NPM dependencies](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / TypeScript v1 / Add NPM dependencies

Add npm dependencies

⚠ The following documentation is specific to TypeScript v1 functions. For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

Functions repositories use [npm](#) for managing dependencies, including packages for generating code based on the Foundry ontology and discovering functions in your code. You can use `npm` to install external dependencies into your repositories, using standard packages for purposes such as manipulating numbers and dates, performing statistical calculations, or working with data formats such as XML.

Note that the functions runtime only supports pure JavaScript libraries—any package that relies on a NodeJS runtime and makes system calls is not supported.

Enable fetching dependencies from the public npm registry

By default, functions repositories do not fetch packages from the public npm registry.

If your repository does not already fetch dependencies from the public npm registry, a banner for enabling it will appear when you open a `package.json` file in Code Repositories.

API Reference ↗

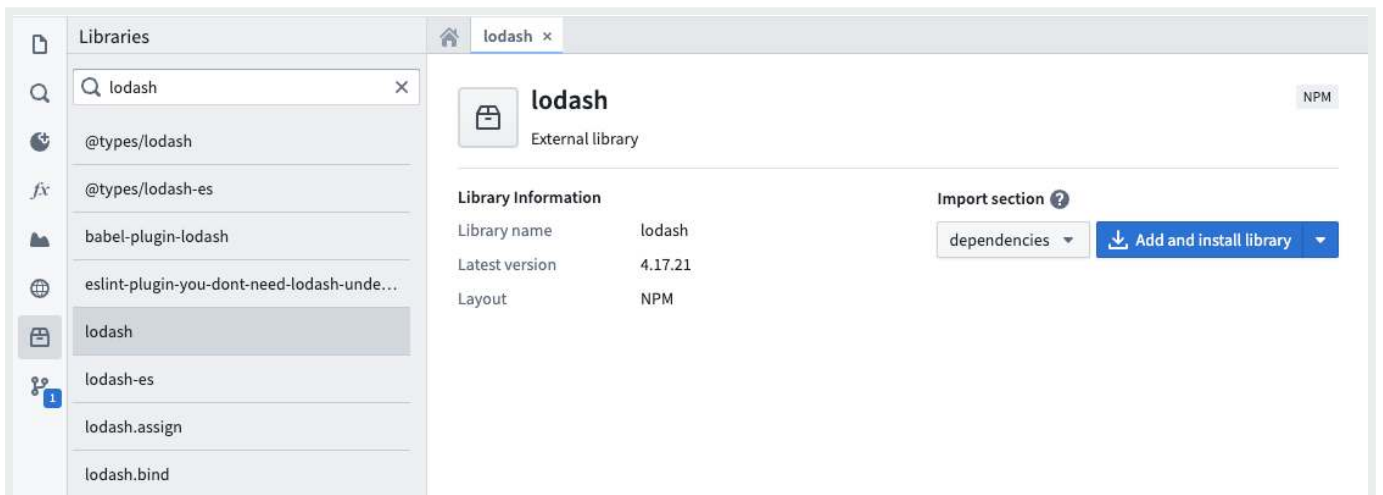


Send feedback


```
package.json x
Enable fetching dependencies from public npm registry.
1 {
2   "name": "my-functions",
3   "version": "0.0.0",
4   "description": "My useful functions",
5   "main": "dist/index.js",
6   "types": "dist/index.d.ts",
```

Add dependencies in Code Repositories

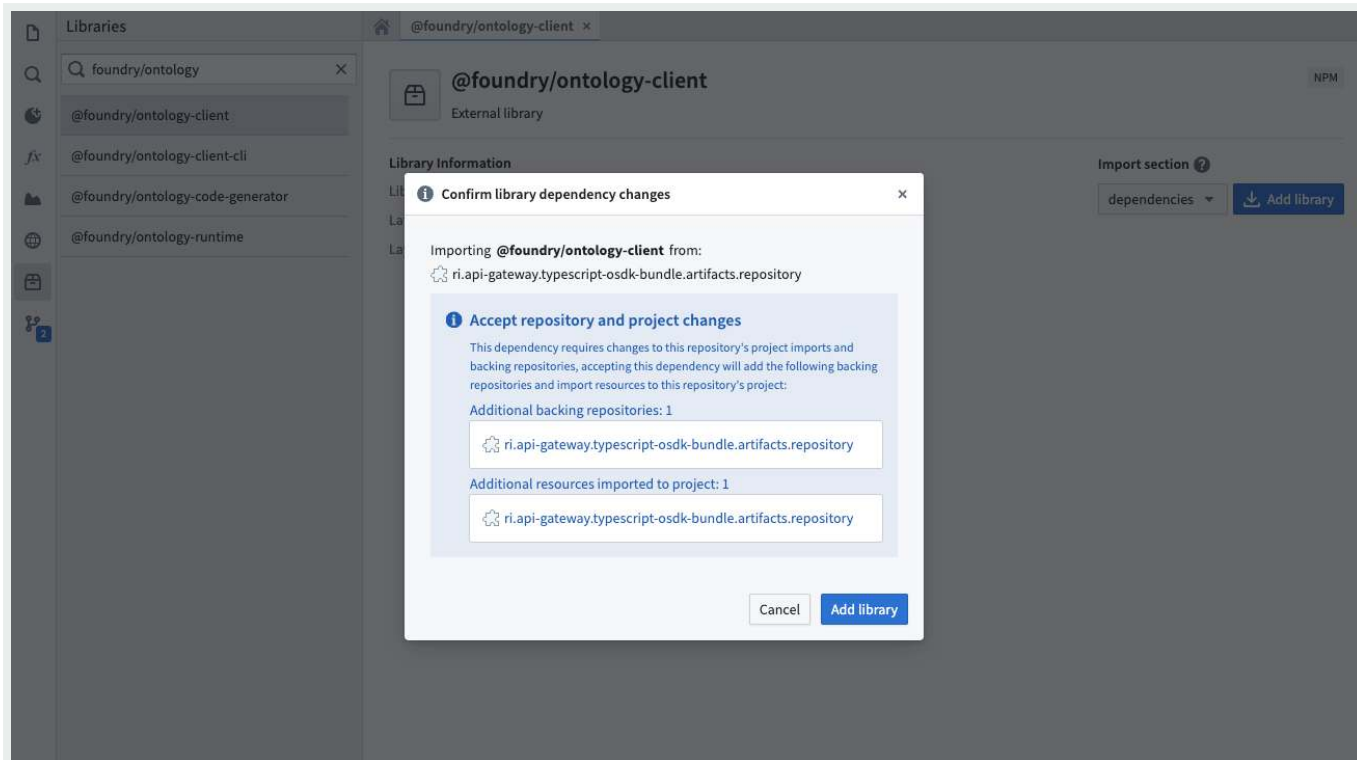
You can add packages to your functions repository using the Libraries sidebar in **Code Repositories**. Search for the desired package, and select a result to view details like the latest version. Results include packages from Foundry and <https://npmjs.com>.



Choose whether to add the package to `dependencies` or `devDependencies` in your `package.json` file. Select **Add and install library** to add the package to your repository.

API Reference ↗

Send feedback



If the package's originating repository is not yet configured as a backing repository, a dialog will prompt you to import additional resources. The **Add and install library** button automatically imports the package and its dependencies into your functions repository, updating your `package.json` and `package-lock.json`.

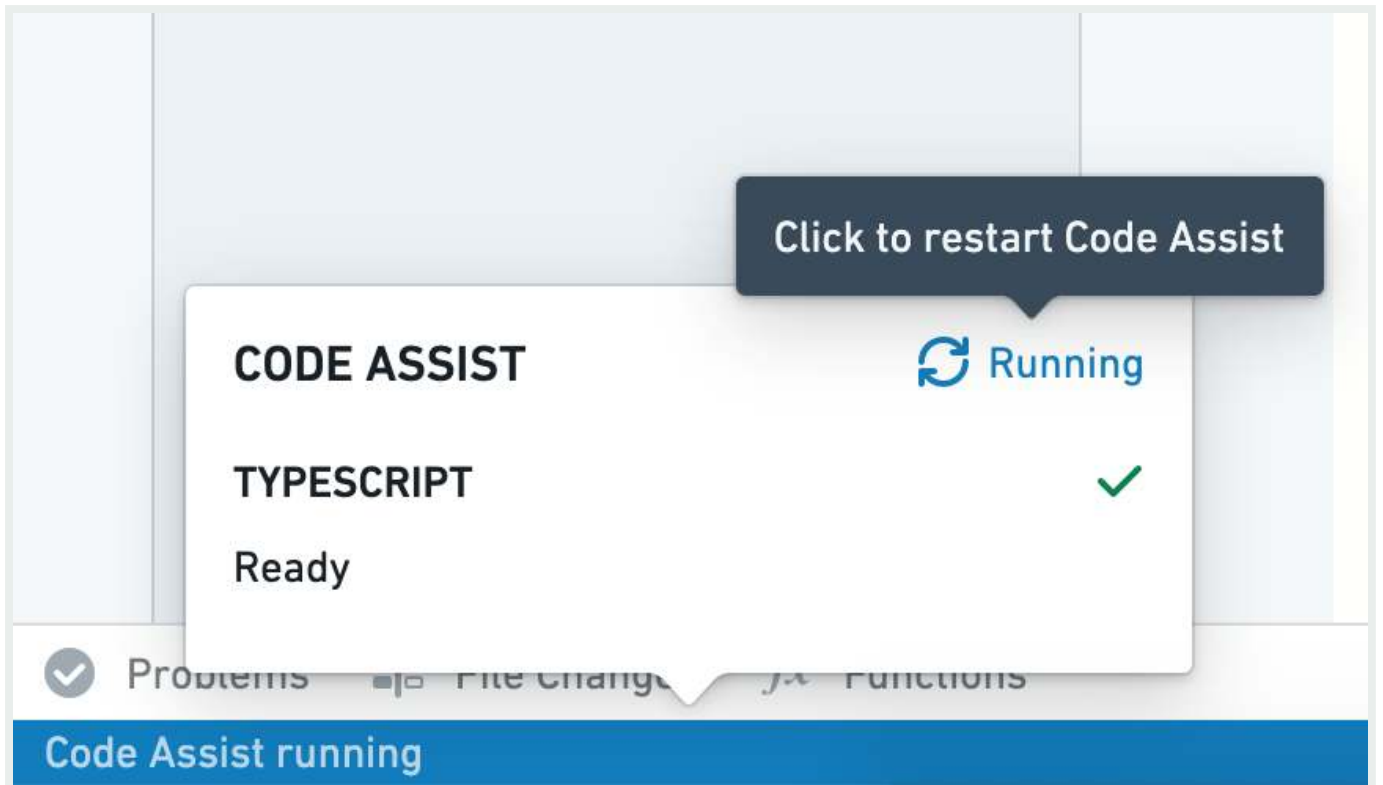
Once the running install tasks have finished, the package will be ready for use within your repository.

If you are using a `typescript-functions` template version lower than 0.520.0, installation through the task runner will be disabled. In this case, commit your updated `package.json` file, ensure checks pass successfully, then restart Code Assist to make the new package available.

Manually add dependencies

You can manually add a package by modifying the `package.json` file in Code Repositories. This can be useful if you need to install a specific package version. Open `package.json`, add your dependency with a relevant version chosen from <https://npmjs.com>, and select **API Reference** . After the checks pass successfully, restart Code Assist to make the new package available.

Send feedback



Below is an example of adding the `d3-array` package manually to the `package.json` file in a repository:

```
1  "dependencies": {
2    ...
3    "d3-array": "^2.3.1"
4  },
5  "devDependencies": {
6    ...
7    "@types/d3-array": "^2.0.0"
8  }
```

PREVIOUS
← Import resources into Code Repositories

Use webhooks
NEXT →

© 2026 Palantir Technologies Inc. All rights reserved.

[API Reference ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[Send feedback](#)

Ontology building / Functions / TypeScript v1 / Use webhooks

Webhooks in functions

⚠ The following documentation is specific to TypeScript v1 functions. For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

i Webhooks can be published as first-class functions. This means you can invoke them from Workshop, the Ontology SDK, Actions, and other functions. To publish a webhook as a function instead, see [Webhook functions](#).

This guide will walk you through setting up a function that can make requests to external systems using [webhooks](#).

i Prerequisites

This guide assumes you have already created a data connection source and a webhook. For more information, see the [documentation how to create a data connection source and webhook](#).

Webhooks backed by outbound applications are supported. For more information, see the [Supported workflows](#) section in the outbound application configuration page.

API Reference ↗ces into a functions repository

B   While following this guide, make sure you already created a function that you can use to understand how to write and publish functions as described in [our tutorial](#).

Send feedback

You must first enable the source to be imported into Code Repositories. To do this, go to the **Enable code imports** menu for your REST API source and enable the option to allow the source to be imported into Code Repositories. Since it is not possible to perform exportable Marking validations in all workflows where functions are used, you must also enable exports to the source without Marking validations in the **Enable exports** menu for each source.

Enable code imports

☒ Allow this source to be imported into code repositories

When a source is imported into a code repository, users with access to the code repository can write custom code using the connection details configured for that source, including accessing secret values.

☐ Allow this source to be imported into compute modules

When a source is imported into a compute module, users with access to the compute module can write and execute arbitrary custom code using the connection details configured for that source, including secret values. Secret values will be available in environment variables as plaintext to the code running in the compute modules.

☐ Allow this source to be imported into pipelines

When a source is imported into a pipeline, the entire pipeline - including any user written code referenced in the pipeline - will be authorized to egress to this source and access secret values.

Enable exports

☐ Enable exports to this source

This will allow people to export datasets from Foundry to this source. By default, data without any markings will be exportable. If you wish to export data with additional [security markings](#), you'll be asked to configure what data is permissible to export to this source. Data marked with an organization will need that organization to be set as exportable.

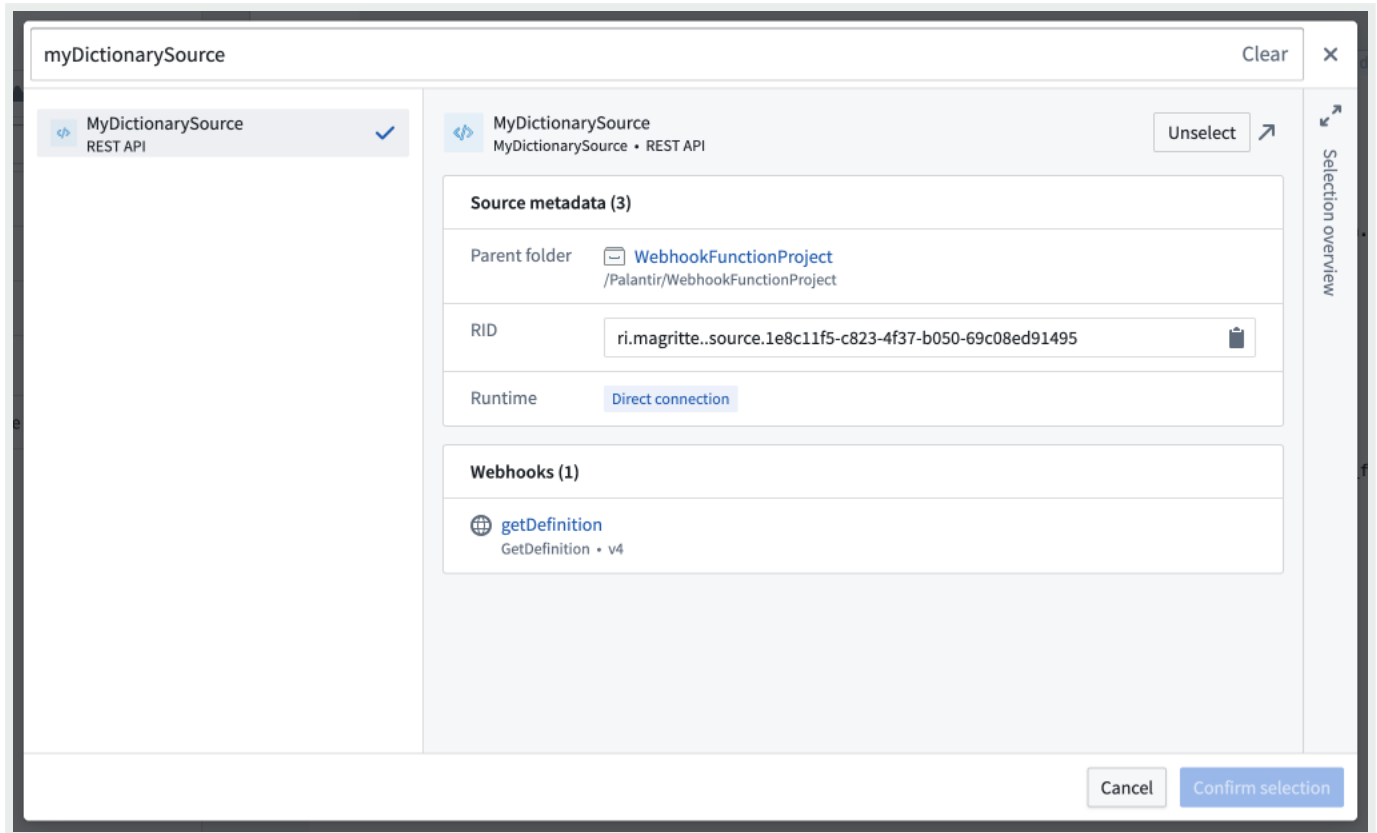
☒ Enable exports to this source without markings validations

In certain interactive workflows the source can be used in, we are currently unable to validate the security markings of the data being exported. By enabling exports without markings validation, you acknowledge that you are responsible for ensuring that the data being exported is compliant with your organization's policies.

[API Reference ↗](#)

Next, to use a webhook in functions, the backing REST API source of [Send feedback](#)  it be imported into the repository. Select the [Resource imports left-side panel](#) to view the

sources imported into the repository. Select **Add > Sources** to display a search dialog where you may select the source you want to import. Only sources with API names may be imported through this dialog.



- i** Source imports into Function repositories for webhook usage work differently than source imports to Python transforms repositories and compute modules. Function repositories that utilize a given source only for webhook usage will *not* be displayed in the list of repositories shown on the source overview. Any user with **Viewer** access to a source will be able to import and use those webhooks in external functions.

Use webhooks in functions

Once you import the REST API source to the functions repository, it will be available in the TypeScript environment and accessible through the namespace of the source:

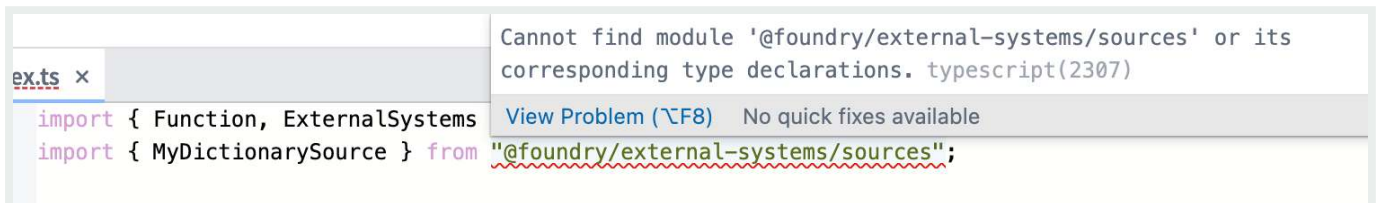
API Reference ↗

```
import { Function } from "@foundry/functions-api";
```

Send feedback

```
2 import { MyDictionarySource } from "@foundry/external-systems/sources";
```

If you get the error `Cannot find module '@foundry/external-systems' or its corresponding type declarations.`, ensure the value for `enableExternalSystems` is set to `true` in the `functions-typescript/functions.json` file. Once you update it and commit the changes, the system should install the necessary packages, including `@foundry/external-systems`.



Example: Make multiple calls from a Function

In the example below, we will explain how to make multiple calls to the dictionary API using a single Function.

If your Function does not make any Ontology edits, you will create a `@Query()` function. If you would like to make Ontology edits, it would instead require the `@OntologyEditFunction` decorator. Learn more about making Ontology edits from functions in our [documentation](#).

Using the standard [TypeScript async/await pattern](#), multiple webhook calls can be made simultaneously from a Function. Check the success of calls using the `isOk` helper function exported from `@foundry/functions-api`.

The following Function accepts a list of words as a TypeScript string array and makes one call for each word:

```
1 import { OntologyEditFunction, isOk } from "@foundry/functions-api";
2 import { MyDictionarySource } from "@foundry/external-systems/sources";
3
4 export class MyFunctions {
5     @OntologyEditFunction()
6     public async defineWords(words: string[]): Promise<void> {
7         const results = await Promise.all(words.map(word => MyDictionarySource.get(word)));
8     }
9 }
```

API Reference ↗

Send feedback


```

10         wordToDefine: word
11     }));
12
13     results.forEach((result, i) => {
14         if (isOk(result)) {
15             const output = result.value.output;
16             output.dictionary_definitions.forEach(definitions_for_word => {
17                 definitions_for_word.meanings.forEach(meaning => {
18                     meaning.definitions.forEach(def_for_part_of_speech => {
19                         console.log(`Found a ${meaning.partOfSpeech} definition for "${wordToDefine}"`);
20                     })
21                 })
22             });
23         }
24     });
25 }
26 }

```

Log output for an input of ["tuba", "cool"]:

```

LOG [2023-07-28T03:16:22.968Z] Found a noun definition for "tuba": A large brass instrument.
LOG [2023-07-28T03:16:22.968Z] Found a noun definition for "tuba": A type of Roman wind instrument.
LOG [2023-07-28T03:16:22.968Z] Found a noun definition for "tuba": A large reed instrument.
LOG [2023-07-28T03:16:22.968Z] Found a noun definition for "tuba": A Malayan plucked string instrument.
LOG [2023-07-28T03:16:22.968Z] Found a noun definition for "tuba": A reddish pale color.
LOG [2023-07-28T03:16:22.968Z] Found a noun definition for "tuba": A tube or tube-like structure.
LOG [2023-07-28T03:16:22.968Z] Found a noun definition for "cool": A moderate or pleasant temperature.
LOG [2023-07-28T03:16:22.968Z] Found a noun definition for "cool": A calm temper

```

Error handling

To help mitigate failures when working with a networked system, functions expose errors propagated from webhooks using Result objects, which give information about the kind of error that occurred:

[API Reference ↗](#)

```

1 import { OntologyEditFunction, isOk } from "@foundry/functions-api";
2 import { MyDictionarySource } from "@foundry/external-dictionary";
3

```

[Send feedback](#)

```

4  export class MyFunctions {
5
6      @OntologyEditFunction()
7      public async defineWords(words: string[]): Promise<void> {
8
9          const results = await Promise.all(words.map(word => MyDictionaryS
10             wordToDefine: word
11         }));
12
13         results.forEach((result, i) => {
14             if (isOk(result)) {
15                 // Extract the response
16             } else {
17                 const errorName = result.error.name;
18
19                 if (errorName === "WebhookExecutionFailedToStart") {
20                     console.log("We were unable to initiate a request to
21                 } else if (errorName === "ParsingResponseFailed") {
22                     console.log("The external request succeeded, but the
23                 } else {
24                     console.log("Something went wrong.");
25                 }
26             }
27         });
28     }
29 }

```

When handling errors, authored code should listen for specific names and react accordingly. Functions currently return the following errors:

Error	Description
WebhookExecutionFailedToStart	The webhook failed to start. If this error is returned, it can be safely assumed that no request was made to the external system.
WebhookExecutionTimedOut	The webhook execution began, but no response was received from the external system within the configured webhook time limit.
API Reference ↗ returnedError	The external system returned an error. Only returned for webhooks configured on a REST API source.

Send feedback

Error	Description
RemoteApiReturnedError	The external system returned an error. Only returned for webhooks configured on a non-REST API source.
ParsingResponseFailed	The webhook execution was successful, but the response from the external system could not be successfully parsed. This can happen if, for example, the response from the external system did not contain an expected field. Since the result of a webhook call will not necessarily be used, it is up to the application builder whether this should be marked as a failure to end users.
ServerError	An internal problem occurred within the webhooks service or the connector.
UnknownError	An error occurred which could not be directly attributed to any Foundry service.

This list of error types may change; users should structure their code to include a default case in the event that the Function executor returns an error with a new name.

Example: Handle errors when making multiple webhook calls from a single Function

The following code describes how to handle multiple webhook calls where some succeed and some fail within the same function. In our example, a `RemoteRestApiReturnedError` is returned in the event that the dictionary server cannot find the definition for a given word.

```

1 import { OntologyEditFunction, isOk } from "@foundry/functions-api";
2 import { MyDictionarySource } from "@foundry/external-systems/sources";
3
4 export class MyFunctions {
5
6     @OntologyEditFunction()
7     async defineWords(words: string[]): Promise<void> {
8
9         const results = await Promise.all(words.map(word => {
10             wordToDefine: word

```

[API Reference ↗](#)
[Send feedback](#)

```

11         }));
12
13         results.forEach((result, i) => {
14             if (isOk(result)) {
15                 const output = result.value.output;
16                 output.dictionary_definitions.forEach(definitions_for_word => {
17                     definitions_for_word.meanings.forEach(meaning => {
18                         meaning.definitions.forEach(def_for_part_of_speech => {
19                             console.log(`Found a ${meaning.partOfSpeech} definition for "${words[i]}"`);
20                         })
21                     })
22                 });
23             } else {
24                 if (result.error.name === "RemoteRestApiReturnedError") {
25                     console.log(`ERROR: ${words[i]} could not be defined`);
26                 }
27             }
28         });
29     }
30 }

```

Inputting `["asdf", "shire"]` to the above Function returns the following result:

```

LOG [2023-07-28T15:38:47.263Z] ERROR: asdf could not be defined Request returned 404
LOG [2023-07-28T15:38:47.264Z] Found a noun definition for "shire": Physical area of land
LOG [2023-07-28T15:38:47.264Z] Found a noun definition for "shire": Former administrative district
LOG [2023-07-28T15:38:47.264Z] Found a noun definition for "shire": The general area of a town
LOG [2023-07-28T15:38:47.264Z] Found a noun definition for "shire": A rural or country district
LOG [2023-07-28T15:38:47.264Z] Found a noun definition for "shire": A shire horse
LOG [2023-07-28T15:38:47.264Z] Found a verb definition for "shire": To (re)construct

```

Limitations

Currently, there are no limits to the number of requests that can be made from within an Ontology edit function, but existing [functions resource limits](#) still apply. [Webhook limits](#) are

a [API Reference ↗](#)

Functions currently support webhooks with the following input and output:

[Send feedback](#)

- Attachments
- Booleans
- Integers
- Longs
- Doubles
- Strings
- Optionals
- Dates
- Timestamps
- Lists
- Enums (list of allowed String type values)
- Records with and without expected fields

When calling webhooks from a `@Query` function, the webhook must perform only `Read API` calls that do not mutate the external system. Query functions are frequently retried or silently executed on pageload, and thus do not provide the same level of structured deliberate execution that is possible with an `@OntologyEditFunction`. When configuring a webhook, you can specify whether it is safe to execute from a Query function by using the option for `Read API` or `Write API`.

Unsupported webhook features

- Webhooks that use the `OR` type as an input or output parameter are currently not supported. No code will be generated for those webhooks.

Handle version changes in functions and webhooks

Functions and webhooks have versions, and callers may invoke any version of a Function or webhook. When a Function is published, the most recent webhook version available at that time will be pinned to it.

When a functions repository is opened in the [Code Repositories](#) application, the generated code for autocomplete will always use the most recent version of the webhook. This webhook version is displayed in the **Resource imports** side panel to the left.

API Reference ↗

Send feedback

📄

🔍

⬇️

📁

🔗

← Back

🌐

getDefinition

Accepts a single English word as a parameter and returns a list of definitions.

API Name

GetDefinition

📋

Source API Name

MyDictionarySource

📋

Input Parameters (1)

🔗 wordToDefine

String

Output Parameters (1)

☰ dictionary_definitions

List<...>

Version

v4

Safety

Read-only ⓘ

Created

Jul 26, 2023, 5:48 AM by Test User

Updated

Jul 28, 2023, 4:08 AM by Test User

[← Back](#)

getDefinition

API Name

Source API Name

Input Parameters (1)

String

Output Parameters (1)

List<...>

Version

Safety

Created

Updated



Make sure your webhook is stable before publishing functions that rely on its functionality.

API Reference ↗

Send feedback

Action	User	Permission required
Import webhook to a functions repository	Function editor	<code>webhooks:editor</code> on the webhook, which is granted to the Editor default role.
Publish Function invoking a webhook	Function editor	<code>webhooks:execute</code> permission on the source, which is only granted to Owner and Editor default roles.
Configure an Action to use an <code>@OntologyEditFunction()</code> that calls webhooks	Action editor	<code>webhooks:grant-action-validated-execution</code> permission on the webhook and <code>Viewer</code> permission on the Function
Execute a <code>@Query()</code> webhook from Workshop	End user	<code>webhooks:execute</code> permission on the source, which is only granted to <code>Owner</code> and <code>Editor</code> default roles.
Execute an <code>@OntologyEditFunction()</code> from an Action	End user	The user must meet the submission criteria for the Action. No permissions on the source, webhook, or Function are checked in this case. Users creating and managing Actions must ensure that submission criteria are configured appropriately.

Monitoring, troubleshooting, and debugging

Use the following platform tools to gain more insight into webhook executions from functions:

- [Webhook execution history](#), which is available in the **History** tab when viewing a single webhook in [Data Connection](#).
- The Function usage history, available in [Ontology Manager](#), shows a history of when functions were executed including the inputs, outputs, and user that triggered the Function.

API Reference ↗ [review for functions](#), which provides performance profiling, debug output, and more.

Send feedback

Best practices

We recommend the following best practices when using external sources to call webhooks from functions:

- Thoroughly test webhooks with the [test webhook side panel](#) in Data Connection before attempting to use those webhooks in functions.
- Use webhook input and output `record` type parameters with expected fields, if possible. Using explicit types, rather than JSON, means that Function code is less likely to throw unexpected runtime errors.
- Use the `isOk` and `isErr` built-in functions exported from `@foundry/functions-api` to check for success and error states, and narrow down the type of error through the name field.
- If users will be writing to both an external system and the Ontology from a single Function call, remember that the write to the Ontology could fail, even if the write to the external system succeeds. Be sure that measures are in place to deal with such inconsistencies, and grant users visibility into the state of their modification to both systems if needed.

PREVIOUS

← [Add NPM dependencies](#)

NEXT

[Functions on objects / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Functions on objects (FOO)

Functions in Foundry natively support accessing and modifying data from objects and links in the Ontology. After defining object and link types in the Ontology, you can import these types into a functions repository to have code bindings automatically generated. These code bindings include support for:

- Passing object and object set types into a function as [parameters](#)
- Searching for object sets on demand using the [Object set APIs](#)
- Modifying objects using [OntologyEditFunctions](#)

Because of this native support for the Ontology, functions in Foundry go far beyond commonly used Functions-as-a-Service (FaaS) platforms by providing native support for data storage, retrieval, and modification—all subject to Foundry's guarantees for data security, lineage, and transparency.

[Learn how to get started with functions on objects.](#)

i The term "functions on objects" (sometimes referred to as "FOO") is used loosely to refer to functions that read object data, either as a parameter or using an object search, but there is no formal notion of a "function on objects" in Foundry as being distinct from any other function.

PREVIOUS

← TypeScript 1 / Use webhooks

API Reference ↗

NEXT

Getting started →



[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Functions on objects / Getting started

Getting started with functions on objects

One of core features of functions is they can easily access data that has been integrated into the Foundry Ontology. The Ontology provides semantic modeling of data for your organization, which makes it easy to access structured data and reuse logic across use cases.

Prerequisites

This tutorial assumes that you have created and set up a TypeScript repository. If you haven't yet, complete the [Getting started](#) tutorial first.

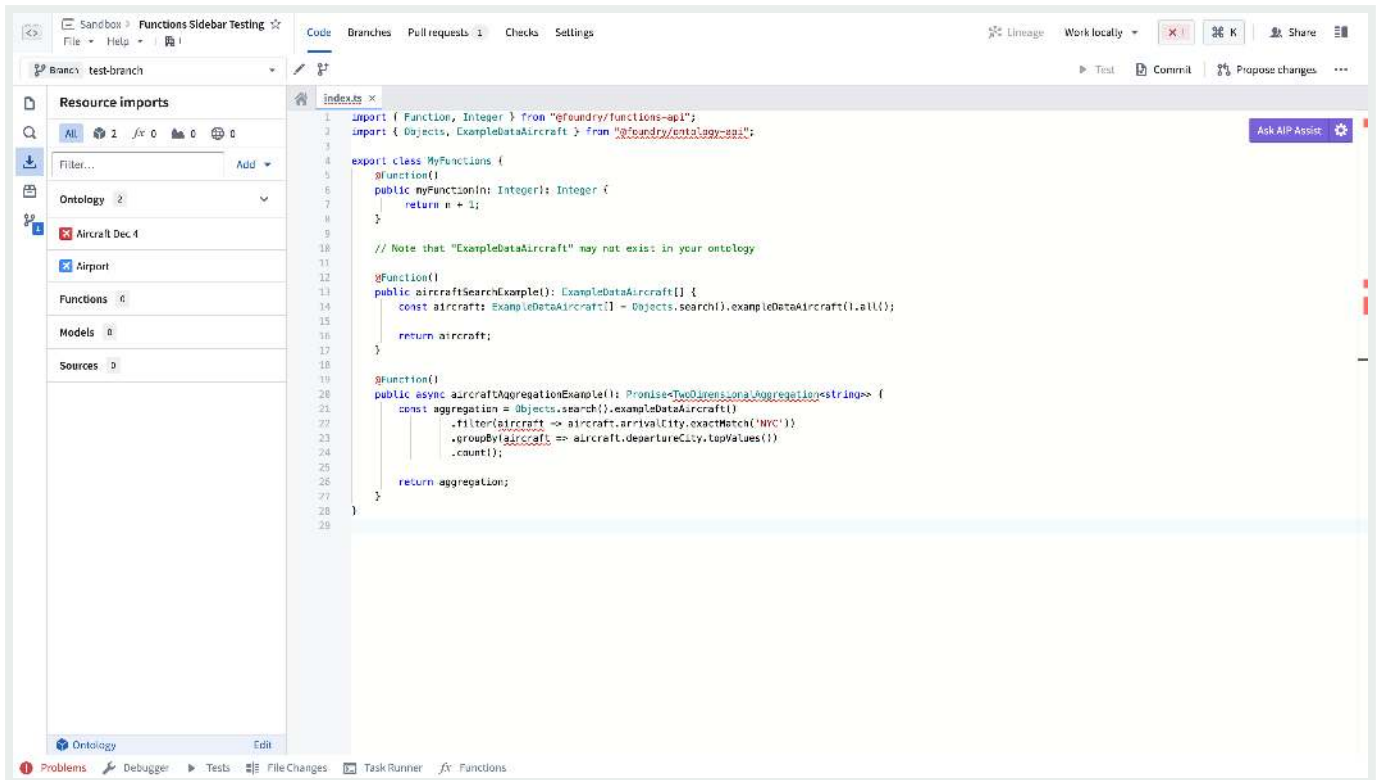
Import Ontology types

Any object, interface, or link types you want to use in your function must be imported into the Project that contains your repository. Selecting the **Resource Imports** side bar shows you the object types which have been imported into the Project.

API Reference ↗



Send feedback



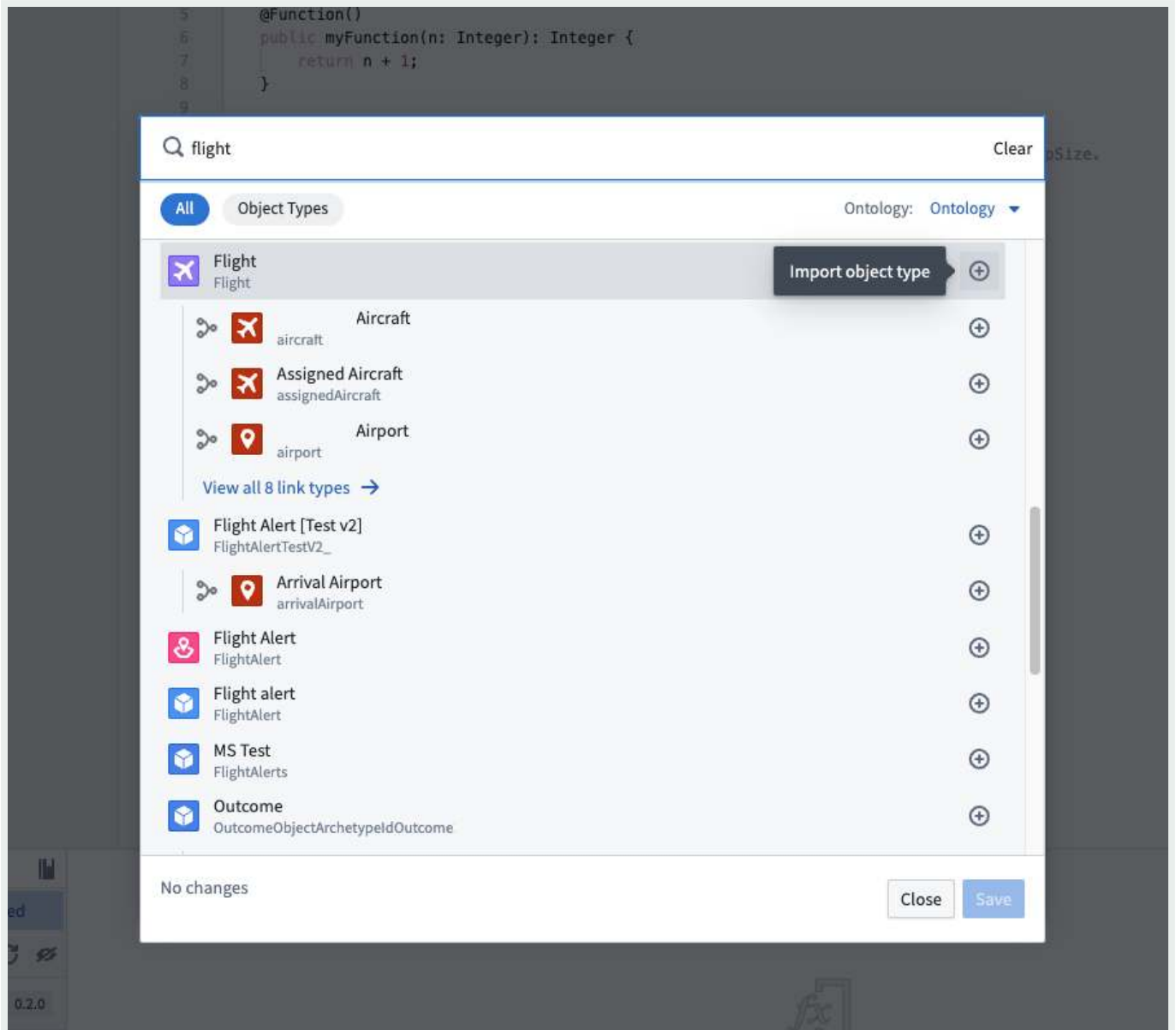
i Your organization may not have the Airport and Flight objects. Use any object types you have access to when following along.

To import additional object types, you will need to select the **Add** button in the **Resource Imports** side bar. If no Ontology has been selected you will be prompted to select an Ontology. If you have at least one imported Ontology type, the selected Ontology will automatically be resolved.

Once an Ontology is selected, a search modal will appear. Your Ontology will depend on the object types available in your organization. Start by selecting a few object types and link types that connect them. In this example, we'll import the Airport and Flight objects, in addition to the link type between them.

[API Reference ↗](#)

[Send feedback](#)



Choose **Confirm selection** to import the Ontology types into the project. Code Assist will automatically be restarted to regenerate code bindings to reflect the new object and link types you have imported.

In your code, you may now import Ontology types from the `@foundry/ontology-api` package. If you are using a private Ontology, the package name will instead be `@foundry/ontology-api/<ontology-api-name>`.

i Private Ontologies

API Reference ↗

For a private Ontology, replace `@foundry/ontology-api` with `foundry-ontology-api/your-private-ontology-api-name-here` in all the following examples.

Send feedback

Add an object-backed function

Next, let's write a function using an object type you just imported. Your code will depend on the object types, properties, and link types available to you. Switch back to the **Code** tab, and try importing one of the object types you just added:

```
1 import { Airport } from "@foundry/ontology-api";
```

Then, write a function that takes that object as input:

```
1 @Function()
2 public myObjectFunction(airport: Airport) {
3     airport.
4 }
```

Once Code Assist has started, simply type `airport.` to see autocomplete for the properties and link types available to you:



In this example, we use a [template string](#) to combine the `city` and `country` fields on an Airport object into a human-readable location:

API Reference ↗

```
1 @Function()
2 public airportLocation(airport: Airport): string {
```

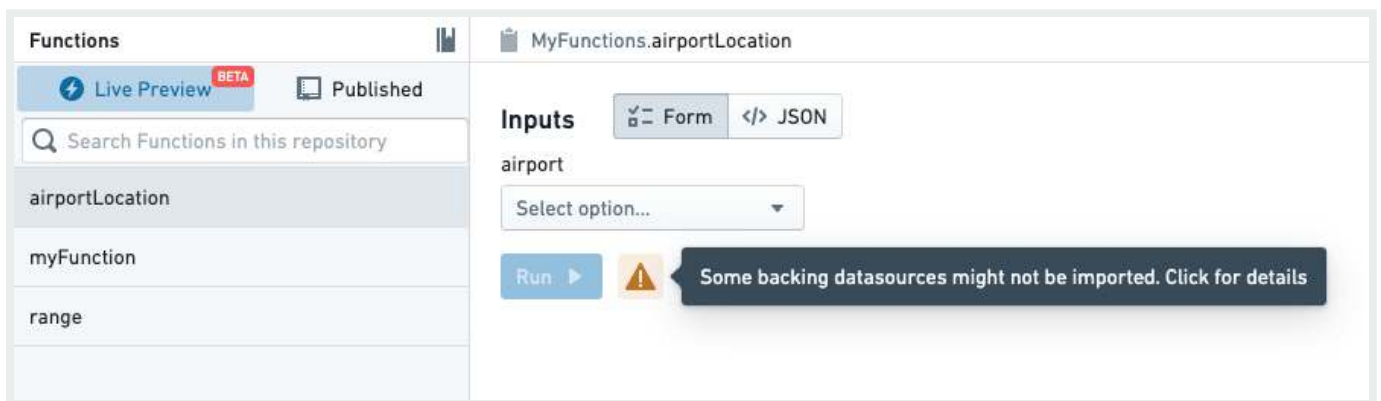
Send feedback

```
3     return `${airport.city}, ${airport.country}`;  
4 }
```

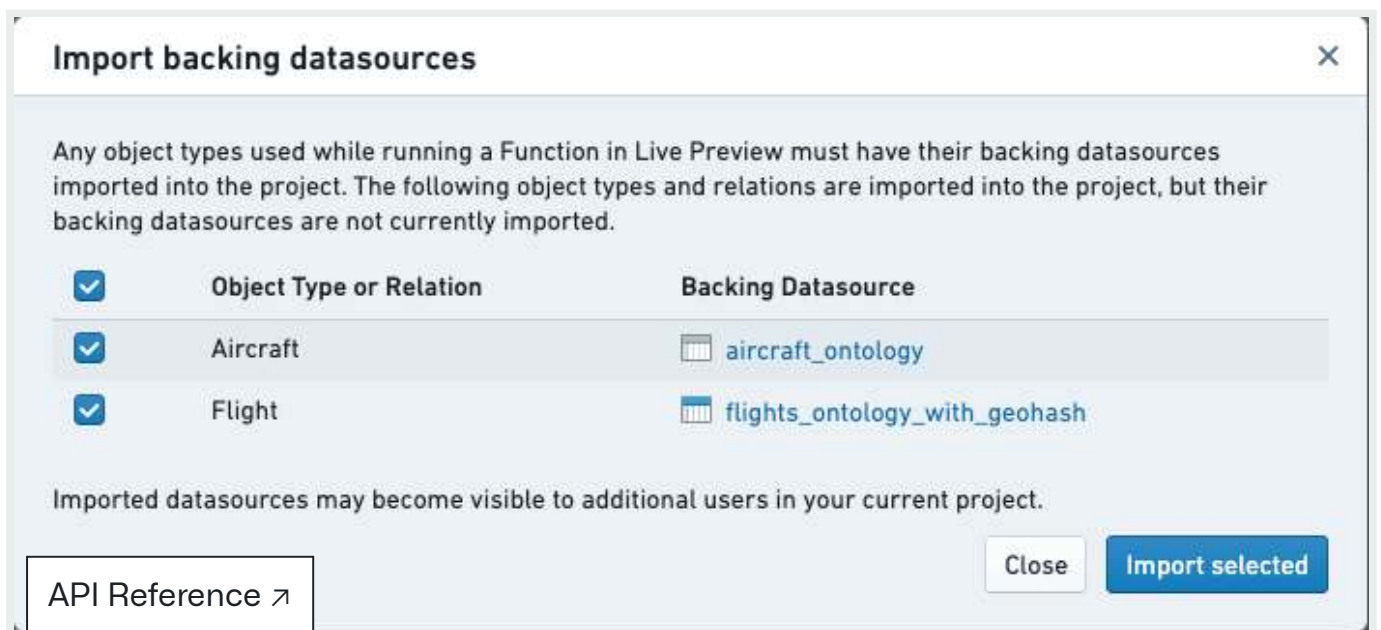
Experiment with the APIs based on your own Ontology and write a function that returns a value based on your object type.

Test in live preview

Open the **functions helper**, toggle to **Live Preview**, and choose the function that you wrote above. To run an object-backed function in live preview, you have to import the backing datasource for the object type. Select the warning icon next to the **Run** option:



Then, use the dialog to import the backing datasources for your object types:



After you've imported the datasources, choose an object and select

Send feedback

Functions

Live Preview

BETA

Published

Search Functions in this repository

airportLocation

myFunction

range

MyFunctions.airportLocation

Inputs

FormJSON

airport

Denver International Airport

Run

Output

Ran in 442ms. Loaded 1 object during execution.

RESULT

Denver, United States

Problems

File Changes

Functions

⚠ Live preview permissions

The permissions on object types in live preview are determined by the [TypeScript repository's permissions on the backing datasources underlying each object type](#). When testing [functions that create notifications](#), the recipients' permissions are not enforced. For this reason, a function that creates a notification may succeed in live preview but fail when used by an action elsewhere in Foundry. Learn more about [configuring notifications for actions](#).

Publish the new function

Publish the new function by committing your code and publishing a new tag using the **Branches** tab. Once your function has been published, you can test it using the **functions helper**

API Reference ↗

Send feedback

The screenshot shows the Visual Studio Code interface for managing Azure Functions. On the left, the 'Functions' sidebar lists three functions: 'airportLocation' (0.2.0), 'myFunction' (0.2.0), and 'range' (0.2.0). The 'airportLocation' function is selected. On the right, the 'MyFunctions.airportLocation' function is shown with its source code 'functions-typescript/src/index.ts'. The 'Inputs' section shows a dropdown menu for 'airport' with 'Denver International Airport' selected. A 'Run' button is visible. The 'Output' section shows the result: 'Denver, United States'. The bottom status bar indicates 'Problems', 'File Changes', and 'Functions'.

After the function has been published, you can start using it in other applications throughout the platform.

Next steps

This tutorial just scratches the surface of what you can do with functions on objects. To learn more, refer to these resources:

- Refer to the [object API documentation](#) to learn what you can do with objects
- Read the [object Sets documentation](#) to learn about searching for objects and aggregations on-demand
- Learn about ways you can [use functions in the platform](#)

PREVIOUS
← Overview

NEXT
Object identifiers →

API Reference ↗ Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

Send feedback

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Object identifiers

The identity of an object in Foundry is represented in a few different ways, and understanding these different representations can be important for writing correct code in functions. This section explains the various ways that objects are identified and the implications for your code.

Types of identifiers

Object RIDs

A "RID" refers to a [Resource Identifier ↗](#), Palantir's open-source specification used to identify an entity. Ontology objects have a RID assigned to them when they are created, either from indexing a backing dataset or as part of an Action.

In functions, every [Ontology object](#) has a `rid` field of type `string | undefined`. The reason a RID may be undefined is that it's possible to create a new object in functions using the [object creation](#) API. Newly created objects always have a `rid` value of `undefined`, while existing objects always have a defined `rid`.

Primary keys

Objects can also be uniquely identified by their object type and primary key. A primary key is a unique `propertyId` and value pair. For example, an Employee object type may be uniquely identified by a `string` property called `employeeId`.

All   objects always have a `typeId` and `primaryKey` field that uniquely identify newly created objects. This is because you are required to provide the primary key when

Send feedback

creating a new object.

Implications for code

Checking for equality

Within functions, each Ontology object is represented using a [JavaScript object](#) ↗. It's possible for one Ontology object to be represented as multiple JavaScript objects. For example, this can happen if you load the Ontology object from an [Object search](#) multiple times, or load an object from an Object search in addition to having it passed in as a parameter:

```
1 public myFunction(employee: Employee): void {
2     const employee2 = Objects.search().employee()
3       .filter(e => e.id.exactMatch(employee.id))
4       .all()[0];
5     console.log(employee == employee2); // false
6     console.log(employee === employee2); // false
7     console.log(employee.id === employee2.id); // true
8 }
```

Even though both `employee` and `employee2` refer to the same conceptual Ontology object in the above example, comparing them using the `==` and `===` operators returns `false` because the variables refer to two distinct JavaScript objects. Simply comparing the `rid` fields can be problematic because newly created objects have a `rid` of `undefined`.

As a result, the best way to compare two Ontology objects for equality is to compare the `typeId` and `primaryKey`:

```
1 function isEqual(o1: OntologyObject, o2: OntologyObject) {
2     return o1.typeId === o2.typeId
3     && JSON.stringify(o1.primaryKey) == JSON.stringify(o2.primaryKey);
```

API Reference ↗

Send feedback

Object mappings

It can often be useful to store a mapping from an object to some value. For example, you may want to iterate through an array of objects and store values for more efficient lookup.

Because of the equality checking issues described above, you cannot simply use a JavaScript Map to store values for each object. Instead, you can use a [FunctionsMap](#) which is specifically designed to support `OntologyObjects` as keys.

PREVIOUS

← [Getting started](#)

NEXT

[Create a custom aggregation](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Functions / Functions on objects / Create a custom aggregation

Create a custom aggregation

Functions can be used to compute custom aggregations based on data in the ontology, which can then be surfaced in a chart widget in Workshop. This guide walks through how to write custom aggregation logic that loads aggregated data from the ontology, manipulates the results to create a projection of future results, and returns the modified results.

These references may be useful while working through this section:

- Reference for [Object Set aggregations](#)
- Reference for [aggregation types](#)

Loading an aggregation

In this example, assume you have an ontology consisting of expenses, with each `expense` object having properties for department name, expense `date`, and expense `amount`. If you want to estimate the monthly spend by department over the next six months, you can begin by loading the aggregated data for the monthly spend:

```
1 const result = await Objects.search()  
2   .expenses()  
3   .groupBy(expense => expense.departmentName.topValues())  
4   .segmentBy(expense => expense.date.byMonth())  
5   .sum(expense => expense.amount);
```

API Reference ↗

Next, you can extrapolate the spend for each department for the next six months. For this example, you can take a simple approach of using the final month's value as the estimate for the next six months.

```
1  const modifiedBuckets = result.buckets.map(bucket => {  
2    // Find the bucket corresponding to the most recent month  
3    const lastBucket = bucket.value[bucket.value.length - 1];  
4  
5    let nextSixMonths: IBaseBucket<IRange<Timestamp>, Double>[] = [];  
6    let currentMonth = lastBucket.key.max!;  
7    // Loop six times  
8    for (let i = 0; i < 6; i++) {  
9      // Find the end of this range (the following month)  
10     const nextMonth = currentMonth.plusMonths(1);  
11     // Add a new bucket which uses the next month as the date range  
12     // and the most recent month as the value  
13     nextSixMonths.push({  
14       key: {  
15         min: currentMonth,  
16         max: nextMonth,  
17       },  
18       value: lastBucket.value,  
19     });  
20     currentMonth = nextMonth;  
21   }  
22  
23   // Return the modified results  
24   return { key: bucket.key, value: nextSixMonths };  
25 });
```

Returning the aggregation

Now that you have created an estimate for the next six months, you can return these estimated values:

```
return { key: bucket.key, value: modifiedBuckets };
```

[API Reference ↗](#)

[Send feedback](#)

The full example code for this function is as follows:

```

1  @Function()
2  public async estimatedDepartmentExpenses(): Promise<ThreeDimensionalAggre
3      const result = await Objects.search()
4          .expenses()
5          .groupBy(expense => expense.departmentName.topValues())
6          .segmentBy(expense => expense.date.byMonths())
7          .sum(expense => expense.amount);
8
9      const modifiedBuckets = result.buckets.map(bucket => {
10         // Find the bucket corresponding to the most recent month
11         const lastBucket = bucket.value[bucket.value.length - 1];
12
13         let nextSixMonths: IBaseBucket<IRange<Timestamp>, Double>[] = [];
14         let currentMonth = lastBucket.key.max!;
15         // Loop six times
16         for (let i = 0; i < 6; i++) {
17             // Find the end of this range (the following month)
18             const nextMonth = currentMonth.plusMonths(1);
19             // Add a new bucket which uses the next month as the date ran
20             // and the most recent month as the value
21             nextSixMonths.push({
22                 key: {
23                     min: currentMonth,
24                     max: nextMonth,
25                 },
26                 value: lastBucket.value,
27             });
28             currentMonth = nextMonth;
29         }
30
31         // Return the modified results
32         return { key: bucket.key, value: nextSixMonths };
33     });
34
35     return { buckets: modifiedBuckets };
36 }

```

The resulting aggregation can be used in a Workshop chart to show the monthly spend estimate for the next six months.

[API Reference ↗](#)

PREVIOUS

← [Object identifiers](#)

[Import object, into](#)

[Send feedback](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

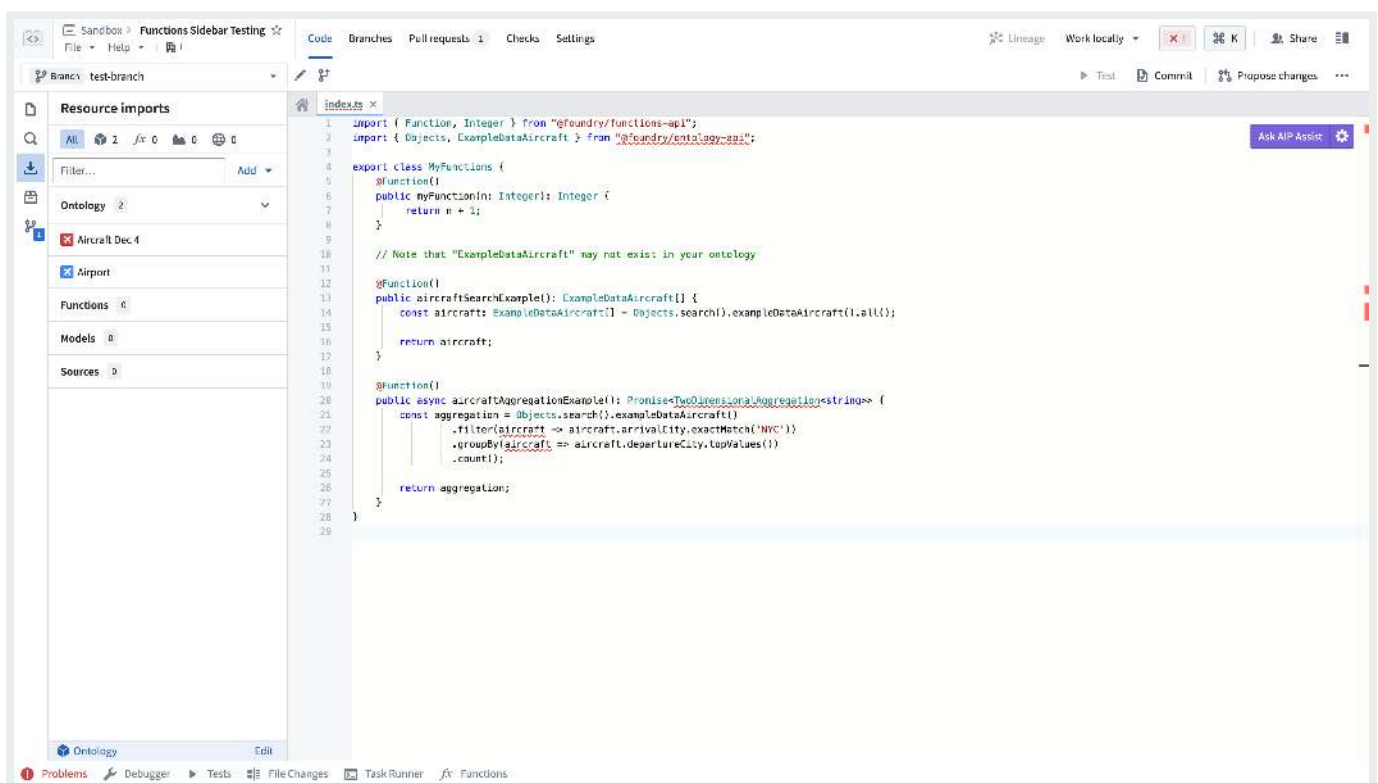
[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Functions on objects / Import object, interface, and link types

Import object, interface, and link types

Any object, interface, or link types you want to use in your function must be imported into the Project that contains your repository. Select the **Resource Imports** sidebar to view the object types which have been imported into the Project.



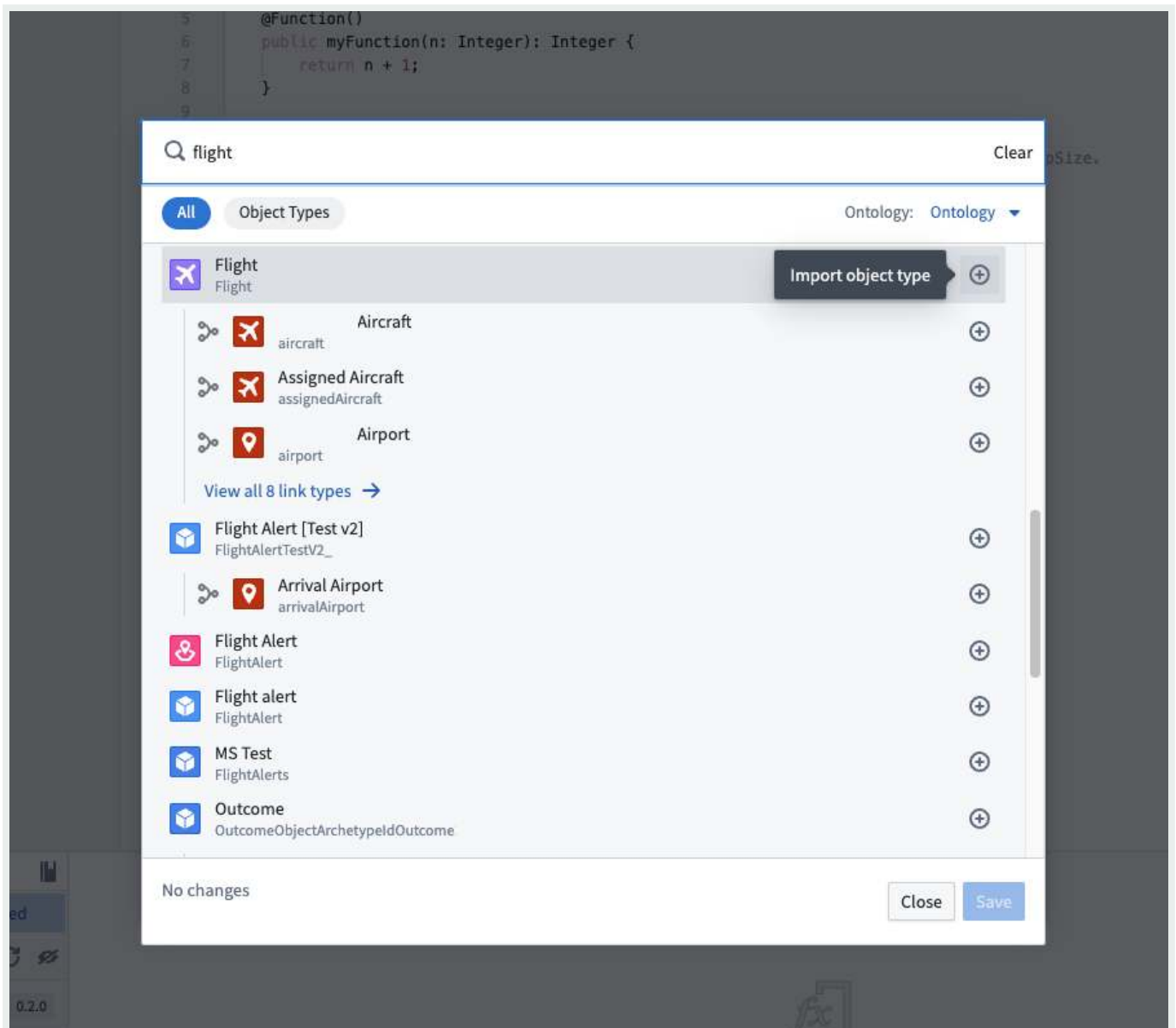
 Your Organization may not have the `Aircraft` and `Flight` objects. Use any object API Reference ↗ access to when following these steps.



Send feedback

To import additional object types, you will need to select the **Add** button in the **Resource Imports** sidebar. If no ontology was selected, you will be prompted to select an ontology. If you have at least one imported ontology type, the selected ontology will automatically be resolved.

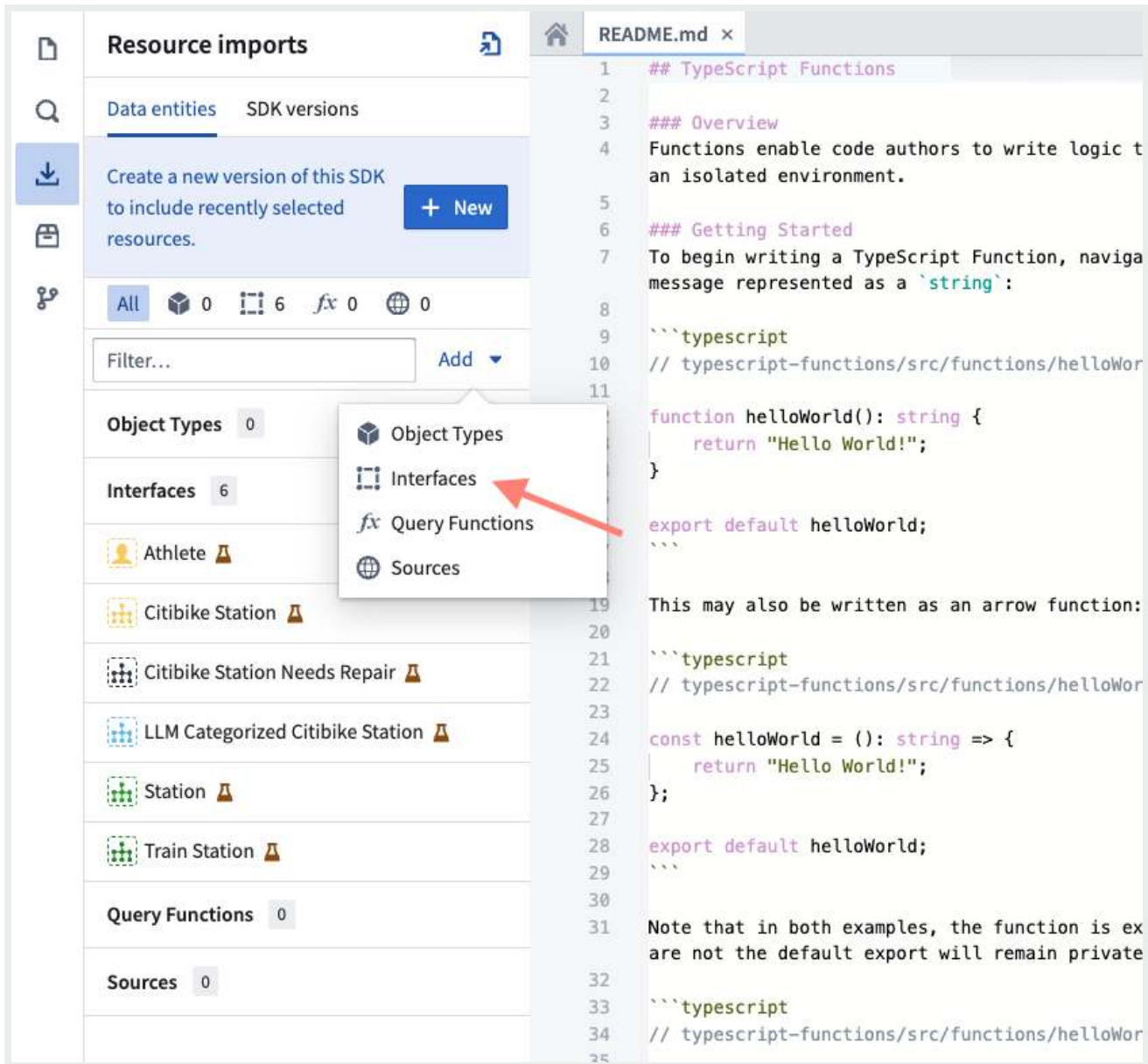
Once an ontology is selected, a search modal will appear. Your ontology will depend on the object types available in your Organization. Start by selecting a few object types and link types that connect them. In this example, we'll import the **Airport** and **Flight** objects, in addition to the link type between them.



You can also import ontology interfaces by selecting **Interfaces** under the **Add** button.

[API Reference ↗](#)

[Send feedback](#)



Choose **Save** to import the ontology types into the Project. Code Assist will automatically restart to regenerate code bindings to reflect the new object and link types you imported.

In your code, you may now import ontology types from the `@foundry/ontology-api` package. If you are using a private ontology, the package name will instead be `@foundry/ontology-api/<ontology-api-name>`.

Once Code Assist starts, you can view all the available object types by using `Ctrl` + click on the `@foundry/ontology-api` package name. The open `index.ts` file shows all of the valid object types you can import into your code:

[API Reference ↗](#)

[Send feedback](#)

```
index.ts x index.ts x
1 export { Aircraft } from "../ontology-api/Aircraft";
2 export { Airport } from "../ontology-api/Airport";
3 export { Assignment } from "../ontology-api/Assignment";
4 export { Counties } from "../ontology-api/Counties";
```

If you have access to more than one ontology, you can use the selector to pick which ontology you would like to use. Currently, importing multiple ontologies into a single project is unsupported.

PREVIOUS
← Create a custom aggregation

NEXT
Objects and links →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Functions on objects / Objects and links

API: Objects and links

Every [object type](#) imported into your project is converted to a TypeScript API so you can easily access and manipulate objects available in Foundry.

Properties

[Properties](#) of each object type are converted to fields on the TypeScript interface generated for each object type. The generated field name uses the API Name specified in the ontology.

You can access the fields for each property with simple dot notation:

```
1 const firstName = employee.firstName;
```

Note that because properties may not have a concrete value set, the returned type when accessing a property value could be `undefined`. The TypeScript compiler will give an error unless you explicitly handle the `undefined` case. See [this guide](#) for more details on this.

Array properties

Array properties on an object type are converted to `ReadOnlyArray` types. This is so that the semantics for [editing](#) an array property are clear—the only way to modify the values of an array property is to update it with an entirely new array value.

If you want to update the values of an array property, make a copy of it:

API Reference ↗



Send feedback

1 // copy to a new array

```
2 let arrayCopy = [...myObject.myArrayProperty];
```

```
3 // Now you can modify the copied array
4 arrayCopy.push(newItem);
```

Link types

[Link types](#) between object types are also converted to fields on the TypeScript interface for each object type. To traverse the link, access the field and then call one of the methods used to load the objects. Link type field names are generated using the API Name specified in the ontology.

The Foundry Ontology supports defining 1-to-1, 1-to-many, and many-to-many link types. When accessing the **1** side of a link, the generated field is of the `SingleLink` type. You can access the linked object using the `get()` or `getAsync()` methods:

```
1 const manager = employee.manager.get();
```

As with properties, when you traverse a 1-to-1 or many-to-1 link, the return value may be `undefined` if there is no linked object. Follow the [guide](#) for handling `undefined` values for these links.

When accessing the **many** side of a link, the generated field is of the `MultiLink` type. You can access an Array of linked objects using the `all()` or `allAsync()` methods. If there are no linked objects, these methods will return an empty Array.

```
1 const employees = employee.reports.all();
```

Traversing links can be expensive because it requires loading which objects are linked in the backend. For details about how to perform link traversals more efficiently, see [this section](#).

The array of linked objects returned from calling `.all()` or `.allAsync()` is a

R API Reference ↗ you want to modify the array, make a copy of it first:

Send feedback

```
1 let copiedEmployees = [...employee.reports.all()];
```



You can traverse links as an `ObjectSet` to avoid loading linked object instances in the memory. When links are created in the Ontology, APIs will be generated on an object set of this type to "search around" to other linked object sets.

```
1 import { ObjectSet, Employee } from "@foundry/ontology-api";
2
3 // Assume you have an object set available:
4 // const employee_id = "123";
5 // const employeeObjectSet : ObjectSet<Employee> = Objects.search().employ
6
7 const linkedObjs: ObjectSet<OtherObjectType> = employeeObjectSet.searchAro
```



If you operate on a single instance of an object and search around from there, you will get a `MultiLink<objectType>`. You cannot convert this `MultiLink` to an `ObjectSet`; you must convert the object instance to an object set to pivot to other object sets.

```
1 // Assuming:
2 // const employee: Employee
3
4 // MultiLink can be loaded in memory to process further.
5
6 const linkedObjs: MultiLink<objectType> = employee.reports
7
8 // Convert a sole object instance to an object set. This statement will t
9 const employeeObjectSet : ObjectSet<Employee> = Objects.search().employee
10
11 // From there, you can use the above "searchAroundToOtherObjectType" to p
```



Ontology metadata

[API Reference ↗](#) Access to the available Ontology by providing the list of objects and properties. Ontology metadata information is available by accessing each object type. See the sections below for more details.

Send feedback

Object property metadata

Object properties also include type metadata, which provides programmatic access to the type of each property. You can use this functionality for advanced workflows like identifying all properties of a given type or validating that a given property name has a specific type.

For instance, for an Ontology that contains an employee object **type**, you can access the type information on that object **type's** property as follows:

```
1 import { Employee } from "@foundry/ontology-api";
2 ...
3 const type = Employee.properties.firstName;
```

In this case, if `firstName` is a string property on the `Employee` object type, then its type will be a `StringPropertyBaseType`.

The following property types are available:

- `BooleanPropertyBaseType`
- `BytePropertyBaseType`
- `DatePropertyBaseType`
- `FloatPropertyBaseType`
- `TimestampPropertyBaseType`
- `ShortPropertyBaseType`
- `GeohashPropertyBaseType` (To be used with `geopoint` properties, previously named `geohash` properties.)
- `DecimalPropertyBaseType`
- `StringPropertyBaseType`
- `LongPropertyBaseType`
- `IntegerPropertyBaseType`
- `DoublePropertyBaseType`
- `ArrayPropertyBaseType`
- `VectorPropertyBaseType`

[API Reference ↗](#)

[Send feedback](#)

[PREVIOUS](#)

← [Import object, interface, and link types](#)

[Object sets](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

API: Object sets

An **object set** represents an unordered collection of objects of a single type. You can use the functions APIs to filter object sets, perform Search Arounds to other object types based on defined link types, and compute aggregated values or retrieve the concrete objects. In addition to passing individual objects as inputs into a function, you can search for object sets at any time using the object search APIs.

- ❗ Filtering, ordering, and aggregations only work on properties that have the `Searchable` render hint enabled in the Ontology app. These properties have been indexed for search. [Learn how to enable the `Searchable` render hint.](#)

- ❗ Object sets are more efficient than object arrays for function inputs because they defer loading until needed. For best practices on using object sets efficiently, see [Optimize performance](#).

Object search

The `objects.search()` interface allows you to initiate a search for any of the object types imported into your project. In this example, the function uses the given `airportCode` to find all flights that departed from that airport. Then, it finds all the distinct destinations of those flights and returns them.

API Reference ↗

```
import { Function } from "@foundry/functions-api";  
rt { Objects } from "@foundry/ontology-api";
```

Send feedback

```

4 export class FlightFunctions {
5   @Function()
6   public currentFlightDestinations(airportCode: string): Set<string> {
7     const flightsFromAirport = Objects.search()
8       .flights()
9       .filter(flight => flight.departureAirportCode.exactMatch(airportCode))
10    .all();
11
12    const destinations = flightsFromAirport.map(flight => flight.arrivalAirportCode);
13
14    return new Set(destinations);
15  }
16 }

```

Object sets can also be created from a list of objects, list of object resource identifiers or an object set resource identifier by passing them as an argument to the searched object type. For example: `Objects.search().flights([flight])`.

Once you have an object set of a given type, you can perform various operations on the set as documented below.

Filtering

The `.filter()` method on an object set allows you to filter the object set based on the searchable properties of the objects. The filter method takes a filter definition, which is based on the type of the property you are filtering on.

- All property types support the `.exactMatch()` filter, which filters to objects with an exact match on that property value. This is useful to filter for exact matches on strings (as in the example above), or to filter on the primary key of an object (for example, `.filter(object => object.primaryKey.exactMatch(PrimaryKey))`).
 - To check whether a property is `null` or `undefined`, use the `hasProperty()` method.
 - To pass multiple values, use the spread operator `.exactMatch(...listVariable)`. If an empty array is passed in, the filter will be ignored.
- String properties support several keyword filters. See the documentation on each

API Reference ↗ Assist for full details.

- `.phrase()` splits the search query into tokens (usually individual words) and then filters values based on whether they contain all of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.containsAll()` filters values based on whether they contain all of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.containsAny()` filters values based on whether they contain any of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.startsWith()` filters values based on whether they start with the given token. If an empty array is passed in, the filter will be ignored.
- `.endsWith()` filters values based on whether they end with the given token. If an empty array is passed in, the filter will be ignored.
- `.contains()` filters values based on whether they contain the given token. If an empty array is passed in, the filter will be ignored.
- `.doesNotContain()` filters values based on whether they do not contain the given token. If an empty array is passed in, the filter will be ignored.
- `.doesNotContainAny()` filters values based on whether they do not contain any of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.doesNotContainAll()` filters values based on whether they do not contain all of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.startsWithAny()` filters values based on whether they start with any of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.startsWithAll()` filters values based on whether they start with all of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.endsWithAny()` filters values based on whether they end with any of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.endsWithAll()` filters values based on whether they end with all of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.containsAnyOf()` filters values based on whether they contain any of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.containsAllOf()` filters values based on whether they contain all of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.doesNotContainAnyOf()` filters values based on whether they do not contain any of the given tokens. If an empty array is passed in, the filter will be ignored.
- `.doesNotContainAllOf()` filters values based on whether they do not contain all of the given tokens. If an empty array is passed in, the filter will be ignored.

Send feedback

periods will be treated as one token. For example, when searching for "banana", an object with the property value "banana_pudding" or "banana.pudding" will not be returned.

- `.phrasePrefix()` is almost identical to `phrase()`, but the last token will also match tokens starting with that token. For example, a `.phrasePrefix()` search for `fresh banana` would match the property value `fresh banana_pudding`, but not the property value `banana_pudding fresh`. A `.phrasePrefix()` search for `pudding` would not match the property value `banana_pudding`.
- `.prefixOnLastToken()` splits the search query into tokens and then filters values based on whether they contain all of the given tokens, where the last token will also match tokens starting with that token. For example, `big app` would match `big apples` as well as `apples from the big tree`, though it would not match `apples from the biggest tree`.
- `.matchAnyToken()`, `.fuzzyMatchAnyToken()` split the search query into tokens and then filter values based on whether they contain any of the given tokens. The `fuzzy` version allows approximate values to match.
- `.matchAllTokens()`, `.fuzzyMatchAllTokens()` split the search query into tokens and then filter values based on whether they contain all of the given tokens. The `fuzzy` version allows approximate values to match.
 - Fuzzy filters can take an optional `Fuzziness` parameter imported from `@foundry/functions-api`.
 - Explanations of the available `Fuzziness` options can be found in the [ElasticSearch documentation](#). More information can also be found below.
- Numbers, dates, and timestamp properties support `.range()` filters.
 - Range filters have a set of `.lt()`, `.lte()`, `.gt()` and `.gte()` methods for performing less than / less than or equal to / greater than / greater than or equal to (respectively) comparisons.
- Boolean properties support `.isTrue()` and `.isFalse()` filters.
- Geopoint properties support `.withinDistanceOf()`, `.withinPolygon()`, and `.withinBoundingBox()` filters.
- GeoShape properties support `.withinBoundingBox()`, `.intersectsBoundingBox()`, `.doesNotIntersectBoundingBox()`, `.withinPolygon()`, `.intersectsPolygon()`, and `.doesNotIntersectPolygon()` filters.
- Link filters can be used to filter objects that do or do not have any linked objects of a given type using the `.isPresent()` method.
- Array properties support the `.contains()` filter, which filters to objects whose array property values contain any of the given values.

API Reference ↗

Send feedback

Combining filters

You can compose filters together using the `Filters` API exported from `@foundry/functions-api`. The available methods are:

- `and()` filters the object set to objects that pass all the given filters
- `or()` filters the object set to objects that pass any of the given filters
- `not()` negates the given filter

In the example below, we can filter an object set of flights by flight destination using `and()`:

```
1 import { Filters } from "@foundry/functions-api";
2
3
4 Objects.search()
5   .flights()
6   .filter(flight => Filters.or(
7     Filters.and(flight.destination.exactMatch("SFO"), flight.passenger
8     Filters.and(flight.destination.exactMatch("LAX"), flight.passenger
9   ))
```

The above code would filter to flights that either arrived at SFO with more than 100 passengers or arrived at LAX with more than 300 passengers.

⚠ Warning

The `.filter()` method on an object set does not use the operators `&&` or `||`. To apply multiple filters, you must use one of the methods on `Filters` listed above (or call `.filter()` multiple times to achieve an `and` condition).

Filtering on string properties with fuzzy search

Search API Reference ↗ The `fuzziness` parameter can provide more fine-tuned control over fuzzy matching behavior. If you do not specify fuzziness, then an automatic edit distance is allowed based on the length of the token you are searching for. You will want to specify `Fuzziness` from `@foundry/functions-api` in order to specify edit distance.

Send feedback

Fuzzy match any token

```
Objects.search().employee().filter(employee => employee.firstName.fuzzyMatchAnyToken(searchTerm, 2))
```

The code above returns any employees with a first name within two edits of the provided search term (with Levenshtein distance of two). In this example, that would include `Michael`, `Micheal`, `Mikhael`, `Michel`, `Mikhail`, `Mihail` (but not `Miguel` for example). If you have more certainty in the accuracy of your search term, you can search with a smaller edit distance (with different Levenshtein distances), refining your search results a little more.

Fuzzy match all tokens

```
Objects.search().employee().filter(employee => employee.fullName.fuzzyMatchAllTokens(searchTerm, 2))
```

You can also use fuzzy filters on a multiple token phrase. The code above would match on employees whose full name contains **both** `Michael` and `Smith` with up to one edit in each token – for example, `Mikhael Smitt` (that is, each with a Levenshtein distance of one each). The ordering of tokens is not taken into account with a `fuzzyMatchAllTokens` or `fuzzyMatchesAllTokens` filter.

Fuzzy match on string array properties

All filters on array-based properties can use the methods available to their underlying type. For example, string array properties can be filtered based on any methods available to string properties, though the naming of the methods may differ slightly. Filtering on array properties requires a single match among the array elements in order for that object to be returned.

Search Around

API Reference ↗

🔍 Search around limits

Object sets loaded into memory `.all()` or `.allAsync()` are all

maximum of 3 search rounds. If more than 3 search rounds are used, an error is

Send feedback

thrown. When performing a search around from object set A to object set B in Object Storage v2, the resulting object set B cannot have more than 10 million object instances, or an error will be thrown. For Object Storage v1, the limit is 100,000 object instances.

Based on the object type of your object set, *Search Around* methods are generated to enable traversing [links](#) based on the object type of your object set. In the below example, we filter to an object set of Flights based on the departure code, then Search Around to the passengers on those flights. This results in an object set of Passengers, which can be further filtered or searched around on.

```
1 const passengersDepartingFromAirport = Objects.search()  
2   .flights()  
3   .filter(flight => flight.departureAirportCode.exactMatch(airportCode))  
4   .searchAroundPassengers();
```

Search Around methods will only be generated for link types that are imported into your project. Refer to [the tutorial](#) for details on how to import link types.

Note that for performance reasons, the number of Search Around operations you can conduct in a single search is currently limited to 3. If you attempt to run a search with more than three levels of Search Around depth, the search will fail at runtime.

K-nearest neighbors (KNN)

i KNN Limits

KNN is only supported on object types indexed into [OSv2](#). The k value is limited to the range $0 < K \leq 100$. Also, the search vector must be the same size as the one used for indexing and has a 2048 dimension limit. An error will be thrown if any of these limits are exceeded.

API Reference ↗

Object types with embedding properties will be available for KNN searches. These searches will return the k value objects that have an embedding property near the provided embedding parameter. The following example returns the most similar movies to a provided

Send feedback

movie script. Embeddings can be generated in transformation tools such as [Pipeline Builder](#) ; or at function query time [using a Palantir-provided embedding model](#) or [your own model in a function](#).

Make sure that your functions repository's `functions.json` configuration file has the `enableVectorProperties` entry set to `true` .

```
1 import { Objects } from "@foundry/ontology-api";
2
3 const kValue: number = 2;
4 // Vector can be generated from FML Live or come from an existing object
5 const vector: Double[] = [0.7, 0.1, 0.3];
6 const movies: Movies[] = Objects.search()
7     .movies()
8     .nearestNeighbors(obj => obj.vectorProperty.near(vector, { kValue
9     .orderByRelevance()
10    .take(kValue);
```

For an example of a full semantic search workflow, review the [semantic search workflow guide](#).

Set operations

Object sets of the same object type can be combined in various ways using set operations:

- `.union()` creates a new object set composed of objects present in any of the given object sets.
- `.intersect()` creates a new object set composed of objects present in all of the given object sets.
- `.subtract()` removes any objects present in the given object sets.

Retrieving all objects

The `findAllAsync()` methods retrieve all objects in the object set. Note that if you attempt to load too many objects at once, your function will fail to execute. The maximum number of objects you can load is 100,000. However, loading more than 10,000

API Reference ↗

Send feedback

objects may also cause your function execution to time out. [Learn more about time and space limits in functions.](#)

You can use the `.allAsync()` method to retrieve a Promise that resolves to all the objects in the object set. This can be useful for loading data from multiple object sets in parallel.

Ordering and limiting

Instead of retrieving all objects, you can load a limited number by applying an ordering clause to your object set, then specifying a specific number of objects to load. To do this, you can use the following methods:

- `.orderBy()` specifies a searchable property to order by, and allows you to specify an ordering direction. Only properties whose types can be ordered (numbers, dates, and strings) are available for selection in this method. You can call `.orderBy()` multiple times to sort by multiple properties.
- `.orderByRelevance()` specifies that the objects should be returned in order of how well they match the provided filters, with the most relevant listed first. Relevance for a query term against a property value on a given object is a complex determination that takes into account the frequency of the term appearing in the property value, the frequency of the term appearing across all objects, and more. Relevance is less appropriate when performing only `.exactMatch()` filters or filtering on non-string properties. Note that only one of `.orderBy()` and `.orderByRelevance()` may be used in a single search.
- `.take()` and `.takeAsync()` enable you to retrieve a specified number of objects from the set. These methods are only available after you have specified an ordering.

For example, the following code would retrieve the ten employees with the earliest start dates:

```
1 Objects.search()  
2   .employees()  
3   .orderBy(e => e.startDate.asc())  
4   .take(10)
```

[API Reference ↗](#)

As another example, imagine an object type `claims` which contains
for an insurance company. We'd like to find a specific claim involving

[Send feedback](#)

Without the `.orderByRelevance()` line, any results containing any of the words `red`, `car`, `collision`, `with`, or `deer` may have been returned in the top 10 results. With the `.orderByRelevance()` line, the first 10 results will be the claims that contain the most search terms, so that the most relevant claims will appear first.

```
1 const results = Objects.search()  
2   .claims()  
3   .filter(doc => doc.text.matchAnyToken("red car collision with deer"))  
4   .orderByRelevance()  
5   .take(10)
```

Computing aggregations

i Aggregation limits

Aggregations returned from the Objects API are limited to **10,000 total buckets**. An error will be thrown if this limit is exceeded.

When bucketing using `.topValues()`, results will be approximate if the data has more than 1,000 distinct values. The list of top values may not be accurate in that case.

Grouping objects by properties

In many cases, it's unnecessary to load all of the objects in your object set. Instead, you can simply load a bucketed aggregation of values to conduct further analysis.

To begin computing an aggregation, call the `.groupBy()` method on an object set. This allows you to specify bucketing on one of the searchable properties of the object type in the object set. For example, this code groups employees by their start date:

```
API Reference > search()  
1 .employees()  
2 .groupBy(e => e.startDate.byDays())
```

Send feedback

When specifying which property to bucket by, you will have to provide additional information about how the bucketing should be done depending on the property type:

- For `boolean` properties, the only option is `.topValues()`. This returns two buckets, one for `true` and one for `false`.
- For string properties, there are two options:
 - `.topValues()`: For rapid response times and properties with a smaller cardinality. This buckets by the top 1,000 values for the string property. This limit is to ensure that the returned aggregation is not excessively large.
 - `.exactValues()`: For more exact aggregations and the possibility to consider up to 10,000 buckets for high cardinality properties. The amount of considered buckets can be specified via `.exactValues({"maxBuckets": numBuckets})` where `numBuckets` must be an integer value between 0 and 10,000. The response time for this method can take longer, as more results have to be considered.
- For numeric properties (e.g. `Integer`, `Long`, `Float`, `Double`), the two bucketing options are:
 - `.byRanges()` allows you to specify the exact ranges that should be used. For example, you could use `.byRanges({ min: 0, max: 50 }, { min: 50, max: 100 })` to bucket objects into the two ranges of `[0, 50]` and `[50, 100]` that you specify here. The `min` of the range is inclusive and the `max` is exclusive. You may omit either `min` or `max` to represent a bucket containing values from $-\infty$ to `max` or `min` to ∞ respectively.
 - `.byFixedWidth()` specifies the width of each bucket. For example, you could use `.byFixedWidth(50)` to bucket objects into ranges that each have a width of 50.
- For `LocalDate` properties, various convenience methods are provided for easy bucketing:
 - `.byYear()`
 - `.byQuarter()`
 - `.byMonth()`
 - `.byWeek()`
 - `.byDays()` buckets values into days. You may pass in a number of days to use for bucket widths.
- For `Timestamp` properties, the same bucketing options apply as for `LocalDate`, as well as the following additions:
 - `.byHours()` buckets values by hours. You may pass in a number of hours to use for bucket widths.
 - `.byMinutes()` buckets values by minutes. You may pass in a number of minutes to use for bucket widths.

API Reference ↗

Send feedback

- `.bySeconds()` buckets values by seconds. You may pass in a number of seconds to use for bucket widths.
- For `Array` properties, the bucketing options are determined by the type of the elements in the array. In particular, you get the same bucketing methods for `Array<PropertyType>` as you would get for the `PropertyType` (for example, `Array<boolean>` gets the same bucketing methods as `boolean`).
 - For example, if you have an `Array<string>` called `employeeSet` consisting of Alice and Bob who have respectively worked in `["US", "UK"]` and `["US"]`. Then `employeeSet.groupBy(e => e.pastCountries.exactValue()).count()` will return `{ "US": 2, "UK": 1 }`.

After grouping by one property, you may optionally call the `.segmentBy()` method to perform further bucketing. This allows you to compute a three-dimensional aggregation bucketed by two searchable properties. For example, you could group employees by their start date as well as their role as follows:

```
1 Objects.search()
2   .employees()
3   .groupBy(e => e.startDate.byDays())
4   .segmentBy(e => e.role.topValues())
```

Choosing an aggregation metric

After grouping your object set, you can call various aggregation methods to compute aggregation metrics on each bucket. Methods that require a property only accept properties marked searchable. Possible aggregation methods are:

- `.count()` simply returns the number of objects in each bucket
- `.average()` returns the average number for the given numeric, timestamp, date property
- `.max()` returns the maximum value for the given numeric, timestamp, date property
- `.min()` returns the minimum value for the given numeric, timestamp, date property
- `.sum()` returns the sum of values for the given numeric property

`.distinct()` returns the approximate number of distinct values for the given

API Reference ↗

Send feedback

Calling one of these methods returns either a `TwoDimensionalAggregation` or `ThreeDimensionalAggregation`. A `ThreeDimensionalAggregation` is returned if you called `.segmentBy()` before calling one of the final aggregation methods.

[Learn more about the structure of these aggregation types, including valid bucketing types.](#)

Note that the resulting aggregations are wrapped in a `Promise`, as computing the aggregation requires loading data from a remote service. You can use the [async/await](#) syntax to unwrap the `Promise` result.

Below is a full example of loading an aggregation and returning it as a result.

```
1 import { Function, ThreeDimensionalAggregation } from "@foundry/functions";
2 import { Objects } from "@foundry/ontology-api";
3
4 export class AggregationFunctions {
5     @Function()
6     public async employeesByRoleAndOffice(): Promise<ThreeDimensionalAggr
7         return Objects.search()
8             .employee()
9             .groupBy(e => e.title.topValues())
10            .segmentBy(e => e.office.topValues())
11            .count();
12 }
13 }
```

Below is a full example of aggregating without `groupBy` statements:

```
1 import { Function } from "@foundry/functions-api";
2 import { Objects } from "@foundry/ontology-api";
3
4 export class AggregationFunctions {
5     @Function()
6     public async employeesStats(): Promise<Double> {
7         // Count of all employees, default to zero if count() returns und
8         return Objects.search().employee().count() ?? 0;
9     }
10 }
```

[API Reference ↗](#)

[Send feedback](#)

You can also perform other aggregations without `groupBy` by replacing the appropriate line in the code example above, such as:

- Count of all employees: `Objects.search().employee().count();` (as seen in example above)
- Average tenure of employees: `Objects.search().employee().average(e => e.tenure);`
- Maximum tenure of employees: `Objects.search().employee().max(e => e.tenure);`
- Minimum tenure of employees: `Objects.search().employee().min(e => e.tenure);`
- Sum of all employee salaries: `Objects.search().employee().sum(e => e.salary);`
- Number of offices: `Objects.search().employee().cardinality(e => e.office);`

For an example of manipulating aggregation results in memory, try the guide for [creating custom aggregations](#).

PREVIOUS

← [Objects and links](#)

NEXT

[Attachments](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Attachments

⚠ TypeScript v1 functions are executed in an environment that has strict memory limits. Exceeding these memory limits can happen quickly when dealing with file data; we recommend only interacting with attachments under 20MB. Python and TypeScript v2 functions have a default limit of 1GB, which can be adjusted in Ontology Manager.

An attachment is a file that acts like an object property. Attachments are uploaded as temporary files and [attached to objects using actions](#). Once attached to an object, an attachment is persisted and can be accessed similarly to other properties.

Attachments in functions

Attachments can be passed into functions as inputs from actions, or accessed as properties on objects. You can also create and return attachments in functions.

When using Python, attachments are managed using the [API Gateway](#). An attachment type is provided in the [Python OSDK](#).

Below is the import syntax for attachments by language:



Read attachment data

A read method is provided on attachments to read their raw data. The signature for the method is as follows:

TYPESCRIPT V1

TYPESCRIPT V2

PYTHON

```
1 // Blob is a standard JavaScript type, representing a file-like object
2 // https://developer.mozilla.org/en-US/docs/Web/API/Blob
3 readAsync(): Promise<Blob>;
```

You may need to use libraries or write your own custom code for handling complex file types. For example, PDFs must be parsed with an appropriate library. [Learn more about adding dependencies to functions repositories.](#)

File parsing in TypeScript v1

TypeScript v1 functions do not offer filesystem support. Often, dependencies related to parsing file data will rely on the `fs` module, which is not available in the functions environment. This restriction may cause `fs` module errors during compilation and execution. To work around this restriction, you can introduce a dependency on an in-memory file system (`memfs`, for example). Then, alias the dependency under the `fs` name.

Below is an example using the NPM dependency `memfs` in a `package.json` file:

```
1 "fs": "npm:memfs@^x.x.x"
```

Create attachments

For [API Reference](#) ↗, see used to create attachments and attach them to objects. For attachments created in functions to be persisted, the function must make an [Ontology edit](#) that links the attachment to an object.

Send feedback

 Attachments that are not attached to an object can only be viewed by the uploader and are automatically deleted after a certain period of time.

TypeScript v2 functions do not support creating attachments.

To create an attachment, use an upload function on the attachment. The signature for the upload function by language is as follows:

TYPESCRIPT V1 PYTHON

```
1 import { Attachments, Attachment } from "@foundry/functions-api";  
2  
3 // On Attachments:  
4 uploadFile(filename: string, blob: Blob): Promise<Attachment>;
```

The following example shows the process for uploading a file and assigning the resulting attachment to an object.

TYPESCRIPT V1 PYTHON

```
1 import { Attachments, Attachment, OntologyEditFunction } from "@foundry/f  
2  
3 @OntologyEditFunction()  
4 public async updateMaintenanceLog(aircraft: Aircraft): Promise<void> {  
5     const aircraftMaintenanceLogData: Blob = await aircraft.maintenanceLo  
6     const completedMaintenanceLogData: Blob = await completedMaintenanceL  
7  
8     // You will likely need to rely on libraries or custom code to create  
9     // passed as a parameter into the `uploadFile` method.  
10  
11     // Compare the current aircraft logs and completed logs and create a  
12     const updatedMaintenanceLogData: Blob;  
13  
14     aircraft.maintenanceLog = await Attachments.uploadFile("maintenance-l
```

[API Reference ↗](#)

[Send feedback](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Functions on objects / Media

Media

Functions enable you to access and modify media in [TypeScript v2](#), [Python](#), and [TypeScript v1](#). TypeScript v2 and Python use the `Media` type to read, upload, and transform media, and support media uploads through [Ontology edits](#) and the OSDK. TypeScript v1 functions provide a `MediaItem` type with built-in operations for working with different kinds of media without external libraries.

If you need any operations that don't currently exist out-of-the-box, you will likely need to use external libraries or write your own custom code. [Learn more about adding dependencies to functions repositories.](#)

TypeScript v2 and Python

Use Ontology edit functions to upload media and create objects in the Ontology. Once uploaded, you can read and download media files from objects for use in your application. [Learn more about media sets in Foundry.](#)

You can construct Ontology edits in TypeScript v2 and Python functions by uploading media to the Ontology to obtain a `Media` instance. The `Media` type wraps a `MediaReference` and exposes [higher-level operations](#) for fetching contents, fetching metadata, and attaching media to objects. You can use the `Media` to construct an Ontology edit, or pass existing media into the function as a parameter.

Use as a function input or output type

API Reference ↗

Functions can take in a `Media` as an input, create temporary media by uploading data with `uploadMedia`, or retrieve `Media` from a media reference property on an object. Functions can return a `Media` type as well, whether it has been temporarily uploaded, or if it came

Send feedback

from an object's media reference property. In a function, you can fetch the byte contents of the `Media`, fetch its metadata, or attach it to an Ontology object via Ontology edits. In Python, you can also fetch the full per-variant metadata; in TypeScript v2, `fetchMetadata` currently exposes only the high-level fields (`mediaType`, `sizeBytes`, `path`).

TYPESCRIPT V2 PYTHON

```
1 import type { Media } from "@osdk/client";
2
3 export default async function echoMedia(media: Media): Promise<Media> {
4     return media;
5 }
```

Upload media

Use the Ontology SDK `uploadMedia` (TypeScript v2) and `client.ontology.media.upload_media` (Python) helpers to upload raw bytes within a function. Both return a `Media`, which you can then edit an Ontology object media property with an Ontology edit or return from the function.

TYPESCRIPT V2 PYTHON PYTHON (ASYNC)

```
1 import type { Client, Media } from "@osdk/client";
2 import { uploadMedia } from "@osdk/functions";
3
4 export default async function uploadMediaItem(
5     client: Client,
6     body: string,
7     fileName: string,
8 ): Promise<Media> {
9     const blob = new Blob([body], { type: "text/plain" });
10    const media: Media = await uploadMedia(
11        client,
12        { data: blob, fileName }
13    );
14
15    return media;
16 }
```

API Reference ↗

Send feedback

- i** Uploading media is temporary, unless set to an Ontology object's media reference property. When the Ontology edits are applied, the media is then persisted on the Ontology object property.

Upload media in Ontology edit functions

Whether you uploaded media within a function or received a `Media` as an input to the function, you can update media properties on existing Ontology objects or create new Ontology objects with `Media` parameters.

TYPESCRIPT V2

PYTHON

```
1 // Ensure you are using TypeScript OSDK 2.16 or greater
2
3 import type { Client, Media } from "@osdk/client";
4 import { Aircraft } from "@ontology-sdk/sdk";
5 import type { Edits } from "@osdk/functions";
6 import { createEditBatch, uploadMedia } from "@osdk/functions";
7
8 async function uploadTextToNewPlane(client: Client): Promise<Edits.Object
9     const batch = createEditBatch<Edits.Object<Aircraft>>(client);
10     const blob = new Blob(["Hello, world"], { type: "text/plain" });
11     const media: Media = await uploadMedia(
12         client,
13         { data: blob, fileName: "/planes/aircraft.txt" }
14     );
15     batch.create(Aircraft, { myMediaProperty: media, /* ... */ });
16     return batch.getEdits();
17 }
18
19 export default uploadTextToNewPlane;
```

- i** In TypeScript OSDK generator versions before 2.20, `uploadMedia` returned a `MediaReference`. Starting in version 2.20, `uploadMedia` returns a `Media`, which is a subclass of `MediaReference` and exposes higher-level operations such as `uploadMedia`.

API Reference ↗

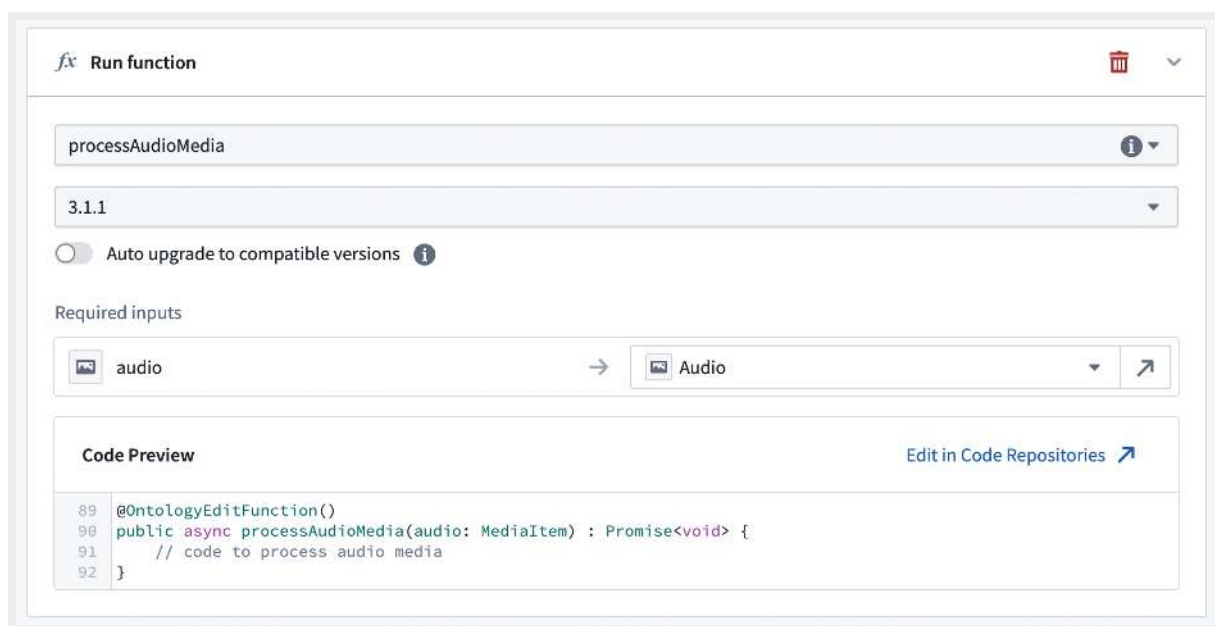
Send feedback

`fetchContents`, `fetchMetadata`, and `getMediaReference()`. You can pass a `Media` directly into `createEditBatch` operations.

Passing a media reference parameter on action type

Action parameters of type media reference can be passed to the function as a parameter.

The screenshot below shows an action passing a media parameter to its backing function.



Media Ontology SDK operations

i The methods below work on any `Media` instance, including those returned from `upload_media` and those exposed as `Media` properties on object types.

Retrieve media bytes data

Y [API Reference ↗](#) aw data stored on the `Media`. The signature for the method is as follows:

[Send feedback](#)

TYPESCRIPT V2 PYTHON

```
1 fetchContents(): Promise<Response>;
2
3 // "Response" is a standard interface on the JavaScript Fetch API
4 // https://developer.mozilla.org/en-US/docs/Web/API/Response
5 const mediaContents: Response = await myAircraft.myMediaProperty.fetchCon
6
7 if (mediaContents.ok) {
8     const mediaMimeType = mediaContents.headers.get("Content-Type");
9
10    // Blob is a standard JavaScript type, representing a file-like object
11    // https://developer.mozilla.org/en-US/docs/Web/API/Blob
12    // https://developer.mozilla.org/en-US/docs/Web/API/Response/blob
13    const mediaBlob: Blob = await mediaContents.blob();
14 }
```

Get media metadata

You can retrieve the metadata of the `Media`:

TYPESCRIPT V2 PYTHON

```
1 fetchMetadata(): Promise<MediaMetadata>;
2
3 // Example usage:
4 const mediaMetadata = await myAircraft.myMediaProperty.fetchMetadata();
5 const sizeBytes = mediaMetadata.sizeBytes;
6 const mediaType = mediaMetadata.mediaType;
```

In Python, `get_media_full_metadata()` returns a `MediaFullMetadata` whose `item_metadata` is a discriminated union over the media type. Narrow on the variant class (or check `item_metadata.type`) to access type-specific fields:

API Reference ↗

PYTHON

Send feedback


```

1 get_media_full_metadata(self) -> MediaFullMetadata: ...
2
3 # Narrow on the variant class (or check item_metadata.type) to access type
4 # Other variants include AudioMediaItemMetadata, VideoMediaItemMetadata,
5 # SpreadsheetMediaItemMetadata, Model3dMediaItemMetadata, DicomMediaItemM
6 # EmailMediaItemMetadata, and UntypedMediaItemMetadata. See the full sche
7 # https://github.com/palantir/foundry-platform-python/blob/develop/docs/v
8
9 from foundry_sdk.v2.media_sets.models import (
10     DocumentMediaItemMetadata,
11     ImageryMediaItemMetadata,
12 )
13 from foundry_sdk_runtime.media import MediaFullMetadata
14
15 full_metadata: MediaFullMetadata = my_aircraft.my_media_property.get_med
16 item = full_metadata.item_metadata
17
18 if isinstance(item, DocumentMediaItemMetadata):
19     page_count = item.pages
20     title = item.title
21 elif isinstance(item, ImageryMediaItemMetadata):
22     dimensions = item.dimensions
23     bands = item.bands

```

Transform media

 Media transformations are in the beta stage of development. Functionality may change during active development.

You can transform media items (such as rotating, resizing, or re-encoding images, slicing or rendering PDF pages, or running OCR) and wait for the result. The transformation job is submitted, it is polled to completion, and the transformed content is returned.

In TypeScript v2, transformations are exposed through `@osdk/api/unstable` as an experimental helper. In Python, call `client.ontology.media.transform_and_wait` on a given `client`. The async variant `async_transform_and_wait` takes the same arguments and can be awaited.

[API Reference ↗](#)

[Send feedback](#)

```

1 // Ensure you are using @osdk/api 2.8.0 or greater for transformAndWait.
2 // "MediaTransformation" is a discriminated union:
3 // each variant (`$image`, `$video`, `$audio`, `$documentToText`, `$documentToText`)
4 // selects a transformation kind, with its own encoding and operation fields.
5 // See the "MediaTransformation" type definition for a full set of variants.
6 // https://github.com/palantir/osdk-ts/blob/main/packages/api/src/experimental
7
8 import {
9     __EXPERIMENTAL__NOT_SUPPORTED_YET__transformAndWait,
10     type MediaTransformation,
11 } from "@osdk/api/unstable";
12 import type { Client, Media } from "@osdk/client";
13 import { uploadMedia } from "@osdk/functions";
14
15 export default async function rotateImage(
16     client: Client,
17     media: Media,
18 ): Promise<Media> {
19     const transformation: MediaTransformation = {
20         $image: {
21             $encoding: "jpg",
22             $operations: [{ $rotate: { $angle: "DEGREE_180" } }],
23         },
24     };
25
26     const result: Response = await client(
27         __EXPERIMENTAL__NOT_SUPPORTED_YET__transformAndWait,
28     ).transformAndWait({
29         mediaReference: media.getMediaReference(),
30         transformation,
31         options: { pollIntervalMs: 3000, pollTimeoutMs: 30000 },
32     });
33
34     if (!result.ok) {
35         // The transformation failed; inspect result.status / result.statusText.
36         throw new Error(`Transformation failed with status ${result.status} and ${result.statusText}`);
37     }
38
39     // Re-upload the transformed bytes so the function returns a Media.
40     return uploadMedia(client, { data: await result.blob(), fileName: "rotated.jpg" });

```

API Reference ↗

Send feedback

Example: Run page-by-page OCR on a PDF with bounding box output

This workflow takes a PDF (uploaded to a media set or attached to an object) and runs OCR on every page, requesting hOCR output. hOCR is HTML with `bbox` attributes on every detected word and line, so you can extract both the recognized text and its bounding box coordinates from the same response. Each `transform_and_wait` call returns the bytes for one page; iterate to cover the whole document.

TYPESCRIPT V2

PYTHON

```
1 import {
2   __EXPERIMENTAL__NOT_SUPPORTED_YET__transformAndWait,
3   type MediaTransformation,
4 } from "@osdk/api/unstable";
5 import type { Client, Media } from "@osdk/client";
6 import type { Integer } from "@osdk/functions";
7
8 export default async function ocrPdfPages(
9   client: Client,
10  media: Media,
11  pageCount: Integer,
12 ): Promise<string[]> {
13   const transformAndWait = client(
14     __EXPERIMENTAL__NOT_SUPPORTED_YET__transformAndWait,
15   ).transformAndWait;
16   const mediaReference = media.getMediaReference();
17
18   const pageResults: string[] = [];
19   for (let pageNumber = 0; pageNumber < pageCount; pageNumber++) {
20     const transformation: MediaTransformation = {
21       $documentToText: {
22         $operation: {
23           $ocrOnPage: {
24             $pageNumber: pageNumber,
25             $parameters: {
26               $outputFormat: { $hocr: {} },
27               $languages: [{ $language: "ENG" }],
28             },
29           },
30         },
31       },
32     };
33
34     const result = await transformAndWait({
```

API Reference ↗

Send feedback

```

35         mediaReference,
36         transformation,
37         options: { pollTimeoutMs: 120_000 },
38     });
39     if (!result.ok) {
40         throw new Error(`OCR failed on page ${pageNumber}: ${result.s
41     }
42     pageResults.push(await result.text());
43 }
44 return pageResults;
45 }

```

Dense pages can push OCR runtime well past the default function timeout. See [Manage published functions](#) to configure function execution timeouts.

Example: Render PDF pages as images and slice ranges

For workflows that need the visual rendering of each page (for downstream image annotation, embedding, or display), use `$documentToImage` with `$renderPage` to get a PNG/JPG image of a specific page. To extract a sub-range of the PDF as its own PDF document, use `$documentToDocument` with `$slicePdfRange`. Each function below re-uploads the transformed bytes so it can return a `Media`. Each function is its own module; a registered function is the module's `export default`.

Render a single page as a PNG image:

TYPESCRIPT V2

PYTHON

```

1  import {
2      __EXPERIMENTAL__NOT_SUPPORTED_YET__transformAndWait,
3      type MediaTransformation,
4  } from "@osdk/api/unstable";
5  import type { Client, Media } from "@osdk/client";
6  import { uploadMedia } from "@osdk/functions";
7
8  export default async function renderFirstPageAsPng(
9      client: Client,
10     media: Media,
11 ): Promise<Media> {
12     const transformation: MediaTransformation = {
13         $documentToImage: {

```

API Reference ↗

Send feedback

```

14         $encoding: "png",
15         $operation: { $renderPage: { $pageNumber: 0, $width: 1200 } }
16     },
17 };
18 const result = await client(
19     __EXPERIMENTAL__NOT_SUPPORTED_YET__transformAndWait,
20 ).transformAndWait({ mediaReference: media.getMediaReference(), trans
21 if (!result.ok) {
22     throw new Error(`Render failed: ${result.status}`);
23 }
24 // Re-upload the rendered page so the function returns a Media.
25 return uploadMedia(client, { data: await result.blob(), fileName: "pa
26 }

```

Slice a page range into a new PDF document:

TYPESCRIPT V2 PYTHON

```

1  import {
2      __EXPERIMENTAL__NOT_SUPPORTED_YET__transformAndWait,
3      type MediaTransformation,
4  } from "@osdk/api/unstable";
5  import type { Client, Media } from "@osdk/client";
6  import { uploadMedia } from "@osdk/functions";
7
8  export default async function sliceFirstTenPages(
9      client: Client,
10     media: Media,
11 ): Promise<Media> {
12     const transformation: MediaTransformation = {
13         $documentToDocument: {
14             $encoding: "pdf",
15             $operation: {
16                 $slicePdfRange: {
17                     $startPageInclusive: 0,
18                     $endPageExclusive: 10,
19                     $strictlyEnforceEndPage: false,
20                 },
21             },
22         },
23     };
24
25     const result = await client(
26         __EXPERIMENTAL__NOT_SUPPORTED_YET__transformAndWait,
27     ).transformAndWait({ mediaReference: media.getMediaReference(), trans

```

API Reference ↗

Send feedback

```

27     if (!result.ok) {
28         throw new Error(`Slice failed: ${result.status}`);
29     }
30     // Re-upload the sliced PDF so the function returns a Media.
31     return uploadMedia(client, { data: await result.blob(), fileName: "sliced.pdf" });
32 }

```

Example: Annotate every page with detected bounding boxes

To produce a visual debugging output (each PDF page rendered with its OCR-detected bounding boxes drawn on top) chain three transformations for every page. For each page, render the page as an image, OCR the same page to recover word/line bounding boxes, then re-upload the rendered image and annotate it with `$image.$annotate`. The page count comes from `get_media_full_metadata()`, which is currently available in Python only. Each step calls `transform_and_wait` and feeds the bytes of the previous step into the next as a fresh upload, and each annotated page is re-uploaded so the function returns one `Media` per page.

PYTHON PYTHON (ASYNC)

```

1  from foundry_sdk.v2.media_sets.models import (
2      AnnotateImageOperation,
3      Annotation,
4      BoundingBox,
5      BoundingBoxGeometry,
6      DocumentMediaItemMetadata,
7      DocumentToImageTransformation,
8      DocumentToTextTransformation,
9      ImageTransformation,
10     OcrHocrOutputFormat,
11     OcrLanguageWrapper,
12     OcrOnPageOperation,
13     OcrParameters,
14     PngFormat,
15     RenderPageOperation,
16 )
17 from foundry_sdk_runtime.media import Media
18 from foundry_sdk_runtime.media_sets import MediaSet
19 from foundry_sdk_runtime.media_sets.api import function
20 from foundry_sdk import FoundryClient

```

API Reference ↗

```

20
21 @function(beta=True)
22 def annotate_pdf_with_ocr_boxes(document: Media) -> List[Media]:

```

Send feedback

```

23 """Render every page of a PDF, OCR each page to find text bounding boxes,
24 draw them on the rendered image, and return one annotated Media per page.
25 client = FoundryClient()
26 media_reference = document.get_media_reference()
27
28 # Use the full metadata (Python only) to discover the page count.
29 metadata = document.get_media_full_metadata().item_metadata
30 if not isinstance(metadata, DocumentMediaItemMetadata) or metadata.page_count is None:
31     raise ValueError("Expected a PDF document with a known page count")
32
33 annotated_pages: list[Media] = []
34 for page_number in range(metadata.pages):
35     # 1. Render the page as a PNG.
36     rendered_png: bytes = client.ontology.media.transform_and_wait(
37         media_reference=media_reference,
38         transformation=DocumentToImageTransformation(
39             encoding=PngFormat(),
40             operation=RenderPageOperation(page_number=page_number, width=width, height=height),
41         ),
42     )
43
44     # 2. OCR the same page in hOCR mode to get word-level bounding boxes.
45     hocr_bytes: bytes = client.ontology.media.transform_and_wait(
46         media_reference=media_reference,
47         transformation=DocumentToTextTransformation(
48             operation=OcrOnPageOperation(
49                 page_number=page_number,
50                 parameters=OcrParameters(
51                     output_format=OcrHocrOutputFormat(),
52                     languages=[OcrLanguageWrapper(language="ENG")],
53                 ),
54             ),
55         ),
56         poll_timeout_seconds=120.0,
57     )
58
59     # 3. Parse hOCR for bounding boxes in image pixels.
60     # The parse_hocr_bounding_boxes helper is omitted here; see the docs for more details.
61     boxes: list[tuple[str, BoundingBox]] = parse_hocr_bounding_boxes(hocr_bytes)
62
63     # 4. Re-upload the rendered PNG as a temporary media item.
64     rendered_media = client.ontology.media.upload_media(
65         body=rendered_png, filename=f"page-{page_number}.png"
66     )
67
68     # 5. Annotate the rendered page with a Media item.
69     annotated_bytes: bytes = client.ontology.media.transform_and_wait(
70         media_reference=rendered_media.get_media_reference(),

```

API Reference ↗

Send feedback

```

71         transformation=ImageTransformation(
72             encoding=PngFormat(),
73             operations=[
74                 AnnotateImageOperation(
75                     annotations=[
76                         Annotation(
77                             geometry=BoundingBoxGeometry(bounding_box
78                             label=label,
79                         )
80                     for label, box in boxes
81                 ],
82             ),
83         ],
84     ),
85 )
86
87     # 6. Re-upload the annotated page so the function returns a Media
88     annotated_pages.append(
89         client.ontology.media.upload_media(
90             body=annotated_bytes, filename=f"page-{page_number}-annot
91         )
92     )
93
94     return annotated_pages

```

The `parse_hocr_bounding_boxes` helper is omitted here. Any HTML parser (such as `lxml` or `BeautifulSoup`) can extract `class="ocrx_word"` elements and their `title="bbox X1 Y1 X2 Y2 ..."` attributes, which you convert into `BoundingBox(left=X1, top=Y1, width=X2-X1, height=Y2-Y1)`.

TypeScript v1

 Foundry enacts strict memory limits when executing TypeScript v1 functions. To ensure you do not exceed those memory limits, you should only interact with media files under 20MB.

[API Reference ↗](#)

 Uploading media within a function is not supported in TypeScript v1. To upload media, use the `upload_media` function below cover passing existing media into Ontology edits and operations.

[Send feedback](#)

properties of object types.

Setting existing media on an object

Use Ontology edit functions to attach existing media items to objects:

TYPESCRIPT V1

```
1 import { OntologyEditFunction, MediaItem } from "@foundry/functions-api";
2 import { Aircraft } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5     @OntologyEditFunction()
6     public async setExistingMediaToObject(
7         aircraft: Aircraft,
8         mediaItem: MediaItem
9     ): Promise<void> {
10         // Ontology Edits with passed in MediaItems are supported
11         aircraft.myMediaProperty = mediaItem;
12     }
13 }
```

Media item parameter on object types

The following example shows the `isAudio` media operations on a media reference property of an object type:

TYPESCRIPT V1

```
1 MediaItem.isAudio(objectType.mediaReferenceProperty)
```

[API Reference ↗](#)

For more information on media data

[Send feedback](#)

You can access a media item by selecting the media reference property on the object. The signature for the method is as follows:

Typescript v1

```
1 // Blob is a standard JavaScript type, representing a file-like object
2 // https://developer.mozilla.org/en-US/docs/Web/API/Blob
3 readAsync(): Promise<Blob>;
```

Get media metadata

You can access a media item's metadata. The signature for the method is as follows:

Typescript v1

```
1 getMetadataAsync(): Promise<IMediaMetadata>;
```

Type guards

Type guards in TypeScript v1 allow you to access functionality that is specific to certain media types. The following type guards can be used on media item metadata:

- `isAudioMetadata()`
- `isDicomMetadata()`
- `isDocumentMetadata()`
- `isImageryMetadata()`
- `isSpreadsheetMetadata()`
- `isUntypedMetadata()`
- `isVideoMetadata()`

An [API Reference ↗](#) could use the imagery type guard to pull out image specific metadata fields:

Send feedback

Typescript v1

```
1 const metadata = await myObject.mediaReference?.getMetadataAsync();
2 if (isImageryMetadata(metadata)) {
3     const imageWidth = metadata.dimensions?.width;
4     ...
5 }
```

You can also use type guards on the media item namespace, which then gives you access to more methods on the type-specific media item. The type guards you can use here are:

- `MediaItem.isAudio()`
- `MediaItem.isDicom()`
- `MediaItem.isDocument()`
- `MediaItem.isImagery()`
- `MediaItem.isSpreadsheet()`
- `MediaItem.isVideo()`

Document-specific operations

Text extraction

To extract text from a document, you can either use optical character recognition (OCR) or extract embedded text on the media item.

For machine-generated PDFs, it may be faster and/or more accurate to extract text embedded digitally in the PDF rather than using optical character recognition (OCR). Below is an example of text extraction usage:

Typescript v1

```
1 extractTextAsync(options: IDocumentExtractTextOptions): Promise<string>;
```

API Reference ↗

When using TypeScript v1, the following can optionally be provided as

Send feedback

- `startPage` : The zero-indexed start page (inclusive, can be empty)
- `endPage` : The zero-indexed end page (exclusive, can be empty).

If both the `startPage` and `endPage` are left empty, the text for all pages in the document will be returned.

For non-machine-generated PDFs, it would be best to use the OCR method for extracting text.

TYPESCRIPT V1

```
1 ocrAsync(options: IDocumentOcrOptions): Promise<string[]>;
```



The following can optionally be provided as a TypeScript object:

- `startPage` : The zero-indexed start page (inclusive).
- `endPage` : The zero-indexed end page (exclusive).
- `languages` : A list of languages to recognize (can be empty).
- `scripts` : A list of scripts to recognize (can be empty).
- `outputType` : Specifies the output type as `text` or `hocr`.

Remember that you need to use type guards in order to access media-type specific operations. Here's an example of using the `isDocument()` type guard to then perform OCR text extraction:

TYPESCRIPT V1

```
1 import { MediaItem } from "@foundry/functions-api";
2 import { ArxivPaper } from "@foundry/ontology-api";
3
4 @Function()
5 public async firstPageText(paper: ArxivPaper): Promise<string | undefined> {
6     if (MediaItem.isDocument(paper.mediaReference)) {
7         const text = (await paper.mediaReference.ocrAsync({ endPage: 1,
8         return text;
9     }
10 }
```



API Reference ↗

Send feedback

```
11     return undefined;
12 }
```

Audio-specific operations

Transcription

Audio media items support transcription using the transcribe method. The signature is as follows:

TYPESCRIPT V1

```
1 transcribeAsync(options: IAudioTranscriptionOptions): Promise<string>;
```

The following can optionally be passed in to specify how the transcription should run:

- `language` : The language to transcribe, passed using the `TranscriptionLanguage` enum.
- `performanceMode` : Runs transcriptions in `More Economical` or `More Performant` mode, passed using the `TranscriptionPerformanceMode` enum.
- `outputFormat` : Specifies the output format by passing an object of type `plainTextNoSegmentData` (plain text) or `pttml`. `pttml` is a [TTML-like](#) format where the object also takes a Boolean `addTimestamps` parameter if the type is `plainTextNoSegmentData`.

An example of providing options for transcription:

TYPESCRIPT V1

```
1 import { Function, MediaItem, TranscriptionLanguage, TranscriptionPerformanceMode } from "@foundry/ontology-api";
2 import { AudioFile } from "@foundry/ontology-api";
3
4 export async function transcribeAudioFile(file: AudioFile): Promise<string|undefined> {
5     if (MediaItem.isAudio(file.mediaReference)) {
6         return await file.mediaReference.transcribeAsync({
7             language: TranscriptionLanguage.ENGLISH,
```

API Reference ↗

Send feedback

```
9         performanceMode: TranscriptionPerformanceMode.MORE_ECONOMICAL
10         outputFormat: {type: "plainTextNoSegmentData", addTimestamps:
11             });
12     }
13
14     return undefined;
15 }
```

PREVIOUS

← Attachments

NEXT

Models / Functions on models →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Functions on models

You can deploy models in the context of the Ontology by using functions that invoke models during their runtime. Model functions are automatically generated wrappers around live model deployments that can be imported into a functions repository and called from your code. This enables you to add custom business logic around model predictions, integrate models with Ontology objects, or orchestrate multiple model calls.

Overview

Model functions can be published from two types of deployments:

- **Direct model deployments**: Published directly from the model page.
- **Modeling Objective live deployments**: Published from the deployment details page in a Modeling Objective.

Both methods create a function with the same input and output API as your model. For details on function behavior, version upgrades, and configuration options, see the [Model functions developer guide](#).

Model functions are fully supported in TypeScript v1, TypeScript v2, and Python functions.

Import a live deployment in a repository

Once a function for a live deployment has been created either [on the model itself](#) or in a repository, it must be imported for usage in a specific repository. Select **Add** and **Functions** in the **Resource Imports** menu on the left side bar. The functions are searchable by the function name that was chosen during

API Reference ↗

Send feedback

Python and TypeScript v2 repositories only support model functions that are tied to an ontology, as described in the section below.

Notes

For TypeScript v1 repositories, it is also possible to import a model function by selecting **Models**, which is functionally equivalent to importing it under **Query Functions**.

Legacy function on models that use the **API Name** card in Modeling Objectives instead of the preferred [model publication dialog](#) can still be imported under the **Modeling Objectives** section, but are being sunset.

Ontology or space-bound functions

Starting from February 2026 onwards, all new model functions will be tied to an ontology. This is a prerequisite for usage in [TypeScript v2 and Python functions](#), which only allow ontology resource imports. Prior to this date, model functions were tied to the model's [space](#). TypeScript v1 allows importing both types of model functions, but the import and usage semantics vary slightly as [detailed below](#).

To check if a function is bound to an ontology, navigate to your model: model functions that are not bound to an ontology will indicate that a migration is available. [Learn more on migrating your function to be ontology-bound](#).

Call a model function from Python or TypeScript v2 functions

Once a function for a live deployment has been created, it must be imported for usage in a specific repository. It can then be queried [like any query function](#).

 API Reference  Python

Send feedback


```

1 from functions.api import function, Double
2 from foundry_sdk_runtime import AllowBetaFeatures
3 from ontology_sdk import FoundryClient
4 from ontology_sdk.ontology.objects import Flight
5
6
7 @function(beta=True)
8 def predict_flight_delays(flight: Flight) -> Double:
9     # Prepare the input to match the model function's API.
10    model_output = FoundryClient().ontology.queries.flight_model_deployme
11        df_in=[
12            {
13                "lastArrivalTime": flight.last_arrival_time,
14                "lastExpectedArrivalTime": flight.last_expected_arrival_t
15            },
16        ]
17    )
18    return model_output.df_out[0].prediction

```

Example code: TypeScript v2

```

1 import { Client } from "@osdk/client";
2 import { Double } from "@osdk/functions";
3 import { flightModelDeployment, Flight } from "@ontology/sdk";
4
5 async function predictFlightDelays(client: Client, flight: Flight): Promi
6     // Prepare the input to match the model function's API.
7     const modelOutput = await client(flightModelDeployment).executeFunci
8         "df_in": [
9             {
10                 "lastArrivalTime": flight.lastArrivalTime,
11                 "lastExpectedArrivalTime": flight.lastExpectedArrivalTime
12             },
13         ]
14     });
15     return modelOutput.df_out[0].prediction;
16 }

```

API Reference ↗

predictFlightDelays;

Send feedback

Write a model-backed TypeScript v1 function

If the model function is registered to a space

We first need to import the function:

```
1 // If the model function is tied to a space and not to an ontology,
2 // copy the import snippet from the Resource imports sidebar.
```

Then, write a function that takes a flight, prepares data for the model, and interprets the result of the model execution. The model is imported as an asynchronous function that respects the [model's input and output specification or API](#). From this, TypeScript can ensure, at compile time, that the correct data structure is sent to and received from the model deployment.

Note that if your model's API expects a single tabular input and output, the associated function will accept single TypeScript objects with properties corresponding to the columns specified for the input if the [Enable row-wise processing option](#) is enabled, which is the default. The `predictFlightDelaysRowWise` function below demonstrates this pattern. Alternatively, consider [using an Object or ObjectSet directly in the model API](#) to facilitate use of your model with objects in functions.

The `predictFlightDelays` function below returns a `FunctionsMap`, which is the TypeScript v1 type used to return a map keyed by objects or scalar values. In this example, it maps each `Flight` object to its predicted delay value.

```
1 import { Function, Double, FunctionsMap } from "@foundry/functions-api";
2 import { Flight } from "@foundry/ontology-api";
3
4 @Function()
5 public async predictFlightDelaysRowWise(flight: Flight): Promise<Double>
6     // Prepare the input to match the model function's API.
7     // The model function expects a single flight.
8     // If you'd like to process multiple flights at a time,
9     // edit your model function and uncheck "Enable row-wise processing".
10
11 // Note you can also use an Object directly in the model API
```

API Reference ➤

Send feedback

```

12 // to avoid tedious mapping between a model API and an object type's
13 const modelInput = {
14     "lastArrivalTime": flight.lastArrivalTime,
15     "lastExpectedArrivalTime": flight.lastExpectedArrivalTime,
16 };
17 // Call the Live deployment.
18 const modelOutput = await FlightModelDeploymentRowWise(modelInput);
19 return modelOutput.prediction;
20 }
21
22 @Function()
23 public async predictFlightDelays(flights: Flight): Promise<FunctionsMap<F
24     let functionsMap = new FunctionsMap();
25     // Prepare the input to match the model function's API,
26     // for the case where row-wise processing is not enabled.
27
28     // Note you can also use an ObjectSet directly in the model API
29     // to avoid tedious mapping between a model API and an object type's
30     const dfIn = flights.map(flight => ({
31         "lastArrivalTime": flight.lastArrivalTime,
32         "lastExpectedArrivalTime": flight.lastExpectedArrivalTime,
33     }));
34     // Call the Live deployment.
35     const modelOutput = await FlightModelDeployment(
36         {"df_in": dfIn}
37     );
38     for (let i = 0; i < flights.length; i++) {
39         functionsMap.set(flights[i], modelOutput.df_out[i].prediction);
40     }
41     return functionsMap;
42 }

```

Note the above example assumes the following Model API:

```

1 import palantir_models as pm
2
3
4 class ExampleModelAdapter(pm.ModelAdapter):
5     ...
6
7     def get_api(cls):
8         inputs = {
9             "df_in": pm.Pandas(columns=["lastArrivalTime", "lastExpectedArrivalTime"])
10         }
11

```

API Reference ↗

Send feedback

```

12     outputs = {
13         "df_out": pm.Pandas(columns=[("prediction", float)])
14     }
15     return inputs, outputs
16     ...

```

If the function is registered to an ontology

In this case, both the import and the query syntax need to be updated as detailed by the comments in the snippet below:

```

1  import { Function, Double } from "@foundry/functions-api";
2  // Add the Queries import to use an ontology-bound model function
3  import { Queries, Flight } from "@foundry/ontology-api";
4
5  export class MyFunctions {
6      @Function()
7      public async predictFlightDelays(flight: Flight): Promise<Double> {
8          // Call your model by API Name from Queries
9          const modelOutput = await Queries.flightModelDeployment({
10              "df_in": [
11                  {
12                      "lastArrivalTime": flight.lastArrivalTime,
13                      "lastExpectedArrivalTime": flight.lastExpectedArrivalTime,
14                  },
15              ],
16          });
17          return modelOutput.df_out[0].prediction;
18      }
19  }

```

Migrating model functions to ontology-bound functions

When you migrate a space-bound model function to an ontology-bound function from the [API Reference](#)  published versions of TypeScript v1 functions that use the model will continue to work. However, the import syntax will no longer be recognized, meaning preview and tagging new releases of the repository consuming the model function will not work.

[Send feedback](#)

To update your TypeScript v1 function after migration:

1. Open your code repository and select the **Resource imports** sidebar.
2. Select the newly created version of the model function, by updating the resources.json file to the new version.
3. Update your function code to use the query function syntax as detailed above, or see the dedicated documentation on [calling a query function](#) for more details.

 Direct model function usage through the [Foundry Platform SDK](#) and the [Functions.Query](#) method will break immediately upon migration. This is because this consumption pattern refers to the function by its API name, which is migrated globally across all model function versions at once. To remediate these consumers, update the API name to its new value after the migration.

Performance considerations

Models are executed as part of the runtime of the function, therefore all standard [limits](#) apply. If your function backs an Action, there are [further limits](#) on the number of resulting edits. When calling live deployments, model input and output data is sent through the network with an upper limit of 50 MB. Including that additional throughput, the total execution time of the function cannot exceed 30 seconds. If you wish to increase this timeout limit per function, contact your Palantir representative.

← PREVIOUS
Functions on objects / Media

NEXT →
Function interfaces

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#)

[Privacy Statement](#)

[API Reference](#)

Send feedback

Function interfaces

Function interfaces allow function authors to integrate their custom logic with native Foundry features and offer a powerful way of defining contracts between consuming applications and functions.

Function interfaces define how an application or user should interact with a function. This includes the function's inputs, outputs, and errors. In other words, a function interface describes a function's signature, but a function interface is not itself a function. Function interfaces are designed to be implemented by functions.

Some Foundry applications use function interfaces to provide specialized behavior when executing functions which implement the interface, given the known inputs, outputs, and errors. Users can provide their own implementations of certain function interfaces, and Foundry can continue providing this specialized behavior. Applications within Foundry which depend on certain function interfaces can discover all functions which implement that interface.

For example, AIP Logic depends on function interfaces to allow users to bring their own LLMs into Logic functions. Specifically, the [Use LLM](#) block in AIP Logic allows users to select Palantir-provided LLMs or *registered* LLMs. Registered models are user-authored functions that have implemented a function interface provided by Foundry; for instance, the chat completion function interface. This allows AIP Logic to discover functions that have been explicitly defined as being an implementation of a chat completion, have a signature typical of a generic LLM, and return errors which AIP Logic can handle appropriately. In the future, user-provided chat completion implementations will be usable in other parts of the platform, such as Pipeline Builder and Model Catalog.

[API Reference ▸](#) [LLMs with the registered models feature](#). For the legacy method using function interfaces, see [Register an LLM using function interfaces](#) [Legacy].

Send feedback

Palantir-provided function interfaces

The following list contains the function interfaces currently provided by Palantir:

- [ChatCompletion](#)

ChatCompletion

Description:

- Functions which generate contextually relevant text responses based on multi-turn and multi-user text conversation history.
- Ideal for conversational use cases.

Foundry integrations:

- The *Use LLM* board in AIP Logic.
- Support in Pipeline Builder coming soon.

Documentation:

- [Register LLMs using function interfaces](#) [\[Legacy\]](#)

Type customization

To provide more flexibility, you are not limited to the provided types when implementing a function interface. In some cases, you may want to create your own [custom types](#). As long as a function is [compatible](#) with the function defined on the function interface, the function will be accepted by the compiler and successfully published. If the function interface defines an input type for which all fields are optional, at least one common optional field must be shared when customizing the type.

API Reference ↗

```
1 ...  
2 interface CustomParams extends GenericCompletionParams  
3     modelSpecificParam?: string  
4 }
```



Send feedback


```

5  ...
6
7  // valid implementation
8  @ChatCompletion()
9  public async myRegisteredModel(
10     messages: FunctionsGenericChatCompletionRequestMessages,
11     params: CustomParams
12 ): Promise<FunctionsGenericChatCompletionResponse> {
13     ...
14 }

```

Troubleshooting

Function interfaces are designed to be flexible and allow for a wide range of implementations. However, you may encounter errors when implementing a function interface. Here are some tips for TypeScript functions to help you avoid these errors when customizing your implementation.

Error: Function input name does not match the required input name of the function interface at the specific input position

The input names of each parameter must match the input names defined on the function interface at each specific input position. As the linting suggests, ensure that each input name has the exact same input name as declared on the function interface at each position.



Send feedback

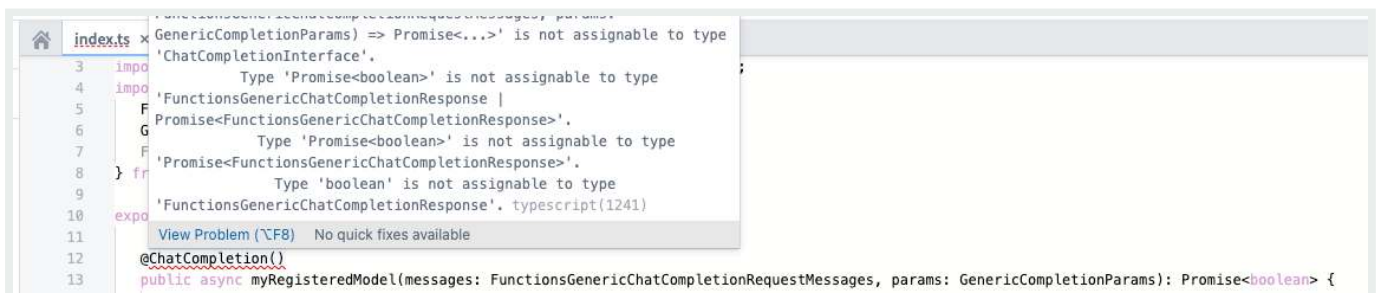
Error: Function is missing input parameter of the function interface

This error arises if the implementing function does not include every required input defined on the function interface. To resolve the error, ensure each input declared on the function interface is included in the implementing function.



Error: Type {type1} is not assignable to type {type2}

The compiler may reject the implementing function as not compatible with the function defined on the interface. If so, ensure your implementing function is compatible with the function defined on the function interface by checking the structure of each type compared to the types defined on the function interface.



PREVIOUS

NEXT

API Reference ↗

Language models in TypeScript v2 and Python functions

Send feedback

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Models / Language models in TypeScript v2 and Python functions

Language models in TypeScript v2 and Python functions

Prerequisites

To use Palantir-provided language models, [AIP must be enabled on your enrollment](#). You must also have permissions to use [AIP builder capabilities](#).

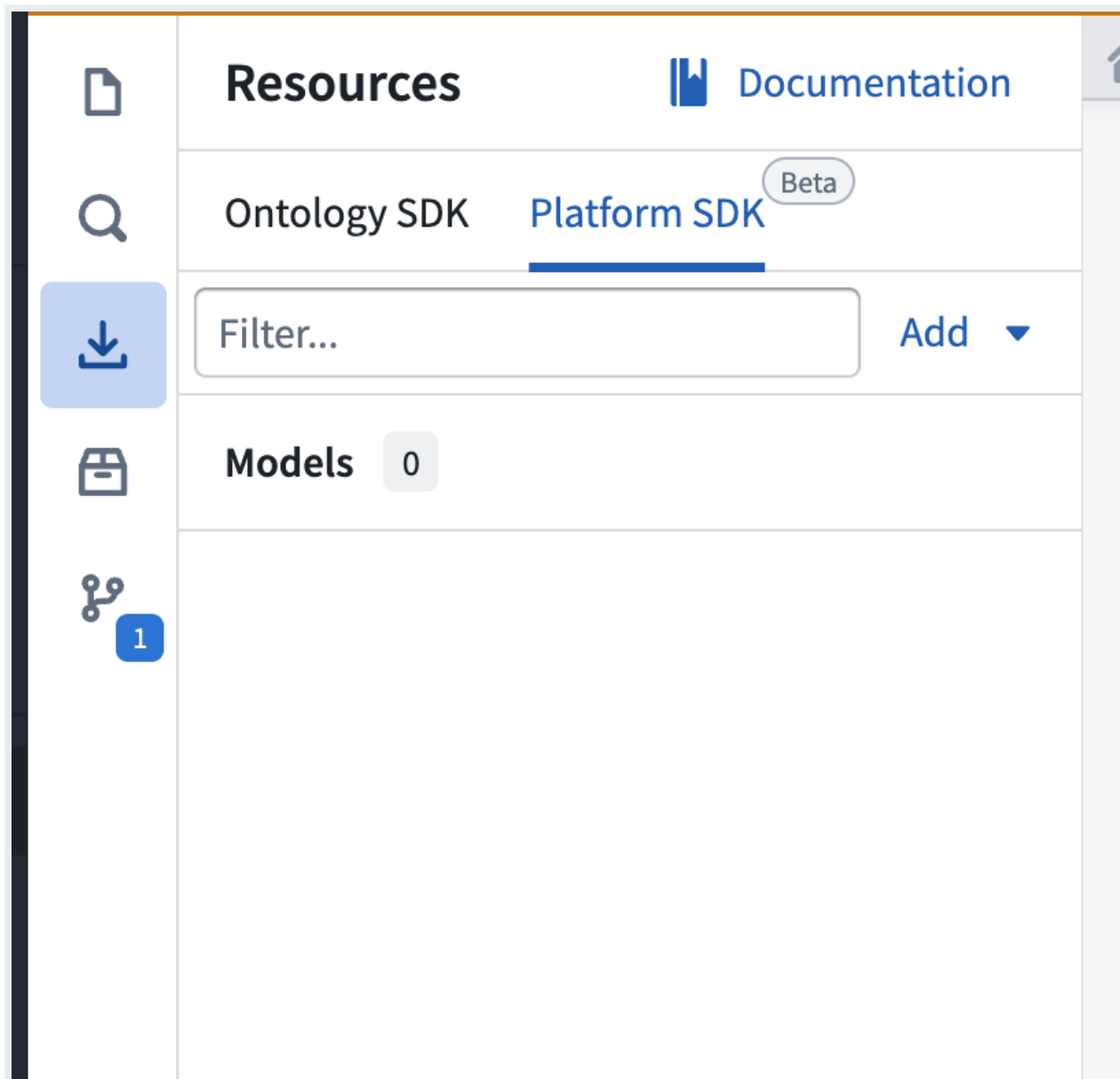
Palantir provides a set of language models that can be used within functions. [Learn more about Palantir-provided LLMs](#).

Import a language model

To begin using a language model, you must import the specific model into your functions code repository by following the steps below:

1. Open the **Platform SDK** tab in the **Resource imports** panel.

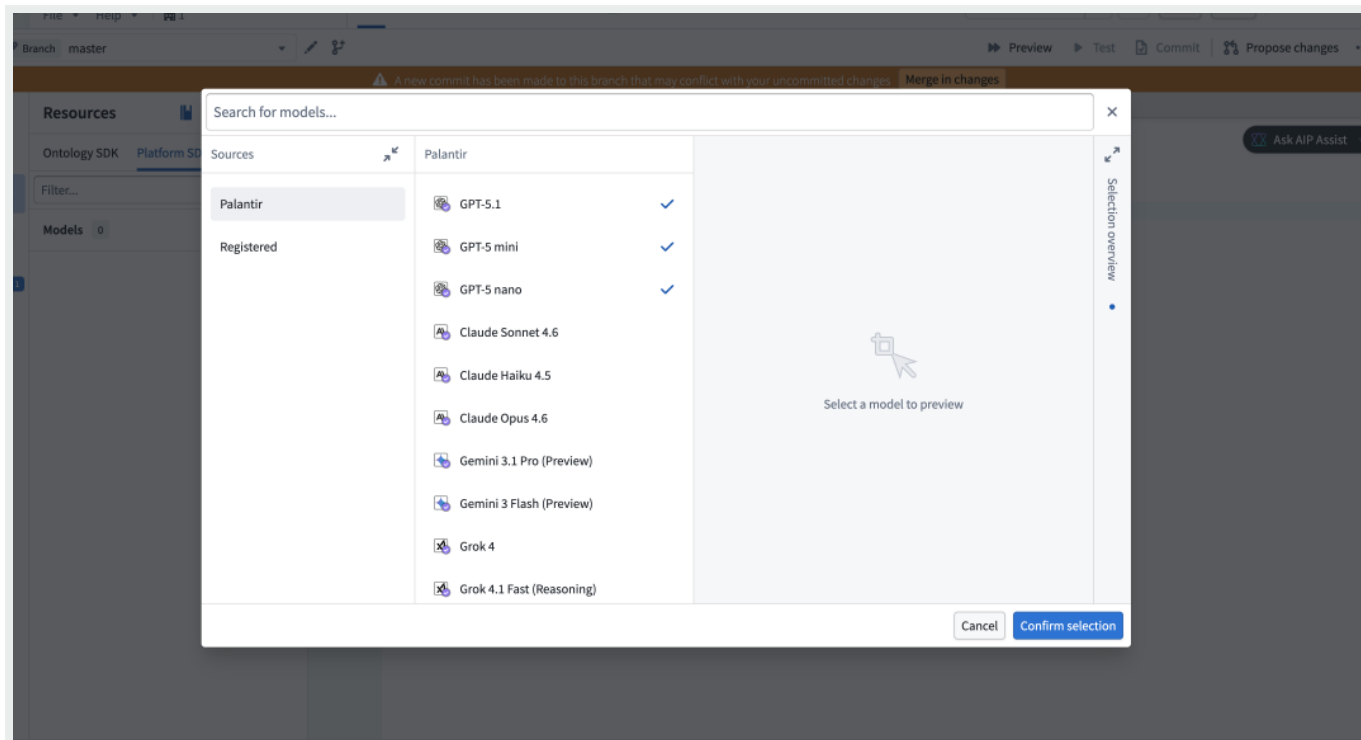
[API Reference ↗](#)[Send feedback](#)



2. To import a new language model, select **Add > Models** in the upper right corner. A window will open in which you can view available Palantir-provided and registered models.

[API Reference ↗](#)

[Send feedback](#)

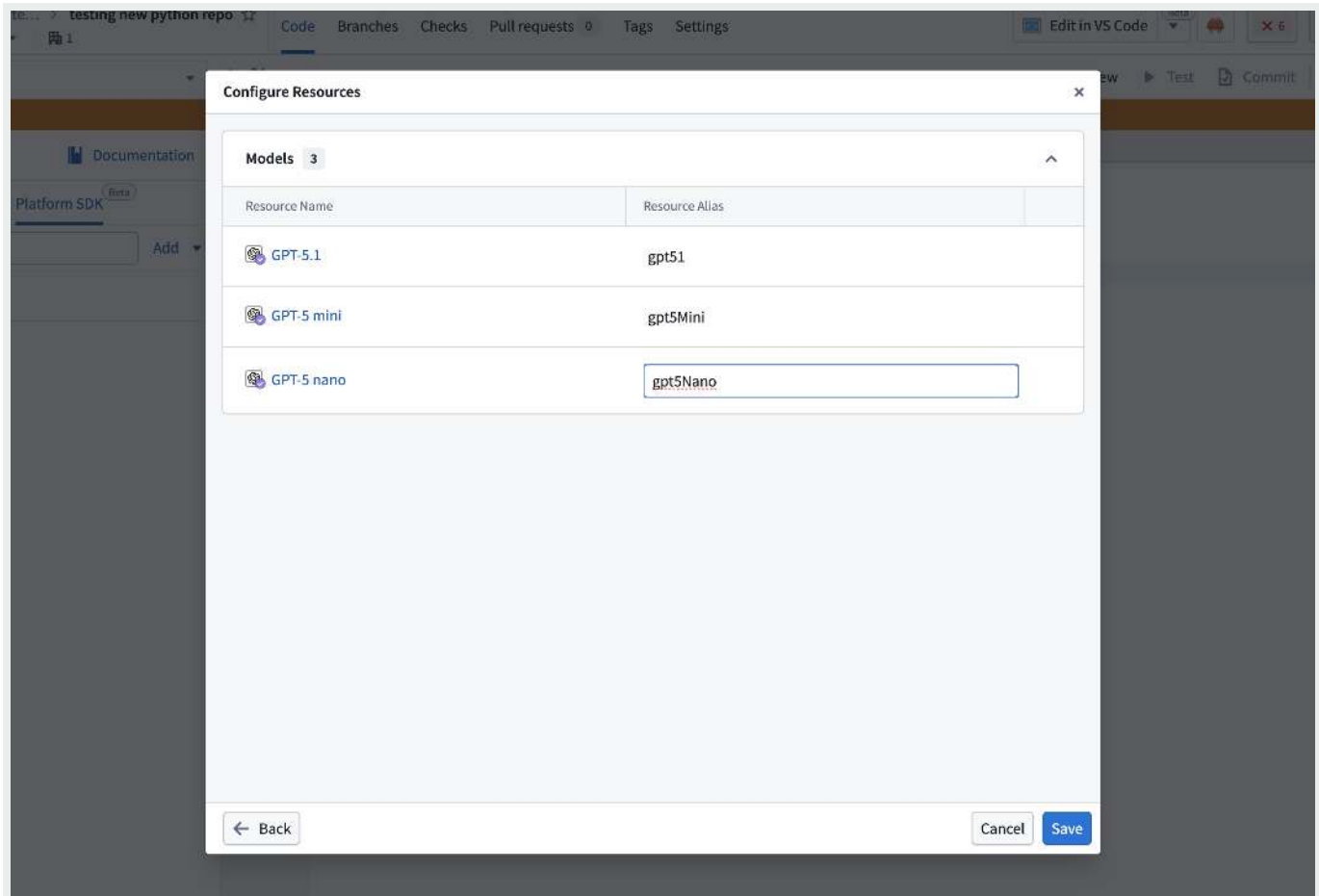


3. Select the models to import, then choose **Confirm selection**. A configuration dialog will open in which you can configure aliases for each selected model. Select the pen icon near the alias to make edits, or choose to keep the defaults.

⚠ Each model must have an alias, and the alias must be unique within the repository.

[API Reference ↗](#)

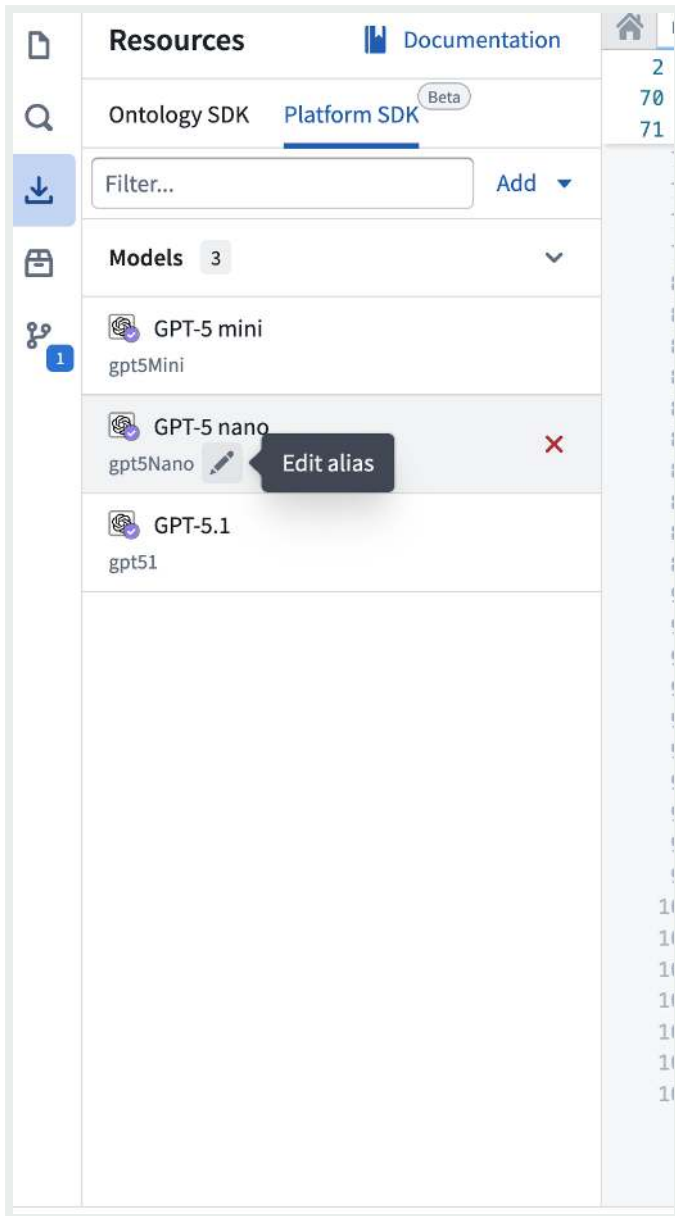
[Send feedback](#)



4. The imported models will appear in the **Platform SDK** tab in the **Resource imports** side panel. You can edit any alias inline by selecting the pen icon next to the alias.

[API Reference ↗](#)

[Send feedback](#)



Write a function that uses a language model

Language models in TypeScript v2 and Python functions use proxy endpoints to interact with models. The following example uses the [OpenAI chat completion proxy endpoint](#). You can select other providers from the documentation side panel.

i Third-party libraries, such as `openai` in the example below, are not pre-installed. Install them from the **Libraries** section of the left side panel.

API Reference ↗

To use an imported language model in your function, begin by importing the following utilities:

Send feedback

TYPESCRIPT V2 PYTHON

```
1 import { PlatformClient } from "@osdk/client";
2 import OpenAI from "openai";
3 import { Aliases } from "@osdk/functions";
4 import { getFoundryToken, getOpenAiBaseUrl, createFetch } from "@osdk/lang";
```

Directly call the model using the model aliases you configured along with the imported utilities. This approach is simpler than the TypeScript v1 workflow and reduces the need to hardcode resource identifiers.

TYPESCRIPT V2 PYTHON

```
1 export default async function callOpenAi(client: PlatformClient, prompt: string): Promise<string> {
2     const oaiClient = new OpenAI({
3         apiKey: await getFoundryToken(client),
4         baseUrl: getOpenAiBaseUrl(client),
5         fetch: createFetch(client),
6     });
7
8     const completion = await oaiClient.chat.completions.create({
9         model: Aliases.model("{MY_ALIAS}").rid,
10        messages: [
11            { role: 'user', content: prompt },
12        ],
13        reasoning_effort: "minimal",
14        max_completion_tokens: 200,
15    });
16
17    return completion.choices[0]?.message.content ?? "";
18 }
```

PREVIOUS

← [Function interfaces](#)

[API Reference ↗](#)

NEXT

[Language models in TypeScript v1](#) →
functions

[Send feedback](#)

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Models / Language models in TypeScript v1 functions

Language models in TypeScript v1 functions

⚠ The following documentation is specific to TypeScript v1 functions. For more [robust capabilities](#), including support for Ontology SDK and configurable resource requests, we recommend [migrating to TypeScript v2](#).

i Prerequisites

To use Palantir-provided language models, [AIP must first be enabled on your enrollment](#). You must also have permissions to use [AIP builder capabilities](#).

Palantir provides a set of language models which can be used within functions. [Read more about Palantir-provided LLMs](#).

Import a language model

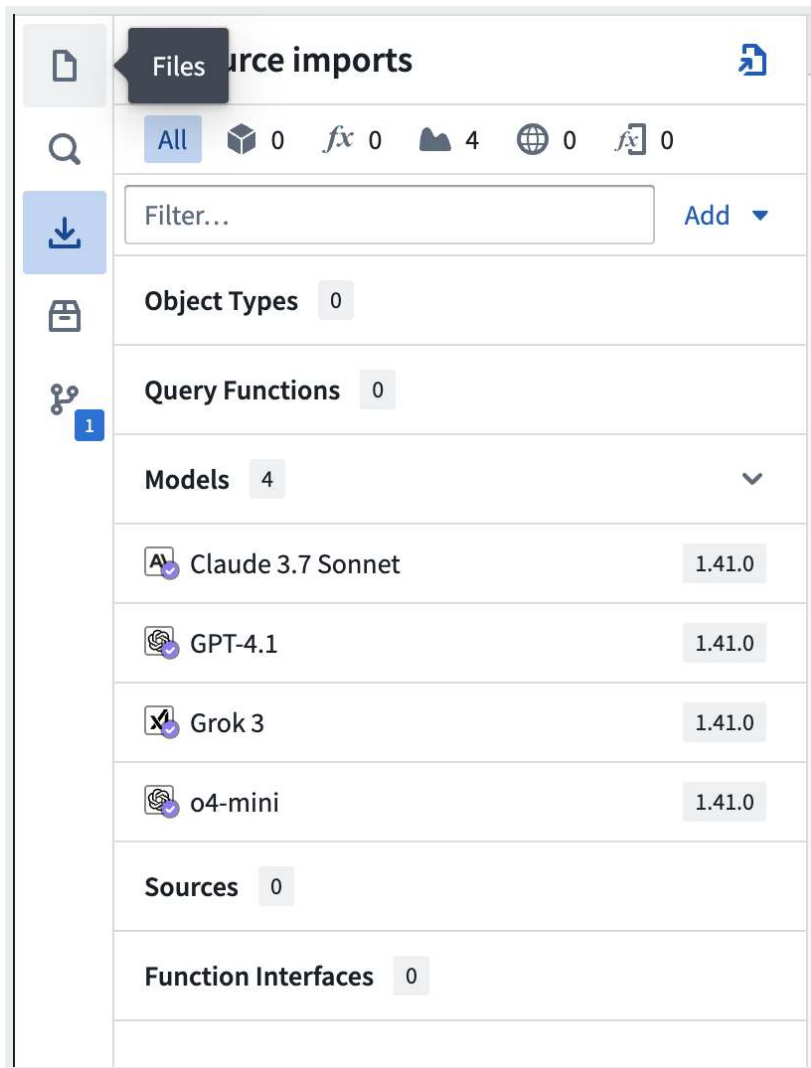
To begin using a language model, you must import the specific model into the code repository where you are writing your functions by following the steps below:

1. Navigate and open the **Model Imports** side panel to see all existing imported models.

API Reference ↗



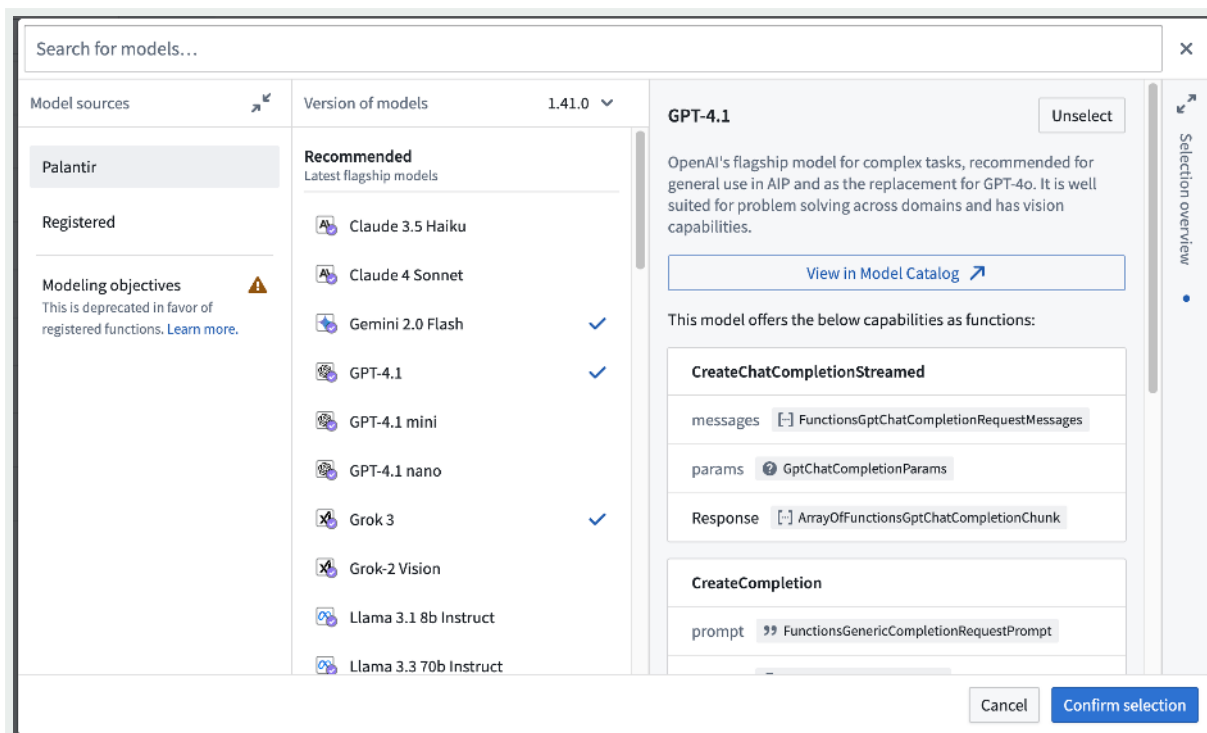
Send feedback



2. To import a new language model, select **Add** in the upper right corner of the **Resource Imports** panel, then select **Models**. This will open a new window where you will be able to see Palantir-provided models that are available to you.

[API Reference ↗](#)

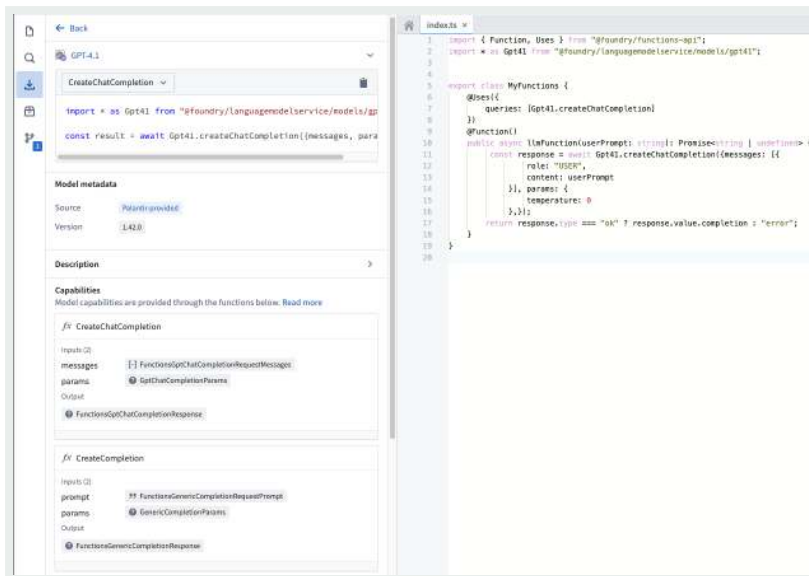
[Send feedback](#)



3. You will also see a tab where you can view custom models created through the Modeling Objectives application or direct model deployments previously. More information on using those models can be found in the [functions on models](#) documentation.
4. Choose the models you would like to import, then select **Confirm selection** to import these models into your repository. Task runner will execute the `localDev` task, generating code bindings to interact with these models.
5. After importing the language models, select the model in the sidebar to view the detailed capabilities offered by this model. You can also copy code snippets to help you import and author functions with the model.

[API Reference ↗](#)

[Send feedback](#)



Write a function that uses a language model

At this stage, you can now write a function that uses the language model you imported. For this example, assume that you imported GPT-4.1.

Begin by adding the following import statement to your file:

```
1 import { Function } from "@foundry/functions-api";
2 import { Gpt41 } from "@foundry/languagemodelservice/models";
```

Each language model will have generated methods available with strongly typed inputs and outputs. For example, the GPT-4.1 model provides `createChatCompletion`, `createChatVisionCompletion`, and `createChatCompletionStreamed` as different APIs to interact with the model. The list of capabilities could expand in later versions of the imported model.

In the following illustrative example, the provided GPT_4o model is used to run a simple sentiment analysis on a piece of text or image provided by a user. The function will classify the text as "Good", "Bad", or "Uncertain".

API Reference ↗

Send feedback

```

1  const SYSTEM_PROMPT =
2      "Provide an estimation of the sentiment the text the user has provide
3  You may respond with either Good, Bad, or Uncertain. Only choose Good or
4  sure that the text is either good or bad. If the text is neutral, or you
5
6  export class MyFunctions {
7      @Function()
8      public async llmFunction_createChatCompletion(userPrompt: string): Pr
9          const response = await Gpt41.createChatCompletion({
10              messages: [
11                  {
12                      role: "SYSTEM",
13                      content: SYSTEM_PROMPT,
14                  },
15                  {
16                      role: "USER",
17                      content: userPrompt,
18                  },
19              ],
20              params: {
21                  temperature: 0,
22              },
23          });
24          return response.type === "ok" ? response.value.completion : "erro
25      }
26
27      @Function()
28      public async llmFunction_createChatVisionCompletion(
29          userPrompt: string,
30          pngBase64String: string,
31      ): Promise<string | undefined> {
32          const response = await Gpt41.createChatVisionCompletion({
33              messages: [
34                  {
35                      role: "USER",
36                      content: [
37                          { type: "text", text: userPrompt },
38                          {
39                              type: "genericMedia",
40                              genericMedia: {
41                                  mimeType: "IMAGE_PNG",
42                                  // Base64 encoded PNG String
43                                  content: pngBase64String,
44                              },
45                          },
46                      ],

```

API Reference ↗

Send feedback

```

47         },
48     ],
49     params: {
50         temperature: 0,
51     },
52 });
53 return response.type === "ok" ? response.value.completion : "error";
54 }
55 }

```

This function can then be used throughout the platform.

Embeddings

Along with generative language models, Palantir also provides models that can be used to generate embeddings. A simple example is as follows:

```

1 @Function()
2 public async llmFunction_embeddings(inputs: string[]): Promise<Double[][]> {
3     const response = await Textembedding3large.createEmbeddings({ inputs });
4     return response.type === "ok" ? response.value.embeddings : [[]];
5 }

```

This is most commonly used to perform [semantic search](#) workflows.

Upgrade from legacy language models within functions

i Skip this step if you are starting from a new repository with no [legacy language models](#) imported.

API Reference ↗

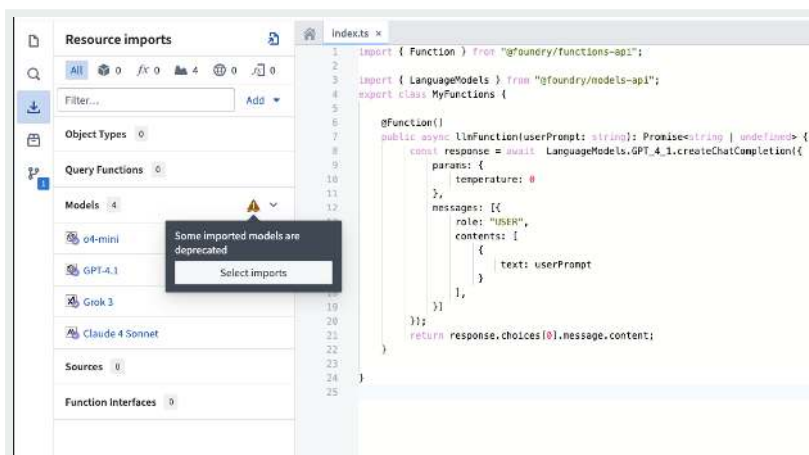
⚠ The following process will result in a compile-time break in your code. The code syntax will be updated. Refer to the code snippets in the sidebar for each.

Send feedback

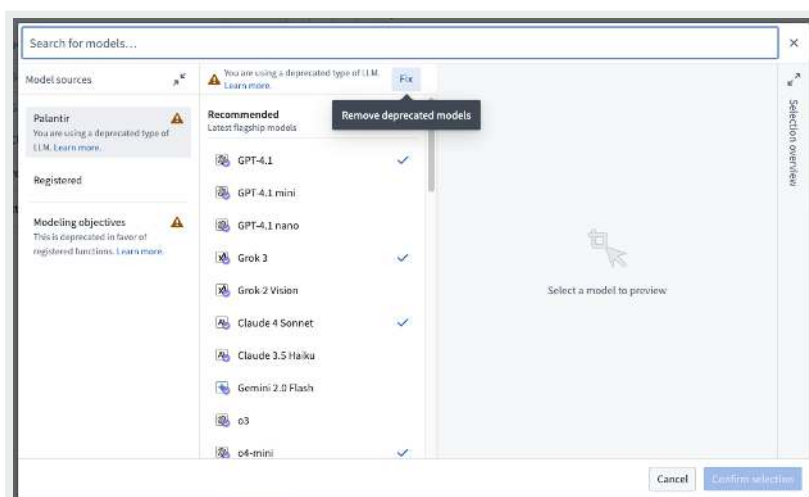
updated model to update your code.

The updated language models in functions offer more advanced capabilities, such as better support for vision and streaming. We highly recommend upgrading your repository to take advantage of the latest AIP offerings.

1. If you have existing [legacy language models](#) imported, a warning icon to upgrade will appear in the sidebar. Choose **Select imports** to open the model import dialog.

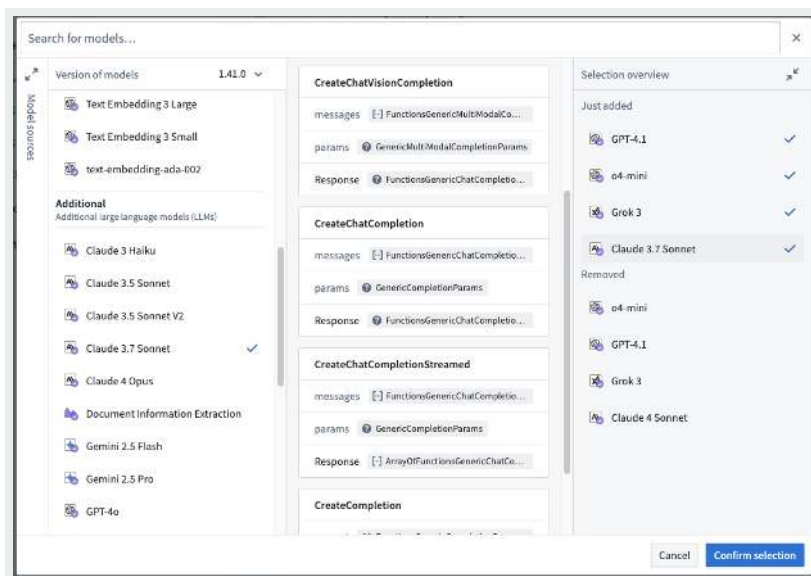


2. In the model import dialog, select **Fix** to remove any deprecated legacy language models.

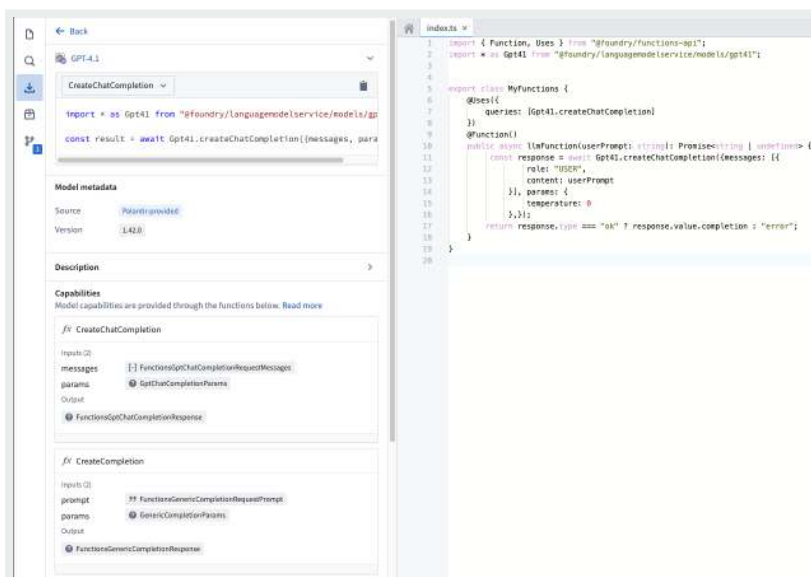


API Reference ↗ els to migrate to the updated language model versions. You can view additional capabilities supported by this version in the details panel of the model import dialog.

Send feedback



4. Now, you can view the updated model imports in the sidebar. Selecting a model will show you a details panel with code snippets to help you update your code to take advantage of the additional capabilities.



Performance considerations

Our API Reference have rate limits applied, limiting the number of tokens that may be passed over a certain time period. This will be enforced along with any standard limits that apply to functions.

Send feedback

Note: AIP feature availability is subject to change and may differ between customers.

PREVIOUS

← [Language models in TypeScript v2 and Python functions](#)

NEXT

[Legacy language models in functions](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Legacy language models within functions

⚠ Legacy

This is documentation for the **legacy** language models within functions. The [updated language models in functions](#) offer more robust capabilities, such as vision and streaming. [Upgrade your language models in functions](#) to take advantage of the latest AIP offerings.

Palantir provides a set of language models which can be used within functions. [Read more about Palantir-provided LLMs.](#)

ℹ Prerequisites

To use Palantir-provided language models, [AIP must first be enabled on your enrollment](#). You also must have permissions to use [AIP builder capabilities](#).

Import a language model

To begin using a language model, you must import the specific model into the code repository where you are writing your functions by following the steps below:

API Reference ↗ in the **Model Imports** side panel to view all existing imported models.



Send feedback

Resource imports

All

8

fx 1

4

3

Filter...

Add ▾

Models

4

GPT-4 (32k)

GPT-4

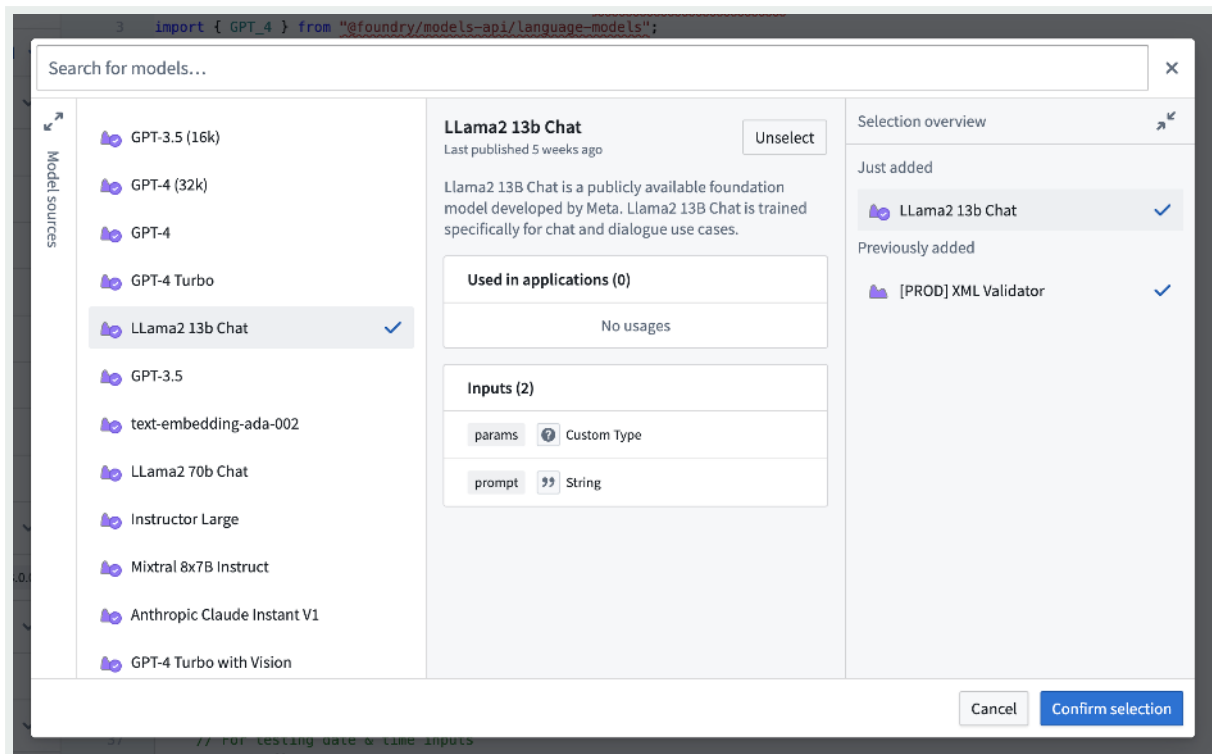
GPT-3.5 (16k)

[PROD] XML Validator

2. To import a new language model, select **Add** in the top-right corner of the **Resource Imports** panel and select **Models**. This will open a new window where you will be able to view the Palantir-provided models that are available to you.

API Reference ↗

Send feedback



3. You will also see a tab where you can view custom models which have been created through the Modeling Objectives app or direct model deployments previously. More information on using those models can be found in the [functions on models](#) documentation.
4. Select the models you would like to import, then select **Confirm selection** to import these models into your repository. Task runner will execute the `localDev` task, generating code bindings to interact with these models.
5. After importing the language models, you can now use them in your repository by adding the following import statement, replacing `GPT_4o` with the name of the language model you have imported into your repository:

```
1 import { GPT_4o } from "@foundry/models-api/language-models"
```

Writing a function that uses a language model

A [API Reference](#) ↗ now write a function that uses a language model we imported. For this example, we assume that we have imported `GPT_4o` as described above.

[Send feedback](#)

We begin by adding the following import statement to our file:

```
1 import { GPT_4o } from "@foundry/models-api/language-models"
```

Each language model will have generated methods available with strongly typed inputs and outputs. For example, the GPT_4o model provides a `createChatCompletion` method which allows the user to pass a set of messages along with additional parameters to modify the model's behavior, such as the temperature or maximum number of tokens.

In the following illustrative example, we use the provided GPT_4o model to run a simple sentiment analysis on a piece of text provided by a user. The function will classify the text as "Good", "Bad", or "Uncertain".

```
1 @Function()
2 public async sentimentAnalysis(userPrompt: string): Promise<string> {
3     const systemPrompt = "Provide an estimation of the sentiment the text
4     You may respond with either Good, Bad, or Uncertain. Only choose Good
5     sure that the text is either good or bad. If the text is neutral, or
6
7     const systemMessage = { role: "SYSTEM", contents: [{ text: systemPrompt }]
8     const userMessage = { role: "USER", contents: [{ text: userPrompt }]
9     const gptResponse = await GPT_4o.createChatCompletion({messages: [sys
10     return gptResponse.choices[0].message.content ?? "Uncertain";
11 }
```

This function can then be used throughout the platform.

Embeddings

Along with generative language models, Palantir also provides models which can be used to generate embeddings. A simple example is as follows:

API Reference ↗

```
2 public async generateEmbeddingsForText(inputs: string[])
3     const response = await TextEmbeddingAda_002.createEmbeddingsForText(inputs)
```

Send feedback

```
4     return response.embeddings;
5 }
```

This is most commonly used to perform [semantic search](#) workflows.

Performance considerations

Certain models may have rate limits applied to them, limiting the number of tokens which may be passed over a certain time period. This will be enforced along with any standard limits that apply to functions.

Note: AIP feature availability is subject to change and may differ between customers.

PREVIOUS

← [Language models in TypeScript v1 functions](#)

NEXT

[Aliases / Custom aliases](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

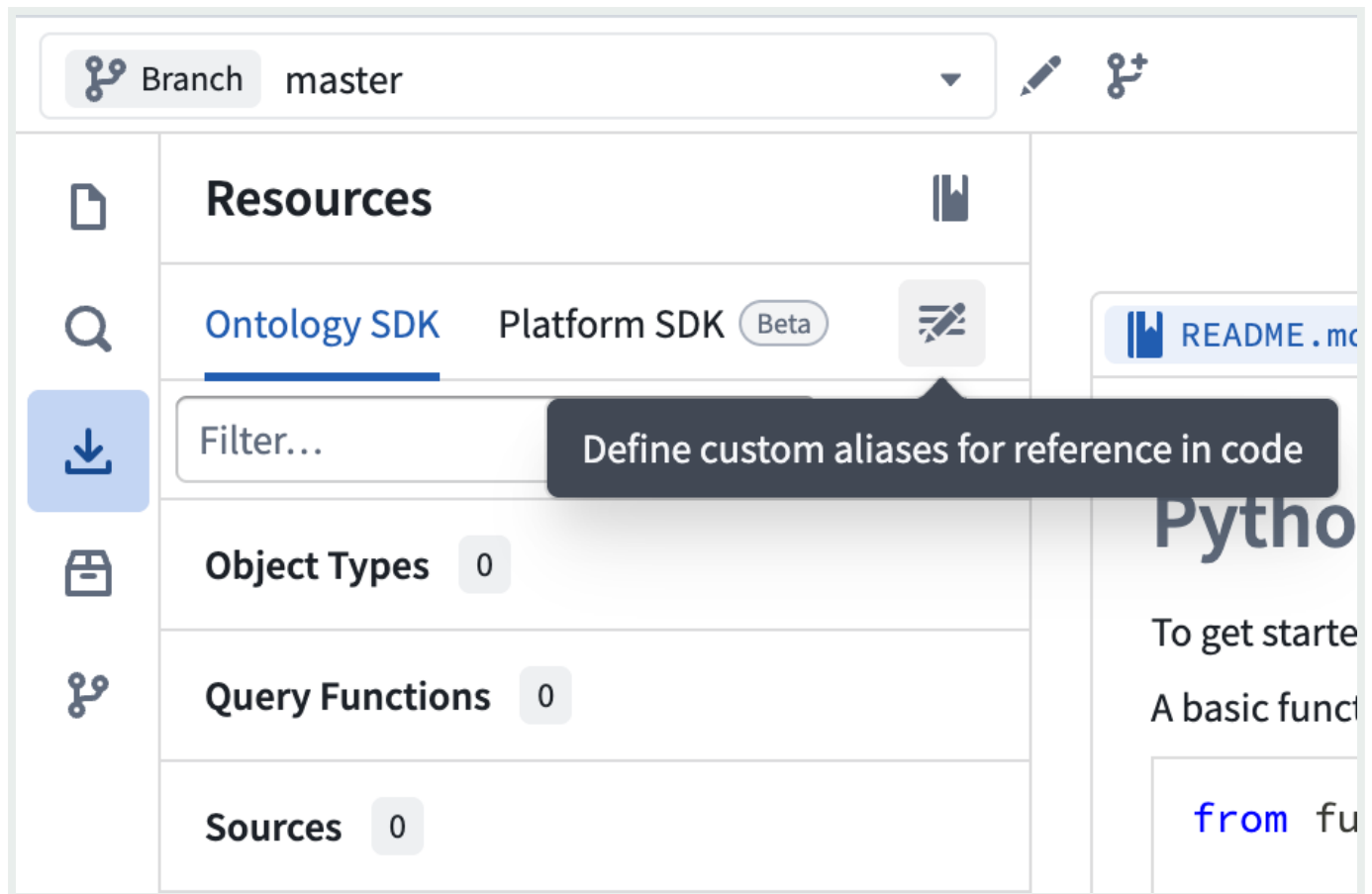
Custom aliases

Custom aliases are named references that store string values such as configuration parameters, feature flags, or environment-specific settings. By using custom aliases instead of hard-coding values, you can decouple your function logic from specific configurations and make your functions portable across environments.

Define a custom alias

To define a custom alias, open a TypeScript v2 or Python code repository and follow the steps below:

1. Open the **Resource imports** side panel and select the **Platform SDK** tab. You will see a section for custom aliases.



2. Select **New alias** to open the alias creation dialog. Provide a **Key** for the alias name and a **Value** to associate with it, then select **Create**.

[API Reference ↗](#)

[Send feedback](#)

Branch master



← Back



New alias



Key

myAlias



Keys must be unique to other custom aliases



Value

someValue

✓ Create

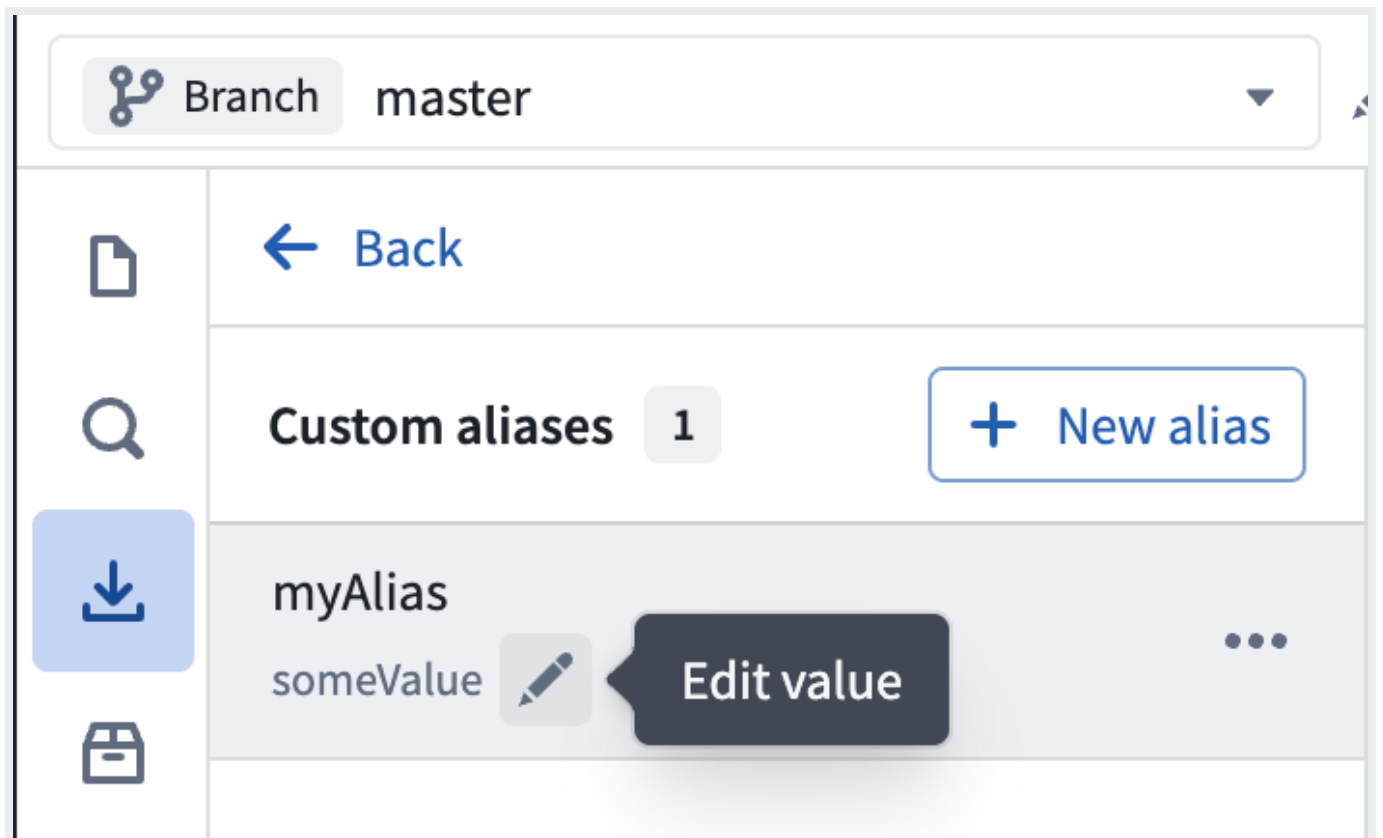
 Alias keys must be unique within the repository.

3. The custom alias will appear in the **Custom aliases** section.

Edit a custom alias

To edit an existing custom alias, navigate to the **Custom aliases** section in the **Platform SDK** tab. Select the pen icon next to the alias to edit its value inline. You can also click on the 3 d API Reference ↗ key or delete the alias altogether.

Send feedback



Use a custom alias in code

To use a custom alias in your function, import the alias utility and reference the alias by its key:

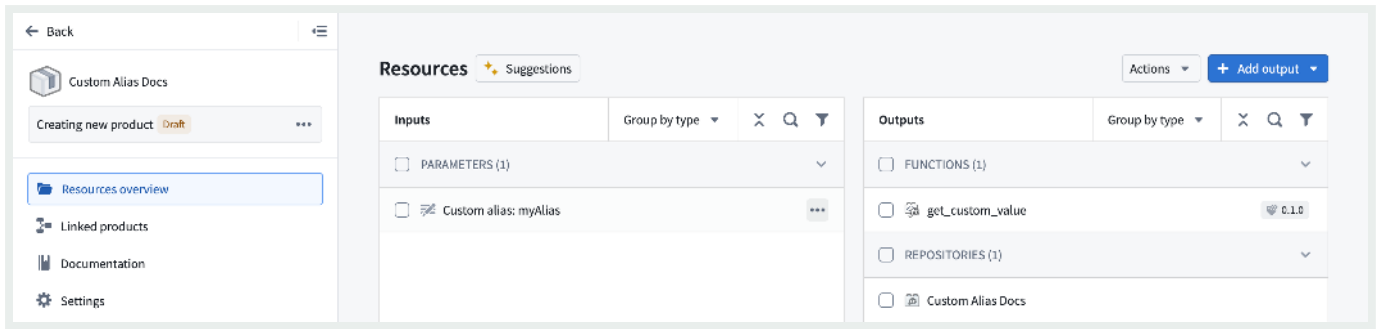


Use custom aliases with Marketplace

When you create a function that uses custom aliases to a [Marketplace product](#), the aliases automatically appear as configurable parameters under **Inputs**. Instead of hardcoding values appropriate for their environment without modifying the function source code.

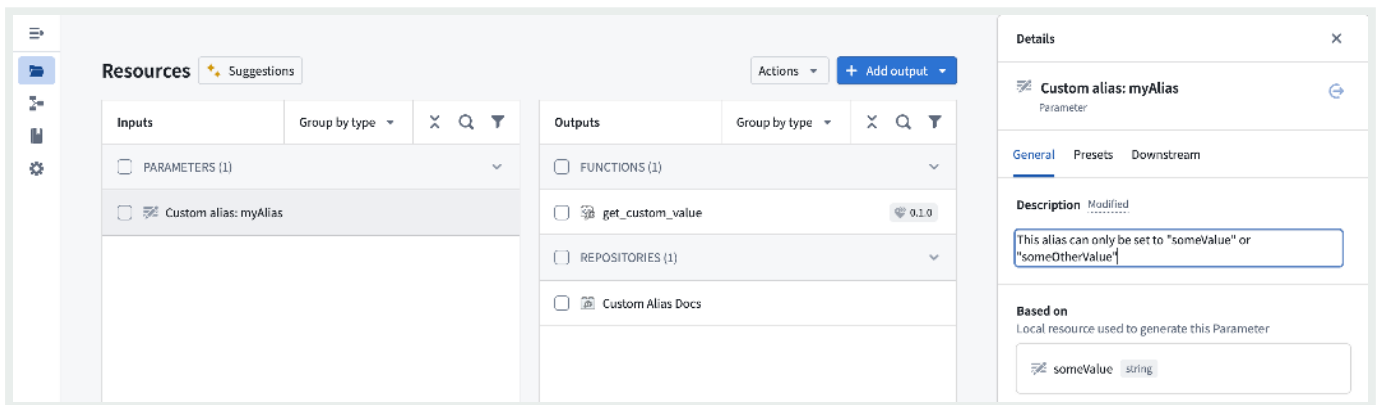
[API Reference](#) ↗

[Send feedback](#)



Set a description

To help installers understand how to configure the alias, you can add a description to the alias parameter. Select the alias under **Inputs** to open the **Details** panel, then enter a description on the **General** tab.

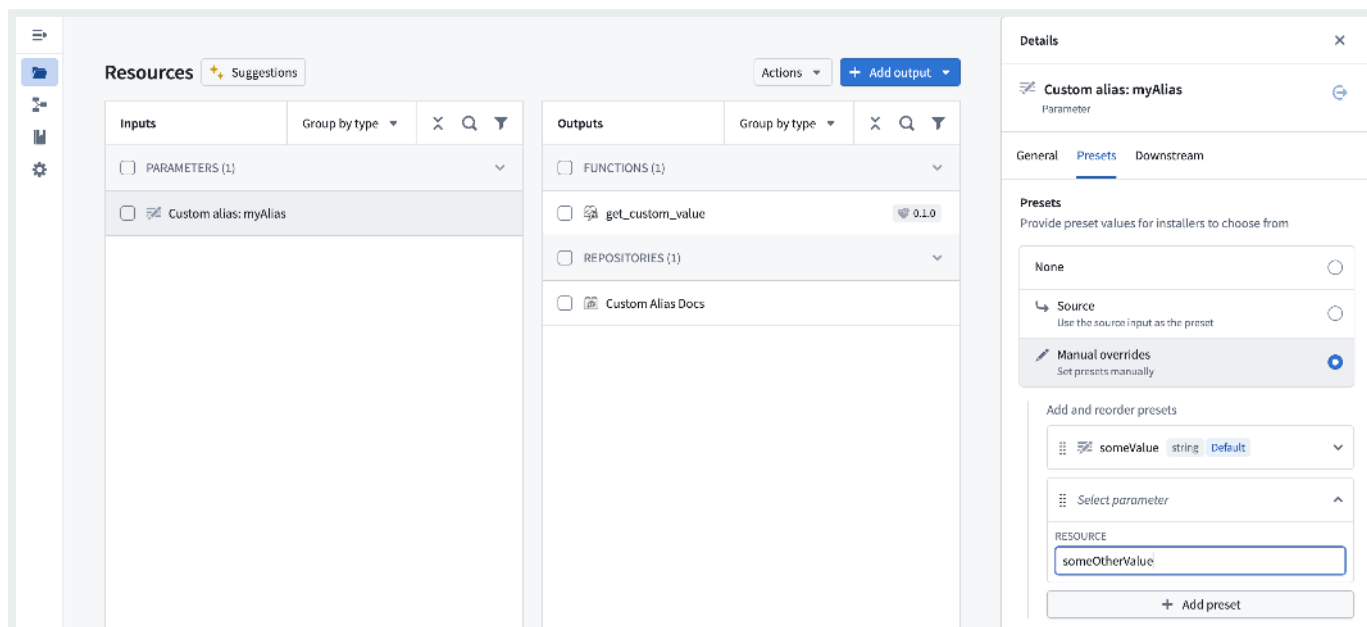


Add presets

You can define preset values for the alias that installers can choose from during installation. In the **Details** panel, select the **Presets** tab and choose **Manual overrides** to define a set of allowed values.

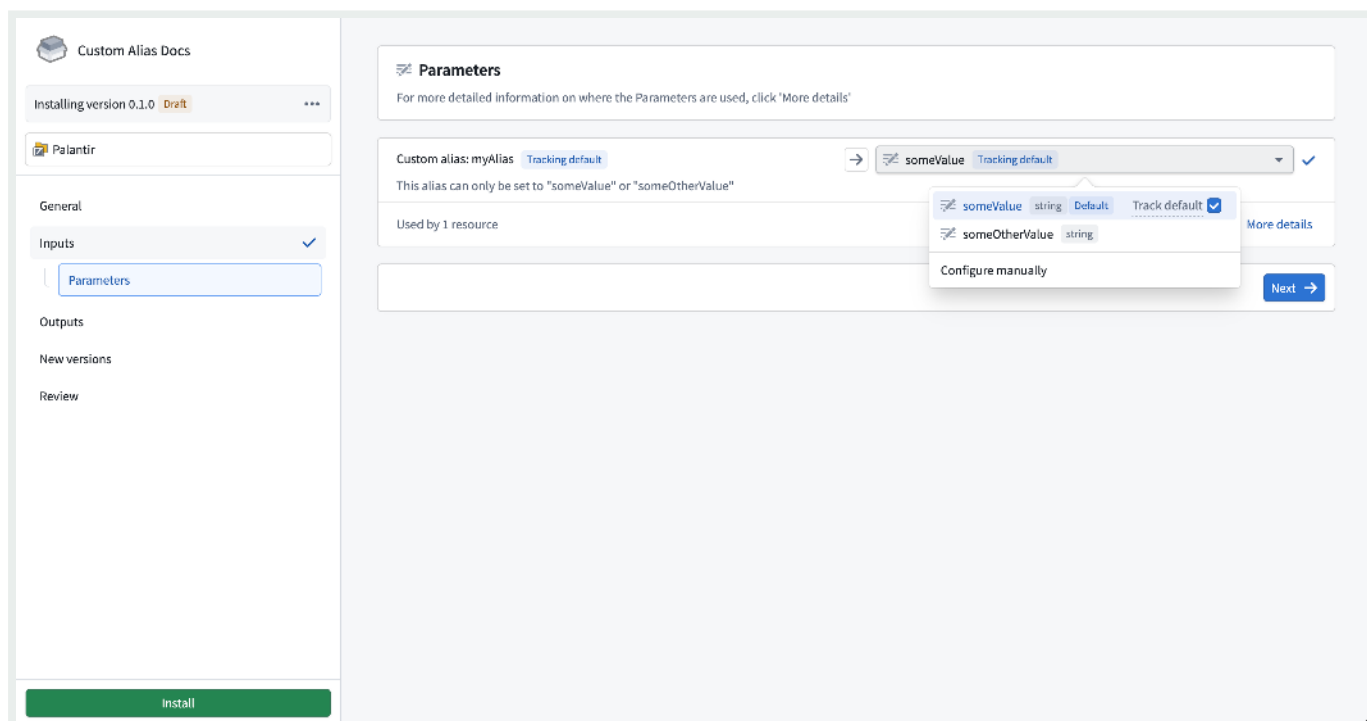
[API Reference ↗](#)

[Send feedback](#)



Installation experience

During installation, the installer will see the alias description and can choose from the preset values or configure the alias manually. After installation, the function resolves the alias to the value configured by the installer.



[API Reference ↗](#)

[PREVIOUS](#)

← [Models / Legacy language models in functions](#)

[Send feedback](#)

[Model aliases](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Aliases / Model aliases

Model aliases

Model aliases are named references to language models that provide a convenient way to reference language models in code.

For a full walkthrough of using language models in functions, see [Language models in TypeScript v2 and Python functions](#).

Define a model alias

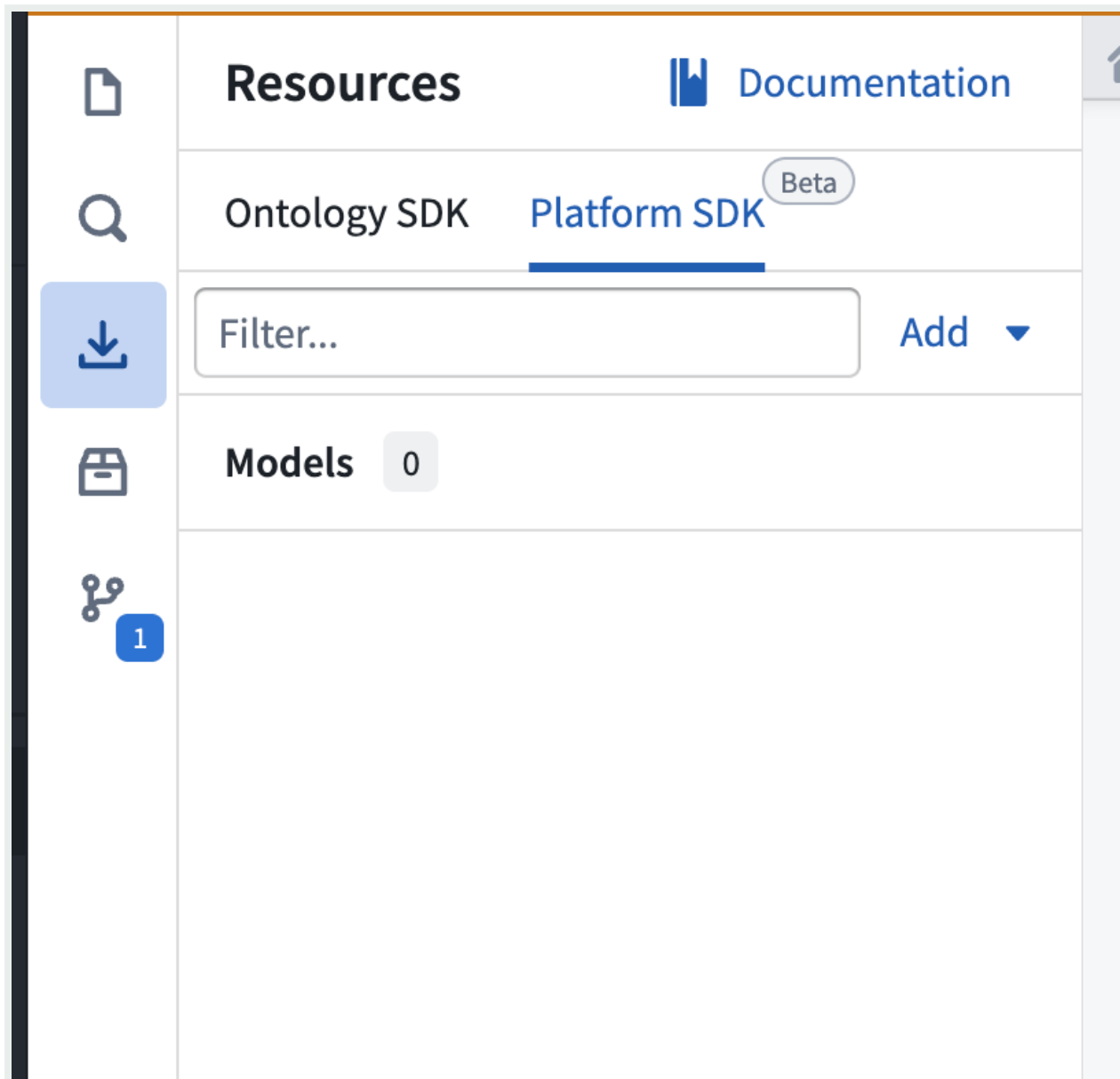
To define a model alias, open a TypeScript v2 or Python code repository and follow the steps below:

1. Open the **Platform SDK** tab in the **Resource imports** panel.

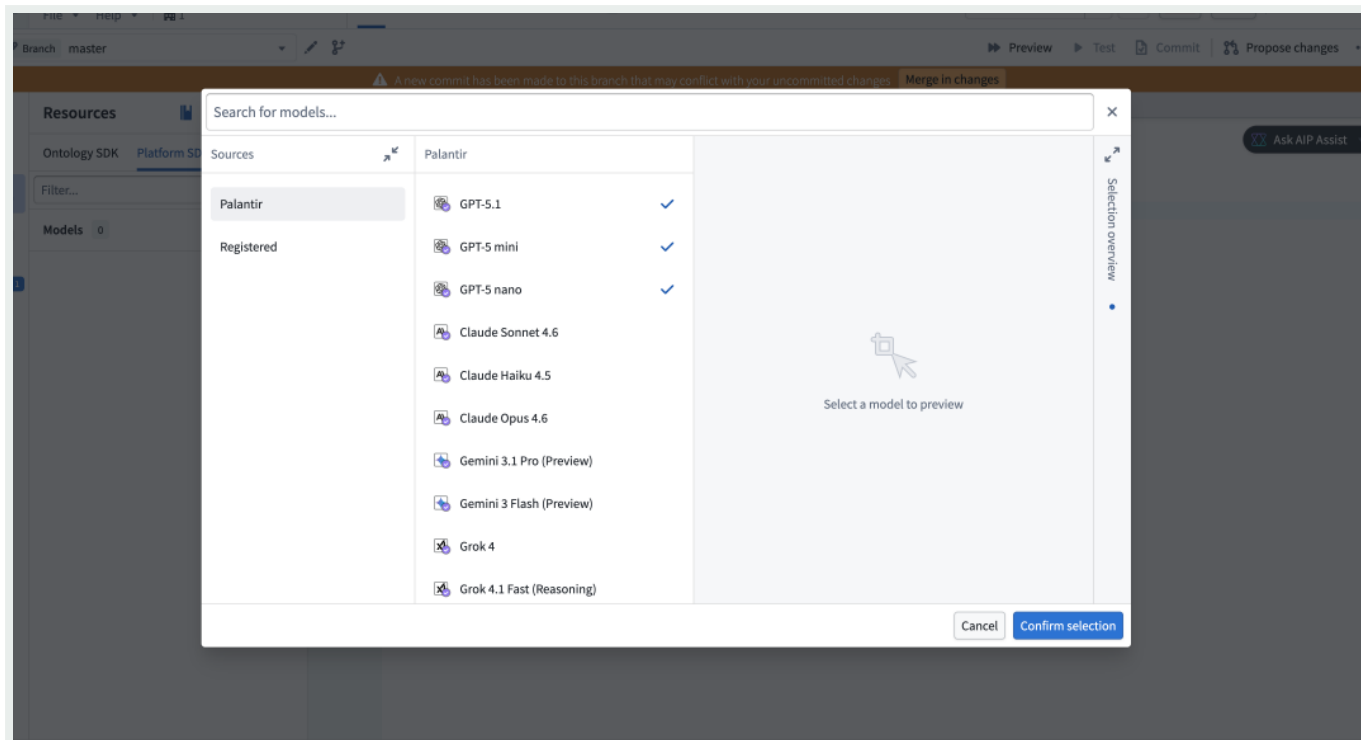
API Reference ↗



Send feedback



2. To import a new language model, select **Add > Models** in the upper right corner. A window will open in which you can view available Palantir-provided and registered models.

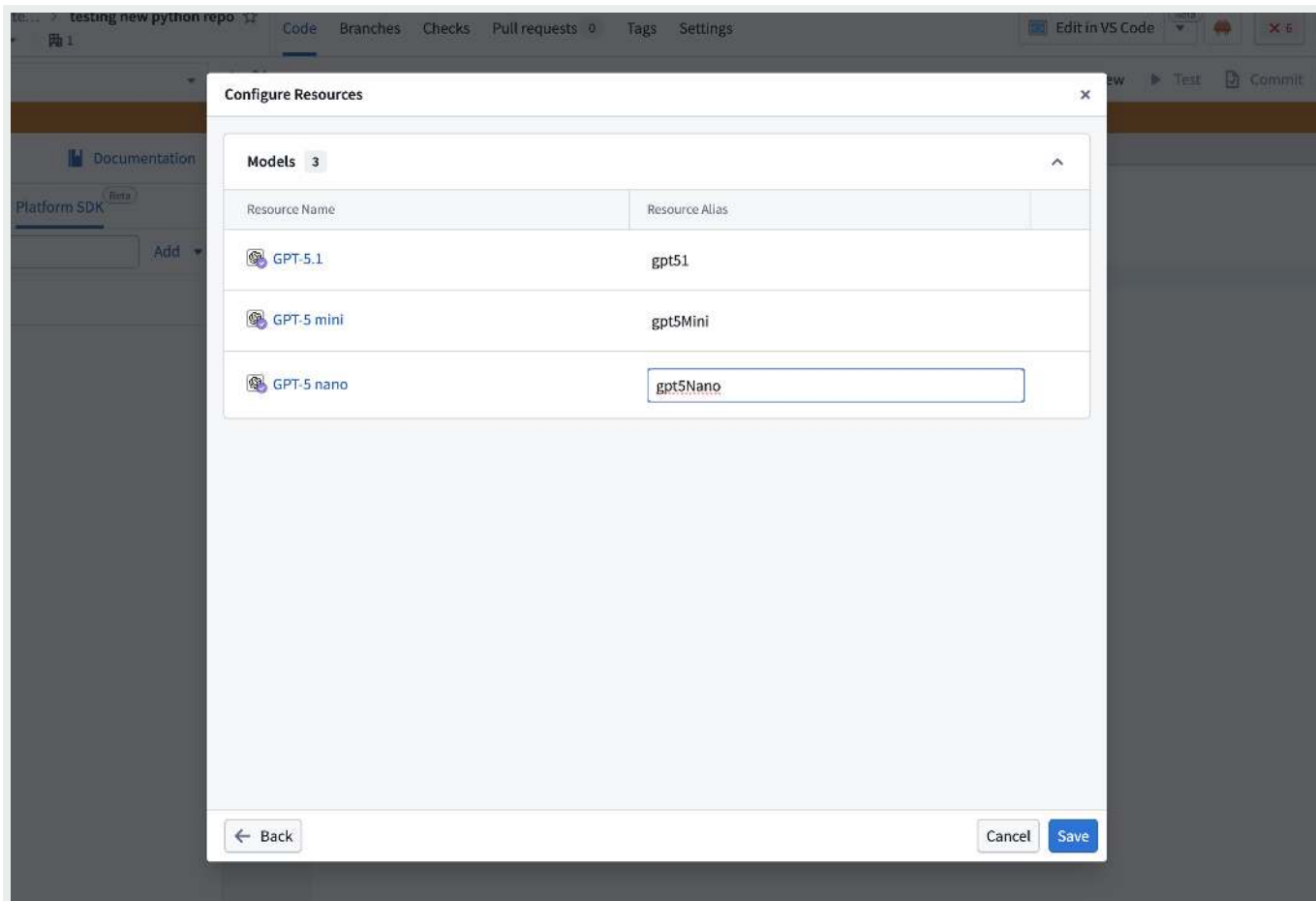


3. Select the models to import, then choose **Confirm selection**. A configuration dialog will open in which you can configure aliases for each selected model. Select the pen icon near the alias to make edits, or choose to keep the defaults.

i Alias keys must be unique within the repository.

[API Reference ↗](#)

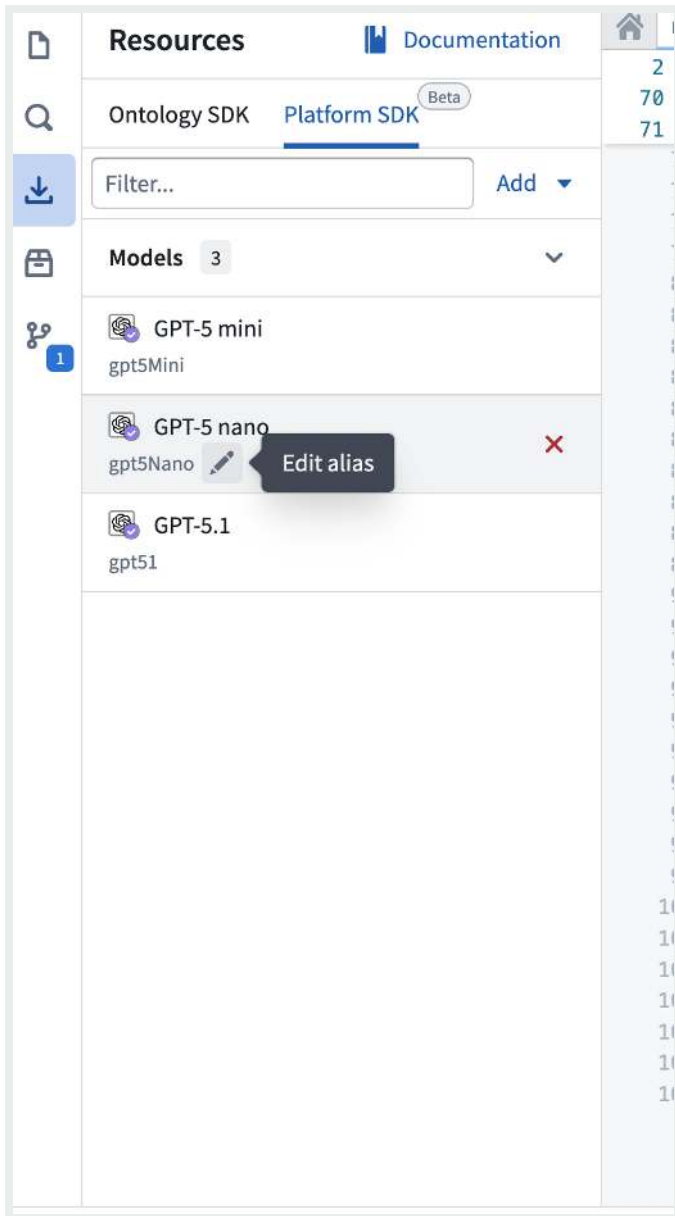
[Send feedback](#)



4. The imported models will appear in the **Platform SDK** tab in the **Resource imports** side panel. You can edit any alias inline by selecting the pen icon next to the alias.

[API Reference ↗](#)

[Send feedback](#)



Use a model alias in code

To use a model alias in your function, import the alias utility and reference the alias by name. The alias resolves to a model RID that you can pass to a model client:



For a complete example of calling a language model using an alias, see [Write a function that uses a language model](#).



Model aliases work in functions added to [Marketplace products](#), but cannot be remapped during installation. If the model referenced by an alias is not available in the target environment, the function will fail to resolve the alias at runtime.

PREVIOUS

← Custom aliases

NEXT

Source aliases →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

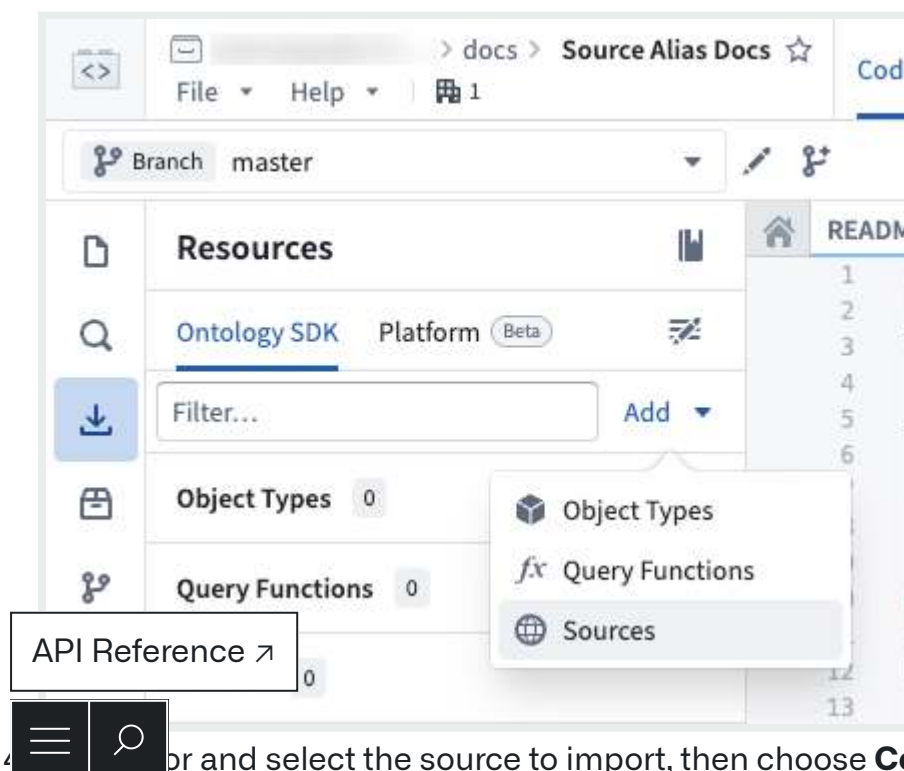
Source aliases

Source aliases are named, portable references to [Data Connection sources](#). By referring to a source by its alias key instead of a specific source identifier, you decouple your function logic from any one source. This keeps your functions portable across environments.

Import a source

To import a source:

1. Open a TypeScript v2 or Python code repository and navigate to the **Resource imports** panel.
2. Select the **Ontology SDK** tab.
3. Select **Add > Sources**.

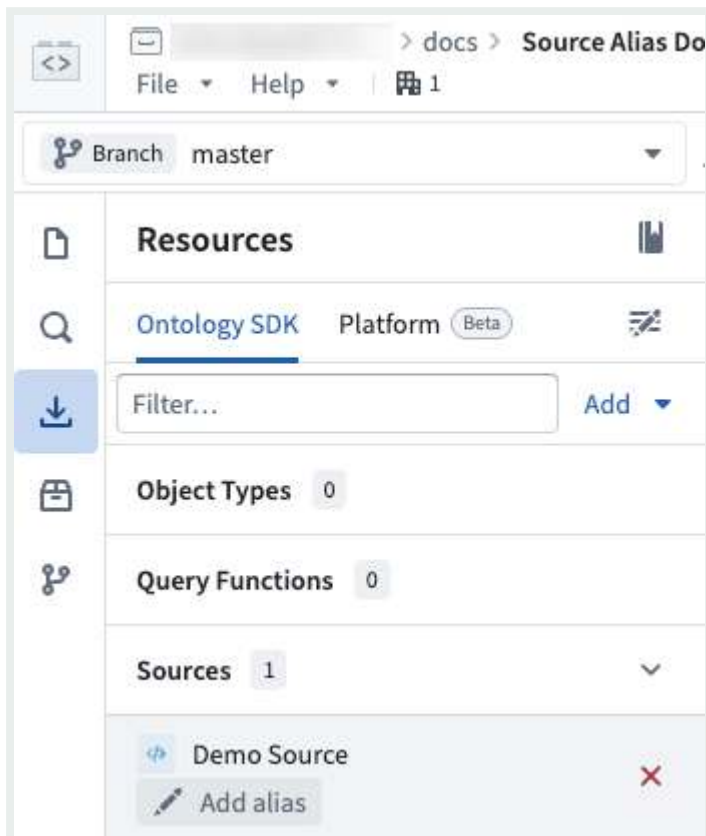


Send feedback

Add a source alias

After importing a source, you can add an alias to it:

1. In the **Resource imports** panel, locate the source under **Sources**.
2. Select **Add alias** below the source name.



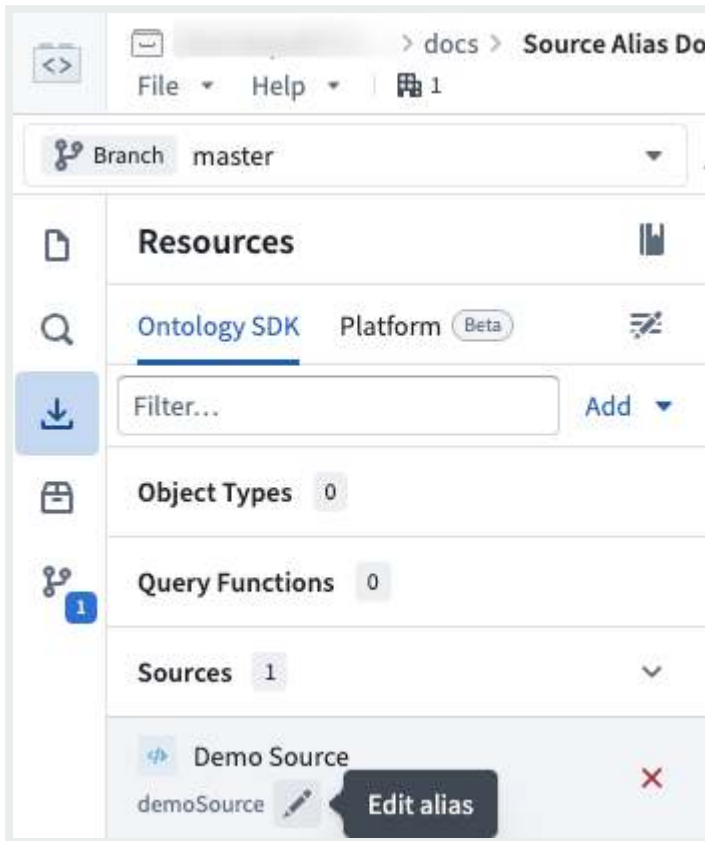
i Alias keys must be unique within the repository.

Edit a source alias

To edit an existing source alias, navigate to the **Sources** section in the **Resource imports** panel. Select the pen icon next to the alias to edit it inline.

[API Reference ↗](#)

[Send feedback](#)



Use a source alias in code

To use a source alias in your function, reference the alias by its key wherever you declare and retrieve the source. Pass the alias key to the `sources` configuration and to `getSource` (TypeScript v2) or `get_source` (Python):

TYPESCRIPT V2 PYTHON

```
1 import { getSource, getHttpsConnection, getFetch } from "@palantir/functi
2
3 export const config = {
4   sources: ["demoSource"]
5 }
6
7 async function myExternalFunction(): Promise<string> {
8   const source = await getSource("demoSource");
9   const { url } = getHttpsConnection(source);
10  fetch = await getFetch(source);
11
12  const response = await fetch(url);
13
```

API Reference ↗

Send feedback


```
14     return response.text();
15 }
```

If you need the resource identifier (RID) of the source that an alias resolves to, use the `Aliases.source` utility in TypeScript v2 or the `source` utility in Python. Read the resolved RID from the `.rid` property:

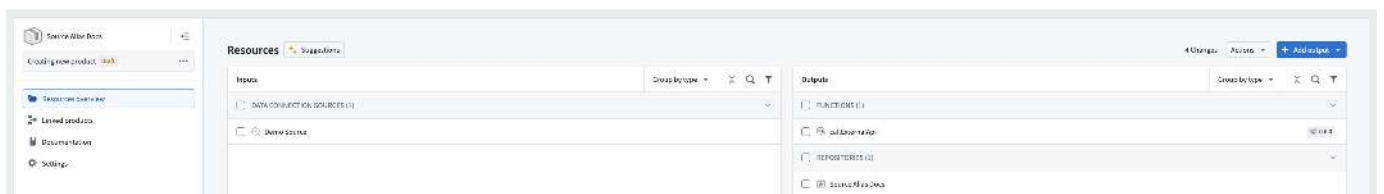
TYPESCRIPT V2 PYTHON

```
1 import { Aliases } from "@osdk/functions";
2
3 const sourceRid = Aliases.source("demoSource").rid;
```

For more information about using sources in functions, including accessing credentials and using pre-configured clients, see [Make API calls from functions](#).

Use source aliases with Marketplace

When you add a function that uses source aliases to a [Marketplace product](#), each aliased source appears as a configurable input under **Data connection sources**. Installers can map each input source to an available source in their environment during installation, without needing to modify the function source code.



Set a description

To help installers understand which source to provide, you can add a description to the source. Click the **API Reference** link next to the source under **Inputs** to open the **Details** panel, then enter a description on the **General** tab.

Send feedback



Installation experience

During installation, installers see the source description and can select a source appropriate for their environment. Marketplace remaps the selected source to the defined alias automatically, so the function code remains untouched.



PREVIOUS
← [Model aliases](#)

NEXT
[Unit testing / Getting started](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Functions / Unit testing / Getting started

Getting started

Functions ships with support for [Jest](#) unit tests. Follow the steps in this guide to get unit testing tools set up for your repository.

By default, functions includes a unit test located in the test file `functions-typescript/src/__tests__/index.ts`. You can create test files anywhere in the `__tests__` folder.

Example

For example, we may want to test the following function `addOne` in `functions-typescript/src/index.ts`:

```
1 import { Function, Integer } from "@foundry/functions-api";
2
3 export class MyFunctions {
4
5     @Function()
6     public addOne(n: Integer): Integer {
7         return n + 1;
8     }
9 }
```

We can test the function `addOne` by writing the following test `test add one`:

API Reference ↗

```
import { MyFunctions } from ".."
describe("example test suite", () => {
```

Send feedback

```

4   const myFunctions = new MyFunctions();
5   test("test add one", () => {
6       expect(myFunctions.addOne(42)).toEqual(43);
7   });
8 });

```

Refer to the [Jest API](#) for details about the full testing API available to you.

Running tests

You can run all your tests by clicking on the **Test** button located on the top right, or run each individual test by clicking on the triangular "Play" button located beside the line number for each test.

The screenshot shows a code editor with a file named `index.ts` containing the following code:

```

1 import { MyFunctions } from ".";
2 import { verifyOntologyEditFunction, whenObjectSet } from "@foundry/functions-testing-lib";
3 import { Objects, Aircraft } from "@foundry/ontology-api";
4
5 describe("my test suite", () => {
6     const myFunctions = new MyFunctions();
7
8     test("test my function", () => {
9         expect(myFunctions.addOne(42)).toEqual(43);
10    });
11
12    test("object property", () => {
13        const UOWC = Objects.create().aircraft(1);
14        UOWC.currentLocation = "TUS";
15        expect(myFunctions.getLocation(UOWC)).toEqual("TUS");
16    });
17 });
18

```

The bottom panel displays the test results:

```

2 of 2 tests done
PASS src/___tests___/index.ts
  my test suite
    ✓ test my function (6 ms)
    ✓ object property (4 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 1.528 s, estimated 2 s
Ran all test suites matching "/\scratch\/standalone\/682d70be-8c62-4476-a73f-628f3b7888fb\/code-assist\/contents\/typescript\/functions-typescript\/src\/___tests___\/index.ts/i with tests matching "(my test suite test my function|my test suite object property)$".

```

A red box highlights the **Test** button in the top right corner of the editor. Another red box highlights the triangular "Play" button next to line 8. A third red box highlights the **API Reference** link in the bottom left corner.

When you click **Commit**, all tests will also run in Checks:

[Send feedback](#)



Next steps

Next, learn about the range of options available for testing functions that interact with the Ontology:

- [Create stub objects](#)
- [Verify Ontology edits](#)
- [Stub Object searches and aggregations](#)

PREVIOUS
← Aliases / Source aliases

NEXT
Create stub objects →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Create stub objects

You can create and define your mock objects using `Objects.create()`, which can be used the same way as if it was a regular function. You can then use these mock objects when writing unit tests. Here is an example:

```
1 import { MyFunctions } from ".."
2
3 import { Objects, ExampleDataAirport } from "@foundry/ontology-api";
4
5 describe("example test suite", () => {
6     const myFunctions = new MyFunctions();
7
8     test("test created objects", () => {
9         const JFK = Objects.create().exampleDataAirport("JFK Test");
10        JFK.displayAirportName = "John F. Kennedy International";
11        expect(myFunctions.getAirportName(JFK)).toEqual("John F. Kennedy");
12    });
13 });
```

For reference, the above example is using the Jest syntax `expect(...).toEqual(...)`.

PREVIOUS

← Getting started

NEXT

Verify Ontology edits →

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Unit testing / Verify Ontology edits

Verify Ontology edits

You can use the `verifyOntologyEditFunction()` API to verify edits performed by your function. You need to import it from `"@foundry/functions-testing-lib"`. This allows you to create unit tests around the workflows listed below.

Verify object creation

You can use the `.createsObjects` method to verify an object creation. Here's an example:

```
1 import { MyFunctions } from ".."
2
3 import { Objects , ExampleDataAirport } from "@foundry/ontology-api";
4 import { verifyOntologyEditFunction } from "@foundry/functions-testing-lib"
5
6 describe("example test suite", () => {
7   const myFunctions = new MyFunctions();
8
9   test("create airport", () => {
10     verifyOntologyEditFunction(() => myFunctions.createAirport("airportCode",
11       .createsObject(
12         {
13           objectType: ExampleDataAirport,
14           properties: {
15             airport: "airportCode",
16             displayAirportName: "airportDisplayName",
17           },
18         });
19   });
20 });
```

[API Reference ↗](#)



used to test the following function:

[Send feedback](#)


```

1 import { Function, OntologyEditFunction, Edits } from "@foundry/functions"
2 import { Objects, ExampleDataAirport } from "@foundry/ontology-api";
3
4 export class MyFunctions {
5
6     @Edits(ExampleDataAirport)
7     @OntologyEditFunction()
8     public createAirport(airport: string, displayName: string): void {
9         const newAirport = Objects.create().exampleDataAirport(airport);
10         newAirport.displayAirportName = displayName;
11     }
12 }

```

Verify edits on a newly created object

You can verify edits that are created involving a newly created object. For example, you may want to create a new `ExampleDataFlight` objects and verify that the link is created to the `new-flight-delay-0`. Here's an example:

```

1 import { MyFunctions } from ".."
2
3 import { Objects, ExampleDataFlight, ExampleFlightDelayEvent } from "@foundry/ontology-api"
4 import { verifyOntologyEditFunction } from "@foundry/functions-testing-lib"
5
6 describe("example test suite", () => {
7     const myFunctions = new MyFunctions();
8
9     test("single key with single created object", () => {
10         const flight = Objects.create().exampleDataFlight("flightTest");
11         verifyOntologyEditFunction(() => myFunctions.createAndLinkDelayEvent(flight, {
12             .createsObject({
13                 objectType: ExampleFlightDelayEvent,
14                 properties: {
15                     eventId: "new-flight-delay-0",
16                 },
17             })
18             .addsLink(edits => ({
19                 link: flight.flightDelayEvent,
20                 linkedObject: edits.createdObjects.byObjectType(ExampleFlightDelayEvent)
21             })))

```

API Reference ↗

Send feedback

```
22     });  
23 });
```

This can be used to test the following function:

```
1  import { Function, Integer, OntologyEditFunction, Edits } from "@foundry/ry/  
2  import { Objects, ExampleDataFlight, ExampleFlightDelayEvent } from "@fou  
3  
4  export class MyFunctions {  
5  
6      @Edits(ExampleDataFlight, ExampleFlightDelayEvent )  
7      @OntologyEditFunction()  
8      public createAndLinkDelays(flight: ExampleDataFlight, numDelay: Integer)  
9          for (let n = 0; n < numDelay; n++) {  
10              const delay = Objects.create().exampleFlightDelayEvent(`new-f  
11              flight.flightDelayEvent.add(delay);  
12          }  
13      }  
14  }
```

Verify object property edits

You can verify edits to the property using `.modifiesObjects`. Here's an example:

```
1  import { MyFunctions } from ".."  
2  
3  import { Objects , ExampleDataFlight, ExampleFlightDelayEvent } from "@fo  
4  import { verifyOntologyEditFunction } from "@foundry/functions-testing-li  
5  
6  describe("example test suite", () => {  
7      const myFunctions = new MyFunctions();  
8  
9      test("modifies aircraft of the flight", () => {  
10          const flight = Objects.create().exampleDataFlight("NY -> LA");  
11          const oldAircraft = Objects.create().exampleDataAircraft("N11111"  
12          flight.aircraft.set(oldAircraft);  
13          const newAircraft = Objects.create().exampleDataAircraft("A00000"  
14          verifyOntologyEditFunction(() => myFunctions.assignAircraftToFlig  
15              .modifiesObject(  
16                  {  
17                      object: flight,
```

API Reference ➤

Send feedback

```

18         properties: {
19             tailNumber: "A00000"
20         }
21     })
22 });
23 });

```

This can be used to test the following function:

```

1  import { Function, OntologyEditFunction, Edits } from "@foundry/functions"
2  import { Objects, ExampleDataFlight, ExampleFlightDelayEvent } from "@foundry/ontology"
3
4  export class MyFunctions {
5
6      @Edits(ExampleDataFlight)
7      @OntologyEditFunction()
8      public assignAircraftToFlight(flight: ExampleDataFlight, aircraft: ExampleDataFlight) {
9          flight.aircraft.clear();
10         aircraft.flight.set(flight);
11         flight.tailNumber = aircraft.tailNumber;
12     }
13 }

```

Verify no other edits to an object

You can ensure there are no other edits using the optional `.hasNoMoreEdits()`. This means that only the specified edits are allowed, and the verification will fail if other edits are detected. Here's an example:

```

1  import { MyFunctions } from ".."
2
3  import { Objects, ExampleDataFlight, ExampleFlightDelayEvent } from "@foundry/ontology"
4  import { verifyOntologyEditFunction } from "@foundry/functions-testing-library"
5
6  describe("example test suite", () => {
7      const myFunctions = new MyFunctions();
8
9      test("single key with linked object", () => {
10         const flight = Objects.create().exampleDataFlight;
11         const delay = Objects.create().exampleFlightDelayEvent;

```

API Reference ↗

Send feedback

```

12         verifyOntologyEditFunction(() => myFunctions.linkDelays(flight
13             .addLink({link: flight.flightDelayEvent, linkedObject: o
14             .hasNoMoreEdits();
15     });
16 });

```

When using `.hasNoMoreEdits()`, you can ignore specific kinds of edits that take place. You do this by passing an object with some or all of the following:

- `ignoreExtraCreatedObjects: true`
- `ignoreExtraModifiedObjects: true`
- `ignoreExtraDeletedObjects: true`
- `ignoreExtraLinkedObjects: true`
- `ignoreExtraUnlinkedObjects: true`

Verify link creation to an object

You can verify link creation on an object using `.addLink`. Here's an example:

```

1  import { MyFunctions } from ".."
2
3  import { Objects , ExampleDataFlight, ExampleFlightDelayEvent } from "@foundry/ontology-testing-lib"
4  import { verifyOntologyEditFunction } from "@foundry/functions-testing-lib"
5
6  describe("example test suite", () => {
7      const myFunctions = new MyFunctions();
8
9      test("single key with linked object", () => {
10         const flight = Objects.create().exampleDataFlight("flightAnother")
11         const delay = Objects.create().exampleFlightDelayEvent("new-flight-delay")
12         verifyOntologyEditFunction(() => myFunctions.linkDelays(flight
13             .addLink({link: flight.flightDelayEvent, linkedObject: delay})
14             .hasNoMoreEdits();
15     });
16 });

```

T [API Reference ↗](#) to testing for the same link going in the opposite direction:

Send feedback

```

1 import { MyFunctions } from ".."
2
3 import { Objects , ExampleDataFlight, ExampleFlightDelayEvent } from "@foundry/functions-testing-lib"
4 import { verifyOntologyEditFunction } from "@foundry/functions-testing-lib"
5
6 describe("example test suite", () => {
7     const myFunctions = new MyFunctions();
8
9     test("single key with linked object reverse", () => {
10         const flight = Objects.create().exampleDataFlight("flightAnot")
11         const delay = Objects.create().exampleFlightDelayEvent("new-flight")
12         verifyOntologyEditFunction(() => myFunctions.linkDelays(flight, delay))
13             .addsLink({link: delay.flight, linkedObject: flight })
14             .hasNoMoreEdits();
15     });
16 });

```

This can be used to test the following function:

```

1 import { Function, OntologyEditFunction, Edits } from "@foundry/functions-testing-lib"
2 import { Objects, ExampleDataFlight, ExampleFlightDelayEvent } from "@foundry/functions-testing-lib"
3
4 export class MyFunctions {
5
6     @Edits(ExampleDataFlight, ExampleFlightDelayEvent )
7     @OntologyEditFunction()
8     public linkDelays(flight: ExampleDataFlight, delay: ExampleFlightDelayEvent) {
9         flight.flightDelayEvent.add(delay);
10     }
11 }

```

Verify link removal from an object

You can verify link removal from an object using `.removesLink`. Here's an example:

API Reference ↗

```

1 import { Functions } from ".."

```

```

2
3 import { Objects , ExampleDataFlight, ExampleFlightDelayEvent } from "@foundry/functions-testing-lib"
4 import { verifyOntologyEditFunction } from "@foundry/functions-testing-lib"
5

```

Send feedback

```

6 describe("example test suite", () => {
7     const myFunctions = new MyFunctions();
8
9     test("test link removal", () => {
10         const flight = Objects.create().exampleDataFlight("flightAnot
11         const delay = Objects.create().exampleFlightDelayEvent("new-f
12         flight.flightDelayEvent.add(delay);
13         verifyOntologyEditFunction(() => myFunctions.removeAllDelays(
14             .removesLink({link: flight.flightDelayEvent, unlinkedObjec
15             .hasNoMoreEdits();
16     });
17 });

```

This can be used to test the following function:

```

1 import { Function, OntologyEditFunction, Edits } from "@foundry/functions"
2 import { Objects, ExampleDataFlight, ExampleFlightDelayEvent } from "@fou
3
4 export class MyFunctions {
5
6     @Edits(ExampleDataFlight, ExampleFlightDelayEvent)
7     @OntologyEditFunction()
8     public removeAllDelays(flight: ExampleDataFlight): void {
9         flight.flightDelayEvent.clear();
10    }
11 }

```

Verify deleting an object

You can verify deleting an object using `.deletesObject`. Here's an example:

```

1 import { MyFunctions } from ".."
2
3 import { Objects, ExampleDataFlight } from "@foundry/ontology-api";
4 import { verifyOntologyEditFunction } from "@foundry/functions-testing-li
5
6 describe("example test suite", () => {
7     const myFunctions = new MyFunctions();
8
9     test("test object deletion", () => {
10         const flight = Objects.create().exampleDataFlight("flightAnot

```

API Reference ↗

Send feedback

```

11         verifyOntologyEditFunction(() => myFunctions.deleteFlight(fli
12             .deletesObject(flight)
13             .hasNoMoreEdits());
14     });
15 });

```

This can be used to test the following function:

```

1  import { Function, OntologyEditFunction, Edits } from "@foundry/functions
2  import { Objects, ExampleDataFlight } from "@foundry/ontology-api";
3
4  export class MyFunctions {
5
6      @Edits(ExampleDataFlight)
7      @OntologyEditFunction()
8      public deleteFlight(flight: ExampleDataFlight): void {
9          flight.delete();
10     }
11 }

```

Verify multiple objects were created

You can use the `.createsObjects` method and pass in a list to create multiple objects to test on. Here's an example:

```

1  import { MyFunctions } from ".."
2
3  import { Objects, ExampleDataFlight, ExampleFlightDelayEvent } from "@fo
4  import { verifyOntologyEditFunction } from "@foundry/functions-testing-li
5
6  describe("example test suite", () => {
7      const myFunctions = new MyFunctions();
8
9      test("single key with many created objects", () => {
10         const flight = Objects.create().exampleDataFlight("flightTest
11         verifyOntologyEditFunction(() => myFunctions.createAndLinkDel
12         .createsObjects(
13             [0, 1, 2].map(i => ({
14                 objectType: ExampleFlightDelay
15                 properties: {
16                     eventId: "new-flight-delay-" + i,

```

API Reference ↗

Send feedback

```

17         },
18     })),
19 )
20     .addsLinks(edits =>
21         edits.createdObjects.byObjectType(ExampleFlightDelayEvent
22             link: flight.flightDelayEvent,
23             linkedObject: event,
24         )),
25 )
26     .hasNoMoreEdits();
27 });
28 });

```

This can be used to test the following function:

```

1  import { Function, Integer, OntologyEditFunction, Edits } from "@foundry/
2  import { Objects, ExampleDataFlight, ExampleFlightDelayEvent } from "@fou
3
4  export class MyFunctions {
5
6      @Edits(ExampleDataFlight, ExampleFlightDelayEvent )
7      @OntologyEditFunction()
8      public createAndLinkDelays(flight: ExampleDataFlight, numDelay: Integer) {
9          for (let n = 0; n < numDelay; n++) {
10             const delay = Objects.create().exampleFlightDelayEvent(`new-f
11             flight.flightDelayEvent.add(delay);
12         }
13     }
14 }

```

Asynchronous ontology edits

You can verify asynchronous ontology edits as follows:

```

1  import { verifyOntologyEditFunction } from "@foundry/functions-testing"
2
3  async edit function", async () => {
4      const obj = Objects.create().objectWithAllProperties()
5      (await verifyOntologyEditFunction(() => myFunction(obj)))
6      object: obj,

```

API Reference ↗

Send feedback


```

7         properties: {
8             timestampProperty: makeTimestamp(),
9         },
10    });
11 });

```

Multiple verifications

As we have seen in the examples above, we can chain verifications. The following pattern illustrates this:

```

1  import { verifyOntologyEditFunction } from "@foundry/functions-testing-lib";
2  import { Objects, ExampleDataObject } from "@foundry/ontology-api";
3
4  test("multiple action edit", () => {
5      verifyOntologyEditFunction(() => myFunctions.multistageEdits("objectID", {
6          .createsObject({...})
7          .modifiesObjects({...})
8          .addsLinks({...})
9          .removesLinks({...})
10         .deletesObject(...)
11         .hasNoMoreEdits();
12    }));

```

PREVIOUS

← Create stub objects

NEXT

Stub object searches and aggregations →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

Send feedback

Ontology building / Functions / Unit testing / Stub object searches and aggregations

Stub object searches and aggregations

When writing unit tests you may want to create canned answers (also called "stubs") for object sets searches or object aggregations to dictate the responses to the calls your code is making when writing unit tests. You need to import `{ whenObjectSet }` from `"@foundry/functions-testing-lib"` to use stubs.

Testing filters on object sets

```
1 import { Objects } from "@foundry/ontology-api";
2
3 const objectSet = Objects.search().objectType();
4
5 expect(myFunctions.filterObjectSet(objectSet))
6   .toEqual(objectSet.filter(s => s.prop.range().gte(0)))
```

Testing aggregations on an object property using stubs

You can define the response to aggregation calls using stubs.

```
1 import { whenObjectSet } from "@foundry/functions-testing-lib"
2
3 (Objects.search().objectType().sum(s => s.property)).thenRetu
```

API Reference ↗



Send feedback

This means that whenever `Objects.search().objectType().sum(s => s.property))` is run, the result will be 55.

Testing objects using stubs

You can also define the response to certain object searches using stubs.

```
1 import { whenObjectSet } from "@foundry/functions-testing-lib";  
2  
3 whenObjectSet(Objects.search().objectType().orderBy().takeAsync(10)).thenReturn(  
4   await expect(myFunctions.aggregateSum(objectSet)).resolves.toEqual(65);
```

This means that whenever this particular objects search aggregation is run, the property sum will resolve to 65.

Testing different object sets using stubs

You can mock multiple specific object set searches by overloading the search constructor. You must give each object a `rid` property.

```
1 import { whenObjectSet } from "@foundry/functions-testing-lib";  
2  
3 const objA = Objects.create().objectType('a');  
4 const objB = Objects.create().objectType('b');  
5  
6 objA.rid = 'ridA';  
7 objB.rid = 'ridB';  
8  
9 whenObjectSet(Objects.search().objType([objA]).all()).thenReturn([objA]);  
10 whenObjectSet(Objects.search().objType([objB, objA]).all()).thenReturn([objB, objA]);
```

[PREVIOUS](#)

← [API Reference ↗](#)

[NEXT](#)

[Mock dates, timestamps, and UUIDs](#) →

[Send feedback](#)

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Unit testing / Mock dates, timestamps, and UUIDs

Mock dates, timestamps, and UUIDs

You can specify the output of non-deterministic functions by utilizing `jest.spyOn()` to inject a mock to run the test.

UUID functions

You can specify the output of `Uuid` by injecting a mock. Here is an example:

```
1 import { MyFunctions } from ".."
2
3 import { Objects, ExampleDataFlight } from "@foundry/ontology-api";
4 import { verifyOntologyEditFunction } from "@foundry/functions-testing-lib";
5 import { Uuid } from "@foundry/functions-utils";
6
7 describe("example test suite", () => {
8   const myFunctions = new MyFunctions();
9
10  test("creates new flight", () => {
11    const makeUuid = () => "my-uuid";
12    jest.spyOn(Uuid, "random").mockImplementation(() => makeUuid());
13
14    verifyOntologyEditFunction(() => myFunctions.createNewFlight())
15      .createsObject({
16        objectType: ExampleDataFlight,
17        properties: {
18          flightId: makeUuid()
19        }
20      })
21  })
22 })
```

API Reference ↗



Send feedback

This can be used to test the following function:

```
1 import { Function, OntologyEditFunction, Edits } from "@foundry/functions";
2 import { Objects, ExampleDataFlight } from "@foundry/ontology-api";
3 import { Uuid } from "@foundry/functions-utils";
4
5 export class MyFunctions {
6     @Edits(ExampleDataFlight)
7     @OntologyEditFunction()
8     public createNewFlight(): void {
9         Objects.create().exampleDataFlight(Uuid.random());
10    }
11 }
```

Advanced UUID functions

There are certain circumstances where you may want full control over the output of the `Uuid`. This requires you to adjust the code of the function you are testing. For example, the `createNewFlight` function above is wrapped in a class `MyFunctions` and you can add a constructor to the class that takes a supplier with a default value. The updated function with the supplier looks like this:

```
1 import { Function, OntologyEditFunction, Edits } from "@foundry/functions";
2 import { Objects, ExampleDataFlight } from "@foundry/ontology-api";
3 import { Uuid } from "@foundry/functions-utils";
4
5 export class MyFunctions {
6     constructor (private UuidSupplier: () => string = Uuid.random){} // t
7
8     @Edits(ExampleDataFlight)
9     @OntologyEditFunction()
10    public createNewFlightWithConstructor(): void {
11        Objects.create().exampleDataFlight(this.UuidSupplier());
12    }
13 }
```

[API Reference ↗](#)

Then can be tested with full control of the output (in this case we set the generated `Uuid` to be `my-other-uuid`):

[Send feedback](#)

```

1 import { MyFunctions } from ".."
2
3 import { Objects , ExampleDataFlight } from "@foundry/ontology-api";
4 import { verifyOntologyEditFunction } from "@foundry/functions-testing-li
5 import { Uuid } from "@foundry/functions-utils";
6
7 describe("example test suite", () => {
8     const myFunctions = new MyFunctions();
9
10    test("creates new flight with supplier", () => {
11        const myNewFunctions = new MyFunctions(() => "my-other-uuid");
12
13        verifyOntologyEditFunction(() => myNewFunctions.createNewFlightWi
14            .createsObject({
15                objectType: ExampleDataFlight,
16                properties: {
17                    flightId: "my-other-uuid"
18                }
19            })
20
21    })
22 });

```

Timestamp.now() functions

You can specify the output of `Timestamp.now()` by injecting a mock. Here is an example:

```

1 import { MyFunctions } from ".."
2
3 import { Objects , ExampleDataFlight } from "@foundry/ontology-api";
4 import { verifyOntologyEditFunction } from "@foundry/functions-testing-li
5 import { Timestamp } from "@foundry/functions-api";
6
7 describe("example test suite", () => {
8     const myFunctions = new MyFunctions();
9
10    test("test timestamp now", () => {
11        const makeTimestamp = () => Timestamp.fromISOString("2018-06-13T1
12        jest.spyOn(Timestamp, "now").mockImplementation(() => makeTimestamp
13
14    const flight = Objects.create().exampleDataFlight
15    verifyOntologyEditFunction(() => myFunctions.startTakeoff(flight)

```

API Reference ↗

Send feedback


```

16         .modifiesObject({
17             object: flight,
18             properties: {
19                 takeoff: makeTimestamp()
20             }
21         })
22     })
23 });

```

This can be used to test the following function:

```

1  import { Function, OntologyEditFunction, Edits, Timestamp } from "@foundry/ontology-api";
2  import { Objects, ExampleDataFlight } from "@foundry/ontology-api";
3
4  export class MyFunctions {
5      @Edits(ExampleDataFlight)
6      @OntologyEditFunction()
7      public startTakeoff(flight: ExampleDataFlight): void {
8          flight.takeoff = Timestamp.now();
9      }
10 }

```

LocalDate.now() functions

You can specify the output of `LocalDate.now()` by injecting a mock. Here is an example:

```

1  import { MyFunctions } from ".."
2
3  import { Objects, ExampleDataFlight } from "@foundry/ontology-api";
4  import { verifyOntologyEditFunction } from "@foundry/functions-testing-lib";
5  import { LocalDate } from "@foundry/functions-api";
6
7  describe("example test suite", () => {
8      const myFunctions = new MyFunctions();
9
10     it("test LocalDate now", () => {
11         const makeLocalDate = () => LocalDate.fromISOString("2018-06-13");
12         jest.spyOn(LocalDate, "now").mockImplementation(makeLocalDate);
13
14         const flight = Objects.create().exampleDataFlight("flightTest"),

```

[API Reference ↗](#)

[Send feedback](#)

```

15         verifyOntologyEditFunction(() => myFunctions.dateTakeoff(flight))
16         .modifiesObject({
17             object: flight,
18             properties: {
19                 date: makeLocalDate()
20             }
21         })
22     })
23 });

```

This can be used to test the following function:

```

1  import { Function, OntologyEditFunction, Edits, LocalDate } from "@foundry/ontology-api";
2  import { Objects, ExampleDataFlight } from "@foundry/ontology-api";
3
4  export class MyFunctions {
5      @Edits(ExampleDataFlight)
6      @OntologyEditFunction()
7      public dateTakeoff(flight: ExampleDataFlight): void {
8          flight.date = LocalDate.now();
9      }
10 }

```

PREVIOUS

← Stub object searches and aggregations

NEXT

Mock users and groups →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Unit testing / Mock users and groups

Mock users and groups

User mocks

You are able to create partial mock of a user using `createUser`, where all properties besides `id` and `username` are optional. You need to import `{ createUser }` from `"@foundry/functions-testing-lib"`.

```
1 import { MyFunctions } from ".."
2
3 import { verifyOntologyEditFunction, createGroup, createUser } from "@foundry/functions-testing-lib"
4
5 describe("example test suite", () => {
6   const myFunctions = new MyFunctions();
7   test("test users and groups", async () => {
8     const group = createGroup({
9       id: "groupId",
10    });
11    const user = createUser({
12      id: "userId",
13      username: "username",
14    });
15    await expect(myFunctions.searchUsers("userId", "groupId")).resolves
16  });
17 });
```

This can be used to test the following function:

API Reference ↗



rt { Function, OntologyEditFunction, Users, Group,

```
3 export class MyFunctions {
```

Send feedback

```

4   @Function()
5   public async searchUsers(userId: string, groupId: string): Promise<Pr
6       const existingPrincipals = await Promise.all([
7           Users.getUserByIdAsync(userId),
8           Users.getGroupByIdAsync(groupId),
9       ]);
10      return existingPrincipals.filter(r => !!r).map(r => r!);
11  }
12 }

```

Group mocks

You are also able to create partial mock of a group using `createGroup`, where all properties besides `id` are optional. You need to import `{ createGroup }` from `"@foundry/functions-testing-lib"`.

```

1  import { MyFunctions } from ".."
2
3  import { verifyOntologyEditFunction, createGroup } from "@foundry/functions-testing-lib"
4
5  describe("example test suite", () => {
6      const myFunctions = new MyFunctions();
7      test("test groups", async () => {
8          const group = createGroup({
9              id: "groupId",
10          });
11          await expect(myFunctions.searchGroups("groupId")).resolves.toEqual([group]);
12      });
13  });

```

This can be used to test the following function:

```

1  import { Function, OntologyEditFunction, Users, Group, Principal } from "@foundry/functions"
2
3  export class MyFunctions {
4      constructor() {
5          // ...
6      }
7      public async searchGroups(groupId: string): Promise<Principal[]> {
8          const existingPrincipals = await Promise.all([
9              Users.getGroupByIdAsync(groupId),

```

API Reference ↗

Send feedback

```
8         });  
9         return existingPrincipals.filter(r => !!r).map(r => r!);  
10    }  
11 }
```

PREVIOUS

← Mock dates, timestamps, and UUIDs

NEXT

Debug →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

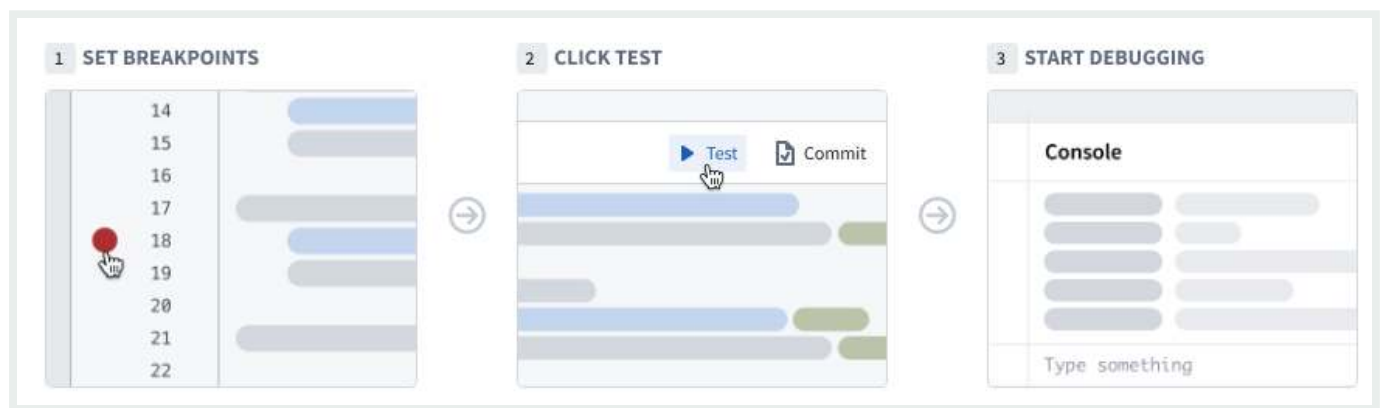
[API Reference ↗](#)

[Send feedback](#)

Ontology building / Functions / Unit testing / Debug

Debug

Use the debugger tool in Code Repositories to examine the behavior of your unit test while it runs. Set breakpoints to pause the execution of the unit test in order to examine variables, and understand functions and libraries.



For detailed information on [setting breakpoints](#), [running the debugger](#), and [examining variables](#), refer to the [function debugging documentation](#).

PREVIOUS

← [Mock users and groups](#)

NEXT

[Interfaces / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[API Reference ↗](#)



Send feedback

404 Error

Page not found... yet

[Go to homepage](#) →

[Cookie Settings ↗](#)

YOUTUBE

X

LINKEDIN

GITHUB

STORE